ANALYSIS PIPELINE · R MARKDOWN · REPRODUCIBLE WORKFLOW

Soil microbiome and physicochemical responses to elephantiasis in banana and plantain

Integrated documentation of the 16S rRNA, ITS2 and soil-chemistry analysis scripts for Gros Michel banana and Dominico Hartón plantain crops in Caldas and Risaralda, Colombia

2
marker genes
(16S rRNA · ITS2)

10
R Markdown scripts:
7 universal + 3 soil

12
soil physicochemical
variables

9,999
permutations, restricted
within farm

AUTHOR
Sergio Pardo-Díaz

AFFILIATION
Corporación Colombiana de Investigación Agropecuaria (AGROSAVIA), Colombia

ASSOCIATED MANUSCRIPT
Comparative analysis of the microbial responses to elephantiasis in banana and plantain. *Frontiers in Soil Science* (under review).

DOCUMENT VERSION
October 2026 · universal microbiome scripts (bacteria + fungi)

LANGUAGES AND TOOLS
R (phyloseq, vegan, lme4, ANCOMBC, randomForest), QIIME 2 outputs

CONTENTS
Pipeline overview (pp. 2–11) · full source code listings as Annexes A–B (pp. 12–309)

1. Study and analytical overview

This repository contains the complete statistical workflow used to compare the soil microbial communities (bacteria and fungi) and the soil physicochemical profile of plants **symptomatic** and **asymptomatic** for elephantiasis in two *Musa* crops: **Gros Michel banana** and **Dominico Hartón plantain**. Samples come from commercial farms in the departments of Caldas and Risaralda, Colombia.

1.1 Sampling design as encoded in the scripts

FACTOR	METADATA COLUMN	ROLE IN THE ANALYSES
Symptom status	`Plant_Condition`	Main factor of interest: diseased (symptomatic) vs. healthy (asymptomatic). It varies *within* each farm.
Crop	`Crop`	Dominico Hartón vs. Gros Michel. It is constant within each farm, so the farm, and not the sample, is the independent unit for crop comparisons.
Farm	`Land`	Blocking unit. Used as a random intercept in mixed models, as permutation strata, or as a fixed covariate.
Department	`Department`	Caldas or Risaralda. Farms are nested within department in the hierarchical PERMANOVA.
Marker	`Marker`	16S rRNA (soil bacteria; 96 samples expected) or ITS2 without truncation (soil fungi; 92 samples expected after filtering).

1.2 Analytical design principles

Farm-aware inference

Every test of symptom status respects the farm structure: as a random intercept `(1 | Farm)` in mixed models, as permutations restricted within `Land` in PERMANOVA, PERMDISP and dbRDA, or as a covariate in ANCOM-BC2. No analysis falls back to unrestricted permutations.

Crop tested at the correct level

Crop is constant within farm, so crop effects are evaluated with farms as the replicates (farm centroids or farm-level contrasts). This keeps the unbalanced sample numbers per crop from being counted as independent replication.

Rarefied vs. unrarefied data by purpose

Diversity, ordination and composition use even-depth rarefied tables (bacteria 5,000 reads; fungi 2,000 reads; seed 123; without replacement). Differential abundance uses the unrarefied, post-filtering counts, as ANCOM-BC2 requires.

Multiple-testing control

Benjamini–Hochberg FDR is applied within explicitly defined families: per model × term, per domain × crop, or per crop. Significance is reported as $q < 0.05$.

Assumption diagnostics

The scripts report Shapiro–Wilk tests, Brown–Forsythe tests, singular-fit checks, PERMDISP, VIF screening, Box's M, Mahalanobis outliers and condition numbers next to the results they qualify. The IQR outliers they flag are for plots only and are never dropped silently.

Parameterised, auditable scripts

All paths, thresholds and variable names live in each YAML `params` block. The scripts stop with explicit errors when inputs, factors or sample counts do not match expectations, and export every table to Excel.

2. Workflow

The microbial pipeline uses the same scripts for bacteria (16S) and fungi (ITS2); each step is run once per domain. The two domains, together with the soil chemistry, are integrated in a final constrained ordination (dbRDA). The physicochemical analyses form an independent module that reads the same sample metadata.

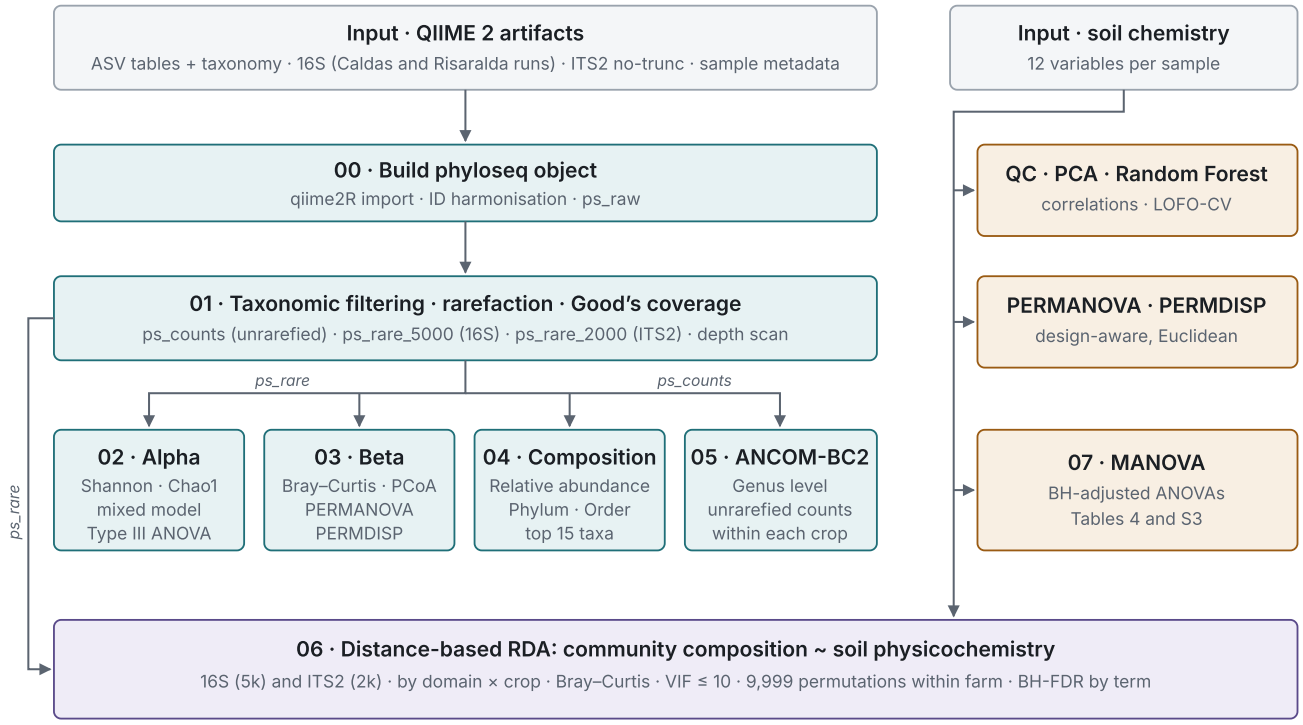


Figure 1. Analysis workflow. Teal: microbial steps; each is one universal script, run once per domain (16S bacteria, ITS2 fungi). Amber: soil physicochemical module. Purple: integration of community and soil data. Step numbers match the script file names.

2.1 Script catalogue

Bacteria and fungi share **one script per step**. The domain is selected with the `domain` parameter (`"Bacterias"` or `"Hongos"`), and every value that differs between them is stored once in `domain_settings.R` .

STEP	SCRIPT	DOMAIN	MAIN ANALYSIS
—	`domain_settings.R`	BOTH	Domain-specific settings (depths, inputs, cut-offs)
—	`run_pipeline.R`	BOTH	Renders steps 00–06 for both domains
00	`00_Build_Phyloseq.Rmd`	PER DOMAIN	QIIME 2 → phyloseq (one or several runs)
01	`01_Filtering_Rarefaction_Coverage.Rmd`	PER DOMAIN	Filtering, rarefaction, Good's coverage, rare-ASV diagnostics
02	`02_Alpha_Diversity.Rmd`	PER DOMAIN	Alpha diversity, farm-aware LMM
03	`03_Beta_Diversity.Rmd`	PER DOMAIN	PCoA, PERMANOVA, PERMDISP
04	`04_Relative_Abundance.Rmd`	PER DOMAIN	Relative abundance (Phylum, Order)
05	`05_DA_ANCOMBC2.Rmd`	PER DOMAIN	Differential abundance (ANCOM-BC2) + LFC-direction audit
06	`06_dbRDA.Rmd`	BOTH	dbRDA with soil chemistry, 16S + ITS2
—	`Analisis_quimico.Rmd`	SOIL	QC, PCA, correlations, Random Forest
—	`PERMANOVA_Fisicoquimicos.Rmd`	SOIL	Design-aware PERMANOVA, PERMDISP
07	`07_MANOVA_Quimica_padjust_TableS3.Rmd`	SOIL	MANOVA, BH-adjusted univariate ANOVAs

The universal scripts merge the most recent fungal (ITS2) scripts with the bacterial (16S) versions. Run on a synthetic test dataset, they reproduce the outputs of the original scripts of each domain: identical phyloseq objects, alpha-diversity models, relative-abundance tables, ANCOM-BC2 results, and PERMANOVA and dbRDA statistics. The only differences were permutation P-values, which are now reproducible because of a fixed seed (`seed = 2026`). Earlier exploratory versions are left out (bacterial Type III ANOVA without a farm term, dbRDA with unrestricted permutations, and MANOVA without p-value adjustment).

3. Microbial community module (16S rRNA and ITS2)

Each step is a single script for both domains. Where the domains differ, the value comes from `domain_settings.R` and is stated below. Annex references point to the full source listing.

STEP 00 Build the phyloseq object

PURPOSE	Import QIIME 2 artifacts and sample metadata into a single, validated `phyloseq` object.
INPUT	ITS2: `table-ITS2-notrunc.qza`, `taxonomy-ITS2-notrunc.qza`, `Metadata_ITS2_notrunc.tsv`. 16S: separate tables and taxonomies for the Caldas and Risaralda runs, merged here, plus `Metadata_Completo.tsv`.
METHODS	`qiime2R::read_qza`; checks the table ↔ taxonomy ↔ metadata ID matching before building; stops if any input file is missing. Phylogenetic trees are not used, because all downstream distances are Bray–Curtis.
OUTPUT	`RDS/ps_raw.rds`
ANNEX	A2

STEP 01 Taxonomic filtering, rarefaction and Good's coverage

PURPOSE	Remove non-target and uninformative features, choose and apply a defensible even sequencing depth, and document coverage.
METHODS	Removes mitochondria and chloroplast sequences, features unassigned at Kingdom or Phylum, and placeholder (alphanumeric) taxon labels. No prevalence or minimum-count filter is applied, matching the lab's endophyte pipeline. Rarefaction uses `rarefy_even_depth` without replacement (`rngseed = 123`). Rarefaction curves are drawn over the full range and as a zoom around the selected depth. Good's coverage $\hat{C} = 1 - f_1/m$ is estimated by resampling (50 replicates) over a depth scan; for fungi the scan covers 500 to 4,000 reads.
DEPTH	Bacteria **5,000** reads per sample · Fungi **2,000** reads per sample, chosen from the post-filtering coverage and depth scan.
OUTPUT	`ps_counts.rds` (unrarefied, filtered) · `ps_filt.rds` · `ps_rel.rds` · `ps_rare_<depth>.rds` · Excel summaries of filtering, coverage and depth.
DOMAIN SETTINGS	Depth and coverage scan come from `domain_settings.R` (bacteria: 1,000–8,000 reads; fungi: 500–4,000 reads).
DIAGNOSTICS	Rare-ASV tables (singletons, doubletons, prevalence) before and after rarefaction, and global vs. within-farm prevalence of rare ASVs.
ANNEX	A3

STEP 02 Alpha diversity: farm-aware linear mixed models

PURPOSE	Test whether within-sample diversity differs between symptom states and crops.
METRICS	Shannon index (analysed untransformed) and Chao1 (analysed as `log1p(Chao1)`), calculated on the rarefied object.
MODEL	`y ~ Crop * Plant_Condition + (1 | Farm)`, fitted by REML with `lmerTest`. Type III ANOVA with Satterthwaite degrees of freedom; estimated marginal means and contrasts with `emmeans`.
DIAGNOSTICS	Singular-fit check (`isSingular`), Shapiro–Wilk test on residuals, Brown–Forsythe test (Levene centred on the median), and verification of the experimental design. IQR outliers (k = 1.5) are flagged for plots only and are not excluded from models.
OUTPUT	Boxplot multipanels per domain × metric, and Excel tables of ANOVA, emmeans, diagnostics and the sample design.
ANNEX	A4

STEP 03 Beta diversity: PCoA, PERMANOVA and PERMDISP

PURPOSE	Test whether community composition differs between symptom states and crops while respecting the farm structure.
METHODS	Bray–Curtis dissimilarity on the rarefied tables; PCoA (`ape`). PERMANOVA (`vegan::adonis2`, 9,999 permutations): • **Primary model:** `Plant_Condition * Crop` with permutations restricted within `Land`, reported both sequentially (*by = terms*) and marginally (*by = margin*). • **Hierarchical model:** `Plant_Condition * Crop + Department/Land`. • **Stratified tests:** symptom status within each crop, within each farm and within each department. • **Crop effect** evaluated at the independent farm level. PERMDISP (`betadisper` + `permutest`, 9,999 permutations within `Land`) tests whether groups differ in multivariate dispersion, which matters for interpreting PERMANOVA. A fixed seed (`seed = 2026`) makes permutation P-values reproducible.
QC	Strict rebuild of the phyloseq object and checks of the expected depth and sample number (96 bacterial, 92 fungal).
ANNEX	A5

STEP 04 Relative abundance (descriptive composition)

METHODS	Sample-wise relative abundance at Phylum and Order. Labels are parsed from either parsed rank columns or QIIME 2 `Taxon` strings, and problematic labels are filtered out. The top 15 taxa are kept, with the rest grouped as “Others” and “Unassigned” shown separately and always last. Values are averaged by Crop × Plant_Condition after renormalisation per sample.
DOMAIN SETTINGS	Fungi: global top-N selection (one consistent legend), computed from `ps_rare_2000`. Bacteria: union of the top-N of each crop, so that a taxon dominant in one crop is not hidden; computed from `ps_rel`. Both options live in the same script (`relabund_top_mode`).
NOTE	These figures are descriptive. Inference on individual taxa is done in step 05.
ANNEX	A6

STEP 05 **Differential abundance with ANCOM-BC2**

PURPOSE	Identify genera whose abundance differs between symptomatic and asymptomatic plants within each crop.
METHODS	Uses unrarefied filtered counts (`ps_counts.rds`) agglomerated at **Genus**. A taxonomic QC step removes labels that are not resolved genera, such as a higher rank posing as a genus. ANCOM-BC2 runs separately within each crop with `fix_formula = Land + Plant_Condition` , so farm enters as a fixed covariate. Settings: `struc_zero = TRUE` , `neg_lb = TRUE` , `lib_cut = 0` , BH adjustment, q < 0.05. Structural zeros, the pseudo-count sensitivity flag (`passed_ss`) and robustness are reported separately. The output includes a top-10 plot by q-value and library-size and design QC.
PREVALENCE	Fungi `prv_cut = 0.10` · Bacteria `prv_cut = 0.01` (domain settings)
AUDIT	An independent descriptive check confirms the LFC sign convention (LFC > 0 = higher in Diseased_Plant) against observed within-farm differences. It does not change the ANCOM-BC2 inference.
ANNEX	A7

STEP 06 **Distance-based redundancy analysis (dbRDA)**

PURPOSE	Quantify how much community variation is explained by soil physicochemistry, separately for each domain × crop.
METHODS	`vegan::dbrda` on Bray–Curtis (bacteria `ps_rare_5000` , fungi `ps_rare_2000`), using predefined soil variables. Collinearity is removed iteratively: the predictor with the highest VIF is dropped while any VIF exceeds 10. Final model: `Community ~ retained predictors` . Global and per-term tests (sequential, Type I) use 9,999 permutations restricted within `Land` , and the script stops rather than falling back if the farm variable is invalid. Per-term BH-FDR is applied within each domain × crop model. Full and adjusted R^2 are exported, and biplots show sample and predictor scores.
INPUT	`Metadata_Completo_plus_edaphic_R_16S_ITS.xlsx` plus the rarefied phyloseq objects for both domains.
ANNEX	A8

3.1 Domain settings (`domain_settings.R`)

PARAMETER	BACTERIA (16S)	FUNGI (ITS2 NO-TRUNC)
Samples analysed	96	92
Rarefaction depth (reads/sample)	5,000	2,000
Random seed / replacement	123 / no	123 / no
Coverage depth scan (reads)	1,000–8,000	500–4,000
Relative-abundance input / Top-N mode	ps_rel / per crop (union)	ps_rare_2000 / global
PERMANOVA, PERMDISP, dbRDA permutations	9,999 within farm	9,999 within farm
ANCOM-BC2 prevalence cut-off	0.01	0.10
ANCOM-BC2 rank / FDR	Genus / BH q < 0.05	Genus / BH q < 0.05
dbRDA collinearity filter	VIF ≤ 10	VIF ≤ 10

4. Soil physicochemical module

The physicochemical analyses use the same sample metadata workbook (`Metadata_Completo_plus_edaphic.xlsx`). Twelve variables were measured: pH, organic matter, P, exchangeable Ca, Mg and K, CEC, Zn, Cu, B, total N and soil respiration (respirometry). The analyses go from exploration (QC, PCA, Random Forest) to design-aware multivariate inference (PERMANOVA) and the manuscript's MANOVA tables.

SOIL · 1 Quality control, PCA, correlations and Random Forest

QC	Sample skewness (moment coefficient, flagged when $\|g\| > 1$) and Tukey-IQR univariate outliers (1.5 × IQR). No automatic transformation is applied; scaling is handled inside each analysis.
PCA	Centred and scaled PCA by crop, with variable vectors coloured by their contribution.
CORRELATIONS	Point-biserial correlations between each variable and symptom status (healthy = 0, diseased = 1), BH-corrected within crop; correlation matrices plotted with `corrplot`.
RANDOM FOREST	`randomForest` with 1,000 trees (`seed = 2026`). Predictive performance is estimated with **leave-one-farm-out cross-validation** (accuracy and balanced accuracy). Variable importance is tested with 499 permutations of the condition label restricted within `Land`.
ANNEX	B1

SOIL · 2 Design-aware PERMANOVA, PERMDISP and univariate analyses

PURPOSE	The definitive multivariate test of the soil profile. It replaces a sample-level factorial MANOVA, so that the 24 vs. 72 sample imbalance between crops is not treated as independent replication of crop type.
REDUNDANCY	Before any distance is calculated, variable pairs with $\|\text{Pearson } r\| \geq 0.90$ are flagged. In each flagged pair, the variable with the higher mean absolute correlation is removed, and this repeats until no pair remains. The condition number of the correlation matrix is reported before and after filtering.
METHODS	Variables standardised once, Euclidean distance, 9,999 permutations (`seed = 2026`): • Symptom status at sample level, permutations restricted within farm, globally and within each crop. • Crop type at the farm level, using one balanced centroid per farm. • Crop × symptom interaction as a farm-level contrast: the multivariate Symptomatic − Asymptomatic difference per farm, compared between crops. PERMDISP and BH-adjusted univariate follow-ups are included.
ANNEX	B2

STEP 07 MANOVA with BH-adjusted univariate ANOVAs (Tables 4 and S3)

VARIABLES	11 responses. CEC is excluded a priori because of severe collinearity with exchangeable Ca (and Mg); it was also removed by the VIF filter in the dbRDA. Responses are centred and scaled.
MODELS	Global `Crop * Plant_Condition` and `Plant_Condition` within each crop. Pillai's trace is the primary statistic; Wilks, Hotelling–Lawley and Roy are also reported. Follow-up univariate ANOVAs are adjusted by BH, with each model × term combination treated as one family.
ASSUMPTIONS	Multivariate outliers by Mahalanobis distance ($\alpha = 0.001$), Box's M test for homogeneity of covariance matrices, and correlation matrix, high-correlation pairs ($\lvert r \rvert \geq 0.90$) and condition number of the responses.
ANNEX	B3

5. Software, reproducibility and repository layout

5.1 R packages used

PURPOSE	PACKAGES
Data import and structures	`qiime2R`, `phyloseq`, `readxl`, `readr`, `fs`
Community ecology	`vegan` (adonis2, betadisper, dbrda, vif.cca), `permute`, `ape`
Mixed and linear models	`lme4`, `lmerTest`, `car`, `emmeans`
Differential abundance	`ANCOMBC` (ANCOM-BC2)
Machine learning	`randomForest`
Data handling	`tidyverse`, `glue`, `stringi`, `conflicted`
Graphics and export	`ggplot2`, `patchwork`, `ggrepel`, `ggnewscale`, `viridis`, `scales`, `corrplot`, `Cairo`, `openxlsx`

Upstream processing (demultiplexing, denoising into ASVs and taxonomic classification) was done in QIIME 2. Its outputs (`.qza`) are the inputs to step 00.

5.2 Suggested repository structure

```
musaceae-elephantiasis-soil/
├── README.md
├── Pipeline_overview.pdf            ← this document
├── microbiome/                      ← one script per step, both domains
│   ├── domain_settings.R
│   ├── run_pipeline.R
│   ├── 00_Build_Phyloseq.Rmd
│   ├── 01_Filtering_Rarefaction_Coverage.Rmd
│   ├── 02_Alpha_Diversity.Rmd
│   ├── 03_Beta_Diversity.Rmd
│   ├── 04_Relative_Abundance.Rmd
│   ├── 05_DA_ANCOMBC2.Rmd
│   └── 06_dbRDA.Rmd                 (16S + ITS2 together)
└── physicochemical/
    ├── Analisis_quimico.Rmd
    ├── PERMANOVA_Fisicoquimicos.Rmd
    └── 07_MANOVA_Quimica_padjust_TableS3.Rmd
```

5.3 Running the pipeline

Data are read from `<project_root>/<domain folder>` (`Bacterias` and `Hongos` by default). Each step writes to its own sub-folder there (`01_Filtering_ASVs/RDS`, `02_Alfa_Diversity`, `03_Beta-Diversity`, `04_Relative_Abundance`, `05_DA`, `06_RDA`), and later steps find their inputs in those folders. To run everything for both domains, set `project_root` in `run_pipeline.R` and execute it. One step can also be rendered for one domain:

```
rmarkdown::render(
  "02_Alpha_Diversity.Rmd",
  params = list(domain = "Bacterias", project_root = "path/to/Soil")
)
```

Every domain-specific value can also be overridden from `params` (e.g. `rare_depth`, `domain_dir`) without editing code.

6. Annex index

Data availability. Raw sequencing data and metadata are not included in the repository. The scripts are released so that the analytical decisions reported in the manuscript can be inspected and reproduced once the data are public.

The annexes reproduce the complete R Markdown source of each final script as a code listing (chunks shown as written, not executed). Page numbers refer to this PDF.

Annex A — Microbiome pipeline (universal: bacteria + fungi)			12
A0	`domain_settings.R`	Domain settings (single source of truth)	13
A1	`run_pipeline.R`	Pipeline driver (both domains)	15
A2	`00_Build_Phyloseq.Rmd`	Build phyloseq	17
A3	`01_Filtering_Rarefaction_Coverage.Rmd`	Filtering, rarefaction, Good's coverage	22
A4	`02_Alpha_Diversity.Rmd`	Alpha diversity (farm-aware LMM)	41
A5	`03_Beta_Diversity.Rmd`	Beta diversity (PCoA, PERMANOVA, PERMDISP)	68
A6	`04_Relative_Abundance.Rmd`	Relative abundance	126
A7	`05_DA_ANCOMBC2.Rmd`	Differential abundance (ANCOM-BC2)	137
A8	`06_dbRDA.Rmd`	dbRDA, 16S + ITS2	212
Annex B — Soil physicochemical analyses			240
B1	`Analisis_quimico.Rmd`	QC, PCA, correlations, Random Forest	241
B2	`PERMANOVA_Fisicoquimicos.Rmd`	Design-aware PERMANOVA, PERMDISP	265
B3	`07_MANOVA_Quimica_padjust_TableS3.Rmd`	MANOVA + BH-adjusted ANOVAs	279

Annex A — Microbiome pipeline (universal: bacteria + fungi)

One script per step for soil bacteria (16S rRNA) and soil fungi (ITS2, no truncation). The domain is chosen with the domain parameter; domain-specific values live in domain_settings.R.

A0	`domain_settings.R`	Domain settings (single source of truth)
A1	`run_pipeline.R`	Pipeline driver (both domains)
A2	`00_Build_Phyloseq.Rmd`	Build phyloseq
A3	`01_Filtering_Rarefaction_Coverage.Rmd`	Filtering, rarefaction, Good's coverage
A4	`02_Alpha_Diversity.Rmd`	Alpha diversity (farm-aware LMM)
A5	`03_Beta_Diversity.Rmd`	Beta diversity (PCoA, PERMANOVA, PERMDISP)
A6	`04_Relative_Abundance.Rmd`	Relative abundance
A7	`05_DA_ANCOMBC2.Rmd`	Differential abundance (ANCOM-BC2)
A8	`06_dbRDA.Rmd`	dbRDA, 16S + ITS2

domain_settings.R

Script: domain_settings.R · source listing (108 lines) · code shown as written; chunks not executed

```
# ======================================================================
# domain_settings.R
# ----------------------------------------------------------------------
# Single source of truth for every value that differs between soil
# bacteria (16S rRNA) and soil fungi (ITS2, no truncation).
#
# All Rmd scripts of the pipeline source this file and call
#   cfg <- domain_settings(domain, overrides)
# so the same script runs for either domain. Change a value here once
# and every step of the pipeline uses it.
#
# Domain codes follow the metadata convention: "Bacterias" | "Hongos".
# ======================================================================

DOMAIN_SETTINGS <- list(

  Bacterias = list(
    label          = "Bacteria",
    label_long     = "Soil bacteria (16S rRNA)",
    marker         = "16S",
    target_kingdom = "Bacteria",

    # Folder (relative to project_root) holding this domain's inputs/outputs
    domain_dir     = "Bacterias",

    # --- Step 00: QIIME 2 inputs (several runs are merged by ASV union) ---
    qiime_tables     = c("table_Caldas.qza", "table_Risaralda.qza"),
    qiime_taxonomies = c("taxonomy_Caldas.qza", "taxonomy_Risaralda.qza"),
    metadata_tsv     = "Metadata_Completo.tsv",
    # The 16S metadata file also lists ITS samples, so extra IDs are allowed
    metadata_must_match_table = FALSE,

    # --- Step 01: rarefaction and coverage ---
    rare_depth      = 5000,
    coverage_depths = c(1000, 2000, 3000, 4000, 5000, 5049, 6000, 7000, 8000),
    rarecurve_zoom_max = 7500,

    # --- Steps 02–05 ---
    expected_n      = 96,
    alpha_ylim      = list(Shannon = c(3.0, 6.5, 0.5, 6.25),   # min, max, step, label y
                           Chao1   = c(0, 1000, 200, 950)),
    relabund_input  = "ps_rel.rds",       # relative abundance of ps_filt
    relabund_top_mode = "per_crop_union", # top N per crop, then union
    ancombc_prv_cut = 0.01,
    counts_expected_n = NA_integer_       # ps_counts.rds: not fixed
  ),

  Hongos = list(
    label          = "Fungi",
    label_long     = "Soil fungi (ITS2, no truncation)",
    marker         = "ITS",
    target_kingdom = "Fungi",

    domain_dir     = "Hongos",

    qiime_tables     = c("table-ITS2-notrunc.qza"),
    qiime_taxonomies = c("taxonomy-ITS2-notrunc.qza"),
    metadata_tsv     = "Metadata_ITS2_notrunc.tsv",
    # IDs in the ITS2 metadata match the DADA2 table one-to-one
    metadata_must_match_table = TRUE,

    rare_depth      = 2000,
    coverage_depths = c(500, 750, 1000, 1250, 1500, 1750, 2000,
                        2250, 2500, 3000, 3500, 4000),
```

```
    rarecurve_zoom_max = 3000,

    expected_n      = 92,
    alpha_ylim      = list(Shannon = c(1.5, 6.0, 0.5, 5.7),
                           Chao1   = c(0, 400, 50, 380)),
    relabund_input  = "ps_rare_2000.rds", # same rarefied object as alpha/beta
    relabund_top_mode = "global",         # one global top N for all groups
    ancombc_prv_cut = 0.10,
    counts_expected_n = 95                # non-zero samples before rarefaction
  )
)

# ------------------------------------------------------------------------
# domain_settings(): returns the settings of one domain. Any non-NULL
# element of `overrides` (typically the Rmd `params`) replaces the
# default value with the same name.
# ------------------------------------------------------------------------
domain_settings <- function(domain, overrides = list()) {
  domain <- as.character(domain)
  if (length(domain) != 1 || !domain %in% names(DOMAIN_SETTINGS)) {
    stop("Unsupported domain: '", paste(domain, collapse = ", "),
         "'. Use one of: ", paste(names(DOMAIN_SETTINGS), collapse = " | "),
         call. = FALSE)
  }
  cfg <- DOMAIN_SETTINGS[[domain]]
  overrides <- as.list(overrides)
  for (nm in intersect(names(overrides), names(cfg))) {
    if (!is.null(overrides[[nm]])) cfg[[nm]] <- overrides[[nm]]
  }
  cfg$domain <- domain
  cfg
}

# ------------------------------------------------------------------------
# domain_path(): folder that holds a domain's data and results.
#   project_root/<domain_dir>   (domain_dir = "." means project_root)
# ------------------------------------------------------------------------
domain_path <- function(project_root, cfg) {
  if (is.null(cfg$domain_dir) || cfg$domain_dir %in% c("", ".")) {
    return(project_root)
  }
  file.path(project_root, cfg$domain_dir)
}

```

run_pipeline.R

Script: run_pipeline.R · source listing (69 lines) · code shown as written; chunks not executed

```r
# ======================================================================
# run_pipeline.R
# ----------------------------------------------------------------------
# Renders the unified soil-microbiome pipeline for both domains.
# Every step is ONE R Markdown script; the domain is chosen with the
# `domain` parameter and all domain-specific values (rarefaction depth,
# QIIME 2 inputs, prevalence cut-off, ...) come from domain_settings.R.
#
# Usage (from this folder):
#   Rscript run_pipeline.R                # all steps, both domains
#   Rscript run_pipeline.R 02 03          # selected steps only
# ======================================================================

# Parent folder that contains one sub-folder per domain
# (folder names are set in domain_settings.R: "Bacterias", "Hongos").
project_root <- "path/to/Soil"

domains <- c("Bacterias", "Hongos")

steps <- c(
  "00" = "00_Build_Phyloseq.Rmd",
  "01" = "01_Filtering_Rarefaction_Coverage.Rmd",
  "02" = "02_Alpha_Diversity.Rmd",
  "03" = "03_Beta_Diversity.Rmd",
  "04" = "04_Relative_Abundance.Rmd",
  "05" = "05_DA_ANCOMBC2.Rmd"
)

args <- commandArgs(trailingOnly = TRUE)
run_steps <- if (length(args)) intersect(c(names(steps), "06"), args) else c(names(steps), "06")

scripts_dir <- normalizePath(".")
reports_dir <- file.path(project_root, "Reports")
dir.create(reports_dir, recursive = TRUE, showWarnings = FALSE)

render_step <- function(rmd, params, out_name) {
  message("▶ ", out_name)
  rmarkdown::render(
    file.path(scripts_dir, rmd),
    params        = params,
    output_file   = out_name,
    output_dir    = reports_dir,
    knit_root_dir = scripts_dir,   # domain_settings.R is found here
    envir         = new.env()
  )
}

# Steps 00–05: once per domain
for (st in intersect(names(steps), run_steps)) {
  for (dom in domains) {
    render_step(
      steps[[st]],
      params   = list(domain = dom, project_root = project_root),
      out_name = paste0(sub("\\.Rmd$", "", steps[[st]]), "_", dom, ".html")
    )
  }
}

# Step 06: dbRDA runs both domains together
if ("06" %in% run_steps) {
  render_step(
    "06_dbRDA.Rmd",
    params   = list(project_root = project_root),
    out_name = "06_dbRDA_Bacteria_Fungi.html"
```

```
  )
}

message("✔ Pipeline finished. Reports in: ", reports_dir)

```

00 — Build phyloseq object from QIIME 2 artifacts | [domain]

Script: 00_Build_Phyloseq.Rmd · source listing (319 lines) · code shown as written; chunks not executed

R chunks

line 30: ```{r setup, message=FALSE, warning=FALSE}
line 56: ```{r check_inputs, message=TRUE, warning=TRUE}
line 78: ```{r import_qiime, message=TRUE, warning=TRUE}
line 169: ```{r metadata, message=TRUE, warning=TRUE}
line 257: ```{r build_phyloseq, message=TRUE, warning=TRUE}
line 316: ```{r session_info}

````
---
title: "00 — Build phyloseq object from QIIME 2 artifacts | `r params$domain`"
author: "Sergio Pardo-Díaz"
date: "`r format(Sys.Date())`"
output:
  html_document:
    toc: true
    toc_depth: 3
    number_sections: true

params:
  # === Domain: "Bacterias" (16S rRNA) or "Hongos" (ITS2 no-trunc) ===
  domain: "Hongos"
  # Parent folder; data are read from <project_root>/<domain_dir>
  project_root: "."
  # NULL = use the default from domain_settings.R
  domain_dir: null
  qiime_tables: null
  qiime_taxonomies: null
  metadata_tsv: null
  settings_file: "domain_settings.R"
---

This script imports one or more QIIME 2 runs of one marker gene and builds a
validated `phyloseq` object (`RDS/ps_raw.rds`). Several runs (e.g. the 16S
Caldas and Risaralda runs) are merged by the **union** of ASVs: an ASV absent
from a run receives zero counts there. Phylogenetic trees are not imported
because all downstream distances are Bray–Curtis.

```{r setup, message=FALSE, warning=FALSE}
suppressPackageStartupMessages({
  library(qiime2R)
  library(phyloseq)
  library(tidyverse)
  library(fs)
})

settings_file <- params$settings_file
if (!file.exists(settings_file)) {
  settings_file <- file.path(dirname(knitr::current_input(dir = TRUE)), settings_file)
}
source(settings_file)

cfg <- domain_settings(params$domain, params)
dom <- cfg$domain

base_dir <- domain_path(params$project_root, cfg)
rds_dir  <- file.path(base_dir, "RDS")
dir_create(rds_dir)

cat("Domain      :", dom, "—", cfg$label_long, "\n")
cat("Data folder :", base_dir, "\n")
cat("QIIME 2 runs:", length(cfg$qiime_tables), "\n")
```
````

````
```{r check_inputs, message=TRUE, warning=TRUE}
if (length(cfg$qiime_tables) != length(cfg$qiime_taxonomies)) {
  stop("❌ qiime_tables and qiime_taxonomies must have the same length (one taxonomy per run).",
       call. = FALSE)
}

p_tabs <- file.path(base_dir, cfg$qiime_tables)
p_taxs <- file.path(base_dir, cfg$qiime_taxonomies)
p_meta <- file.path(base_dir, cfg$metadata_tsv)

paths_needed <- c(p_tabs, p_taxs, p_meta)
if (!all(file_exists(paths_needed))) {
  missing <- paths_needed[!file_exists(paths_needed)]
  stop(
    paste("❌ The following input files are missing for", dom, ":\n",
          paste(missing, collapse = "\n")),
    call. = FALSE
  )
}
message("✔ All input files were found.")
```

```{r import_qiime, message=TRUE, warning=TRUE}
## ------------------------------------------------
## Import every run (table + taxonomy)
## ------------------------------------------------
tabs <- lapply(p_tabs, function(p) {
  message("📥 Importing ASV table: ", basename(p))
  read_qza(p)$data
})

taxs <- lapply(p_taxs, function(p) {
  message("📥 Importing taxonomy: ", basename(p))
  read_qza(p)$data %>% column_to_rownames("Feature.ID")
})

## ------------------------------------------------
## Merge runs: UNION of ASVs (not intersection)
## ------------------------------------------------
fill_zero_asv <- function(tab, all_asv_ids) {
  missing_asv <- setdiff(all_asv_ids, rownames(tab))
  if (length(missing_asv) > 0) {
    zeros <- matrix(0L, nrow = length(missing_asv), ncol = ncol(tab),
                    dimnames = list(missing_asv, colnames(tab)))
    tab <- rbind(tab, zeros)
  }
  tab[all_asv_ids, , drop = FALSE]
}

fill_na_tax <- function(tax, all_asv_ids) {
  missing_asv <- setdiff(all_asv_ids, rownames(tax))
  if (length(missing_asv) > 0) {
    na_mat <- matrix(NA_character_, nrow = length(missing_asv), ncol = ncol(tax),
                     dimnames = list(missing_asv, colnames(tax)))
    tax <- rbind(as.matrix(tax), na_mat)
  }
  as.matrix(tax)[all_asv_ids, , drop = FALSE]
}

all_asv <- Reduce(union, lapply(tabs, rownames))
tab_all <- do.call(cbind, lapply(tabs, fill_zero_asv, all_asv_ids = all_asv))

if (any(duplicated(colnames(tab_all)))) {
  stop("❌ The same sample ID appears in more than one QIIME 2 run.", call. = FALSE)
}

tax_asv  <- Reduce(union, lapply(taxs, rownames))
````
````

```
tax_list <- lapply(taxs, fill_na_tax, all_asv_ids = tax_asv)

# Warn when runs disagree on the taxonomy of a shared ASV (does not stop)
if (length(tax_list) > 1) {
  for (k in 2:length(tax_list)) {
    a <- tax_list[[1]]; b <- tax_list[[k]]
    if (any(!is.na(a) & !is.na(b) & a != b)) {
      warning("⚠️ Taxonomy differs between run 1 and run ", k, " for some ASVs. ",
              "Check that the same classifier/reference database was used.")
    }
  }
}

# Policy: the first run has priority; gaps are filled from the next runs
tax_all <- tax_list[[1]]
if (length(tax_list) > 1) {
  for (k in 2:length(tax_list)) {
    fill <- is.na(tax_all)
    tax_all[fill] <- tax_list[[k]][fill]
  }
}

## ------------------------------------------------
## Table ↔ taxonomy check
## ------------------------------------------------
message("\n🔎 Table ↔ taxonomy check")
message("  - ASVs in table(s):     ", length(all_asv))
message("  - ASVs in taxonomy(ies): ", nrow(tax_all))

asv_missing_tax <- setdiff(all_asv, rownames(tax_all))
if (length(asv_missing_tax) > 0) {
  stop(paste0("❌ ASVs in the table without taxonomy (", length(asv_missing_tax), "):\n",
              paste(head(asv_missing_tax, 20), collapse = "\n")),
       call. = FALSE)
}
message("✔ Every ASV of the table has a taxonomy entry.")

tax_extra <- setdiff(rownames(tax_all), all_asv)
if (length(tax_extra) > 0) {
  message("ℹ ", length(tax_extra), " ASVs in the taxonomy are not present in the table.")
}

# Keep exactly the ASVs, in the order of the table
tax_all <- tax_all[rownames(tab_all), , drop = FALSE]
```

```{r metadata, message=TRUE, warning=TRUE}
## ------------------------------------------------
## Metadata: robust SampleID detection + QIIME 2 cleanup
## ------------------------------------------------
meta_raw <- read_tsv(p_meta, show_col_types = FALSE)

id_candidates <- c("SampleID", "#SampleID", "sampleid", "sample-id",
                   "Sample_Id", "Sample_Name", "ID")
id_col <- id_candidates[id_candidates %in% colnames(meta_raw)]
if (length(id_col) == 0) {
  stop(paste("❌ No sample ID column found in the metadata.",
             "\nAvailable columns:", paste(colnames(meta_raw), collapse = ", ")),
       call. = FALSE)
}
id_col <- id_col[1]
message("✔ Using metadata column as SampleID: ", id_col)

# Sample IDs are used exactly as written (they must match the DADA2 table)
meta <- meta_raw %>%
  rename(SampleID = !!rlang::sym(id_col)) %>%
  mutate(SampleID = as.character(SampleID)) %>%
  filter(!is.na(SampleID), SampleID != "#q2:types")
```

```

duplicated_ids <- meta$SampleID[duplicated(meta$SampleID)]
if (length(duplicated_ids) > 0) {
  stop(paste0("❌ Duplicated IDs in the metadata:\n",
              paste(unique(duplicated_ids), collapse = "\n")),
       call. = FALSE)
}
meta <- meta %>% column_to_rownames("SampleID")
meta_ids <- rownames(meta)

## ------------------------------------------------
## Confirm the orientation of the QIIME 2 table
## ------------------------------------------------
samples_in_cols <- sum(colnames(tab_all) %in% meta_ids)
samples_in_rows <- sum(rownames(tab_all) %in% meta_ids)

message("\n🔎 Table orientation:")
message("  - Metadata IDs found in columns: ", samples_in_cols)
message("  - Metadata IDs found in rows:    ", samples_in_rows)

if (samples_in_rows > samples_in_cols) {
  message("✔ Samples are in rows. Transposing the table...")
  tab_all <- t(tab_all)
} else {
  message("✔ Samples are in columns.")
}
# From here on: rows = ASVs, columns = samples
tab_ids <- colnames(tab_all)

## ------------------------------------------------
## Strict check: table ↔ metadata
## ------------------------------------------------
message("\n🔎 Table ↔ metadata check")
message("  - Samples in table:    ", length(tab_ids))
message("  - Samples in metadata: ", length(meta_ids))

missing_in_meta <- setdiff(tab_ids, meta_ids)
if (length(missing_in_meta) > 0) {
  stop(paste0("❌ Samples in the table that are not in the metadata (",
              length(missing_in_meta), "):\n",
              paste(head(missing_in_meta, 50), collapse = "\n")),
       call. = FALSE)
}

extra_in_meta <- setdiff(meta_ids, tab_ids)
if (length(extra_in_meta) > 0) {
  if (isTRUE(cfg$metadata_must_match_table)) {
    stop(paste0("❌ Samples in the metadata that are not in the table (",
                length(extra_in_meta), "):\n",
                paste(head(extra_in_meta, 50), collapse = "\n")),
         call. = FALSE)
  }
  message("ℹ The metadata has ", length(extra_in_meta),
          " IDs that are not in the table; they are dropped.")
}

# Reorder metadata exactly as the table
meta <- meta[tab_ids, , drop = FALSE]

stopifnot(
  identical(rownames(meta), colnames(tab_all)),
  identical(rownames(tax_all), rownames(tab_all))
)
message("✔ Table, taxonomy and metadata are aligned.")
```

```{r build_phyloseq, message=TRUE, warning=TRUE}
## ------------------------------------------------
```

```r
## Build phyloseq (ps_raw) — without tree
## ------------------------------------------------
ps_raw <- phyloseq(
  otu_table(tab_all, taxa_are_rows = TRUE),
  tax_table(as.matrix(tax_all)),
  sample_data(meta)
)

if (nsamples(ps_raw) != length(tab_ids)) {
  stop("❌ The final number of samples in phyloseq does not match the table.", call. = FALSE)
}
if (ntaxa(ps_raw) != nrow(tab_all)) {
  stop("❌ The final number of ASVs in phyloseq does not match the table.", call. = FALSE)
}

## ------------------------------------------------
## Diagnostics
## ------------------------------------------------
depths <- sample_sums(ps_raw)

message("\n🔎 INSPECTION ps_raw — ", dom)
message("------------------------------------------------")
message("  - Samples: ", nsamples(ps_raw))
message("  - ASVs:    ", ntaxa(ps_raw))
message("  - Reads:   ", sum(depths))
message("\n📊 Sequencing depth:")
print(summary(depths))
message("  - Samples with 0 reads: ", sum(depths == 0))
message("  - Samples with ≥", format(cfg$rare_depth, big.mark = ","),
        " reads (planned rarefaction depth): ",
        sum(depths >= cfg$rare_depth), " / ", length(depths))

meta_ps <- data.frame(sample_data(ps_raw), check.names = FALSE)
message("\n📋 Variables in sample_data():\n  ", paste(colnames(meta_ps), collapse = ", "))

for (v in c("Plant_Condition", "Plant-Condition", "Condition", "Land",
            "Crop", "Department", "Crop_system")) {
  if (v %in% colnames(meta_ps)) {
    message("\n🌱 Variable: ", v)
    print(table(meta_ps[[v]], useNA = "ifany"))
  }
}

message("\n🧬 Taxonomy columns: ",
        paste(colnames(tax_table(ps_raw)), collapse = ", "))
message("✔ phyloseq object validated.")

## ------------------------------------------------
## Save
## ------------------------------------------------
output_rds <- file.path(rds_dir, "ps_raw.rds")
saveRDS(ps_raw, output_rds)
cat("✔ ps_raw.rds saved to:", output_rds, "\n")
```

Session information

```{r session_info}
sessionInfo()
```

01 — Taxonomic filtering, rarefaction and Good's coverage | [domain]

Script: 01_Filtering_Rarefaction_Coverage.Rmd · source listing (1258 lines) · code shown as written; chunks not executed

R chunks

line 44: ```{r setup, message=FALSE, warning=FALSE}
line 84: ```{r helpers, message=TRUE, warning=TRUE}
line 538: ```{r process_domains, message=TRUE, warning=TRUE}
line 970: ```{r run_pipeline, message=TRUE, warning=TRUE}
line 980: ```{r validate_and_summarize, message=TRUE, warning=TRUE}
line 1023: ```{r rare_asv_diagnostics, message=TRUE, warning=FALSE}
line 1127: ```{r rare_asv_within_farm_diagnostics, message=TRUE, warning=FALSE}
line 1255: ```{r session_info}

````
---
title: "01 — Taxonomic filtering, rarefaction and Good's coverage | `r params$domain`"
author: "Sergio Pardo-Díaz"
date: "`r format(Sys.time(), '%Y-%m-%d %H:%M')`"
output:
  html_document:
    toc: true
    toc_depth: 3
    number_sections: true

params:
  # === Domain: "Bacterias" (16S rRNA) or "Hongos" (ITS2 no-trunc) ===
  domain: "Hongos"
  # Parent folder; data are read from <project_root>/<domain_dir>
  project_root: "."
  # NULL = use the default from domain_settings.R
  domain_dir: null
  rare_depth: null
  coverage_depths: null
  rarecurve_zoom_max: null

  # === Good's coverage by resampling ===
  coverage_reps: 50

  # === TAXONOMIC FILTERS (endophyte-pipeline style — option 2) ===
  remove_mitochloro: true
  remove_unassigned: true
  remove_alphanumeric_taxa: true

  # Ranks to scan (NULL = all ranks present in tax_table)
  ranks_to_scan: null

  # === OUTPUT SUBDIR ===
  out_subdir: "01_Filtering_ASVs"

  # === Export / reproducibility ===
  pdf_dpi: 300
  rngseed: 123
  rare_replace: false
  settings_file: "domain_settings.R"
---

## 🟢 CHUNK 1 — SETUP
```{r setup, message=FALSE, warning=FALSE}
suppressPackageStartupMessages({
  library(phyloseq)
  library(tidyverse)
  library(fs)
  library(glue)
  library(vegan)
````
````

```
  library(viridis)
  library(Cairo)
  library(openxlsx)
  library(readr)
})

# ---- Domain settings (single source of truth: domain_settings.R)
settings_file <- params$settings_file
if (!file.exists(settings_file)) {
  settings_file <- file.path(dirname(knitr::current_input(dir = TRUE)), settings_file)
}
source(settings_file)

cfg <- domain_settings(params$domain, params)
dom <- cfg$domain
base_dir <- domain_path(params$project_root, cfg)

log_file <- file.path(base_dir, "pipeline_progress.log")

log_step <- function(step){
  msg <- sprintf("[%s] ▶ %s", format(Sys.time(), "%H:%M:%S"), step)
  cat(msg, "\n")
  flush.console()
  write(msg, file = log_file, append = TRUE)
}

log_step(glue("Pipeline started (01_Filtering – endophyte-pipeline criteria) | {dom}"))
tibble(Domain = dom, Label = cfg$label_long, Data_folder = base_dir,
       Rarefaction_depth = cfg$rare_depth)
```

🟢 CHUNK 2 – Helper functions (aligned with the endophyte pipeline)

```{r helpers, message=TRUE, warning=TRUE}
# ---- Safe rebuild
rebuild_phyloseq_safe <- function(ps) {
  otu <- as(otu_table(ps), "matrix")
  if (!taxa_are_rows(ps)) otu <- t(otu)
  tax <- as(tax_table(ps), "matrix")
  sam <- sample_data(ps)

  if (is.null(rownames(otu)) || any(is.na(rownames(otu)) | rownames(otu) == "")) {
    stop("❌ OTU table has missing/NA/empty taxa_names.")
  }
  if (is.null(rownames(tax)) || any(is.na(rownames(tax)) | rownames(tax) == "")) {
    stop("❌ TAX table has missing/NA/empty taxa_names.")
  }

  if (any(duplicated(rownames(otu)))) {
    otu <- rowsum(otu, group = rownames(otu), reorder = FALSE)
  }
  if (any(duplicated(rownames(tax)))) {
    tax <- tax[!duplicated(rownames(tax)), , drop = FALSE]
  }

  common <- intersect(rownames(otu), rownames(tax))
  if (length(common) == 0) stop("❌ No ASVs shared between OTU and TAX tables.")

  otu2 <- otu[common, , drop = FALSE]
  tax2 <- tax[common, , drop = FALSE]

  ps2 <- phyloseq(
    otu_table(otu2, taxa_are_rows = TRUE),
    tax_table(tax2),
    sam
  )
  validObject(ps2)
  ps2
```

```r
}

# ---- Tax helpers
get_tax_df <- function(ps) {
  if (is.null(tax_table(ps, errorIfNULL = FALSE))) {
    stop("❌ The phyloseq object has no tax_table().")
  }
  as.data.frame(tax_table(ps), stringsAsFactors = FALSE) %>%
    rownames_to_column("ASV")
}

rank_cols <- function(tax_df) {
  setdiff(colnames(tax_df), "ASV")
}

# ---- Robust sanitization
sanitize_text <- function(x) {
  x <- as.character(x)
  x[x %in% c("NA", "NaN", "")] <- NA_character_
  x <- iconv(x, from = "", to = "UTF-8", sub = "byte")
  x <- gsub("[\r\n\t]", " ", x)
  x <- gsub("\\s+", " ", x)
  trimws(x)
}

# ---- Detectors (identical to the endophyte pipeline)
is_alphanumeric_taxon <- function(x) {
  x <- sanitize_text(x)
  out <- rep(FALSE, length(x))
  ok  <- !is.na(x)
  out[ok] <- stringr::str_detect(x[ok], "[0-9]") &
    stringr::str_detect(x[ok], "^[A-Za-z0-9_]+$") &
    !stringr::str_detect(stringr::str_to_lower(x[ok]), "^na$|^nan$")
  out
}

is_mito_or_chloro <- function(x) {
  x <- sanitize_text(x)
  out <- rep(FALSE, length(x))
  ok  <- !is.na(x)
  out[ok] <- stringr::str_detect(stringr::str_to_lower(x[ok]), "mitochondria|chloroplast")
  out
}

is_unassigned <- function(x) {
  x <- sanitize_text(x)
  out <- rep(FALSE, length(x))
  ok  <- !is.na(x)
  if (!any(ok)) return(out)

  y <- stringr::str_to_lower(x[ok])
  y <- stringr::str_replace_all(y, "[_\\-]+", " ")
  y <- stringr::str_squish(y)

  token_hit <- stringr::str_detect(
    y,
    stringr::regex("\\b(unassigned|unclassified|uncultured|unknown|unidentified|uncertain)\\b",
                   ignore_case = TRUE)
  )

  empty_rank_hit <- stringr::str_detect(
    y,
    stringr::regex(
      "\\b[dkpcofgs]__\\s*(na|nan|unassigned|unclassified|uncultured|unknown|unidentified)?\\b\\s*$",
      ignore_case = TRUE
    )
  )

```

```r
  placeholder_hit <- stringr::str_detect(
    y,
    stringr::regex(
      "\\b(incertae\\s+sedis|metagenome|environmental\\s+sample|uncharacterized|ambiguous\\s+taxa)\\b",
      ignore_case = TRUE
    )
  )

  out[ok] <- token_hit | empty_rank_hit | placeholder_hit
  out
}

# ============================================================
# Rarefaction curves
# Produces:
#   A) full range
#   B) zoom around the selected depth
#   + the same two panels exported as separate files
#
# Keeps the original plotting style.
# ============================================================

plot_rarecurve_pdf <- function(
  ps,
  out_pdf,
  vline_depth,
  zoom_max,
  title_text = ""
) {

  # ------------------------------------------------------------
  # Extract OTU matrix
  # ------------------------------------------------------------

  otu <- as(otu_table(ps), "matrix")

  if (taxa_are_rows(ps)) {
    otu <- t(otu)
  }

  sample_totals <- rowSums(otu)

  step_size <- max(
    1,
    floor(max(sample_totals) / 120)
  )


  # ------------------------------------------------------------
  # Compute rarefaction curves
  # ------------------------------------------------------------

  rare_list <- lapply(
    seq_len(nrow(otu)),
    function(i) {

      total_reads <- sample_totals[i]

      depths <- seq(
        1,
        total_reads,
        by = step_size
      )

      if (tail(depths, 1) != total_reads) {
        depths <- c(
          depths,
          total_reads
```

```
        )
      }

      asvs <- vegan::rarefy(
        otu[i, , drop = FALSE],
        depths
      )

      tibble::tibble(
        Sample = rownames(otu)[i],
        Depth  = depths,
        ASVs   = as.numeric(asvs)
      )
    }
  )

  rare_df <- dplyr::bind_rows(
    rare_list
  )


  # ------------------------------------------------------------
  # Shared theme
  # Keeps the original plain style
  # ------------------------------------------------------------

  rare_theme <- ggplot2::theme_bw(
    base_size = 10
  ) +
    ggplot2::theme(

      plot.background =
        ggplot2::element_rect(
          fill = "white",
          color = NA
        ),

      panel.background =
        ggplot2::element_rect(
          fill = "white",
          color = NA
        ),

      panel.grid.major =
        ggplot2::element_line(
          color = "grey92"
        ),

      panel.grid.minor =
        ggplot2::element_blank(),

      plot.title =
        ggplot2::element_text(
          size = 10,
          face = "plain"
        ),

      axis.text =
        ggplot2::element_text(
          size = 8,
          color = "black"
        ),

      axis.title =
        ggplot2::element_text(
          size = 9,
          color = "black"
        ),
```

```
      panel.border =
        ggplot2::element_rect(
          color = "black",
          fill = NA
        )
    )


# ------------------------------------------------------------
# Base function to avoid style differences
# between the full and zoom panels
# ------------------------------------------------------------

make_plot <- function(
  plot_title,
  x_limits = NULL
) {

  p <- ggplot2::ggplot(
    rare_df,
    ggplot2::aes(
      x = Depth,
      y = ASVs,
      group = Sample
    )
  ) +

    ggplot2::geom_line(
      color = "black",
      linewidth = 0.4,
      alpha = 0.7
    ) +

    ggplot2::geom_vline(
      xintercept = vline_depth,
      linetype = "dashed",
      color = "gray40",
      linewidth = 0.6
    ) +

    ggplot2::labs(
      title = plot_title,
      x = "Sequencing depth (reads)",
      y = "Number of ASVs"
    ) +

    rare_theme


  # The zoom is applied with coord_cartesian().
  # It does NOT remove data from rare_df.
  if (!is.null(x_limits)) {

    p <- p +
      ggplot2::coord_cartesian(
        xlim = x_limits
      )
  }

  p
}


# ------------------------------------------------------------
# Panel A — full range
# ------------------------------------------------------------

```

```r
  p_full <- make_plot(
    plot_title = paste0(
      title_text,
      " | Full range"
    )
  )


  # ------------------------------------------------------------
  # Panel B — zoom
  # ------------------------------------------------------------

  p_zoom <- make_plot(
    plot_title = paste0(
      title_text,
      " | Zoom"
    ),
    x_limits = c(
      0,
      zoom_max
    )
  )


  # ------------------------------------------------------------
  # Single-domain multipanel
  # ------------------------------------------------------------

 p_combined <- patchwork::wrap_plots(
  p_full,
  p_zoom,
  ncol = 2,
  guides = "keep"
)


  # ------------------------------------------------------------
  # Export PDF
  # ------------------------------------------------------------

  ggplot2::ggsave(
    filename = out_pdf,
    plot = p_combined,
    width = 15,
    height = 5.5,
    units = "in",
    device = cairo_pdf
  )


  # ------------------------------------------------------------
  # Export PNG 300 dpi
  # ------------------------------------------------------------

  out_png <- sub(
    "\\.pdf$",
    "_300dpi.png",
    out_pdf
  )

  ggplot2::ggsave(
    filename = out_png,
    plot = p_combined,
    width = 15,
    height = 5.5,
    units = "in",
    dpi = 300,
    device = "png",
```

```
    bg = "white"
  )


  # ----------------------------------------------------------
  # Full-only and zoom-only panels (7.5 × 5.5 in)
  # ----------------------------------------------------------

  for (panel in c("FULL", "ZOOM")) {
    p_single <- if (panel == "FULL") p_full else p_zoom
    f_single <- sub("\\.pdf$", paste0("_", panel, ".pdf"), out_pdf)
    ggplot2::ggsave(f_single, p_single, width = 7.5, height = 5.5,
                    units = "in", device = cairo_pdf)
    ggplot2::ggsave(sub("\\.pdf$", "_300dpi.png", f_single), p_single,
                    width = 7.5, height = 5.5, units = "in", dpi = 300,
                    device = "png", bg = "white")
  }


  # ----------------------------------------------------------
  # Return objects for optional reuse
  # ----------------------------------------------------------

  invisible(
    list(
      data = rare_df,
      full = p_full,
      zoom = p_zoom,
      combined = p_combined
    )
  )
}

# ---- Good's coverage by resampling
# C-hat = 1 - f1/m, averaged over `coverage_reps` rarefied replicates
# (subsampling without replacement, vegan::rrarefy).
# If m == n the full sample is used; coverage is NOT forced to 1.
# References: Good (1953) Biometrika 40:237–264;
#             Chao & Jost (2012) Ecology 93:2533–2547.
mc_coverage_single <- function(x, m, n_rep = params$coverage_reps, seed = params$rngseed) {
  x <- as.numeric(x)
  x <- x[x > 0]
  n <- sum(x)
  if (is.na(m) || m <= 0 || m > n) return(NA_real_)
  if (m == n) {
    f1 <- sum(x == 1)
    return(1 - f1 / m)
  }
  set.seed(seed)
  covs <- numeric(n_rep)
  for (i in seq_len(n_rep)) {
    sub <- vegan::rrarefy(x, sample = m)
    f1 <- sum(sub == 1)
    covs[i] <- 1 - f1 / m
  }
  mean(covs)
}

coverage_at_depth <- function(otu_mat, depth, n_rep = params$coverage_reps) {
  vapply(seq_len(ncol(otu_mat)), function(j) {
    mc_coverage_single(otu_mat[, j], depth, n_rep = n_rep,
                       seed = params$rngseed + j + depth)
  }, numeric(1))
}

ps_diag <- function(ps, label="phyloseq") {
  cat("\n====================================\n")
  cat("DIAGNOSTICS:", label, "\n")
```

```
  cat("====================================\n")
  cat("nsamples :", nsamples(ps), "\n")
  cat("ntaxa    :", ntaxa(ps), "\n")
  cat("min reads:", min(sample_sums(ps)), "\n")
  cat("max reads:", max(sample_sums(ps)), "\n")
  invisible(NULL)
}
```

🟢 CHUNK 3 — MAIN FUNCTION (endophyte-pipeline criteria: no prevalence filter, no min_counts)

```{r process_domains, message=TRUE, warning=TRUE}
process_domain <- function(dom, target_kingdom){

  log_step(paste("Starting domain:", dom))

  # ================================
  # Directories
  # ================================
  # base_dir = <project_root>/<domain_dir> (see domain_settings.R)
  outdir_root <- file.path(base_dir, params$out_subdir)

  dirs <- c(
    outdir_root,
    file.path(outdir_root, "RDS"),
    file.path(outdir_root, "QC"),
    file.path(outdir_root, "Figures"),
    file.path(outdir_root, "Tables"),
    file.path(base_dir, "Multipanels", "rarefaction")
  )
  walk(dirs, dir_create)

  rds_dir <- file.path(outdir_root, "RDS")
  tab_dir <- file.path(outdir_root, "Tables")
  fig_dir <- file.path(outdir_root, "Figures")

  # ================================
  # Load RAW phyloseq
  # ================================
  ps_raw_path <- file.path(base_dir, "RDS", "ps_raw.rds")
  if (!file.exists(ps_raw_path)) stop(glue("❌ File not found: {ps_raw_path}"))

  log_step("Loading ps_raw")
  ps_raw <- readRDS(ps_raw_path)
  ps_diag(ps_raw, paste0("ps_raw — ", dom))

  rare_depth <- cfg$rare_depth
  log_step(glue("Target rarefaction depth: {rare_depth} (domain = {dom})"))

  # ================================
  # TAXONOMIC FILTERING — ENDOPHYTE-PIPELINE STYLE
  #  - mitochondria / chloroplast
  #  - unassigned (robust)
  #  - alphanumeric
  #  - NO prevalence filter, NO min_counts
  # ================================
  log_step("Taxonomic filtering (mito/chloro + unassigned + alphanumeric)")

  tax_df <- get_tax_df(ps_raw)
  ranks  <- rank_cols(tax_df)

  if (!is.null(params$ranks_to_scan)) {
    ranks <- intersect(ranks, params$ranks_to_scan)
    if (length(ranks) == 0) stop("params$ranks_to_scan did not match any rank.")
  }

  flag_mitochloro <- rep(FALSE, nrow(tax_df))
  flag_unassigned <- rep(FALSE, nrow(tax_df))
```

```
  flag_alphanum   <- rep(FALSE, nrow(tax_df))

  if (isTRUE(params$remove_mitochloro)) {
    flag_mitochloro <- apply(tax_df[, ranks, drop = FALSE], 1, function(z) {
      any(is_mito_or_chloro(z), na.rm = TRUE)
    })
  }
  if (isTRUE(params$remove_unassigned)) {
  # Soil taxonomy comes as a full string in the "Taxon" column
  # Ej: "d__Bacteria;p__Proteobacteria;...;g__uncultured;s__uncultured"
  # Only Kingdom (d__) and Phylum (p__) are checked — the first 2 tokens
  # "Uncultured" at Genus/Species level is valid and must NOT be removed

  if ("Taxon" %in% ranks) {
    # Soil case: Taxon is a semicolon-separated string
    flag_unassigned <- apply(tax_df[, "Taxon", drop = FALSE], 1, function(z) {
      taxon_str <- as.character(z)
      if (is.na(taxon_str) || taxon_str == "") return(TRUE)
      tokens    <- strsplit(taxon_str, ";\\s*")[[1]]
      top2      <- gsub("^[a-zA-Z]__\\s*", "", head(tokens, 2))
      any(is_unassigned(top2), na.rm = TRUE)
    })
  } else {
    # Plant case: separate rank columns — Kingdom and Phylum only
    high_ranks <- intersect(c("Kingdom", "Phylum"), ranks)
    if (length(high_ranks) == 0) high_ranks <- head(ranks, 2)
    flag_unassigned <- apply(tax_df[, high_ranks, drop = FALSE], 1, function(z) {
      any(is_unassigned(z), na.rm = TRUE)
    })
  }
  cat("Unassigned check: applied to Kingdom/Phylum only\n")
}
  if (isTRUE(params$remove_alphanumeric_taxa)) {
    flag_alphanum <- apply(tax_df[, ranks, drop = FALSE], 1, function(z) {
      any(is_alphanumeric_taxon(z), na.rm = TRUE)
    })
  }

  tax_df_flags <- tax_df %>%
    mutate(
      Flag_MitoChloro   = as.logical(flag_mitochloro),
      Flag_Unassigned   = as.logical(flag_unassigned),
      Flag_Alphanumeric = as.logical(flag_alphanum),
      Remove            = Flag_MitoChloro | Flag_Unassigned | Flag_Alphanumeric
    )

  cat("\n--- Filtering summary [", dom, "] ---\n", sep = "")
  cat("Total ASVs :", nrow(tax_df_flags), "\n")
  cat("Mito/Chloro:", sum(tax_df_flags$Flag_MitoChloro, na.rm = TRUE), "\n")
  cat("Unassigned :", sum(tax_df_flags$Flag_Unassigned, na.rm = TRUE), "\n")
  cat("Alphanum   :", sum(tax_df_flags$Flag_Alphanumeric, na.rm = TRUE), "\n")
  cat("Removed    :", sum(tax_df_flags$Remove, na.rm = TRUE), "\n")
  cat("Kept       :", sum(!tax_df_flags$Remove, na.rm = TRUE), "\n")

  keep_asvs <- tax_df_flags %>% filter(!Remove) %>% pull(ASV)
  ps_filt <- prune_taxa(keep_asvs, ps_raw)
  ps_filt <- prune_samples(sample_sums(ps_filt) > 0, ps_filt)
  ps_filt <- rebuild_phyloseq_safe(ps_filt)

  ps_diag(ps_filt, paste0("ps_filt — ", dom))

  # ps_counts = final filtered (unrarefied) counts, used by ANCOM-BC2
  ps_counts <- ps_filt
  saveRDS(ps_counts, file.path(rds_dir, "ps_counts.rds"))

  # ================================
  # Depth QC (ps_filt)
  # ================================
```

```r
log_step("Per-sample depth QC (ps_filt)")

meta_filt <- data.frame(sample_data(ps_filt), check.names = FALSE)
depth_df_filt <- tibble(
  SampleID = sample_names(ps_filt),
  Reads    = as.numeric(sample_sums(ps_filt)),
  Plant_Condition = if ("Plant_Condition" %in% colnames(meta_filt))
    as.character(meta_filt$Plant_Condition) else NA_character_
) %>% arrange(Reads)

p_depth <- ggplot(depth_df_filt, aes(Reads, fct_reorder(SampleID, Reads))) +
  geom_col(aes(fill = Plant_Condition), width = 0.7, na.rm = TRUE) +
  scale_fill_viridis_d(option = "C", end = 0.85, na.translate = FALSE) +
  geom_vline(xintercept = rare_depth, linetype = 2) +
  theme_bw() +
  labs(
    title = paste("QC — Sequencing depth by sample (ps_filt):", dom),
    x = "Reads", y = NULL, fill = "Condition"
  ) +
  theme(axis.text.y = element_text(size = 6))

ggsave(
  file.path(fig_dir, glue("QC_Depth_bySample_{dom}.pdf")),
  p_depth,
  width = 7,
  height = max(4, 0.12 * nrow(depth_df_filt)),
  device = cairo_pdf
)

p_hist <- ggplot(depth_df_filt, aes(Reads)) +
  geom_histogram(bins = 40, fill = "grey70", color = "white") +
  geom_vline(xintercept = rare_depth, linetype = 2) +
  theme_bw() +
  labs(title = paste("QC — Depth distribution (ps_filt):", dom),
       x = "Reads", y = "Count")

ggsave(
  file.path(fig_dir, glue("QC_Depth_Hist_{dom}.pdf")),
  p_hist, width = 7, height = 4, device = cairo_pdf
)

# ================================
# Retention at the rarefaction cutoff
# ================================
ret_global <- depth_df_filt %>%
  mutate(Keep = Reads >= rare_depth) %>%
  summarise(
    rare_depth = rare_depth,
    n_total  = n(),
    n_keep   = sum(Keep),
    n_drop   = sum(!Keep),
    pct_keep = round(100 * n_keep / n_total, 1),
    min_reads = min(Reads),
    p10_reads = as.numeric(quantile(Reads, 0.10)),
    median_reads = median(Reads)
  )

cat("\n--- Global retention at cutoff [", dom, "] ---\n", sep = "")
print(ret_global)

write_tsv(ret_global,
          file.path(tab_dir, glue("Retention_Global_depth_{rare_depth}_{dom}.tsv")))
write_tsv(depth_df_filt %>% arrange(Reads),
          file.path(tab_dir, glue("Depth_BySample_depth_{rare_depth}_{dom}.tsv")))

# ================================
# GOOD'S COVERAGE DEPTH SCAN
# Computed on ps_filt, before the final rarefaction.
```

```r
# ================================
log_step("Good's Coverage depth scan (ps_filt)")

otu_cov <- as(otu_table(ps_filt), "matrix")
if (!taxa_are_rows(ps_filt)) otu_cov <- t(otu_cov)
reads_per_sample <- colSums(otu_cov)

# The selected depth is always part of the scan
depths_scan <- sort(unique(as.integer(c(cfg$coverage_depths, rare_depth))))

coverage_results <- purrr::map_dfr(depths_scan, function(d) {
  keep <- reads_per_sample >= d
  n_keep <- sum(keep)

  if (n_keep < 1) {
    return(tibble(
      Domain = dom, Depth = d,
      N_samples_total = ncol(otu_cov),
      N_samples_retained = 0L,
      Pct_samples_retained = 0,
      N_ASVs_retained = NA_integer_,
      Coverage_mean = NA_real_, Coverage_median = NA_real_,
      Coverage_min = NA_real_, Coverage_p10 = NA_real_
    ))
  }

  otu_keep <- otu_cov[, keep, drop = FALSE]
  cov_vals <- coverage_at_depth(otu_keep, d, n_rep = params$coverage_reps)

  set.seed(params$rngseed + d)
  otu_rare_scan <- vegan::rrarefy(t(otu_keep), sample = d)
  n_asvs_retained <- sum(colSums(otu_rare_scan) > 0)

  tibble(
    Domain = dom, Depth = d,
    N_samples_total = ncol(otu_cov),
    N_samples_retained = n_keep,
    Pct_samples_retained = round(100 * n_keep / ncol(otu_cov), 1),
    N_ASVs_retained = n_asvs_retained,
    Coverage_mean = round(mean(cov_vals, na.rm = TRUE), 4),
    Coverage_median = round(median(cov_vals, na.rm = TRUE), 4),
    Coverage_min = round(min(cov_vals, na.rm = TRUE), 4),
    Coverage_p10 = round(as.numeric(quantile(cov_vals, 0.10, na.rm = TRUE)), 4)
  )
})

print(coverage_results, n = Inf)
write_tsv(coverage_results,
          file.path(tab_dir, glue("Rarefaction_DepthScan_GoodsCoverage_{dom}.tsv")))

p_cov <- ggplot(coverage_results, aes(Depth, Coverage_mean)) +
  geom_line(linewidth = 0.8) + geom_point(size = 2) +
  geom_vline(xintercept = rare_depth, linetype = "dashed") +
  geom_hline(yintercept = c(0.90, 0.95), linetype = "dotted", color = "gray50") +
  scale_y_continuous(labels = scales::percent, limits = c(0, 1)) +
  theme_bw(base_size = 11) +
  labs(title = paste("Expected Good's coverage vs. rarefaction depth —", dom),
       subtitle = paste0("Selected depth = ", rare_depth, " reads"),
       x = "Rarefaction depth (reads)", y = "Expected Good's coverage")

ggsave(file.path(fig_dir, glue("Coverage_vs_Depth_{dom}.pdf")),
       p_cov, width = 8, height = 5, device = cairo_pdf)
ggsave(file.path(fig_dir, glue("Coverage_vs_Depth_{dom}_300dpi.png")),
       p_cov, width = 8, height = 5, dpi = params$pdf_dpi, bg = "white")

p_ret <- ggplot(coverage_results, aes(Depth, Pct_samples_retained)) +
  geom_line(linewidth = 0.8) + geom_point(size = 2) +
  geom_vline(xintercept = rare_depth, linetype = "dashed") +
```

```
    scale_y_continuous(limits = c(0, 100), labels = function(x) paste0(x, "%")) +
    theme_bw(base_size = 11) +
    labs(title = paste("Sample retention vs. rarefaction depth –", dom),
         x = "Rarefaction depth (reads)", y = "Samples retained")

  ggsave(file.path(fig_dir, glue("SampleRetention_vs_Depth_{dom}.pdf")),
         p_ret, width = 8, height = 5, device = cairo_pdf)

  selected_coverage <- coverage_results %>% filter(Depth == rare_depth)
  if (nrow(selected_coverage) != 1) {
    stop("❌ The selected depth must be included in the coverage depth scan.")
  }
  cat("\n--- Coverage/retention at selected depth [", dom, "] ---\n", sep = "")
  print(selected_coverage)

 # ================================
# Rarefaction curves (visual QC)
# Full range + zoom
# ================================

log_step(
  "Rarefaction curves (QC visual: full + zoom)"
)

# ------------------------------------------------------------
# Domain-specific upper limit of the zoom window
# ------------------------------------------------------------

zoom_max <- if (is.null(cfg$rarecurve_zoom_max)) rare_depth * 1.5 else cfg$rarecurve_zoom_max

# ------------------------------------------------------------
# Output file
# ------------------------------------------------------------

out_rc <- file.path(
  base_dir,
  "Multipanels",
  "rarefaction",
  glue(
    "Rarefaction_Curves_{dom}_QC.pdf"
  )
)

# ------------------------------------------------------------
# Build + export
# ------------------------------------------------------------

rare_plot_obj <- plot_rarecurve_pdf(
  ps = ps_filt,
  out_pdf = out_rc,
  vline_depth = rare_depth,
  zoom_max = zoom_max,
  title_text = glue(
    "Rarefaction curves | {dom} | depth = {rare_depth} reads"
  )
)

log_step(
  glue(
    "Rarefaction figure exported: {out_rc} | ",
    "zoom = 0-{zoom_max} reads"
  )
)

  # ================================
  # ps_rare
  # ================================
  log_step(glue("Rarefying to {rare_depth} reads"))
```

```r
  ps_keep <- prune_samples(sample_sums(ps_filt) >= rare_depth, ps_filt)

  if (nsamples(ps_keep) < 2) {
    stop(glue("❌ [{dom}] With rare_depth={rare_depth}, fewer than 2 samples remain."))
  }

  set.seed(params$rngseed)
  ps_rare <- rarefy_even_depth(
    ps_keep,
    sample.size = rare_depth,
    rngseed     = params$rngseed,
    replace     = isTRUE(params$rare_replace),
    verbose     = FALSE
  )
  ps_rare <- prune_samples(sample_sums(ps_rare) > 0, ps_rare)
  ps_rare <- rebuild_phyloseq_safe(ps_rare)
  ps_diag(ps_rare, paste0("ps_rare — ", dom))

  # ================================
  # Metadata inspection
  # ================================
  message("\n🔎 INSPECTION sample_data() — ps_filt — ", dom)
  if ("Plant_Condition" %in% colnames(meta_filt)) print(table(meta_filt$Plant_Condition, useNA = "ifany"))
  if ("Land" %in% colnames(meta_filt)) print(table(meta_filt$Land, useNA = "ifany"))

  meta_rare <- data.frame(sample_data(ps_rare), check.names = FALSE)
  message("\n🔎 INSPECTION sample_data() — ps_rare — ", dom)
  if ("Plant_Condition" %in% colnames(meta_rare)) print(table(meta_rare$Plant_Condition, useNA = "ifany"))
  if ("Land" %in% colnames(meta_rare)) print(table(meta_rare$Land, useNA = "ifany"))

  # ================================
  # Save objects
  # ================================
  log_step("Saving phyloseq objects")
  saveRDS(ps_filt, file.path(rds_dir, "ps_filt.rds"))

  ps_rel <- transform_sample_counts(ps_filt, function(x) x / sum(x))
  ps_rel <- rebuild_phyloseq_safe(ps_rel)
  saveRDS(ps_rel, file.path(rds_dir, "ps_rel.rds"))

  saveRDS(ps_rare, file.path(rds_dir, glue("ps_rare_{rare_depth}.rds")))
  saveRDS(ps_rare, file.path(rds_dir, "ps_rare.rds"))

  # ================================
  # Export Excel (raw vs filt vs rare + flags)
  # ================================
  tbl_samples_raw <- tibble(
    SampleID = sample_names(ps_raw),
    Reads = as.numeric(sample_sums(ps_raw))
  ) %>% arrange(Reads)
  tbl_samples_filt <- depth_df_filt %>% select(SampleID, Reads)
  tbl_samples_rare <- tibble(
    SampleID = sample_names(ps_rare),
    Reads = as.numeric(sample_sums(ps_rare))
  ) %>% arrange(Reads)

  tbl_taxa_raw  <- tibble(ASV = taxa_names(ps_raw),  TotalCount = as.numeric(taxa_sums(ps_raw)))  %>% arrange(desc(TotalCount))
  tbl_taxa_filt <- tibble(ASV = taxa_names(ps_filt), TotalCount = as.numeric(taxa_sums(ps_filt))) %>% arrange(desc(TotalCount))
  tbl_taxa_rare <- tibble(ASV = taxa_names(ps_rare), TotalCount = as.numeric(taxa_sums(ps_rare))) %>% arrange(desc(TotalCount))

  flag_summary <- tax_df_flags %>%
    select(ASV, all_of(ranks), Flag_MitoChloro, Flag_Unassigned, Flag_Alphanumeric, Remove)

  sum_tbl <- tibble(
    Object    = c("ps_raw", "ps_filt", paste0("ps_rare_", rare_depth)),
```

```
    nsamples  = c(nsamples(ps_raw), nsamples(ps_filt), nsamples(ps_rare)),
    ntaxa     = c(ntaxa(ps_raw), ntaxa(ps_filt), ntaxa(ps_rare)),
    total_reads = c(sum(sample_sums(ps_raw)), sum(sample_sums(ps_filt)), sum(sample_sums(ps_rare))),
    min_reads = c(min(sample_sums(ps_raw)), min(sample_sums(ps_filt)), min(sample_sums(ps_rare))),
    max_reads = c(max(sample_sums(ps_raw)), max(sample_sums(ps_filt)), max(sample_sums(ps_rare)))
  )

  wb <- createWorkbook()
  addWorksheet(wb, "Summary");      writeDataTable(wb, "Summary",      sum_tbl)
  addWorksheet(wb, "Coverage_DepthScan"); writeDataTable(wb, "Coverage_DepthScan", coverage_results)
  addWorksheet(wb, "Samples_raw");  writeDataTable(wb, "Samples_raw",  tbl_samples_raw)
  addWorksheet(wb, "Samples_filt"); writeDataTable(wb, "Samples_filt", tbl_samples_filt)
  addWorksheet(wb, "Samples_rare"); writeDataTable(wb, "Samples_rare", tbl_samples_rare)
  addWorksheet(wb, "Taxa_raw");     writeDataTable(wb, "Taxa_raw",     tbl_taxa_raw)
  addWorksheet(wb, "Taxa_filt");    writeDataTable(wb, "Taxa_filt",    tbl_taxa_filt)
  addWorksheet(wb, "Taxa_rare");    writeDataTable(wb, "Taxa_rare",    tbl_taxa_rare)
  addWorksheet(wb, "Filter_flags"); writeDataTable(wb, "Filter_flags", flag_summary)

  xlsx_path <- file.path(tab_dir,
                         glue("Filtering_Rarefaction_Summary_depth_{rare_depth}_{dom}.xlsx"))
  saveWorkbook(wb, xlsx_path, overwrite = TRUE)
  cat("Saved Excel:", xlsx_path, "\n")

  tibble(
    Domain       = dom,
    Rare_depth   = rare_depth,
    Samples_raw  = nsamples(ps_raw),
    Samples_filt = nsamples(ps_filt),
    Samples_rare = nsamples(ps_rare),
    ASVs_raw     = ntaxa(ps_raw),
    ASVs_filt    = ntaxa(ps_filt),
    ASVs_rare    = ntaxa(ps_rare)
  )
}
```

🟢 CHUNK 4 — RUN PIPELINE

```{r run_pipeline, message=TRUE, warning=TRUE}
all_summaries <- process_domain(dom, cfg$target_kingdom)

log_step("Filtering and rarefaction complete")
all_summaries
```

🟢 CHUNK 5 — FINAL RAREFACTION VALIDATION AND OBJECT SUMMARY

```{r validate_and_summarize, message=TRUE, warning=TRUE}
rds_dir <- file.path(base_dir, params$out_subdir, "RDS")
tab_dir <- file.path(base_dir, params$out_subdir, "Tables")

ps_raw_chk  <- readRDS(file.path(base_dir, "RDS", "ps_raw.rds"))
ps_filt_chk <- readRDS(file.path(rds_dir, "ps_filt.rds"))
ps_rare_chk <- readRDS(file.path(rds_dir, "ps_rare.rds"))

observed_depths <- unique(as.numeric(sample_sums(ps_rare_chk)))
if (length(observed_depths) != 1 || observed_depths != cfg$rare_depth) {
  stop(glue("❌ Rarefaction validation failed for {dom}: expected {cfg$rare_depth}, ",
            "observed {paste(observed_depths, collapse = ', ')}"))
}
cat("✅ Rarefaction depth validated:", dom, "=", cfg$rare_depth, "reads/sample |",
    nsamples(ps_rare_chk), "samples |", ntaxa(ps_rare_chk), "ASVs\n")

summarize_ps <- function(ps, obj_name) {
  reads <- sample_sums(ps)
  tibble(Domain = dom, Object = obj_name, nsamples = nsamples(ps), ntaxa = ntaxa(ps),
         total_reads = sum(reads), min_reads = min(reads), max_reads = max(reads))
```

````
}

summary_table <- bind_rows(
  summarize_ps(ps_raw_chk,  "ps_raw"),
  summarize_ps(ps_filt_chk, "ps_filt"),
  summarize_ps(ps_rare_chk, paste0("ps_rare_", cfg$rare_depth))
)
print(summary_table)

out_xlsx <- file.path(tab_dir, glue("Phyloseq_Object_Summary_{cfg$label}.xlsx"))
wb <- createWorkbook()
addWorksheet(wb, "Summary"); writeDataTable(wb, "Summary", summary_table)
saveWorkbook(wb, out_xlsx, overwrite = TRUE)
log_step(glue("Object summary exported: {out_xlsx}"))
```

## 🟢 CHUNK 6 — RARE ASV DIAGNOSTICS: ABUNDANCE AND PREVALENCE

Describes how many ASVs are singletons, doubletons or low-prevalence, and how
many reads they carry, before (`ps_filt`) and after (`ps_rare`) rarefaction.
No filter is applied here: the table documents why no prevalence filter is
needed before the diversity analyses.

```{r rare_asv_diagnostics, message=TRUE, warning=FALSE}
diagnose_rare_asvs <- function(ps, object_name) {
  otu <- as(otu_table(ps), "matrix")
  if (!taxa_are_rows(ps)) otu <- t(otu)   # ASVs in rows
  n_samples <- ncol(otu)

  asv_stats <- tibble(
    ASV = rownames(otu),
    Total_Count = as.numeric(rowSums(otu)),
    Samples_Present = as.integer(rowSums(otu > 0))
  ) %>%
    mutate(Prevalence = Samples_Present / n_samples,
           Prevalence_pct = 100 * Prevalence) %>%
    arrange(Total_Count)

  total_asvs  <- nrow(asv_stats)
  total_reads <- sum(asv_stats$Total_Count)
  if (total_asvs == 0)  stop(glue("❌ [{dom} | {object_name}] No ASVs found."))
  if (total_reads == 0) stop(glue("❌ [{dom} | {object_name}] No reads found."))

  # Count and reads carried by ASVs matching a condition
  n_of <- function(cond) sum(cond)
  r_of <- function(cond) sum(asv_stats$Total_Count[cond])
  tc <- asv_stats$Total_Count; sp <- asv_stats$Samples_Present; pv <- asv_stats$Prevalence

  abundance_summary <- tibble(
    Domain = dom, Object = object_name, Samples = n_samples,
    Total_ASVs = total_asvs, Total_Reads = total_reads,
    Singletons = n_of(tc == 1), Doubletons = n_of(tc == 2),
    ASVs_Count_LE5 = n_of(tc <= 5), ASVs_Count_LE10 = n_of(tc <= 10),
    Pct_ASVs_Singletons  = 100 * n_of(tc == 1)  / total_asvs,
    Pct_ASVs_Doubletons  = 100 * n_of(tc == 2)  / total_asvs,
    Pct_ASVs_Count_LE5   = 100 * n_of(tc <= 5)  / total_asvs,
    Pct_ASVs_Count_LE10  = 100 * n_of(tc <= 10) / total_asvs,
    Reads_from_Singletons = r_of(tc == 1), Reads_from_Doubletons = r_of(tc == 2),
    Reads_from_Count_LE5  = r_of(tc <= 5), Reads_from_Count_LE10 = r_of(tc <= 10),
    Pct_Reads_from_Singletons = 100 * r_of(tc == 1)  / total_reads,
    Pct_Reads_from_Doubletons = 100 * r_of(tc == 2)  / total_reads,
    Pct_Reads_from_Count_LE5  = 100 * r_of(tc <= 5)  / total_reads,
    Pct_Reads_from_Count_LE10 = 100 * r_of(tc <= 10) / total_reads
  )

  prevalence_summary <- tibble(
    Domain = dom, Object = object_name, Samples = n_samples, Total_ASVs = total_asvs,
    Present_1_sample = n_of(sp == 1), Present_LE2_samples = n_of(sp <= 2),
````

```
    Prevalence_LT5pct = n_of(pv < 0.05), Prevalence_LT10pct = n_of(pv < 0.10),
    Prevalence_GE10pct = n_of(pv >= 0.10),
    Pct_Present_1_sample    = 100 * n_of(sp == 1)    / total_asvs,
    Pct_Present_LE2_samples = 100 * n_of(sp <= 2)    / total_asvs,
    Pct_Prevalence_LT5pct   = 100 * n_of(pv < 0.05)  / total_asvs,
    Pct_Prevalence_LT10pct  = 100 * n_of(pv < 0.10)  / total_asvs,
    Pct_Prevalence_GE10pct  = 100 * n_of(pv >= 0.10) / total_asvs
  )

  prevalence_reads <- tibble(
    Domain = dom, Object = object_name, Total_Reads = total_reads,
    Reads_ASVs_1_sample = r_of(sp == 1), Reads_ASVs_LE2_samples = r_of(sp <= 2),
    Reads_ASVs_LT5pct = r_of(pv < 0.05), Reads_ASVs_LT10pct = r_of(pv < 0.10),
    Reads_ASVs_GE10pct = r_of(pv >= 0.10),
    Pct_Reads_ASVs_1_sample    = 100 * r_of(sp == 1)    / total_reads,
    Pct_Reads_ASVs_LE2_samples = 100 * r_of(sp <= 2)    / total_reads,
    Pct_Reads_ASVs_LT5pct      = 100 * r_of(pv < 0.05)  / total_reads,
    Pct_Reads_ASVs_LT10pct     = 100 * r_of(pv < 0.10)  / total_reads,
    Pct_Reads_ASVs_GE10pct     = 100 * r_of(pv >= 0.10) / total_reads
  )

  list(asv_stats = asv_stats, abundance_summary = abundance_summary,
       prevalence_summary = prevalence_summary, prevalence_reads = prevalence_reads)
}

diag_filt <- diagnose_rare_asvs(ps_filt_chk, "ps_filt")
diag_rare <- diagnose_rare_asvs(ps_rare_chk, paste0("ps_rare_", cfg$rare_depth))

abundance_summary  <- bind_rows(diag_filt$abundance_summary,  diag_rare$abundance_summary)
prevalence_summary <- bind_rows(diag_filt$prevalence_summary, diag_rare$prevalence_summary)
prevalence_reads   <- bind_rows(diag_filt$prevalence_reads,   diag_rare$prevalence_reads)
stopifnot(nrow(abundance_summary) == 2, nrow(prevalence_summary) == 2, nrow(prevalence_reads) == 2)

print(abundance_summary,  width = Inf)
print(prevalence_summary, width = Inf)
print(prevalence_reads,   width = Inf)

out_xlsx <- file.path(tab_dir, "Rare_ASV_Abundance_Prevalence_Diagnostics.xlsx")
wb <- createWorkbook()
sheets <- list(
  Abundance_Summary   = abundance_summary,
  Prevalence_Summary  = prevalence_summary,
  Reads_by_Prevalence = prevalence_reads,
  ASVs_ps_filt        = diag_filt$asv_stats
)
sheets[[paste0("ASVs_ps_rare", cfg$rare_depth)]] <- diag_rare$asv_stats
for (nm in names(sheets)) {
  addWorksheet(wb, nm); writeDataTable(wb, nm, as.data.frame(sheets[[nm]]))
}
saveWorkbook(wb, out_xlsx, overwrite = TRUE)
log_step(glue("Rare ASV diagnostics exported: {out_xlsx}"))
```

🟢 CHUNK 7 — GLOBAL vs. WITHIN-FARM PREVALENCE OF RARE ASVs

ASVs that are rare across all samples may still be recurrent inside one farm.
This chunk compares global prevalence with the maximum prevalence reached
within any farm (rarefied data).

```{r rare_asv_within_farm_diagnostics, message=TRUE, warning=FALSE}
ps   <- ps_rare_chk
meta <- data.frame(sample_data(ps), check.names = FALSE)

farm_candidates <- c("Land", "Farm", "farm", "Finca", "finca",
                     "Farm_ID", "farm_id", "Finca_ID", "finca_id")
farm_var <- farm_candidates[farm_candidates %in% colnames(meta)][1]
if (is.na(farm_var)) {
  stop(paste0("❌ The farm variable could not be identified automatically.\n",
```

```
              "Available variables:\n", paste(colnames(meta), collapse = ", ")))
}
cat("Farm variable identified:", farm_var, "\n")

otu <- as(otu_table(ps), "matrix")
if (!taxa_are_rows(ps)) otu <- t(otu)
meta <- meta[colnames(otu), , drop = FALSE]
stopifnot(identical(rownames(meta), colnames(otu)))

farm <- as.character(meta[[farm_var]])
if (any(is.na(farm))) stop("❌ Some samples have no farm information.")
farm_levels <- unique(farm)

global_stats <- tibble(
  ASV = rownames(otu),
  Total_Count = as.numeric(rowSums(otu)),
  Global_Samples_Present = as.integer(rowSums(otu > 0))
) %>%
  mutate(Global_Prevalence = Global_Samples_Present / ncol(otu),
         Global_Prevalence_pct = 100 * Global_Prevalence)

farm_prevalence <- map_dfr(farm_levels, function(f) {
  idx <- which(farm == f)
  otu_farm <- otu[, idx, drop = FALSE]
  tibble(ASV = rownames(otu_farm), Farm = f, Farm_n_samples = length(idx),
         Farm_Samples_Present = as.integer(rowSums(otu_farm > 0))) %>%
    mutate(Farm_Prevalence = Farm_Samples_Present / length(idx),
           Farm_Prevalence_pct = 100 * Farm_Prevalence)
})

max_farm_prevalence <- farm_prevalence %>%
  group_by(ASV) %>%
  summarise(Max_Farm_Prevalence = max(Farm_Prevalence),
            Max_Farm_Prevalence_pct = 100 * Max_Farm_Prevalence,
            Farms_Present = sum(Farm_Samples_Present > 0),
            .groups = "drop")

asv_comparison <- global_stats %>%
  left_join(max_farm_prevalence, by = "ASV") %>%
  mutate(
    Global_1_sample    = Global_Samples_Present == 1,
    Global_LE2_samples = Global_Samples_Present <= 2,
    Global_LT5pct      = Global_Prevalence < 0.05,
    Global_LT10pct     = Global_Prevalence < 0.10,
    Farm_GE25pct = Max_Farm_Prevalence >= 0.25,
    Farm_GE50pct = Max_Farm_Prevalence >= 0.50,
    Farm_GE75pct = Max_Farm_Prevalence >= 0.75,
    Farm_100pct  = Max_Farm_Prevalence == 1
  )

summarize_global_rare_group <- function(df, filter_vector, group_name) {
  tmp <- df[filter_vector, , drop = FALSE]
  n_asvs <- nrow(tmp)
  pct <- function(x) if (n_asvs == 0) NA_real_ else 100 * sum(x) / n_asvs
  tibble(
    Global_Rarity_Group = group_name, ASVs = n_asvs, Reads = sum(tmp$Total_Count),
    Farm_GE25pct = sum(tmp$Farm_GE25pct), Farm_GE50pct = sum(tmp$Farm_GE50pct),
    Farm_GE75pct = sum(tmp$Farm_GE75pct), Farm_100pct  = sum(tmp$Farm_100pct),
    Pct_ASVs_Farm_GE25pct = pct(tmp$Farm_GE25pct),
    Pct_ASVs_Farm_GE50pct = pct(tmp$Farm_GE50pct),
    Pct_ASVs_Farm_GE75pct = pct(tmp$Farm_GE75pct),
    Pct_ASVs_Farm_100pct  = pct(tmp$Farm_100pct)
  )
}

within_farm_summary <- bind_rows(
  summarize_global_rare_group(asv_comparison, asv_comparison$Global_1_sample,    "Present in 1 sample globally"),
  summarize_global_rare_group(asv_comparison, asv_comparison$Global_LE2_samples, "Present in <=2 samples globally"),
```

```
  summarize_global_rare_group(asv_comparison, asv_comparison$Global_LT5pct,      "Global prevalence <5%"),
  summarize_global_rare_group(asv_comparison, asv_comparison$Global_LT10pct,     "Global prevalence <10%")
)

farm_distribution <- asv_comparison %>%
  count(Farms_Present, name = "ASVs") %>%
  mutate(Pct_ASVs = 100 * ASVs / sum(ASVs)) %>%
  arrange(Farms_Present)

reads_by_farms <- asv_comparison %>%
  group_by(Farms_Present) %>%
  summarise(ASVs = n(), Reads = sum(Total_Count), .groups = "drop") %>%
  mutate(Pct_ASVs = 100 * ASVs / sum(ASVs), Pct_Reads = 100 * Reads / sum(Reads)) %>%
  arrange(Farms_Present)

farm_sample_sizes <- tibble(Farm = farm) %>%
  count(Farm, name = "Samples") %>%
  mutate(One_sample_pct = 100 / Samples, Two_samples_pct = 200 / Samples) %>%
  arrange(Farm)

print(farm_sample_sizes,   n = Inf, width = Inf)
print(within_farm_summary, n = Inf, width = Inf)
print(farm_distribution,   n = Inf, width = Inf)
print(reads_by_farms,      n = Inf, width = Inf)

stopifnot(
  nrow(asv_comparison) == ntaxa(ps),
  sum(asv_comparison$Total_Count) == sum(sample_sums(ps)),
  all(asv_comparison$Farms_Present >= 1)
)

out_xlsx <- file.path(tab_dir, "Rare_ASV_Global_vs_WithinFarm_Diagnostics.xlsx")
wb <- createWorkbook()
sheets <- list(
  Global_vs_WithinFarm  = within_farm_summary,
  Samples_per_Farm      = farm_sample_sizes,
  ASVs_by_Farms         = farm_distribution,
  Reads_by_Farms        = reads_by_farms,
  ASV_Global_Farm_Stats = asv_comparison,
  ASV_x_Farm            = farm_prevalence
)
for (nm in names(sheets)) {
  addWorksheet(wb, nm); writeDataTable(wb, nm, as.data.frame(sheets[[nm]]))
}
saveWorkbook(wb, out_xlsx, overwrite = TRUE)
log_step(glue("Global vs. within-farm diagnostics exported: {out_xlsx}"))
log_step("Pipeline finished successfully")
```

Session information

```{r session_info}
sessionInfo()
```

02 — Alpha diversity: farm-aware linear mixed models (Shannon, Chao1) | [domain]

Script: 02_Alpha_Diversity.Rmd · source listing (1774 lines) · code shown as written; chunks not executed

R chunks

line 43: ```{r setup, message=FALSE, warning=FALSE}
line 88: ```{r helpers, message=FALSE, warning=FALSE}
line 519: ```{r alpha_models, message=FALSE, warning=FALSE}
line 1689: ```{r rarefaction_qc_summary, message=TRUE, warning=FALSE}
line 1731: ```{r alpha_multipanels, message=TRUE, warning=FALSE}
line 1771: ```{r session_info}

```
---
title: "02 — Alpha diversity: farm-aware linear mixed models (Shannon, Chao1) | `r params$domain`"
author: "Sergio Pardo-Díaz"
date: "`r format(Sys.Date())`"

output:
  html_document:
    toc: true
    toc_depth: 3
    number_sections: true

params:
  # === Domain: "Bacterias" (16S rRNA) or "Hongos" (ITS2 no-trunc) ===
  domain: "Hongos"
  # Parent folder; data are read from <project_root>/<domain_dir>
  project_root: "."
  # NULL = use the default from domain_settings.R
  domain_dir: null
  rare_depth: null

  filtering_subdir: "01_Filtering_ASVs"
  filtering_rds_subdir: "RDS"
  analysis_subdir: "02_Alfa_Diversity"

  # Variables
  state_var: "Plant_Condition"
  crop_var: "Crop"
  farm_var: "Land"   # farm/block; change only if your metadata uses another name

  # Crops
  target_crops:
    - "Dominico_Harton"
    - "Banano_Gros_Michael"

  # Outliers: visual only
  iqr_k: 1.5

  multipanel_root: "Multipanels"
  settings_file: "domain_settings.R"
---

## Chunk 1 — Setup
```{r setup, message=FALSE, warning=FALSE}

suppressPackageStartupMessages({

  library(phyloseq)
  library(tidyverse)
  library(openxlsx)
  library(patchwork)
  library(glue)

```
```

```r
  # Statistics
  library(car)
  library(emmeans)
  library(lme4)
  library(lmerTest)
})

theme_set(
  theme_bw(base_size = 12)
)

# Reproducibility
set.seed(123)

# ---- Domain settings (single source of truth: domain_settings.R)
settings_file <- params$settings_file
if (!file.exists(settings_file)) {
  settings_file <- file.path(dirname(knitr::current_input(dir = TRUE)), settings_file)
}
source(settings_file)

cfg <- domain_settings(params$domain, params)
domain_base_dir <- domain_path(params$project_root, cfg)

message("Domain: ", cfg$domain, " — ", cfg$label_long,
        " | data folder: ", domain_base_dir,
        " | rarefaction depth: ", cfg$rare_depth)
```

---

3. CHUNK 2 — Helpers

```{r helpers, message=FALSE, warning=FALSE}

# ============================================================
# Palette
# ============================================================

col_state <- c(
  "Diseased_Plant" = "#F8766D",
  "Healthy_Plant"  = "#00BFC4"
)


# ============================================================
# Significance
# ============================================================

sig_label <- function(p) {

  dplyr::case_when(
    is.na(p)  ~ "NS",
    p < 0.001 ~ "***",
    p < 0.01  ~ "**",
    p < 0.05  ~ "*",
    TRUE      ~ "NS"
  )
}


# ============================================================
# UTF-8
# ============================================================

utf8_sanitize_vec <- function(x) {
```

```
121
122   if (is.factor(x)) {
123     x <- as.character(x)
124   }
125
126   if (!is.character(x)) {
127     return(x)
128   }
129
130   y <- iconv(
131     x,
132     from = "",
133     to   = "UTF-8",
134     sub  = ""
135   )
136
137   y[is.na(y)] <- ""
138
139   y
140 }
141
142
143 utf8_sanitize_df <- function(df) {
144
145   df[] <- lapply(
146     df,
147     utf8_sanitize_vec
148   )
149
150   df
151 }
152
153
154 # ============================================================
155 # Normalize IDs
156 # ============================================================
157
158 normalize_id <- function(x) {
159
160   x <- utf8_sanitize_vec(x)
161   x <- tolower(x)
162
163   gsub(
164     "[^a-z0-9]+",
165     "",
166     x
167   )
168 }
169
170
171 # ============================================================
172 # Directories
173 # ============================================================
174
175 make_analysis_dirs <- function(dom) {
176
177   base_dir <- domain_base_dir
178
179   analysis_dir <- file.path(
180     base_dir,
181     params$analysis_subdir
182   )
183
184   dir.create(
185     file.path(
186       analysis_dir,
187       "Tables"
188     ),
```

```
    recursive = TRUE,
    showWarnings = FALSE
  )

  dir.create(
    file.path(
      analysis_dir,
      "Figures"
    ),
    recursive = TRUE,
    showWarnings = FALSE
  )

  list(
    base_dir = base_dir,
    analysis_dir = analysis_dir
  )
}


# ============================================================
# Target crops
# ============================================================

get_target_crops <- function() {

  if (
    !is.null(params$target_crops) &&
    length(params$target_crops) > 0
  ) {

    return(
      as.character(
        params$target_crops
      )
    )
  }

  NULL
}


# ============================================================
# Locate RDS
# ============================================================

find_rds <- function(dom, filename) {

  expected <- file.path(
    domain_base_dir,
    params$filtering_subdir,
    params$filtering_rds_subdir,
    filename
  )

  if (file.exists(expected)) {

    return(expected)
  }

  hits <- list.files(
    domain_base_dir,
    pattern = paste0(
      "^",
      filename,
      "$"
    ),
    recursive = TRUE,
```

```
    full.names = TRUE
  )

  if (length(hits) >= 1) {

    return(
      hits[1]
    )
  }

  stop(
    glue(
      "❌ {filename} was not found within ",
      "{domain_base_dir}"
    )
  )
}


# ============================================================
# Expected depth
# ============================================================

expected_depth <- function(dom) {

  if (dom != cfg$domain) {
    stop("Unrecognized domain: ", dom)
  }

  as.numeric(cfg$rare_depth)
}


# ============================================================
# Load rarefied object
#
# Depth comes from domain_settings.R
# (bacteria = 5,000; fungi = 2,000 reads/sample)
# ============================================================

load_ps <- function(dom) {

  depth <- expected_depth(dom)

  filename <- glue(
    "ps_rare_{depth}.rds"
  )

  ps_path <- find_rds(
    dom,
    filename
  )

  message(
    "📦 Loading: ",
    ps_path
  )

  ps <- readRDS(
    ps_path
  )

  # ------------------------------------------------------------
  # Sanitize metadata
  # ------------------------------------------------------------

  sd <- data.frame(
    sample_data(ps),
```

```
    check.names = FALSE,
    stringsAsFactors = FALSE
  )

  rn <- rownames(sd)

  sd <- utf8_sanitize_df(
    sd
  )

  rownames(sd) <- utf8_sanitize_vec(
    rn
  )

  sample_data(ps) <- phyloseq::sample_data(
    sd
  )


  # ------------------------------------------------------------
  # CRITICAL DEPTH VALIDATION
  # ------------------------------------------------------------

  depths <- as.numeric(
    sample_sums(ps)
  )

  if (
    any(depths != depth)
  ) {

    stop(
      glue(
        "❌ {dom}: object {filename} does not contain exactly ",
        "{format(depth, big.mark=',')} reads in every sample. ",
        "Observed range: {min(depths)}–{max(depths)}."
      )
    )
  }


  message(
    "✔ ",
    dom,
    ": ",
    nsamples(ps),
    " samples | ",
    ntaxa(ps),
    " ASVs | ",
    format(
      depth,
      big.mark = ","
    ),
    " reads/sample"
  )

  ps
}


# ============================================================
# Align metadata with alpha
# ============================================================

align_meta_to_alpha <- function(
  ps,
  alpha_rownames
) {
```

```r

  meta <- data.frame(
    sample_data(ps),
    check.names = FALSE,
    stringsAsFactors = FALSE
  )

  meta <- utf8_sanitize_df(
    meta
  )

  id_alpha <- utf8_sanitize_vec(
    alpha_rownames
  )

  id_meta <- utf8_sanitize_vec(
    rownames(meta)
  )

  key_a <- normalize_id(
    id_alpha
  )

  key_m <- normalize_id(
    id_meta
  )

  if (anyDuplicated(key_a) > 0) {

    stop(
      "Duplicated IDs in alpha."
    )
  }

  if (anyDuplicated(key_m) > 0) {

    stop(
      "Duplicated IDs in metadata."
    )
  }

  if (!setequal(
    key_a,
    key_m
  )) {

    stop(
      "IDs do NOT match between alpha and metadata."
    )
  }

  meta2 <- meta[
    match(
      key_a,
      key_m
    ),
    ,
    drop = FALSE
  ]

  rownames(meta2) <- id_alpha

  meta2
}


# ============================================================
# Outliers IQR
```

```
# Visual ONLY.
# They are not excluded from the model.
# ============================================================

flag_outliers_iqr <- function(
  v,
  k = 1.5
) {

  out <- rep(
    FALSE,
    length(v)
  )

  vv <- v[
    is.finite(v)
  ]

  if (length(vv) < 4) {

    return(out)
  }

  q1 <- quantile(
    vv,
    0.25,
    na.rm = TRUE
  )

  q3 <- quantile(
    vv,
    0.75,
    na.rm = TRUE
  )

  iqr <- q3 - q1

  idx <- which(
    is.finite(v)
  )

  out[idx] <-
    v[idx] < (q1 - k * iqr) |
    v[idx] > (q3 + k * iqr)

  out
}
```

Chunk 3 — Alpha diversity
Shannon + Chao1
Farm-aware linear mixed models
Type III ANOVA (Satterthwaite) + Shapiro-Wilk +
Brown-Forsythe + emmeans

```{r alpha_models, message=FALSE, warning=FALSE}
alpha_plots <- list()
depth_qc_all <- list()

k_iqr <- as.numeric(params$iqr_k)


for (dom in cfg$domain) {

  message("")
```

```r
message("================================================")
message("📌 Alpha diversity — ", dom)
message("================================================")


# ============================================================
# Directories
# ============================================================

dirs <- make_analysis_dirs(dom)
analysis_dir <- dirs$analysis_dir


# ============================================================
# Load phyloseq
# ============================================================

ps <- load_ps(dom)

rare_depth <- expected_depth(dom)


# ============================================================
# Depth QC
# ============================================================

depth_qc <- tibble::tibble(

  Domain = dom,

  Rarefaction_depth = rare_depth,

  Samples = nsamples(ps),

  ASVs = ntaxa(ps),

  Total_reads = sum(sample_sums(ps)),

  Min_depth = min(sample_sums(ps)),

  Median_depth = median(sample_sums(ps)),

  Max_depth = max(sample_sums(ps))
)

depth_qc_all[[dom]] <- depth_qc

print(depth_qc)


# ============================================================
# Shannon + Chao1
#
# Both are recalculated from the current rarefied object.
#
# Depth set in domain_settings.R
# ============================================================

alpha_df <- estimate_richness(
  ps,
  measures = c(
    "Shannon",
    "Chao1"
  )
)


# ============================================================
```

```
# Metadata
# ============================================================

meta2 <- align_meta_to_alpha(
  ps,
  rownames(alpha_df)
)


df0 <- cbind(
  alpha_df,
  meta2
) %>%

  as.data.frame() %>%

  tibble::rownames_to_column(
    "SampleID"
  ) %>%

  dplyr::mutate(

    Crop = factor(
      utf8_sanitize_vec(
        .data[[params$crop_var]]
      )
    ),

    Plant_Condition = factor(
      utf8_sanitize_vec(
        .data[[params$state_var]]
      ),
      levels = c(
        "Healthy_Plant",
        "Diseased_Plant"
      )
    ),

    Farm = factor(
      utf8_sanitize_vec(
        .data[[params$farm_var]]
      )
    )
  ) %>%

  dplyr::filter(
    !is.na(Crop),
    Crop != "",
    !is.na(Plant_Condition),
    !is.na(Farm),
    Farm != ""
  )


# ============================================================
# Target crops
# ============================================================

tc <- get_target_crops()

if (!is.null(tc)) {

  df_try <- df0 %>%
    dplyr::filter(
      as.character(Crop) %in% tc
    )

  if (nrow(df_try) > 0) {
```

```
      df0 <- df_try

    } else {

      warning(
        "⚠ target_crops did not match the data. ",
        "All crops are kept."
      )
    }
  }


  df0 <- df0 %>%
    dplyr::mutate(
      Crop = droplevels(Crop),
      Plant_Condition = droplevels(Plant_Condition),
      Farm = droplevels(Farm)
    )


  if (nrow(df0) == 0) {

    warning(
      "⚠ df0 is empty in ",
      dom
    )

    next
  }


  # ============================================================
  # Check hierarchical structure
  # ============================================================

  if (nlevels(df0$Farm) < 2) {

    stop(
      "At least two farms are required to fit ",
      "the mixed model in ",
      dom
    )
  }


# ============================================================
# Check experimental design
# ============================================================

design_table <- with(
  df0,
  table(
    Crop,
    Plant_Condition,
    Farm
  )
)

cat(
  "\nExperimental design:\n"
)

print(design_table)


# ============================================================
# Experimental design in long format
```

```r
# for export to Excel
# ============================================================

design_export <- df0 %>%

  dplyr::count(
    Crop,
    Farm,
    Plant_Condition,
    name = "N_samples"
  ) %>%

  dplyr::arrange(
    Crop,
    Farm,
    Plant_Condition
  ) %>%

  dplyr::mutate(
    Crop = as.character(Crop),
    Farm = as.character(Farm),
    Plant_Condition = as.character(Plant_Condition)
  ) %>%

  as.data.frame()


cat(
  "\nExperimental design for export:\n"
)

print(design_export)
  # ============================================================
  # Descriptive statistics (RAW)
  # ============================================================

  summary_crop <- df0 %>%

    dplyr::group_by(
      Crop,
      Plant_Condition
    ) %>%

    dplyr::summarise(

      n = dplyr::n(),

      Shannon_mean = mean(
        Shannon,
        na.rm = TRUE
      ),

      Shannon_sd = sd(
        Shannon,
        na.rm = TRUE
      ),

      Shannon_median = median(
        Shannon,
        na.rm = TRUE
      ),

      Chao1_mean = mean(
        Chao1,
        na.rm = TRUE
      ),

      Chao1_sd = sd(
```

```
        Chao1,
        na.rm = TRUE
      ),

      Chao1_median = median(
        Chao1,
        na.rm = TRUE
      ),

      .groups = "drop"
    )


# ============================================================
# Result objects
# ============================================================

anova_all      <- list()
shapiro_all    <- list()
levene_all     <- list()
tukey_all      <- list()
outlier_all    <- list()
random_all     <- list()
diagnostic_all <- list()


# ============================================================
# LOOP: Shannon and Chao1
# ============================================================

for (metric in c(
  "Shannon",
  "Chao1"
)) {

  message("")
  message("------------------------------------------------")
  message("📊 Metric: ", metric)


  df_metric <- df0 %>%

    dplyr::filter(
      is.finite(
        .data[[metric]]
      )
    ) %>%

    dplyr::mutate(
      Crop = droplevels(Crop),
      Plant_Condition = droplevels(Plant_Condition),
      Farm = droplevels(Farm)
    )


  if (nrow(df_metric) == 0) {

    warning(
      "No finite values for ",
      metric,
      " in ",
      dom
    )

    next
  }
```

```r
  # ========================================================
  # Pre-model checks
  # ========================================================

  if (nlevels(df_metric$Crop) < 2) {

    stop(
      "The Crop variable has fewer than two levels for ",
      metric,
      " in ",
      dom
    )
  }


  if (nlevels(df_metric$Plant_Condition) < 2) {

    stop(
      "Plant_Condition has fewer than two levels for ",
      metric,
      " in ",
      dom
    )
  }


  if (nlevels(df_metric$Farm) < 2) {

    stop(
      "Farm has fewer than two levels for ",
      metric,
      " in ",
      dom
    )
  }


  # ========================================================
  # Outliers
  # Visual identification ONLY
  # They are not removed from the analysis.
  # ========================================================

  df_plot <- df_metric %>%

    dplyr::group_by(
      Crop,
      Plant_Condition
    ) %>%

    dplyr::mutate(

      is_outlier = flag_outliers_iqr(
        .data[[metric]],
        k = k_iqr
      )
    ) %>%

    dplyr::ungroup()


  outlier_all[[metric]] <- df_plot %>%

    dplyr::filter(
      is_outlier
    ) %>%

    dplyr::transmute(
```

```
      SampleID,
      Crop,
      Plant_Condition,
      Farm,
      Metric = metric,
      Value = .data[[metric]]
    )


# ============================================================
# Model response variable
#
# Shannon -> original scale
# Chao1   -> log1p
#
# RAW Chao1 is kept for figures and descriptive statistics.
# ============================================================

df_model <- df_metric


if (metric == "Chao1") {

  df_model <- df_model %>%
    dplyr::mutate(
      log1p_Chao1 = log1p(Chao1)
    )

  y_lm <- "log1p_Chao1"

} else {

  y_lm <- "Shannon"
}


# ============================================================
# Contrasts for Type III ANOVA
# ============================================================

contrasts(
  df_model$Crop
) <- contr.sum(
  nlevels(df_model$Crop)
)


contrasts(
  df_model$Plant_Condition
) <- contr.sum(
  nlevels(df_model$Plant_Condition)
)


# ============================================================
# Linear mixed model
#
# Farm = random intercept
#
# Crop is constant within Farm.
# Plant_Condition varies within Farm.
# ============================================================

f_int <- stats::as.formula(
  glue::glue(
    "{y_lm} ~ Crop * Plant_Condition + (1 | Farm)"
  )
)
```

```
    fit_int <- lmerTest::lmer(
      formula = f_int,
      data = df_model,
      REML = TRUE
    )


    # ========================================================
    # Singularity diagnostic
    # ========================================================

    singular_fit <- lme4::isSingular(
      fit_int,
      tol = 1e-4
    )


    diagnostic_all[[metric]] <- tibble::tibble(

      Domain = dom,

      Metric = metric,

      y_variable = y_lm,

      N_samples = nrow(df_model),

      N_farms = nlevels(df_model$Farm),

      Singular_fit = singular_fit
    )


    if (singular_fit) {

      warning(
        "⚠ Singular fit for ",
        dom,
        " — ",
        metric,
        ". Check the variance of the Farm random effect."
      )
    }

    # ========================================================
# Type III ANOVA
# Satterthwaite degrees of freedom
# ========================================================

anova_raw <- anova(
  fit_int,
  type = 3,
  ddf = "Satterthwaite"
)

anova_tab <- anova_raw %>%
  as.data.frame() %>%
  tibble::rownames_to_column("Term") %>%
  tibble::as_tibble() %>%
  dplyr::mutate(
    Domain = dom,
    Metric = metric,
    y_variable = y_lm,
    Model = "Crop * Plant_Condition + (1 | Farm)",
    SS_type = "III",
    DDF_method = "Satterthwaite",
```

```
    Random_effect = "Farm",
    Singular_fit = singular_fit
  ) %>%
  dplyr::relocate(
    Domain,
    Metric,
    y_variable,
    Model,
    SS_type,
    DDF_method,
    Random_effect,
    Singular_fit
  )

anova_all[[metric]] <- anova_tab


    # ========================================================
    # Variance of the Farm random effect
    # ========================================================

    vc <- as.data.frame(
      lme4::VarCorr(
        fit_int
      )
    )


    random_all[[metric]] <- vc %>%

      tibble::as_tibble() %>%

      dplyr::mutate(
        Domain = dom,
        Metric = metric,
        y_variable = y_lm
      ) %>%

      dplyr::relocate(
        Domain,
        Metric,
        y_variable
      )


    # ========================================================
    # Shapiro-Wilk on mixed-model residuals
    # ========================================================

    resid_model <- residuals(
      fit_int
    )


    sw <- stats::shapiro.test(
      resid_model
    )


    shapiro_all[[metric]] <- tibble::tibble(

      Domain = dom,

      Metric = metric,

      y_variable = y_lm,

      Model =
```

```
      "Crop * Plant_Condition + (1 | Farm)",

    n = length(
      resid_model
    ),

    SW_W = unname(
      sw$statistic
    ),

    SW_p = sw$p.value,

    normal =
      sw$p.value >= 0.05
  )


# ============================================================
# Brown-Forsythe
#
# Median-centered Levene test.
# Kept as a descriptive diagnostic of dispersion
# between Crop × Plant_Condition groups.
# ============================================================

group_interaction <- interaction(
  df_model$Crop,
  df_model$Plant_Condition,
  drop = TRUE
)


lev <- car::leveneTest(
  df_model[[y_lm]],
  group = group_interaction,
  center = median
)


levene_all[[metric]] <- tibble::tibble(

  Domain = dom,

  Metric = metric,

  y_variable = y_lm,

  Test =
    "Brown-Forsythe (median-centered Levene)",

  F_value =
    lev[1, "F value"],

  df1 =
    lev[1, "Df"],

  df2 =
    lev[2, "Df"],

  p_value =
    lev[1, "Pr(>F)"],

  homogeneous_variance =
    lev[1, "Pr(>F)"] >= 0.05
)


# ============================================================
```

```
# EMMEANS
#
# Healthy vs Diseased within each crop.
# Marginal means are obtained from the mixed model,
# so Farm is already accounted for in the model.
# ========================================================

emm <- emmeans::emmeans(
  fit_int,
  ~ Plant_Condition | Crop
)


tukey_raw <- emmeans::contrast(
  emm,
  method = "pairwise",
  adjust = "tukey"
) %>%

  as.data.frame() %>%

  tibble::as_tibble() %>%

  dplyr::mutate(
    Domain = dom,
    Metric = metric,
    y_variable = y_lm,
    sig = sig_label(p.value)
  ) %>%

  dplyr::relocate(
    Domain,
    Metric,
    y_variable,
    Crop
  )


tukey_all[[metric]] <- tukey_raw


# ========================================================
# Extract global P-values
# ========================================================

p_column <- grep(
  "^Pr\\(",
  names(anova_tab),
  value = TRUE
)


if (length(p_column) != 1) {

  stop(
    "Could not uniquely identify the P-value column ",
    "of the Type III ANOVA. Available columns: ",
    paste(
      names(anova_tab),
      collapse = ", "
    )
  )
}


p_condition <- anova_tab %>%

  dplyr::filter(
```

```
      Term == "Plant_Condition"
    ) %>%

    dplyr::pull(
      dplyr::all_of(
        p_column
      )
    )


  p_interaction <- anova_tab %>%

    dplyr::filter(
      Term == "Crop:Plant_Condition"
    ) %>%

    dplyr::pull(
      dplyr::all_of(
        p_column
      )
    )


  if (length(p_condition) == 0) {
    p_condition <- NA_real_
  }


  if (length(p_interaction) == 0) {
    p_interaction <- NA_real_
  }


  p_condition_txt <- ifelse(

    is.na(p_condition),

    "NA",

    formatC(
      p_condition,
      format = "g",
      digits = 3
    )
  )


  p_interaction_txt <- ifelse(

    is.na(p_interaction),

    "NA",

    formatC(
      p_interaction,
      format = "g",
      digits = 3
    )
  )


  # ========================================================
  # Y-AXIS SCALES
  #
  # Specific to domain × metric (alpha_ylim in domain_settings.R):
  # c(min, max, break step, y position of the significance label).
  #
  # IMPORTANT: coord_cartesian() only changes the visible window.
```

```
# It does NOT remove observations or affect boxplots, models or tests.
# ============================================================

ylim_cfg <- cfg$alpha_ylim[[metric]]

if (!is.null(ylim_cfg)) {

  y_min_plot <- ylim_cfg[1]
  y_max_plot <- ylim_cfg[2]
  y_breaks   <- seq(ylim_cfg[1], ylim_cfg[2], by = ylim_cfg[3])
  y_annot    <- ylim_cfg[4]

} else {

  y_min_plot <- min(df_metric[[metric]], na.rm = TRUE)
  y_max_plot <- max(df_metric[[metric]], na.rm = TRUE)
  y_breaks   <- waiver()
  y_annot    <- y_max_plot
}

# ============================================================
# EMMeans annotation
# ============================================================

ann_y <- df_metric %>%

  dplyr::distinct(
    Crop
  ) %>%

  dplyr::mutate(
    y = y_annot
  )


stats_annot <- tukey_raw %>%

  dplyr::left_join(
    ann_y,
    by = "Crop"
  ) %>%

  dplyr::filter(
    is.finite(y)
  )


# ============================================================
# Subtitle
# ============================================================

model_label <- if (
  metric == "Chao1"
) {

  "log1p(Chao1)"

} else {

  "Shannon"
}


subtitle_txt <- glue::glue(

  "Farm-aware mixed model [{model_label} ~ Crop × Plant_Condition + (1|Farm)] | ",

  "Plant_Condition p = {p_condition_txt} | ",
```

```
    "Interaction p = {p_interaction_txt} | ",

    "EMMeans pairwise comparison | ",

    "× = IQR outlier"
  )


  # ========================================================
  # Plot
  # ========================================================

  pplot <- ggplot(

    df_plot,

    aes(
      x = Plant_Condition,
      y = .data[[metric]],
      fill = Plant_Condition
    )
  ) +

    geom_boxplot(
      outlier.shape = NA,
      alpha = 0.85
    ) +

    geom_jitter(

      aes(
        color = Plant_Condition,
        shape = is_outlier
      ),

      width = 0.15,
      size = 5,
      alpha = 0.7,
      show.legend = FALSE
    ) +

    scale_shape_manual(
      values = c(
        "FALSE" = 16,
        "TRUE"  = 4
      )
    ) +

    facet_wrap(
      ~ Crop,
      scales = "fixed"
    ) +

    scale_fill_manual(
      values = col_state,
      drop = FALSE
    ) +

    scale_color_manual(
      values = col_state,
      drop = FALSE
    ) +

    scale_y_continuous(

      breaks = y_breaks,
```

```
      expand = expansion(
        mult = c(
          0,
          0.02
        )
      )
    ) +

    coord_cartesian(
      ylim = c(
        y_min_plot,
        y_max_plot
      ),
      clip = "on"
    ) +

    geom_text(

      data = stats_annot,

      aes(
        x = 1.5,
        y = y,
        label = sig
      ),

      inherit.aes = FALSE,
      size = 6
    ) +

    labs(

      title = glue::glue(
        "{dom} — {metric}"
      ),

      subtitle = subtitle_txt,

      x = NULL,

      y = metric
    ) +

    theme(

      legend.position = "none",

      panel.grid = element_blank(),

      strip.text = element_text(
        face = "bold"
      )
    )


  # ========================================================
  # Export figures
  # ========================================================

  ggsave(

    file.path(
      analysis_dir,
      "Figures",
      glue::glue(
        "Alpha_{metric}_{dom}.pdf"
      )
    ),
```

```
      pplot,

      width = 7.8,
      height = 4.4,
      device = cairo_pdf
    )


    ggsave(

      file.path(
        analysis_dir,
        "Figures",
        glue::glue(
          "Alpha_{metric}_{dom}.png"
        )
      ),

      pplot,

      width = 7.8,
      height = 4.4,
      dpi = 300
    )


    alpha_plots[[
      paste(
        dom,
        metric,
        sep = "_"
      )
    ]] <- pplot
  }


  # ============================================================
  # Final tables
  # ============================================================

  anova_final <- dplyr::bind_rows(
    anova_all
  )

  shapiro_final <- dplyr::bind_rows(
    shapiro_all
  )

  levene_final <- dplyr::bind_rows(
    levene_all
  )

  tukey_final <- dplyr::bind_rows(
    tukey_all
  )

  outlier_final <- dplyr::bind_rows(
    outlier_all
  )

  random_final <- dplyr::bind_rows(
    random_all
  )

  diagnostic_final <- dplyr::bind_rows(
    diagnostic_all
  )
```

```
  # ============================================================
  # Export Excel
  # ============================================================

  out_xlsx <- file.path(

    analysis_dir,

    "Tables",

    glue::glue(
      "Alpha_Diversity_{dom}.xlsx"
    )
  )


  openxlsx::write.xlsx(

    list(

      Rarefaction_QC =
        depth_qc,

      Sample_Design =
  design_export,

      Alpha_Data_Raw =
        df0,

      Summary_Raw =
        summary_crop,

      Outliers_IQR_visual =
        outlier_final,

      ANOVA_TypeIII =
        anova_final,

      MixedModel_Diagnostics =
        diagnostic_final,

      Random_Effect_Farm =
        random_final,

      Shapiro_Residuals =
        shapiro_final,

      Brown_Forsythe =
        levene_final,

      EMMeans_byCrop =
        tukey_final
    ),

    out_xlsx,

    overwrite = TRUE
  )


  message(
    "✔ Excel: ",
    out_xlsx
  )
}
```

````
## Chunk 4 — Rarefaction QC


```{r rarefaction_qc_summary, message=TRUE, warning=FALSE}

depth_qc_final <- dplyr::bind_rows(
  depth_qc_all
)

print(
  depth_qc_final,
  n = Inf
)


qc_out <- file.path(
  domain_base_dir,
  params$analysis_subdir,
  "Tables",
  glue("Alpha_Diversity_Rarefaction_QC_{cfg$domain}.xlsx")
)


openxlsx::write.xlsx(

  list(
    Rarefaction_QC =
      depth_qc_final
  ),

  qc_out,

  overwrite = TRUE
)


message(
  "✔ Overall QC: ",
  qc_out
)

```

## Chunk 5 — Alpha diversity multipanel (Shannon | Chao1)

```{r alpha_multipanels, message=TRUE, warning=FALSE}
multipanel_dir <- file.path(domain_base_dir, params$multipanel_root)
dir.create(multipanel_dir, recursive = TRUE, showWarnings = FALSE)

dom <- cfg$domain
required_plots <- paste(dom, c("Shannon", "Chao1"), sep = "_")
missing_plots  <- setdiff(required_plots, names(alpha_plots))
if (length(missing_plots) > 0) {
  stop("❌ Missing plots (run Chunk 3 first): ", paste(missing_plots, collapse = ", "))
}

p_shannon <- alpha_plots[[required_plots[1]]]
p_chao1   <- alpha_plots[[required_plots[2]]]
stopifnot(inherits(p_shannon, "ggplot"), inherits(p_chao1, "ggplot"))

# Each panel keeps its own y scale; the original ggplots are not modified
p_export <- patchwork::wrap_plots(plots = list(p_shannon, p_chao1), ncol = 2)

pdf_file <- file.path(multipanel_dir, paste0(dom, "_Alpha_Diversity_Multipanel.pdf"))
png_file <- file.path(multipanel_dir, paste0(dom, "_Alpha_Diversity_Multipanel.png"))

ggplot2::ggsave(pdf_file, p_export, device = grDevices::cairo_pdf,
````
````

```
                width = 15.6, height = 4.4, units = "in", limitsize = FALSE)
ggplot2::ggsave(png_file, p_export, width = 15.6, height = 4.4, units = "in",
                dpi = 300, bg = "white", limitsize = FALSE)

if (!file.exists(pdf_file) || !file.exists(png_file)) {
  stop("❌ The multipanel files were not written.")
}

cat("\n✔ Multipanel exported\n",
    "  PDF: ", pdf_file, " | ", round(file.info(pdf_file)$size / 1024, 1), " KB\n",
    "  PNG: ", png_file, " | ", round(file.info(png_file)$size / 1024, 1), " KB\n",
    sep = "")

print(p_export)
```

Session information

```{r session_info}
sessionInfo()
```

03 — Beta diversity: Bray–Curtis PCoA, PERMANOVA and PERMDISP | [domain]

Script: 03_Beta_Diversity.Rmd · source listing (3896 lines) · code shown as written; chunks not executed

R chunks

line 43: ```{r setup, message=FALSE, warning=FALSE}
line 124: ```{r helpers, message=TRUE, warning=TRUE}
line 2155: ```{r diagnostics_load_ps, message=TRUE, warning=FALSE}
line 2170: ```{r beta_diversity_ps_rare_only, message=TRUE, warning=TRUE}
line 3123: ```{r crop_farm_level_test, message=TRUE, warning=TRUE}
line 3280: ```{r final_permanova_permdisp_tables, message=TRUE, warning=TRUE}
line 3893: ```{r session_info}

```
---
title: "03 — Beta diversity: Bray–Curtis PCoA, PERMANOVA and PERMDISP | `r params$domain`"
author: "Sergio Pardo-Díaz"
date: "`r format(Sys.Date())`"
output:
  html_document:
    toc: true
    toc_depth: 3
    number_sections: true
  pdf_document:
    toc: true
    number_sections: true

params:
  # === Domain: "Bacterias" (16S rRNA) or "Hongos" (ITS2 no-trunc) ===
  domain: "Hongos"
  # Parent folder; data are read from <project_root>/<domain_dir>
  project_root: "."
  # NULL = use the default from domain_settings.R
  domain_dir: null
  rare_depth: null
  expected_n: null

  filtering_subdir: "01_Filtering_ASVs"
  filtering_rds_subdir: "RDS"
  analysis_subdir: "03_Beta-Diversity"

  state_var: "Plant_Condition"
  crop_var: "Crop"
  strata_var: "Land"
  department_var: "Department"
  # Random seed for permutation tests (reproducible P-values)
  seed: 2026
  settings_file: "domain_settings.R"
---

> **Definitive beta-diversity workflow (bacteria and fungi).**
> Input: `01_Filtering_ASVs/RDS/ps_rare_<depth>.rds` (bacteria 5,000 reads, 96 samples;
> fungi 2,000 reads, 92 samples; set in `domain_settings.R`).
> Bray–Curtis analyses use 9,999 permutations. Symptom-status inference is restricted within farm (`Land`), while crop type is evaluated separately at the independent farm level.

## 🔹 CHUNK 1 — Setup
```{r setup, message=FALSE, warning=FALSE}

suppressPackageStartupMessages({
  library(phyloseq)
  library(tidyverse)
  library(vegan)
  library(ape)
  library(glue)
```
```

```r
  library(openxlsx)
  library(patchwork)
})

knitr::opts_chunk$set(
  echo = TRUE,
  message = FALSE,
  warning = FALSE,
  fig.width = 9,
  fig.height = 4.5,
  dpi = 300
)

# ============================================================
# Variables metadata
# ============================================================

state_var      <- params$state_var
crop_var       <- params$crop_var
strata_var     <- params$strata_var
department_var <- params$department_var

# ============================================================
# Number of permutations
# ============================================================

nperm_permanova <- 9999

# Fixed seed so that permutation P-values are reproducible between runs
set.seed(params$seed)

# ============================================================
# Colors
# ============================================================

col_state <- c(
  "Diseased_Plant" = "#F8766D",
  "Healthy_Plant"  = "#00BFC4"
)

# ============================================================
# Domain settings (single source of truth: domain_settings.R)
# ============================================================

settings_file <- params$settings_file
if (!file.exists(settings_file)) {
  settings_file <- file.path(dirname(knitr::current_input(dir = TRUE)), settings_file)
}
source(settings_file)

cfg <- domain_settings(params$domain, params)

# Rarefied object and depth used by every analysis of this script
ps_file_rare   <- paste0("ps_rare_", cfg$rare_depth, ".rds")
expected_depth_dom <- cfg$rare_depth

# ============================================================
# Domain folder (<project_root>/<domain_dir>)
# ============================================================

project_root <- normalizePath(
  domain_path(params$project_root, cfg),
  winslash = "/",
  mustWork = FALSE
)

message("📌 project_root = ", project_root)
message("📌 getwd()      = ", getwd())
```

````
```
---

## 🔹 CHUNK 2 — Helpers (paths + strict rebuild + stats + export)

```{r helpers, message=TRUE, warning=TRUE}

# ============================================================
# OUTPUT DIRECTORIES
# ============================================================

make_outdirs <- function(domain){

  # Outputs are written to <domain folder>/03_Beta-Diversity.
  root <- file.path(
    project_root,
    params$analysis_subdir
  )

  dir.create(
    file.path(root, "Figures"),
    recursive = TRUE,
    showWarnings = FALSE
  )

  dir.create(
    file.path(root, "Tables"),
    recursive = TRUE,
    showWarnings = FALSE
  )

  dir.create(
    file.path(root, "RDS"),
    recursive = TRUE,
    showWarnings = FALSE
  )

  root
}


make_multipanel_dir <- function(){

  path <- file.path(
    project_root,
    "Multipanels",
    "03_Beta_Diversity"
  )

  dir.create(
    path,
    recursive = TRUE,
    showWarnings = FALSE
  )

  dir.create(
    file.path(path, "Figures"),
    recursive = TRUE,
    showWarnings = FALSE
  )

  dir.create(
    file.path(path, "Tables"),
    recursive = TRUE,
    showWarnings = FALSE
  )

  path
````

```r
}


# ============================================================
# FIND ps_rare
# ============================================================

find_ps_file <- function(domain, filename){

  dom_dir <- project_root

  expected <- file.path(
    project_root,
    params$filtering_subdir,
    params$filtering_rds_subdir,
    filename
  )

  if (file.exists(expected)) {
    return(expected)
  }

  hits <- list.files(
    dom_dir,
    pattern = paste0("^", filename, "$"),
    recursive = TRUE,
    full.names = TRUE
  )

  if (length(hits) == 1) {
    return(hits[1])
  }

  if (length(hits) > 1) {
    stop(
      paste0(
        "❌ Multiple files named ", filename,
        " were found within ", dom_dir,
        ".\nNone will be selected automatically.\n",
        paste(hits, collapse = "\n")
      )
    )
  }

  stop(
    glue(
      "❌ {filename} was not found within {dom_dir}."
    )
  )
}


# ============================================================
# phyloseq DIAGNOSTICS
# ============================================================

diagnose_taxa_mismatch <- function(ps, label = "") {

  cat(
    "\n================ phyloseq DIAGNOSTICS ================\n"
  )

  cat("Label:", label, "\n")
  cat("taxa_are_rows(ps):", taxa_are_rows(ps), "\n")

  otu_ids <- phyloseq::taxa_names(
    phyloseq::otu_table(ps)
  )
```

```
255
256   tax_ids <- phyloseq::taxa_names(
257     phyloseq::tax_table(ps)
258   )
259
260   ps_ids <- tryCatch(
261     phyloseq::taxa_names(ps),
262     error = function(e) NA_character_
263   )
264
265   cat("length(otu_ids):", length(otu_ids), "\n")
266   cat("length(tax_ids):", length(tax_ids), "\n")
267   cat("length(ps_ids): ", length(ps_ids), "\n")
268
269   cat(
270     "identical(otu_ids, tax_ids):",
271     identical(otu_ids, tax_ids),
272     "\n"
273   )
274
275   cat(
276     "identical(otu_ids, ps_ids): ",
277     identical(otu_ids, ps_ids),
278     "\n"
279   )
280
281   otu <- as(
282     otu_table(ps),
283     "matrix"
284   )
285
286   if (!taxa_are_rows(ps)) {
287     otu <- t(otu)
288   }
289
290   tax <- as(
291     tax_table(ps),
292     "matrix"
293   )
294
295   cat(
296     "\nOTU dims:",
297     paste(dim(otu), collapse = "x"),
298     "\n"
299   )
300
301   cat(
302     "TAX dims:",
303     paste(dim(tax), collapse = "x"),
304     "\n"
305   )
306
307   cat(
308     "identical(rownames(otu), rownames(tax)):",
309     identical(
310       rownames(otu),
311       rownames(tax)
312     ),
313     "\n"
314   )
315
316   cat(
317     "=====================================================\n"
318   )
319 }
320
321
322 sanitize_ids <- function(x){
```

```r

  x <- enc2utf8(x)
  x <- trimws(x)
  x <- gsub(
    "[\t\r\n]",
    "",
    x
  )

  x
}


# ============================================================
# STRICT REBUILD
# ============================================================

rebuild_phyloseq_strict <- function(ps){

  if (
    is.null(
      otu_table(
        ps,
        errorIfNULL = FALSE
      )
    )
  ) {
    stop("❌ ps has no otu_table().")
  }

  if (
    is.null(
      tax_table(
        ps,
        errorIfNULL = FALSE
      )
    )
  ) {
    stop("❌ ps has no tax_table().")
  }

  if (
    is.null(
      sample_data(
        ps,
        errorIfNULL = FALSE
      )
    )
  ) {
    stop("❌ ps has no sample_data().")
  }

  otu <- as(
    otu_table(ps),
    "matrix"
  )

  if (!taxa_are_rows(ps)) {
    otu <- t(otu)
  }

  tax <- as(
    tax_table(ps),
    "matrix"
  )

  sam <- sample_data(ps)

```

```
rownames(otu) <- sanitize_ids(
  rownames(otu)
)

rownames(tax) <- sanitize_ids(
  rownames(tax)
)

common <- intersect(
  rownames(otu),
  rownames(tax)
)

if (length(common) == 0) {
  stop(
    "❌ No ASVs shared between OTU and TAX tables."
  )
}

common <- rownames(otu)[
  rownames(otu) %in% common
]

otu2 <- otu[
  common,
  ,
  drop = FALSE
]

tax2 <- tax[
  common,
  ,
  drop = FALSE
]

ps2 <- phyloseq(
  otu_table(
    otu2,
    taxa_are_rows = TRUE
  ),
  tax_table(tax2),
  sam
)

taxa_names(ps2) <- taxa_names(
  otu_table(ps2)
)

if (
  !identical(
    taxa_names(
      otu_table(ps2)
    ),
    taxa_names(
      tax_table(ps2)
    )
  )
) {
  stop(
    "❌ OTU vs TAX mismatch after rebuild."
  )
}

if (
  !identical(
    taxa_names(
      otu_table(ps2)
    ),
```

```
      taxa_names(ps2)
    )
  ) {
    stop(
      "❌ OTU vs taxa_names(ps2) mismatch after rebuild."
    )
  }

  validObject(ps2)

  ps2
}


# ============================================================
# LOAD PHYLOSEQ
# ============================================================

load_ps <- function(
    domain,
    force_rebuild = TRUE
){

  ps_file <- ps_file_rare

  path <- find_ps_file(
    domain,
    ps_file
  )

  message(
    "\n====================================================="
  )

  message(
    "📦 Loading phyloseq: ",
    domain,
    " | Path: ",
    path
  )

  ps <- readRDS(path)

  # ------------------------------------------------------------
  # QC: verify the definitive rarefaction depth
  # ------------------------------------------------------------
  expected_depth <- expected_depth_dom

  if (!is.null(expected_depth) && !is.na(expected_depth)) {
    ss <- phyloseq::sample_sums(ps)
    bad_depth <- ss != expected_depth

    if (any(bad_depth)) {
      stop(
        paste0(
          "❌ The object loaded for ", domain,
          " does NOT match the definitive rarefaction depth.\n",
          "Expected depth: ", expected_depth, " reads/sample.\n",
          "Observed range: ", min(ss), "–", max(ss), ".\n",
          "Check the ps_rare path before continuing."
        )
      )
    }

    message(
      "✅ Rarefaction QC — ", domain,
      ": all samples have ", expected_depth,
      " reads."
```

```
    )
  }

  if (isTRUE(force_rebuild)) {

    ps <- rebuild_phyloseq_strict(ps)

    message(
      "✅ Strict rebuild applied: ",
      path
    )
  }

  diagnose_taxa_mismatch(
    ps,
    label = paste(
      "DENTRO load_ps() -",
      domain
    )
  )

  validObject(ps)

  meta <- data.frame(
    sample_data(ps),
    check.names = FALSE
  )

  needed <- c(
    state_var,
    crop_var,
    strata_var,
    department_var
  )

  miss <- setdiff(
    needed,
    colnames(meta)
  )

  if (length(miss) > 0) {

    stop(
      "❌ Missing columns in sample_data(): ",
      paste(
        miss,
        collapse = ", "
      )
    )
  }

  message(
    "🔢 Samples: ",
    nsamples(ps),
    " | ASVs: ",
    ntaxa(ps)
  )

  message(
    "====================================================\n"
  )

  ps
}


# ============================================================
# PCoA
```

```r
# ============================================================

pcoa_df <- function(
    dist_obj,
    meta
){

  pc <- ape::pcoa(
    dist_obj
  )

  scores <- as.data.frame(
    pc$vectors[, 1:2]
  )

  colnames(scores) <- c(
    "PCoA1",
    "PCoA2"
  )

  # Ensure the same sample order in PCoA scores and metadata
  meta2 <- meta[
    rownames(scores),
    ,
    drop = FALSE
  ]

  df <- cbind(
    scores,
    meta2
  )

  list(
    df = df,
    var1 = round(
      pc$values$Relative_eig[1] * 100,
      2
    ),
    var2 = round(
      pc$values$Relative_eig[2] * 100,
      2
    )
  )
}


# ============================================================
# CLEAN METADATA
# ============================================================

clean_meta <- function(meta){

  meta %>%
    mutate(

      Plant_Condition = factor(
        .data[[state_var]],
        levels = c(
          "Healthy_Plant",
          "Diseased_Plant"
        )
      ),

      Crop = factor(
        .data[[crop_var]]
      ),

      Land = factor(
```

```
        .data[[strata_var]]
      ),

      Department = factor(
        .data[[department_var]]
      )
    )
}


# ============================================================
# HELPER: FORMAT ADONIS TABLE
# ============================================================

tidy_adonis <- function(
    ad,
    model_name
){

  as.data.frame(ad) %>%

    tibble::rownames_to_column(
      "Term"
    ) %>%

    dplyr::mutate(
      Model = model_name
    ) %>%

    dplyr::relocate(
      Model,
      Term
    )
}


# ============================================================
# PERMANOVA
#
# IMPORTANT:
#
# PRIMARY MODEL
# Plant_Condition * Crop
# permutations restricted within Land.
#
# FULL HIERARCHICAL MODEL
# Plant_Condition * Crop + Department/Land
#
# Department/Land expands to:
# Department + Department:Land
#
# This explicitly represents:
# - Plant_Condition
# - Crop
# - Plant_Condition × Crop
# - Department
# - Land nested within Department
# ============================================================

run_permanova_models <- function(
    dist_obj,
    meta,
    nperm = 9999
){

  stopifnot(
    all(
      c(
```

```
        "Plant_Condition",
        "Crop",
        "Land",
        "Department"
      ) %in% colnames(meta)
    )
  )

  # ------------------------------------------------------------
  # Ensure metadata and distance matrix share the same sample order
  # ------------------------------------------------------------

  ids_dist <- attr(
    dist_obj,
    "Labels"
  )

  meta <- meta[
    ids_dist,
    ,
    drop = FALSE
  ]

  if (
    !identical(
      rownames(meta),
      ids_dist
    )
  ) {
    stop(
      "❌ Metadata order does not match the distance matrix."
    )
  }

  .d <- dist_obj

  res <- list()


  # ============================================================
  # 1. PRIMARY GLOBAL MODEL
  # Plant_Condition * Crop
  # permutations restricted within Land
  # ============================================================

  ad <- vegan::adonis2(
    .d ~ Plant_Condition * Crop,
    data = meta,
    permutations = nperm,
    strata = meta$Land,
    by = "terms"
  )

  res$global_terms <- tidy_adonis(
    ad,
    "Primary: Plant_Condition * Crop | strata = Land | by = terms"
  )


  # ============================================================
  # 2. PRIMARY GLOBAL MODEL — marginal tests
  # ============================================================

  ad <- vegan::adonis2(
    .d ~ Plant_Condition * Crop,
    data = meta,
    permutations = nperm,
    strata = meta$Land,
```

```
  by = "margin"
)

res$global_margin <- tidy_adonis(
  ad,
  "Primary: Plant_Condition * Crop | strata = Land | by = margin"
)


# ============================================================
# 3. ADDITIVE MODEL + LAND
# ============================================================

ad <- vegan::adonis2(
  .d ~ Plant_Condition + Crop + Land,
  data = meta,
  permutations = nperm,
  by = "terms"
)

res$full_additive_land <- tidy_adonis(
  ad,
  "Plant_Condition + Crop + Land | by = terms"
)


# ============================================================
# 4. INTERACTION MODEL + LAND
# ============================================================

ad <- vegan::adonis2(
  .d ~ Plant_Condition * Crop + Land,
  data = meta,
  permutations = nperm,
  by = "terms"
)

res$full_interaction_land <- tidy_adonis(
  ad,
  "Plant_Condition * Crop + Land | by = terms"
)


# ============================================================
# 5. DEFINITIVE FULL HIERARCHICAL MODEL
#
# ALL VARIABLES:
#
# Plant_Condition
# Crop
# Plant_Condition:Crop
# Department
# Department:Land
#
# ============================================================

ad <- vegan::adonis2(
  .d ~
    Plant_Condition *
    Crop +
    Department / Land,
  data = meta,
  permutations = nperm,
  by = "terms"
)

res$full_hierarchical <- tidy_adonis(
  ad,
```

```
    paste0(
      "FULL HIERARCHICAL: ",
      "Plant_Condition * Crop + Department/Land | by = terms"
    )
  )


  # ============================================================
  # 6. FULL HIERARCHICAL — overall test
  # ============================================================

  ad <- vegan::adonis2(
    .d ~
      Plant_Condition *
      Crop +
      Department / Land,
    data = meta,
    permutations = nperm,
    by = NULL
  )

  res$full_hierarchical_overall <- tidy_adonis(
    ad,
    paste0(
      "FULL HIERARCHICAL OVERALL: ",
      "Plant_Condition * Crop + Department/Land"
    )
  )


  # ============================================================
  # 7. UNIVARIATE MODELS
  # ============================================================

  uni_list <- list()

  for (
    v in c(
      "Plant_Condition",
      "Crop",
      "Land",
      "Department"
    )
  ) {

    frm <- reformulate(
      v,
      response = ".d"
    )

    # Condition and Crop:
    # permutations restricted within Land

    if (
      v %in% c(
        "Plant_Condition",
        "Crop"
      )
    ) {

      ad_uni <- vegan::adonis2(
        frm,
        data = meta,
        permutations = nperm,
        strata = meta$Land,
        by = "terms"
      )

```

```
      lab <- paste0(
        "Univariate: ",
        v,
        " | strata = Land"
      )

    } else {

      # Land and Department themselves cannot be
      # tested using Land as strata.

      ad_uni <- vegan::adonis2(
        frm,
        data = meta,
        permutations = nperm,
        by = "terms"
      )

      lab <- paste0(
        "Univariate: ",
        v
      )
    }

    uni_list[[v]] <- tidy_adonis(
      ad_uni,
      lab
    )
  }

  res$univariate <- bind_rows(
    uni_list
  )


  # ============================================================
  # 8. WITHIN CROP
  # Plant_Condition | strata = Land
  # ============================================================

  by_crop_list <- list()

  for (
    cr in levels(
      droplevels(
        meta$Crop
      )
    )
  ) {

    keep <- (
      meta$Crop == cr &
      complete.cases(
        meta[
          ,
          c(
            "Plant_Condition",
            "Land",
            "Crop"
          )
        ]
      )
    )

    if (
      sum(keep) < 4
    ) {
      next
```

```
  }

  if (
    nlevels(
      droplevels(
        meta$Plant_Condition[keep]
      )
    ) < 2
  ) {
    next
  }

  dist_cr <- as.dist(
    as.matrix(.d)[
      keep,
      keep
    ]
  )

  meta_cr <- droplevels(
    meta[
      keep,
      ,
      drop = FALSE
    ]
  )

  ad_cr <- vegan::adonis2(
    dist_cr ~ Plant_Condition,
    data = meta_cr,
    permutations = nperm,
    strata = meta_cr$Land,
    by = "terms"
  )

  by_crop_list[[cr]] <- tidy_adonis(
    ad_cr,
    paste0(
      "Within Crop: ",
      cr,
      " | Plant_Condition | strata = Land"
    )
  )
}

res$by_crop <- bind_rows(
  by_crop_list
)


# ============================================================
# 9. WITHIN LAND
# ============================================================

by_land_list <- list()

for (
  ln in levels(
    droplevels(
      meta$Land
    )
  )
) {

  keep <- (
    meta$Land == ln &
    complete.cases(
      meta[
```

```
        ,
        c(
          "Plant_Condition",
          "Crop",
          "Land"
        )
      ]
    )
  )

  if (
    sum(keep) < 4
  ) {
    next
  }

  if (
    nlevels(
      droplevels(
        meta$Plant_Condition[keep]
      )
    ) < 2
  ) {
    next
  }

  dist_ln <- as.dist(
    as.matrix(.d)[
      keep,
      keep
    ]
  )

  meta_ln <- droplevels(
    meta[
      keep,
      ,
      drop = FALSE
    ]
  )

  if (
    nlevels(
      meta_ln$Crop
    ) > 1
  ) {

    ad_ln <- vegan::adonis2(
      dist_ln ~
        Plant_Condition +
        Crop,
      data = meta_ln,
      permutations = nperm,
      by = "terms"
    )

    lab <- paste0(
      "Within Land: ",
      ln,
      " | Plant_Condition + Crop"
    )

  } else {

    ad_ln <- vegan::adonis2(
      dist_ln ~ Plant_Condition,
      data = meta_ln,
      permutations = nperm,
```

```
        by = "terms"
      )

      lab <- paste0(
        "Within Land: ",
        ln,
        " | Plant_Condition"
      )
    }

    by_land_list[[ln]] <- tidy_adonis(
      ad_ln,
      lab
    )
  }

  res$by_land <- bind_rows(
    by_land_list
  )


  # ============================================================
  # 10. WITHIN DEPARTMENT
  #
  # Plant_Condition (+ Crop)
  # permutations restricted within Land
  # ============================================================

  by_dept_list <- list()

  for (
    dp in levels(
      droplevels(
        meta$Department
      )
    )
  ) {

    keep <- (
      meta$Department == dp &
      complete.cases(
        meta[
          ,
          c(
            "Plant_Condition",
            "Crop",
            "Land",
            "Department"
          )
        ]
      )
    )

    if (
      sum(keep) < 4
    ) {
      next
    }

    if (
      nlevels(
        droplevels(
          meta$Plant_Condition[keep]
        )
      ) < 2
    ) {
      next
    }
```

```
1207
1208      dist_dp <- as.dist(
1209        as.matrix(.d)[
1210          keep,
1211          keep
1212        ]
1213      )
1214
1215      meta_dp <- droplevels(
1216        meta[
1217          keep,
1218          ,
1219          drop = FALSE
1220        ]
1221      )
1222
1223      if (
1224        nlevels(
1225          meta_dp$Crop
1226        ) > 1
1227      ) {
1228
1229        ad_dp <- vegan::adonis2(
1230          dist_dp ~
1231            Plant_Condition +
1232            Crop,
1233          data = meta_dp,
1234          permutations = nperm,
1235          strata = meta_dp$Land,
1236          by = "terms"
1237        )
1238
1239        lab <- paste0(
1240          "Within Department: ",
1241          dp,
1242          " | Plant_Condition + Crop | strata = Land"
1243        )
1244
1245      } else {
1246
1247        ad_dp <- vegan::adonis2(
1248          dist_dp ~ Plant_Condition,
1249          data = meta_dp,
1250          permutations = nperm,
1251          strata = meta_dp$Land,
1252          by = "terms"
1253        )
1254
1255        lab <- paste0(
1256          "Within Department: ",
1257          dp,
1258          " | Plant_Condition | strata = Land"
1259        )
1260      }
1261
1262      by_dept_list[[dp]] <- tidy_adonis(
1263        ad_dp,
1264        lab
1265      )
1266    }
1267
1268    res$by_dept <- bind_rows(
1269      by_dept_list
1270    )
1271
1272
1273    # ============================================================
1274    # CHECK:
```

```
  # Make sure all variables are represented
  # in full hierarchical model
  # ============================================================

  expected_terms <- c(
    "Plant_Condition",
    "Crop",
    "Department",
    "Plant_Condition:Crop",
    "Department:Land"
  )

  observed_terms <- res$full_hierarchical$Term

  missing_terms <- setdiff(
    expected_terms,
    observed_terms
  )

  if (
    length(missing_terms) > 0
  ) {

    warning(
      "⚠️ Terms missing from the hierarchical model: ",
      paste(
        missing_terms,
        collapse = ", "
      )
    )
  }

  res
}


# ============================================================
# BETADISPER / PERMDISP
#
# Plant-condition tests use 9,999 permutations restricted
# within Land, matching the primary PERMANOVA design.
#
# IMPORTANT:
# - PERMDISP tests differences in multivariate dispersion.
# - Interaction groups are diagnostic and are NOT equivalent
#   to testing a factorial interaction in PERMANOVA.
# ============================================================

run_betadisper_models <- function(
    dist_obj,
    meta,
    nperm = 9999
){

  stopifnot(
    all(
      c(
        "Plant_Condition",
        "Crop",
        "Land",
        "Department"
      ) %in% colnames(meta)
    )
  )

  # ------------------------------------------------------------
  # Align metadata with distance matrix
  # ------------------------------------------------------------
```

```r

ids_dist <- attr(
  dist_obj,
  "Labels"
)

meta <- meta[
  ids_dist,
  ,
  drop = FALSE
]

out <- list()


# ============================================================
# Helper: permutation design blocked by Land
# ============================================================

make_land_perm <- function(meta_sub){

  ctrl <- permute::how(
    nperm = nperm
  )

  permute::setBlocks(ctrl) <- meta_sub$Land

  ctrl
}


# ============================================================
# Helper: extract PERMDISP permutation test
# ============================================================

run_perm_betadisper <- function(
    dist_sub,
    group,
    meta_sub,
    model_name,
    block_by_land = TRUE
){

  group <- droplevels(
    as.factor(group)
  )

  # Need at least two groups
  if (
    length(group) < 4 ||
    nlevels(group) < 2
  ) {

    return(
      tibble::tibble()
    )
  }


  # ----------------------------------------------------------
  # Betadisper based on spatial median
  # ----------------------------------------------------------

  bd <- vegan::betadisper(
    dist_sub,
    group = group,
    type = "median"
  )
```

```
1411
1412
1413     # ------------------------------------------------------------
1414     # Permutation test
1415     # ------------------------------------------------------------
1416
1417     if (block_by_land) {
1418
1419       perm_design <- make_land_perm(
1420         meta_sub
1421       )
1422
1423       test <- vegan::permutest(
1424         bd,
1425         permutations = perm_design
1426       )
1427
1428     } else {
1429
1430       test <- vegan::permutest(
1431         bd,
1432         permutations = nperm
1433       )
1434     }
1435
1436
1437     # ------------------------------------------------------------
1438     # Extract ANOVA-style table
1439     # ------------------------------------------------------------
1440
1441     tab <- as.data.frame(
1442       test$tab
1443     ) %>%
1444       tibble::rownames_to_column(
1445         "Term"
1446       )
1447
1448
1449     # vegan versions may use either F or F.Model
1450     f_col <- intersect(
1451       c(
1452         "F",
1453         "F.Model"
1454       ),
1455       colnames(tab)
1456     )
1457
1458     p_col <- intersect(
1459       c(
1460         "Pr(>F)",
1461         "Pr(>F.)"
1462       ),
1463       colnames(tab)
1464     )
1465
1466
1467     if (length(f_col) == 0) {
1468       tab$F <- NA_real_
1469     } else {
1470       tab$F <- tab[[f_col[1]]]
1471     }
1472
1473
1474     if (length(p_col) == 0) {
1475       tab$`Pr(>F)` <- NA_real_
1476     } else {
1477       tab$`Pr(>F)` <- tab[[p_col[1]]]
1478     }
```

```
    tab %>%
      mutate(
        Model = model_name,
        Permutations = nperm,
        Permutation_scheme = ifelse(
          block_by_land,
          "Restricted within Land",
          "Unrestricted"
        )
      )
  }


  # ============================================================
  # 1. GLOBAL Plant Condition
  #
  # This is the PERMDISP directly corresponding to the
  # principal biological contrast.
  #
  # Permutations restricted within Land.
  # ============================================================

  out$bd_condition <- run_perm_betadisper(

    dist_sub = dist_obj,

    group =
      meta$Plant_Condition,

    meta_sub =
      meta,

    model_name =
      "Global PERMDISP: Plant_Condition | permutations restricted within Land",

    block_by_land =
      TRUE
  )


  # ============================================================
  # 2. GLOBAL Crop × Plant Condition
  #
  # Diagnostic dispersion test among the combined groups.
  # This DOES NOT represent a formal PERMDISP interaction test.
  #
  # We retain it as a diagnostic analysis.
  #
  # Since Crop does not vary within Land, it should NOT be
  # interpreted as a blocked test of Crop itself.
  # ============================================================

  grp <- interaction(
    meta$Crop,
    meta$Plant_Condition,
    drop = TRUE,
    sep = " | "
  )


  out$bd_global <- run_perm_betadisper(

    dist_sub =
      dist_obj,

    group =
```

```
    grp,

  meta_sub =
    meta,

  model_name =
    "Global PERMDISP: Crop × Plant_Condition groups",

  block_by_land =
    FALSE
)

out$bd_crop <- run_perm_betadisper(

  dist_sub =

    dist_obj,

  group =

    meta$Crop,

  meta_sub =

    meta,

  model_name =

    "Global PERMDISP: Crop",

  block_by_land =

    FALSE

)


# ============================================================
# 3. Department × Plant Condition
#
# Diagnostic only.
# ============================================================

grp_dept <- interaction(
  meta$Department,
  meta$Plant_Condition,
  drop = TRUE,
  sep = " | "
)


out$bd_global_dept <- run_perm_betadisper(

  dist_sub =
    dist_obj,

  group =
    grp_dept,

  meta_sub =
    meta,

  model_name =
    "Global PERMDISP: Department × Plant_Condition groups",

  block_by_land =
    FALSE
)
```

```

# ============================================================
# 4. Land
#
# Here Land itself is the grouping variable.
# It cannot simultaneously be used as the permutation block.
# Therefore this remains an unrestricted diagnostic test of
# differences in dispersion among farms.
# ============================================================

out$bd_by_land_group <- run_perm_betadisper(

  dist_sub =
    dist_obj,

  group =
    meta$Land,

  meta_sub =
    meta,

  model_name =
    "Global PERMDISP: Land",

  block_by_land =
    FALSE
)


# ============================================================
# 5. WITHIN CROP — Plant Condition
#
# Bray dispersion:
# Symptomatic vs Non symptomatic
#
# Permutations restricted within Land.
# ============================================================

by_crop_list <- list()


for (
  cr in levels(
    droplevels(
      meta$Crop
    )
  )
) {

  keep <- (
    meta$Crop == cr &
    complete.cases(
      meta[
        ,
        c(
          "Plant_Condition",
          "Crop",
          "Land"
        )
      ]
    )
  )


  if (
    sum(keep) < 4 ||
    nlevels(
```

```
        droplevels(
          meta$Plant_Condition[keep]
        )
      ) < 2
    ) {
      next
    }


    dist_cr <- as.dist(
      as.matrix(dist_obj)[
        keep,
        keep
      ]
    )


    meta_cr <- droplevels(
      meta[
        keep,
        ,
        drop = FALSE
      ]
    )


    by_crop_list[[cr]] <- run_perm_betadisper(

      dist_sub =
        dist_cr,

      group =
        meta_cr$Plant_Condition,

      meta_sub =
        meta_cr,

      model_name =
        paste0(
          "Within Crop: ",
          cr,
          " | PERMDISP Plant_Condition | permutations restricted within Land"
        ),

      block_by_land =
        TRUE
    )
  }


  out$bd_by_crop <- dplyr::bind_rows(
    by_crop_list
  )


  # ============================================================
  # 6. WITHIN LAND — Plant Condition
  #
  # Since each subset contains only one Land, there is no need
  # for an additional blocking variable.
  # ============================================================

  by_land_list <- list()


  for (
    ln in levels(
      droplevels(
```

```
        meta$Land
      )
    )
  ) {

    keep <- (
      meta$Land == ln &
      complete.cases(
        meta[
          ,
          c(
            "Plant_Condition",
            "Land"
          )
        ]
      )
    )


    if (
      sum(keep) < 4 ||
      nlevels(
        droplevels(
          meta$Plant_Condition[keep]
        )
      ) < 2
    ) {
      next
    }


    dist_ln <- as.dist(
      as.matrix(dist_obj)[
        keep,
        keep
      ]
    )


    meta_ln <- droplevels(
      meta[
        keep,
        ,
        drop = FALSE
      ]
    )


    by_land_list[[ln]] <- run_perm_betadisper(

      dist_sub =
        dist_ln,

      group =
        meta_ln$Plant_Condition,

      meta_sub =
        meta_ln,

      model_name =
        paste0(
          "Within Land: ",
          ln,
          " | PERMDISP Plant_Condition"
        ),

      block_by_land =
        FALSE
```

```
  )
}


out$bd_by_land <- dplyr::bind_rows(
  by_land_list
)


# ============================================================
# 7. WITHIN DEPARTMENT — Plant Condition
#
# Multiple farms may occur within each Department.
# Therefore permutations remain restricted within Land.
# ============================================================

by_dept_list <- list()


for (
  dp in levels(
    droplevels(
      meta$Department
    )
  )
) {

  keep <- (
    meta$Department == dp &
    complete.cases(
      meta[
        ,
        c(
          "Plant_Condition",
          "Department",
          "Land"
        )
      ]
    )
  )


  if (
    sum(keep) < 4 ||
    nlevels(
      droplevels(
        meta$Plant_Condition[keep]
      )
    ) < 2
  ) {
    next
  }


  dist_dp <- as.dist(
    as.matrix(dist_obj)[
      keep,
      keep
    ]
  )


  meta_dp <- droplevels(
    meta[
      keep,
      ,
      drop = FALSE
    ]
```

```
    )


    by_dept_list[[dp]] <- run_perm_betadisper(

      dist_sub =
        dist_dp,

      group =
        meta_dp$Plant_Condition,

      meta_sub =
        meta_dp,

      model_name =
        paste0(
          "Within Department: ",
          dp,
          " | PERMDISP Plant_Condition | permutations restricted within Land"
        ),

      block_by_land =
        TRUE
    )
  }


  out$bd_by_dept <- dplyr::bind_rows(
    by_dept_list
  )


  # ============================================================
  # RETURN
  # ============================================================

  out
}

# ============================================================
# EXPORT ALL BETA TABLES
# ============================================================

export_beta_tables <- function(
    perm_res,
    bd_res,
    out_root,
    domain
){

  df_all_perm <- bind_rows(

    perm_res$global_terms,
    perm_res$global_margin,

    perm_res$full_additive_land,
    perm_res$full_interaction_land,

    perm_res$full_hierarchical,
    perm_res$full_hierarchical_overall,

    perm_res$univariate,

    perm_res$by_crop,
    perm_res$by_land,
    perm_res$by_dept
  )
```

```
wb <- createWorkbook()


# ------------------------------------------------------------
# All models
# ------------------------------------------------------------

addWorksheet(
  wb,
  "PERMANOVA_All_Models"
)

writeData(
  wb,
  "PERMANOVA_All_Models",
  df_all_perm
)


# ------------------------------------------------------------
# Individual sheets
# ------------------------------------------------------------

sheet_map <- list(

  "PERM_Primary_Terms" =
    perm_res$global_terms,

  "PERM_Primary_Margin" =
    perm_res$global_margin,

  "PERM_Additive_Land" =
    perm_res$full_additive_land,

  "PERM_Inter_Land" =
    perm_res$full_interaction_land,

  "PERM_FULL_Hierarchical" =
    perm_res$full_hierarchical,

  "PERM_FULL_Overall" =
    perm_res$full_hierarchical_overall,

  "PERM_Univariate" =
    perm_res$univariate,

  "PERM_ByCrop" =
    perm_res$by_crop,

  "PERM_ByLand" =
    perm_res$by_land,

  "PERM_ByDepartment" =
    perm_res$by_dept,

  "Betadisper_Condition" =
    bd_res$bd_condition,

  "Betadisper_Crop" =
    bd_res$bd_crop,

  "Betadisper_Global" =
    bd_res$bd_global,

  "Betadisper_Global_Dept" =
    bd_res$bd_global_dept,

```

```
    "Betadisper_Land" =
      bd_res$bd_by_land_group,

    "Betadisper_ByCrop" =
      bd_res$bd_by_crop,

    "Betadisper_ByLand" =
      bd_res$bd_by_land,

    "Betadisper_ByDept" =
      bd_res$bd_by_dept
  )


  for (
    nm in names(sheet_map)
  ) {

    addWorksheet(
      wb,
      nm
    )

    writeData(
      wb,
      nm,
      sheet_map[[nm]]
    )
  }


  # ------------------------------------------------------------
  # Excel
  # ------------------------------------------------------------

  saveWorkbook(
    wb,
    file.path(
      out_root,
      "Tables",
      glue(
        "BetaDiversity_PERMANOVA_Betadisper_{domain}_Bray.xlsx"
      )
    ),
    overwrite = TRUE
  )


  # ------------------------------------------------------------
  # CSVs
  # ------------------------------------------------------------

  write_csv(
    df_all_perm,
    file.path(
      out_root,
      "Tables",
      glue(
        "PERMANOVA_AllModels_{domain}_Bray.csv"
      )
    )
  )


  write_csv(
    perm_res$global_terms,
    file.path(
      out_root,
```

```
      "Tables",
      glue(
        "PERMANOVA_Primary_{domain}_Bray.csv"
      )
    )
  )


  write_csv(
    perm_res$full_hierarchical,
    file.path(
      out_root,
      "Tables",
      glue(
        "PERMANOVA_FULL_Hierarchical_{domain}_Bray.csv"
      )
    )
  )


  write_csv(
    perm_res$by_crop,
    file.path(
      out_root,
      "Tables",
      glue(
        "PERMANOVA_ByCrop_{domain}_Bray.csv"
      )
    )
  )


  write_csv(
    perm_res$by_land,
    file.path(
      out_root,
      "Tables",
      glue(
        "PERMANOVA_ByLand_{domain}_Bray.csv"
      )
    )
  )


  write_csv(
    perm_res$by_dept,
    file.path(
      out_root,
      "Tables",
      glue(
        "PERMANOVA_ByDepartment_{domain}_Bray.csv"
      )
    )
  )

  invisible(
    df_all_perm
  )
}
```

🔹 **CHUNK 3 — Loading diagnostics (optional; recommended once)**

```{r diagnostics_load_ps, message=TRUE, warning=FALSE}
for(dom in cfg$domain){
  message("==================================")
  message("🔍 Diagnostics for domain: ", dom)
```

```
  ps <- load_ps(dom, force_rebuild = TRUE)

  diagnose_taxa_mismatch(ps, label = paste("FUERA load_ps() -", dom))

  d <- phyloseq::distance(ps, "bray")
  print(as.matrix(d)[1:3, 1:3])
}
```

🔹 CHUNK 4 — Beta diversity (ps_rare) + PCoA by Crop + PERMANOVA + betadisper

```{r beta_diversity_ps_rare_only, message=TRUE, warning=TRUE}

plots_pcoa <- list()

permanova_results_all <- list()
betadisper_results_all <- list()


# ============================================================
# MAIN LOOP
# ============================================================

for(dom in cfg$domain){

  message(
    "\n================================================"
  )

  message(
    "Processing domain: ",
    dom
  )

  message(
    "================================================"
  )


  # ============================================================
  # OUTPUT
  # ============================================================

  out_root <- make_outdirs(
    dom
  )


  # ============================================================
  # LOAD PHYLOSEQ
  # ============================================================

  ps <- load_ps(
    dom,
    force_rebuild = TRUE
  )


  # ============================================================
  # METADATA
  # ============================================================

  meta_raw <- data.frame(
    sample_data(ps),
    check.names = FALSE
  )

  meta <- clean_meta(
```

```
  meta_raw
) %>%
  as.data.frame()

rownames(meta) <- rownames(
  meta_raw
)


# ============================================================
# REMOVE INCOMPLETE METADATA
# ============================================================

keep <- complete.cases(
  meta[
    ,
    c(
      "Plant_Condition",
      "Crop",
      "Land",
      "Department"
    )
  ]
)

sample_keep <- rownames(
  meta
)[keep]


message(
  "📌 Samples before metadata filtering: ",
  nsamples(ps)
)

message(
  "📌 Samples with complete metadata: ",
  length(sample_keep)
)


ps <- prune_samples(
  sample_keep,
  ps
)


meta <- meta[
  sample_names(ps),
  ,
  drop = FALSE
]


if (
  !identical(
    rownames(meta),
    sample_names(ps)
  )
) {

  stop(
    "❌ Metadata and phyloseq samples are not in the same order."
  )
}


# ============================================================
```

```
# DROP UNUSED LEVELS
# ============================================================

meta$Plant_Condition <- droplevels(
  factor(meta$Plant_Condition)
)

meta$Crop <- droplevels(
  factor(meta$Crop)
)

meta$Land <- droplevels(
  factor(meta$Land)
)

meta$Department <- droplevels(
  factor(meta$Department)
)


# ============================================================
# SAMPLE-SIZE QC
# ============================================================

message(
  "✅ Final samples for beta diversity — ",
  dom,
  ": ",
  nrow(meta)
)

message(
  "📌 Distribution by plant condition:"
)

print(
  table(
    meta$Plant_Condition,
    useNA = "ifany"
  )
)

message(
  "📌 Distribution by crop:"
)

print(
  table(
    meta$Crop,
    useNA = "ifany"
  )
)

message(
  "📌 Distribution Crop × Plant condition:"
)

print(
  table(
    meta$Crop,
    meta$Plant_Condition
  )
)

message(
  "📌 Distribution Land × Plant condition:"
)

```

```r
print(
  table(
    meta$Land,
    meta$Plant_Condition
  )
)


# ============================================================
# CROP ORDER / LABELS
# ============================================================

crop_levels_target <- c(
  "Banano_Gros_Michael",
  "Dominico_Harton"
)

crop_labels_map <- c(
  "Banano_Gros_Michael" =
    "Gros Michel",

  "Dominico_Harton" =
    "Dominico Harton"
)


crop_levels_present <- intersect(
  crop_levels_target,
  unique(
    as.character(
      meta$Crop
    )
  )
)


if (
  length(crop_levels_present) > 0
) {

  meta$Crop <- factor(
    as.character(
      meta$Crop
    ),
    levels = crop_levels_present
  )
}


shape_crop <- c(
  "Banano_Gros_Michael" = 16,
  "Dominico_Harton" = 17
)

shape_crop_use <- shape_crop[
  crop_levels_present
]


# ============================================================
# 1. GLOBAL BRAY-CURTIS
# ============================================================

dist_bray <- phyloseq::distance(
  ps,
  method = "bray"
)
```

```

if (
  !identical(
    attr(dist_bray, "Labels"),
    rownames(meta)
  )
) {

  stop(
    "❌ Bray-Curtis matrix and metadata are not in the same order."
  )
}


if (
  attr(dist_bray, "Size") != nrow(meta)
) {

  stop(
    "❌ Bray-Curtis matrix size does not match metadata."
  )
}


message(
  "📌 Bray-Curtis size — ",
  dom,
  ": N = ",
  attr(dist_bray, "Size")
)


message(
  "📌 Bray GLOBAL (3x3 matrix) — ",
  dom
)


n_show <- min(
  3,
  attr(dist_bray, "Size")
)


print(
  as.matrix(dist_bray)[
    1:n_show,
    1:n_show
  ]
)


saveRDS(
  dist_bray,
  file.path(
    out_root,
    "RDS",
    glue(
      "dist_Bray_{dom}.rds"
    )
  )
)


# ============================================================
# 2. GLOBAL PCoA
# ============================================================

```

```r
pcoa_global <- pcoa_df(
  dist_bray,
  meta
)


saveRDS(
  pcoa_global,
  file.path(
    out_root,
    "RDS",
    glue(
      "PCoA_Global_{dom}_Bray.rds"
    )
  )
)


# ============================================================
# 3. PCoA WITHIN CROP
# ============================================================

crops_use <- levels(
  droplevels(
    meta$Crop
  )
)


pcoa_crop_list <- list()
crop_plot_list <- list()


for(cr in crops_use){

  message(
    "  - PCoA within Crop: ",
    cr
  )


  keep_cr <- (
    meta$Crop == cr &
    complete.cases(
      meta[
        ,
        c(
          "Plant_Condition",
          "Crop",
          "Land"
        )
      ]
    )
  )


  if (
    sum(keep_cr) < 4
  ) {

    message(
      "    ⚠️ Skipping ",
      cr,
      ": fewer than 4 samples."
    )

    next
  }
```

```r


  meta_cr <- droplevels(
    meta[
      keep_cr,
      ,
      drop = FALSE
    ]
  )


  ps_cr <- prune_samples(
    rownames(meta_cr),
    ps
  )


  meta_cr <- meta_cr[
    sample_names(ps_cr),
    ,
    drop = FALSE
  ]


  if (
    nsamples(ps_cr) < 4
  ) {

    message(
      "    ⚠️ Skipping ",
      cr,
      ": nsamples(ps_cr) < 4."
    )

    next
  }


  if (
    nlevels(
      droplevels(
        meta_cr$Plant_Condition
      )
    ) < 2
  ) {

    message(
      "    ⚠️ Skipping ",
      cr,
      ": only one level in Plant_Condition."
    )

    next
  }


  dist_cr <- phyloseq::distance(
    ps_cr,
    method = "bray"
  )


  message(
    "    N within Crop — ",
    cr,
    ": ",
    attr(dist_cr, "Size")
  )
```

```
2635
2636
2637      saveRDS(
2638        dist_cr,
2639        file.path(
2640          out_root,
2641          "RDS",
2642          glue(
2643            "dist_Bray_{dom}_{make.names(cr)}.rds"
2644          )
2645        )
2646      )
2647
2648
2649      pcoa_cr <- pcoa_df(
2650        dist_cr,
2651        meta_cr
2652      )
2653
2654
2655      df_cr <- pcoa_cr$df %>%
2656        tibble::rownames_to_column(
2657          "SampleID"
2658        ) %>%
2659        mutate(
2660          Crop = factor(
2661            as.character(Crop),
2662            levels = crop_levels_present
2663          )
2664        )
2665
2666
2667      pcoa_crop_list[[cr]] <- pcoa_cr
2668
2669
2670      p_cr <- ggplot(
2671        df_cr,
2672        aes(
2673          PCoA1,
2674          PCoA2,
2675          color = Plant_Condition,
2676          shape = Crop
2677        )
2678      ) +
2679
2680        geom_point(
2681          size = 3,
2682          alpha = 0.9
2683        ) +
2684
2685        scale_color_manual(
2686          values = col_state,
2687          drop = FALSE
2688        ) +
2689
2690        scale_shape_manual(
2691          values = shape_crop_use,
2692          drop = FALSE
2693        ) +
2694
2695        labs(
2696          title =
2697            crop_labels_map[[cr]],
2698
2699          x = glue(
2700            "PCoA1 ({pcoa_cr$var1}%)"
2701          ),
2702
```

```
        y = glue(
          "PCoA2 ({pcoa_cr$var2}%)"
        ),

        color =
          "Plant condition",

        shape =
          "Crop"
      ) +

      theme_bw(
        base_size = 12
      ) +

      theme(
        panel.grid =
          element_blank(),

        plot.title =
          element_text(
            face = "bold",
            hjust = 0.5
          ),

        legend.position =
          "right"
      )


    crop_plot_list[[cr]] <- p_cr


    ggsave(
      file.path(
        out_root,
        "Figures",
        glue(
          "PCoA_Bray_ps_rare_{dom}_{make.names(cr)}.pdf"
        )
      ),
      p_cr,
      width = 5.5,
      height = 4.5
    )


    ggsave(
      file.path(
        out_root,
        "Figures",
        glue(
          "PCoA_Bray_ps_rare_{dom}_{make.names(cr)}.png"
        )
      ),
      p_cr,
      width = 5.5,
      height = 4.5,
      dpi = 300
    )
  }


  # ============================================================
  # MULTIPANEL PCoA
  # ============================================================

  crop_plot_list <- crop_plot_list[
```

```
    crop_levels_present
  ]


  crop_plot_list <- crop_plot_list[
    !vapply(
      crop_plot_list,
      is.null,
      logical(1)
    )
  ]


  if (
    length(crop_plot_list) > 0
  ) {

    p_pcoa <- patchwork::wrap_plots(
      plots = crop_plot_list,
      nrow = 1,
      guides = "collect"
    )


    plots_pcoa[[dom]] <- p_pcoa


    ggsave(
      file.path(
        out_root,
        "Figures",
        glue(
          "Multipanel_PCoA_Bray_ps_rare_{dom}_SideBySide.pdf"
        )
      ),
      p_pcoa,
      width = 11.5,
      height = 4.8
    )


    ggsave(
      file.path(
        out_root,
        "Figures",
        glue(
          "Multipanel_PCoA_Bray_ps_rare_{dom}_SideBySide.png"
        )
      ),
      p_pcoa,
      width = 11.5,
      height = 4.8,
      dpi = 300
    )

  } else {

    message(
      "⚠️ No per-crop PCoA was generated for ",
      dom
    )

    plots_pcoa[[dom]] <- NULL
  }


  # ============================================================
  # 4. PERMANOVA
```

```
# ============================================================

message(
  "\n📊 Running PERMANOVA — ",
  dom
)


perm_res <- run_permanova_models(
  dist_obj = dist_bray,
  meta = meta,
  nperm = nperm_permanova
)


permanova_results_all[[dom]] <- perm_res


saveRDS(
  perm_res,
  file.path(
    out_root,
    "RDS",
    glue(
      "PERMANOVA_results_{dom}_Bray.rds"
    )
  )
)


# ============================================================
# PERMANOVA QC
# ============================================================

message(
  "\n📌 PRIMARY BLOCKED PERMANOVA — ",
  dom
)

print(
  perm_res$global_terms
)


primary_total <- perm_res$global_terms %>%
  dplyr::filter(
    Term == "Total"
  )


if (
  nrow(primary_total) == 1
) {

  total_df_primary <- as.numeric(
    primary_total$Df[1]
  )

  inferred_n_primary <- total_df_primary + 1


  message(
    "✅ Primary model inferred N = ",
    inferred_n_primary
  )


  if (
```

```
    inferred_n_primary != nrow(meta)
  ) {

    stop(
      paste0(
        "❌ PRIMARY PERMANOVA N mismatch: metadata N = ",
        nrow(meta),
        ", PERMANOVA inferred N = ",
        inferred_n_primary
      )
    )
  }
}


message(
  "\n📌 FULL HIERARCHICAL PERMANOVA — ",
  dom
)

print(
  perm_res$full_hierarchical
)


hierarchical_total <- perm_res$full_hierarchical %>%
  dplyr::filter(
    Term == "Total"
  )


if (
  nrow(hierarchical_total) == 1
) {

  total_df_hierarchical <- as.numeric(
    hierarchical_total$Df[1]
  )

  inferred_n_hierarchical <-
    total_df_hierarchical + 1


  message(
    "✅ Hierarchical model inferred N = ",
    inferred_n_hierarchical
  )


  if (
    inferred_n_hierarchical != nrow(meta)
  ) {

    stop(
      paste0(
        "❌ HIERARCHICAL PERMANOVA N mismatch: metadata N = ",
        nrow(meta),
        ", PERMANOVA inferred N = ",
        inferred_n_hierarchical
      )
    )
  }
}


# ============================================================
# 5. PERMDISP
#
```

```
# IMPORTANT:
# This is the ONLY PERMDISP calculation used.
#
# Plant-condition tests:
# - global
# - within Crop
# - within Department
#
# use permutations restricted within Land.
# ============================================================

message(
  "\n📊 Running PERMDISP — ",
  dom
)


bd_res <- run_betadisper_models(
  dist_obj = dist_bray,
  meta = meta,
  nperm = nperm_permanova
)


betadisper_results_all[[dom]] <- bd_res


saveRDS(
  bd_res,
  file.path(
    out_root,
    "RDS",
    glue(
      "Betadisper_results_{dom}_Bray.rds"
    )
  )
)


# ============================================================
# PERMDISP QC
# ============================================================

message(
  "\n📌 GLOBAL PERMDISP — Plant condition — ",
  dom
)

print(
  bd_res$bd_condition
)


message(
  "\n📌 WITHIN-CROP PERMDISP — ",
  dom
)

print(
  bd_res$bd_by_crop
)


# ============================================================
# 6. EXPORT FULL ANALYTICAL TABLES
# ============================================================

export_beta_tables(
```

```
  perm_res = perm_res,
  bd_res = bd_res,
  out_root = out_root,
  domain = dom
)


# ============================================================
# ADD EXPLICIT BLOCKED PERMDISP CSVs
# ============================================================

readr::write_csv(
  bd_res$bd_condition,
  file.path(
    out_root,
    "Tables",
    glue(
      "PERMDISP_PlantCondition_BlockedLand_{dom}_Bray.csv"
    )
  )
)


readr::write_csv(
  bd_res$bd_by_crop,
  file.path(
    out_root,
    "Tables",
    glue(
      "PERMDISP_ByCrop_BlockedLand_{dom}_Bray.csv"
    )
  )
)


# ============================================================
# FINAL DOMAIN QC
# ============================================================

expected_n <- if (is.null(cfg$expected_n)) NA_real_ else cfg$expected_n


if (
  !is.na(expected_n) &&
  nrow(meta) != expected_n
) {

  warning(
    paste0(
      "⚠️ ",
      dom,
      ": expected ",
      expected_n,
      " samples, but the analysis contains ",
      nrow(meta),
      "."
    )
  )

} else {

  message(
    "✅ CONTROL N FINAL — ",
    dom,
    ": ",
    nrow(meta),
    " samples."
  )
```

````
  }


  message(
    "\n✔ Finished domain: ",
    dom
  )
}
```

## 🔹 CHUNK 4B — Crop test at farm level (independent unit = Land)

```{r crop_farm_level_test, message=TRUE, warning=TRUE}

# ============================================================
# CROP TEST AT FARM LEVEL
#
# Crop is constant within Land, so it cannot be tested with
# permutations restricted within Land (sample-level model).
#
# Here each farm is one observation:
#   1) relative abundance per sample
#   2) mean profile per Land × Plant_Condition cell
#   3) farm centroid = simple mean of its two condition cells
#      (balanced; only farms with both conditions are kept)
#   4) Bray–Curtis among farm centroids
#   5) adonis2(dist_land ~ Department + Crop),
#      farms permuted within Department
# ============================================================

set.seed(2026)

crop_farm_results_all <- list()

for (dom in cfg$domain) {

  message("\n================================================")
  message("🌾 FARM-LEVEL CROP TEST — ", dom)
  message("================================================")

  out_root <- make_outdirs(dom)

  # ---- Load (no rebuild / no overwrite of the input file) ----
  ps <- load_ps(dom, force_rebuild = FALSE)

  meta_raw <- data.frame(sample_data(ps), check.names = FALSE)
  meta <- clean_meta(meta_raw) %>% as.data.frame()
  rownames(meta) <- rownames(meta_raw)

  keep <- complete.cases(meta[, c("Plant_Condition", "Crop", "Land", "Department")])
  ps   <- prune_samples(rownames(meta)[keep], ps)
  meta <- droplevels(meta[sample_names(ps), , drop = FALSE])

  # ---- Relative abundance per sample ----
  otu <- as(otu_table(ps), "matrix")
  if (taxa_are_rows(ps)) otu <- t(otu)
  otu <- otu[rownames(meta), , drop = FALSE]

  rs <- rowSums(otu)
  if (any(rs <= 0)) stop("❌ Samples with 0 reads in ", dom)
  otu_rel <- sweep(otu, 1, rs, "/")

  # ---- Mean per Land × Plant_Condition cell ----
  key  <- interaction(meta$Land, meta$Plant_Condition, drop = TRUE, sep = "|")
  cell <- rowsum(otu_rel, key, reorder = TRUE)
  cell <- cell / as.numeric(table(key)[rownames(cell)])

  cell_land <- sub("\\|.*$", "", rownames(cell))
````

```r
cell_cond <- sub("^.*\\|", "", rownames(cell))

# Keep only farms with both plant conditions
complete_lands <- names(which(
  tapply(cell_cond, cell_land, function(x) length(unique(x))) == 2
))
keep_c <- cell_land %in% complete_lands
excluded_lands <- setdiff(unique(cell_land), complete_lands)

# Farm centroid = simple mean of its condition cells
land_centroids <- rowsum(cell[keep_c, , drop = FALSE], cell_land[keep_c])
land_centroids <- land_centroids /
  as.numeric(table(cell_land[keep_c])[rownames(land_centroids)])

# ---- Farm metadata ----
land_meta <- meta %>%
  dplyr::group_by(Land) %>%
  dplyr::summarise(
    Crop         = dplyr::first(as.character(Crop)),
    Department   = dplyr::first(as.character(Department)),
    n_samples    = dplyr::n(),
    n_crop       = dplyr::n_distinct(Crop),
    n_department = dplyr::n_distinct(Department),
    .groups = "drop"
  ) %>%
  as.data.frame()

if (any(land_meta$n_crop != 1))       stop("❌ Crop is not constant within at least one Land.")
if (any(land_meta$n_department != 1)) stop("❌ Department is not constant within at least one Land.")

rownames(land_meta) <- as.character(land_meta$Land)
land_meta$Crop       <- droplevels(factor(land_meta$Crop))
land_meta$Department <- droplevels(factor(land_meta$Department))

land_meta <- droplevels(land_meta[rownames(land_centroids), , drop = FALSE])
stopifnot(identical(rownames(land_meta), rownames(land_centroids)))

dist_land <- vegan::vegdist(land_centroids, method = "bray")

cat("\nNumber of farms =", nrow(land_meta), "\n")
cat("Farms excluded (missing one condition):",
    if (length(excluded_lands)) paste(excluded_lands, collapse = ", ") else "none", "\n")
tab_dc <- with(land_meta, table(Department, Crop))
print(tab_dc)

# Distinct crop assignments -> minimum attainable P
n_assign <- if (nlevels(land_meta$Department) > 1) {
  prod(apply(tab_dc, 1, function(x) choose(sum(x), x[1])))
} else {
  choose(nrow(land_meta), sum(land_meta$Crop == levels(land_meta$Crop)[1]))
}
cat("Distinct crop assignments:", n_assign,
    "| minimum attainable P ≈", round(1 / n_assign, 4), "\n")

# ---- PERMANOVA at farm level ----
set.seed(2026)

if (nlevels(land_meta$Department) > 1) {
  ctrl_farm <- permute::how(nperm = nperm_permanova,
                            blocks = land_meta$Department)
  ad_farm <- vegan::adonis2(dist_land ~ Department + Crop,
                            data = land_meta,
                            permutations = ctrl_farm,
                            by = "terms")
  model_lab <- "Farm level: Department + Crop | farms permuted within Department"
} else {
  ad_farm <- vegan::adonis2(dist_land ~ Crop,
                            data = land_meta,
```

```
                              permutations = nperm_permanova,
                              by = "terms")
    model_lab <- "Farm level: Crop"
  }

  crop_farm_tab <- tidy_adonis(ad_farm, model_lab)

  # Department is not tested (permutations are within Department)
  crop_farm_tab[["Pr(>F)"]][crop_farm_tab$Term == "Department"] <- NA_real_

  crop_farm <- list(
    tab            = crop_farm_tab,
    land_meta      = land_meta,
    land_centroids = land_centroids,
    excluded_lands = excluded_lands,
    n_farms        = nrow(land_meta),
    n_assign       = n_assign,
    min_p          = 1 / n_assign
  )

  crop_farm_results_all[[dom]] <- crop_farm

  saveRDS(crop_farm, file.path(out_root, "RDS", glue("CropFarmTest_{dom}_Bray.rds")))
  readr::write_csv(crop_farm_tab,
                   file.path(out_root, "Tables", glue("PERMANOVA_CropFarmLevel_{dom}_Bray.csv")))

  cat("\nFARM-LEVEL CROP TEST —", dom, "\n")
  print(as.data.frame(crop_farm_tab))
}
```

```
# 🔹 CHUNK 5 — Reviewer-ready PERMANOVA + PERMDISP tables (separate sections)
```{r final_permanova_permdisp_tables, message=TRUE, warning=TRUE}

# ============================================================
# PURPOSE
#
#   PERMANOVA — GLOBAL (sample level; Crop P not tested)
#   PERMANOVA — CROP TYPE TEST BETWEEN FARMS (farm level)
#   PERMANOVA — WITHIN CROP
#   PERMDISP (Betadisper) — GLOBAL
#   PERMDISP (Betadisper) — WITHIN CROP
#
# Each PERMANOVA block reports its own Residual and Total and
# is checked so that SS terms + residual = Total and ΣR² = 1.
# Models from CHUNK 4 and CHUNK 4B are NOT changed here.
# ============================================================


# ============================================================
# 1. BASIC FORMATTERS
# ============================================================

sig_code <- function(p){
  if (length(p) == 0 || is.na(p)) return("")
  dplyr::case_when(
    p < 0.001 ~ "***",
    p < 0.01  ~ "**",
    p < 0.05  ~ "*",
    TRUE      ~ "ns"
  )
}

format_p_plain <- function(p){
  if (length(p) == 0 || is.na(p)) return("")
  if (p < 0.001) return("<0.001")
  formatC(p, format = "f", digits = 4)
```
```

```
}

safe_num <- function(x){
  if (length(x) == 0) return(NA_real_)
  suppressWarnings(as.numeric(x[1]))
}

first_existing_col <- function(x, candidates){
  hit <- intersect(candidates, colnames(x))
  if (length(hit) == 0) return(NULL)
  hit[1]
}


# ============================================================
# 2. LOAD RESULTS FROM CHUNK 4 AND CHUNK 4B
# ============================================================

load_permanova_result <- function(domain){
  if (exists("permanova_results_all") && !is.null(permanova_results_all[[domain]])) {
    return(permanova_results_all[[domain]])
  }
  path <- file.path(make_outdirs(domain), "RDS", glue("PERMANOVA_results_{domain}_Bray.rds"))
  if (!file.exists(path)) stop("❌ File not found: ", path, "\nRun CHUNK 4 first.")
  readRDS(path)
}

load_betadisper_result <- function(domain){
  if (exists("betadisper_results_all") && !is.null(betadisper_results_all[[domain]])) {
    return(betadisper_results_all[[domain]])
  }
  path <- file.path(make_outdirs(domain), "RDS", glue("Betadisper_results_{domain}_Bray.rds"))
  if (!file.exists(path)) stop("❌ File not found: ", path, "\nRun CHUNK 4 first.")
  readRDS(path)
}

load_crop_farm_result <- function(domain){
  if (exists("crop_farm_results_all") && !is.null(crop_farm_results_all[[domain]])) {
    return(crop_farm_results_all[[domain]])
  }
  path <- file.path(make_outdirs(domain), "RDS", glue("CropFarmTest_{domain}_Bray.rds"))
  if (!file.exists(path)) stop("❌ File not found: ", path, "\nRun CHUNK 4B first.")
  readRDS(path)
}


# ============================================================
# 3. STANDARD ROW STRUCTURE
# ============================================================

blank_result_row <- function(comparison = "", term = ""){
  tibble::tibble(
    Comparison = comparison, Term = term,
    Df = NA_real_, SS = NA_real_, R2 = NA_real_, F = NA_real_,
    p_raw = NA_real_, sig = ""
  )
}

section_row <- function(label){
  tibble::tibble(
    Comparison = label, Term = "",
    Df = NA_real_, SS = NA_real_, R2 = NA_real_, F = NA_real_,
    p_raw = NA_real_, sig = "", Row_type = "section"
  )
}

# Mark a row as "not tested" (P shown as —)
mark_not_tested <- function(row){
```

```
  row %>% dplyr::mutate(p_raw = NA_real_, sig = "—")
}


# ============================================================
# 4. EXTRACT PERMANOVA ROW
# ============================================================

extract_permanova_row <- function(tab, term, comparison = "", term_label = term){

  tmp <- tab %>% dplyr::filter(Term == term)

  if (nrow(tmp) == 0) {
    warning("⚠️ PERMANOVA term not found: ", term, " | ", comparison)
    return(blank_result_row(comparison, term_label) %>% dplyr::mutate(Row_type = "data"))
  }

  ss_col <- first_existing_col(tmp, c("SumOfSqs", "Sum Sq", "SumsOfSqs", "SS"))
  p_col  <- first_existing_col(tmp, c("Pr(>F)", "Pr(>F.)"))

  tibble::tibble(
    Comparison = comparison,
    Term = term_label,
    Df = safe_num(tmp$Df),
    SS = if (is.null(ss_col)) NA_real_ else safe_num(tmp[[ss_col]]),
    R2 = if ("R2" %in% colnames(tmp)) safe_num(tmp$R2) else NA_real_,
    F  = if ("F" %in% colnames(tmp)) safe_num(tmp$F) else NA_real_,
    p_raw = if (is.null(p_col)) NA_real_ else safe_num(tmp[[p_col]]),
    sig = if (is.null(p_col)) "" else sig_code(safe_num(tmp[[p_col]])),
    Row_type = "data"
  )
}


# ============================================================
# 5. EXTRACT PERMDISP ANOVA-LIKE ROWS
# ============================================================

extract_permdisp_rows <- function(tab, comparison, groups_label){

  if (is.null(tab) || nrow(tab) == 0) {
    return(blank_result_row(comparison, groups_label) %>% dplyr::mutate(Row_type = "data"))
  }

  dat <- as.data.frame(tab, check.names = FALSE)
  if (!"Term" %in% colnames(dat)) dat$Term <- rownames(dat)
  dat <- dat[!is.na(dat$Term) & nzchar(as.character(dat$Term)), , drop = FALSE]

  ss_col <- first_existing_col(dat, c("Sum Sq", "SumOfSqs", "SumsOfSqs", "SS"))
  f_col  <- first_existing_col(dat, c("F", "F.Model"))
  p_col  <- first_existing_col(dat, c("Pr(>F)", "Pr(>F.)"))

  get_one <- function(target_terms, comparison_value, label_value){
    idx <- which(tolower(as.character(dat$Term)) %in% tolower(target_terms))
    if (length(idx) == 0) return(NULL)
    z <- dat[idx[1], , drop = FALSE]
    p_val <- if (is.null(p_col)) NA_real_ else safe_num(z[[p_col]])
    tibble::tibble(
      Comparison = comparison_value,
      Term = label_value,
      Df = if ("Df" %in% colnames(z)) safe_num(z$Df) else NA_real_,
      SS = if (is.null(ss_col)) NA_real_ else safe_num(z[[ss_col]]),
      R2 = NA_real_,
      F  = if (is.null(f_col)) NA_real_ else safe_num(z[[f_col]]),
      p_raw = p_val,
      sig = sig_code(p_val),
      Row_type = "data"
    )
```

```
  }

  groups_row <- get_one(c("Groups", "Group"), comparison, groups_label)

  if (is.null(groups_row)) {
    z <- dat[1, , drop = FALSE]
    p_val <- if (is.null(p_col)) NA_real_ else safe_num(z[[p_col]])
    groups_row <- tibble::tibble(
      Comparison = comparison,
      Term = groups_label,
      Df = if ("Df" %in% colnames(z)) safe_num(z$Df) else NA_real_,
      SS = if (is.null(ss_col)) NA_real_ else safe_num(z[[ss_col]]),
      R2 = NA_real_,
      F  = if (is.null(f_col)) NA_real_ else safe_num(z[[f_col]]),
      p_raw = p_val,
      sig = sig_code(p_val),
      Row_type = "data"
    )
  }

  residual_row <- get_one(c("Residuals", "Residual"), "", "Residuals")
  if (is.null(residual_row)) return(groups_row)

  dplyr::bind_rows(groups_row, residual_row)
}


# ============================================================
# 6. HELPER TO SUBSET A RESULT TABLE BY MODEL LABEL
# ============================================================

subset_model <- function(tab, pattern){
  if (is.null(tab) || nrow(tab) == 0 || !"Model" %in% colnames(tab)) return(tibble::tibble())
  tab %>% dplyr::filter(grepl(pattern, Model, fixed = TRUE))
}


# ============================================================
# 7. BUILD MANUSCRIPT-STYLE MAIN TABLE
# ============================================================

crop_specs <- list(
  list(internal = "Banano_Gros_Michael", label = "Banana"),
  list(internal = "Dominico_Harton",     label = "Plantain")
)

build_main_table <- function(domain, perm_res, bd_res, crop_farm){

  out <- list()

  # ---------------- PERMANOVA — GLOBAL (sample level) ----------------
  out[[length(out) + 1]] <- section_row("PERMANOVA — GLOBAL (sample level)")

  out[[length(out) + 1]] <- extract_permanova_row(
    perm_res$global_terms, "Plant_Condition", "All samples", "Symptom status*")

  # Crop: Df, SS, R², F kept (block sums to Total); P not tested
  out[[length(out) + 1]] <- extract_permanova_row(
    perm_res$global_terms, "Crop", "", "Crop type**") %>% mark_not_tested()

  out[[length(out) + 1]] <- extract_permanova_row(
    perm_res$global_terms, "Plant_Condition:Crop", "", "Symptom status × Crop type")

  out[[length(out) + 1]] <- extract_permanova_row(
    perm_res$global_terms, "Residual", "", "Residual")

  out[[length(out) + 1]] <- extract_permanova_row(
    perm_res$global_terms, "Total", "", "Total")
```

```
3519
3520   # ---------------- PERMANOVA — CROP TEST BETWEEN FARMS ----------------
3521   out[[length(out) + 1]] <- section_row(
3522     sprintf("PERMANOVA — CROP TYPE TEST BETWEEN FARMS (farm level, n = %d farms)",
3523             crop_farm$n_farms))
3524
3525   if ("Department" %in% crop_farm$tab$Term) {
3526     out[[length(out) + 1]] <- extract_permanova_row(
3527       crop_farm$tab, "Department", "All farms", "Department") %>% mark_not_tested()
3528     out[[length(out) + 1]] <- extract_permanova_row(
3529       crop_farm$tab, "Crop", "", "Crop type")
3530   } else {
3531     out[[length(out) + 1]] <- extract_permanova_row(
3532       crop_farm$tab, "Crop", "All farms", "Crop type")
3533   }
3534
3535   out[[length(out) + 1]] <- extract_permanova_row(crop_farm$tab, "Residual", "", "Residual")
3536   out[[length(out) + 1]] <- extract_permanova_row(crop_farm$tab, "Total", "", "Total")
3537
3538   # ---------------- PERMANOVA — WITHIN CROP ----------------
3539   out[[length(out) + 1]] <- section_row("PERMANOVA — WITHIN CROP")
3540
3541   for (sp in crop_specs) {
3542     tab_cr <- subset_model(perm_res$by_crop, paste0("Within Crop: ", sp$internal))
3543     if (nrow(tab_cr) == 0) next
3544     out[[length(out) + 1]] <- extract_permanova_row(tab_cr, "Plant_Condition", sp$label, "Symptom status")
3545     out[[length(out) + 1]] <- extract_permanova_row(tab_cr, "Residual", "", "Residual")
3546     out[[length(out) + 1]] <- extract_permanova_row(tab_cr, "Total", "", "Total")
3547   }
3548
3549   # ---------------- PERMDISP — GLOBAL ----------------
3550   out[[length(out) + 1]] <- section_row("PERMDISP (Betadisper) — GLOBAL")
3551
3552   out[[length(out) + 1]] <- extract_permdisp_rows(
3553     bd_res$bd_condition, "Symptom status", "Asymptomatic vs. Symptomatic")
3554
3555   out[[length(out) + 1]] <- extract_permdisp_rows(
3556     bd_res$bd_crop, "Crop type", "Banana vs. Plantain")
3557
3558   out[[length(out) + 1]] <- extract_permdisp_rows(
3559     bd_res$bd_global, "Symptom status × Crop type", "Groups (4 groups)")
3560
3561   # ---------------- PERMDISP — WITHIN CROP ----------------
3562   out[[length(out) + 1]] <- section_row("PERMDISP (Betadisper) — WITHIN CROP")
3563
3564   for (sp in crop_specs) {
3565     tab_cr <- subset_model(bd_res$bd_by_crop, paste0("Within Crop: ", sp$internal))
3566     if (nrow(tab_cr) == 0) next
3567     out[[length(out) + 1]] <- extract_permdisp_rows(tab_cr, sp$label, "Asymptomatic vs. Symptomatic")
3568   }
3569
3570   dplyr::bind_rows(out)
3571 }
3572
3573
3574 # ============================================================
3575 # 8. BUILD HIERARCHICAL / SUPPLEMENTARY TABLE
3576 # ============================================================
3577
3578 build_hierarchical_table <- function(domain, perm_res, bd_res){
3579
3580   out <- list()
3581
3582   out[[length(out) + 1]] <- section_row("PERMANOVA — HIERARCHICAL SPATIAL MODEL")
3583
3584   hier_terms <- list(
3585     c("Plant_Condition",      "Symptom status"),
3586     c("Crop",                 "Crop type"),
```

```
    c("Plant_Condition:Crop", "Symptom status × Crop type"),
    c("Department",           "Department"),
    c("Department:Land",      "Land (nested within Department)"),
    c("Residual",             "Residual"),
    c("Total",                "Total")
  )

  for (x in hier_terms) {
    out[[length(out) + 1]] <- extract_permanova_row(
      perm_res$full_hierarchical, x[1],
      if (x[1] == "Plant_Condition") "All samples" else "",
      x[2])
  }

  out[[length(out) + 1]] <- section_row("PERMDISP (Betadisper) — DIAGNOSTIC ANALYSES")
  out[[length(out) + 1]] <- extract_permdisp_rows(bd_res$bd_condition, "Symptom status", "Groups")
  out[[length(out) + 1]] <- extract_permdisp_rows(bd_res$bd_global, "Crop type × Symptom status groups", "Groups")
  out[[length(out) + 1]] <- extract_permdisp_rows(bd_res$bd_global_dept, "Department × Symptom status groups",
"Groups")
  out[[length(out) + 1]] <- extract_permdisp_rows(bd_res$bd_by_land_group, "Land", "Groups")

  dplyr::bind_rows(out)
}


# ============================================================
# 9. FORMAT TABLE FOR EXCEL / MANUSCRIPT
# ============================================================

format_manuscript_table <- function(tab){

  tab %>%
    dplyr::mutate(
      Df_display = dplyr::if_else(is.na(Df), "", as.character(as.integer(round(Df)))),
      SS_display = dplyr::if_else(is.na(SS), "", formatC(SS, format = "f", digits = 3)),
      R2_display = dplyr::if_else(is.na(R2), "", formatC(R2, format = "f", digits = 3)),
      F_display  = dplyr::if_else(is.na(F),  "", formatC(F,  format = "f", digits = 3)),
      p_display  = ifelse(sig == "—", "—", vapply(p_raw, format_p_plain, character(1)))
    ) %>%
    dplyr::transmute(
      Comparison,
      Term,
      Df = Df_display,
      SS = SS_display,
      `R²` = R2_display,
      F = F_display,
      p = p_display,
      sig,
      Row_type
    )
}


# ============================================================
# 10. QC CHECKS
# ============================================================

qc_permanova_table <- function(perm_res, domain){

  primary  <- perm_res$global_terms
  total_df <- primary %>% dplyr::filter(Term == "Total") %>% dplyr::pull(Df)
  r2_sum   <- primary %>% dplyr::filter(Term != "Total") %>%
    dplyr::summarise(value = sum(R2, na.rm = TRUE)) %>% dplyr::pull(value)

  message("\n📌 QC PERMANOVA — ", domain)
  message("R² sum excluding Total = ", round(r2_sum, 8))
  if (length(total_df) == 1) message("Total df = ", total_df, " | inferred N = ", total_df + 1)

```

```
  invisible(tibble::tibble(
    Domain = domain,
    R2_sum_excluding_Total = r2_sum,
    Total_Df = ifelse(length(total_df) == 1, total_df, NA_real_),
    Inferred_N = ifelse(length(total_df) == 1, total_df + 1, NA_real_)
  ))
}

# Each PERMANOVA block: SS terms + residual = Total and ΣR² = 1
qc_block_sums <- function(tab, block_label){
  ss_col <- first_existing_col(tab, c("SumOfSqs", "Sum Sq", "SumsOfSqs", "SS"))
  tab %>%
    dplyr::group_by(Model) %>%
    dplyr::summarise(
      Block    = block_label,
      SS_sum   = sum(.data[[ss_col]][Term != "Total"], na.rm = TRUE),
      SS_total = .data[[ss_col]][Term == "Total"][1],
      R2_sum   = sum(R2[Term != "Total"], na.rm = TRUE),
      .groups  = "drop"
    ) %>%
    dplyr::mutate(OK = abs(SS_sum - SS_total) < 1e-6 & abs(R2_sum - 1) < 1e-6)
}


# ============================================================
# 11. STYLED WORKBOOK WRITER
# ============================================================

write_styled_result_sheet <- function(wb, sheet, table_formatted, title_text, note_text){

  openxlsx::addWorksheet(wb, sheet)
  n_cols <- 8

  title_style <- openxlsx::createStyle(fontName = "Times New Roman", fontSize = 11,
                                       textDecoration = "bold", valign = "center", wrapText = TRUE)
  header_style <- openxlsx::createStyle(fontName = "Times New Roman", fontSize = 10,
                                        textDecoration = "bold", fgFill = "#D9D9D9",
                                        border = "TopBottomLeftRight", borderStyle = "thin",
                                        halign = "center", valign = "center", wrapText = TRUE)
  section_style <- openxlsx::createStyle(fontName = "Times New Roman", fontSize = 10,
                                         textDecoration = c("bold", "italic"), fgFill = "#F2F2F2",
                                         border = "TopBottomLeftRight", borderStyle = "thin",
                                         halign = "left", valign = "center", wrapText = TRUE)
  body_left_style <- openxlsx::createStyle(fontName = "Times New Roman", fontSize = 10,
                                           border = "TopBottomLeftRight", borderStyle = "thin",
                                           halign = "left", valign = "center", wrapText = TRUE)
  body_center_style <- openxlsx::createStyle(fontName = "Times New Roman", fontSize = 10,
                                             border = "TopBottomLeftRight", borderStyle = "thin",
                                             halign = "center", valign = "center", wrapText = TRUE)
  note_style <- openxlsx::createStyle(fontName = "Times New Roman", fontSize = 9,
                                      textDecoration = "italic", halign = "left",
                                      valign = "top", wrapText = TRUE)

  # Title
  openxlsx::writeData(wb, sheet, title_text, startRow = 1, startCol = 1, colNames = FALSE)
  openxlsx::mergeCells(wb, sheet, cols = 1:n_cols, rows = 1:2)
  openxlsx::addStyle(wb, sheet, title_style, rows = 1:2, cols = 1:n_cols, gridExpand = TRUE)

  # Header
  header_row <- 4
  headers <- c("Comparison", "Term", "Df", "SS", "R²", "F", "p", "sig")
  openxlsx::writeData(wb, sheet, t(headers), startRow = header_row, startCol = 1,
                      colNames = FALSE, rowNames = FALSE)
  openxlsx::addStyle(wb, sheet, header_style, rows = header_row, cols = 1:n_cols, gridExpand = TRUE)

  # Body
  current_row <- header_row + 1

```

```
  for (i in seq_len(nrow(table_formatted))) {

    rr <- table_formatted[i, , drop = FALSE]

    if (identical(rr$Row_type[[1]], "section")) {

      openxlsx::writeData(wb, sheet, rr$Comparison[[1]], startRow = current_row,
                          startCol = 1, colNames = FALSE)
      openxlsx::mergeCells(wb, sheet, cols = 1:n_cols, rows = current_row)
      openxlsx::addStyle(wb, sheet, section_style, rows = current_row,
                         cols = 1:n_cols, gridExpand = TRUE)

    } else {

      vals <- unlist(rr[, c("Comparison", "Term", "Df", "SS", "R²", "F", "p", "sig"),
                        drop = FALSE], use.names = FALSE)

      openxlsx::writeData(wb, sheet, t(vals), startRow = current_row, startCol = 1,
                          colNames = FALSE, rowNames = FALSE)
      openxlsx::addStyle(wb, sheet, body_left_style, rows = current_row,
                         cols = 1:2, gridExpand = TRUE)
      openxlsx::addStyle(wb, sheet, body_center_style, rows = current_row,
                         cols = 3:n_cols, gridExpand = TRUE)
    }

    current_row <- current_row + 1
  }

  # Footnote
  note_start <- current_row + 2
  openxlsx::writeData(wb, sheet, note_text, startRow = note_start, startCol = 1, colNames = FALSE)
  openxlsx::mergeCells(wb, sheet, cols = 1:n_cols, rows = note_start:(note_start + 6))
  openxlsx::addStyle(wb, sheet, note_style, rows = note_start:(note_start + 6),
                     cols = 1:n_cols, gridExpand = TRUE)

  # Dimensions
  openxlsx::setColWidths(wb, sheet, cols = 1,   widths = 25)
  openxlsx::setColWidths(wb, sheet, cols = 2,   widths = 36)
  openxlsx::setColWidths(wb, sheet, cols = 3,   widths = 7)
  openxlsx::setColWidths(wb, sheet, cols = 4:7, widths = 11)
  openxlsx::setColWidths(wb, sheet, cols = 8,   widths = 7)
  openxlsx::setRowHeights(wb, sheet, rows = 1:2, heights = 28)
  openxlsx::freezePane(wb, sheet, firstActiveRow = 5)

  invisible(list(last_data_row = current_row - 1, note_start = note_start))
}


# ============================================================
# 12. EXPORT LOOP
# ============================================================

reviewer_tables <- list()

for (dom in cfg$domain) {

  message("\n================================================")
  message("📄 REVIEWER-READY TABLE — ", dom)
  message("================================================")

  out_root   <- make_outdirs(dom)
  tables_dir <- file.path(out_root, "Tables")

  perm_res  <- load_permanova_result(dom)
  bd_res    <- load_betadisper_result(dom)
  crop_farm <- load_crop_farm_result(dom)

  qc <- qc_permanova_table(perm_res, dom)
```

```
3790
3791  # ---- QC per PERMANOVA block ----
3792  qc_blocks <- dplyr::bind_rows(
3793    qc_block_sums(perm_res$global_terms, "PERMANOVA — GLOBAL (sample level)"),
3794    qc_block_sums(crop_farm$tab,         "PERMANOVA — CROP TEST BETWEEN FARMS"),
3795    qc_block_sums(perm_res$by_crop,      "PERMANOVA — WITHIN CROP")
3796  )
3797  print(as.data.frame(qc_blocks))
3798  if (!all(qc_blocks$OK)) stop("❌ At least one PERMANOVA block does not sum to Total / R² ≠ 1 — ", dom)
3799
3800  main_raw <- build_main_table(dom, perm_res, bd_res, crop_farm)
3801  hier_raw <- build_hierarchical_table(dom, perm_res, bd_res)
3802
3803  main_fmt <- format_manuscript_table(main_raw)
3804  hier_fmt <- format_manuscript_table(hier_raw)
3805
3806  reviewer_tables[[dom]] <- list(main = main_fmt, hierarchical = hier_fmt,
3807                                 qc = qc, qc_blocks = qc_blocks)
3808
3809  # ---- Title / note ----
3810  if (dom == "Bacterias") {
3811    title_text <- paste0(
3812      "Table 1. PERMANOVA and PERMDISP (betadisper) results for soil bacterial ",
3813      "community structure based on Bray–Curtis dissimilarities (rarefied at ",
3814      expected_depth_dom, " reads per sample).")
3815  } else {
3816    title_text <- paste0(
3817      "Table 2. PERMANOVA and PERMDISP (betadisper) results for soil fungal ",
3818      "community structure based on Bray–Curtis dissimilarities (rarefied at ",
3819      expected_depth_dom, " reads per sample).")
3820  }
3821
3822  note_text <- sprintf(paste0(
3823    "*Symptom status: asymptomatic vs. symptomatic plants. **Crop type: banana vs. plantain. ",
3824    "PERMANOVA used Bray–Curtis dissimilarities and 9,999 permutations. The permutations were ",
3825    "restricted within farm identity to account for the hierarchical sampling design. Because crop ",
3826    "type was constant within farm, its P value is not reported in the sample-level model; its effect ",
3827    "was evaluated separately at the independent farm level (n = %d farms; one balanced centroid per ",
3828    "farm) after accounting for department, with farms permuted within department (minimum attainable ",
3829    "P ≈ %.3f). PERMDISP is reported separately from PERMANOVA; crop-type and combined-group PERMDISP ",
3830    "tests are diagnostic. PERMDISP numerator degrees of freedom equal k − 1, where k is the number of ",
3831    "groups compared. Df, degrees of freedom; SS, sum of squares; R², proportion of total variation ",
3832    "attributed to a term in the corresponding PERMANOVA model. Significance levels: *** p < 0.001; ",
3833    "** p < 0.01; * p < 0.05; ns, not significant; –, not tested or not reported."),
3834    crop_farm$n_farms, crop_farm$min_p)
3835
3836  # ---- Workbook ----
3837  wb <- openxlsx::createWorkbook()
3838
3839  write_styled_result_sheet(wb = wb, sheet = "Main_Table", table_formatted = main_fmt,
3840                            title_text = title_text, note_text = note_text)
3841
3842  write_styled_result_sheet(
3843    wb = wb, sheet = "Hierarchical_Table", table_formatted = hier_fmt,
3844    title_text = paste0(
3845      "Supplementary analysis. Hierarchical spatial PERMANOVA and PERMDISP diagnostic analyses — ",
3846      ifelse(dom == "Bacterias", "soil bacteria", "soil fungi"), "."),
3847    note_text = paste0(
3848      "Hierarchical model: Bray ~ Symptom status * Crop type + Department/Land, reported as a ",
3849      "descriptive variance decomposition. PERMDISP diagnostics are presented separately because they ",
3850      "test dispersion among grouping structures rather than term-by-term factorial effects. ",
3851      "Symptom-status PERMDISP uses permutations restricted within Land; combined-group, Department ",
3852      "and Land diagnostics are unrestricted.")
3853  )
3854
3855  # Raw / QC sheets
3856  openxlsx::addWorksheet(wb, "Raw_Main");          openxlsx::writeData(wb, "Raw_Main", main_raw)
3857  openxlsx::addWorksheet(wb, "Raw_Hierarchical");  openxlsx::writeData(wb, "Raw_Hierarchical", hier_raw)
```

```
  openxlsx::addWorksheet(wb, "Raw_Crop_FarmLevel")
  openxlsx::writeData(wb, "Raw_Crop_FarmLevel", crop_farm$tab)
  openxlsx::writeData(wb, "Raw_Crop_FarmLevel", crop_farm$land_meta,
                      startRow = nrow(crop_farm$tab) + 4)
  openxlsx::addWorksheet(wb, "QC")
  openxlsx::writeData(wb, "QC", qc)
  openxlsx::writeData(wb, "QC", qc_blocks, startRow = nrow(qc) + 4)

  excel_path <- file.path(tables_dir, glue("Table_PERMANOVA_PERMDISP_Separated_{dom}_Bray.xlsx"))
  openxlsx::saveWorkbook(wb, excel_path, overwrite = TRUE)

  # CSV exports
  readr::write_csv(main_raw %>% dplyr::select(-Row_type),
                   file.path(tables_dir, glue("Table_PERMANOVA_PERMDISP_Separated_{dom}_Bray.csv")))
  readr::write_csv(hier_raw %>% dplyr::select(-Row_type),
                   file.path(tables_dir, glue("Table_Hierarchical_PERMANOVA_PERMDISP_{dom}_Bray.csv")))
  readr::write_csv(qc_blocks,
                   file.path(tables_dir, glue("QC_PERMANOVA_blocks_{dom}_Bray.csv")))

  message("✅ Excel: ", excel_path)
  print(as.data.frame(main_fmt %>% dplyr::select(-Row_type)))
}


# ============================================================
# 13. FINAL CONSISTENCY MESSAGE
# ============================================================

message("\n✅ CHUNK 5 complete.")
message("Crop type P reported only from the farm-level test (CHUNK 4B).")
message(cfg$label, " rarefaction QC = ", expected_depth_dom, " reads/sample.")
```

Session information

```{r session_info}
sessionInfo()
```

04 — Relative abundance (Phylum and Order) | [domain]

Script: 04_Relative_Abundance.Rmd · source listing (728 lines) · code shown as written; chunks not executed

R chunks

line 54: ```{r setup, message=FALSE, warning=FALSE}
line 88: ```{r helpers, message=FALSE, warning=FALSE}
line 517: ```{r tax_levels_selection, message=TRUE, warning=TRUE}
line 544: ```{r relabund_loop_bars, message=TRUE, warning=TRUE}
line 688: ```{r interpretation_note, message=TRUE, warning=FALSE}
line 701: ```{r verification, message=TRUE}
line 725: ```{r session_info}

```
---
title: "04 — Relative abundance (Phylum and Order) | `r params$domain`"
author: "Sergio Pardo-Díaz"
date: "`r format(Sys.Date())`"
output:
  html_document:
    toc: true
    toc_depth: 3
    number_sections: true
params:
  # === Domain: "Bacterias" (16S rRNA) or "Hongos" (ITS2 no-trunc) ===
  domain:            "Hongos"
  # Parent folder; data are read from <project_root>/<domain_dir>
  project_root:      "."
  # NULL = use the default from domain_settings.R
  domain_dir:        null
  relabund_input:    null   # bacteria: ps_rel.rds | fungi: ps_rare_2000.rds
  relabund_top_mode: null   # bacteria: "per_crop_union" | fungi: "global"
  expected_n:        null
  rare_depth:        null

  filtering_subdir:  "01_Filtering_ASVs"
  filtering_rds_subdir: "RDS"
  analysis_subdir:   "04_Relative_Abundance"
  state_var:         "Plant_Condition"
  crop_var:          "Crop"
  farm_var:          "Land"
  target_crops:
    - "Dominico_Harton"
    - "Banano_Gros_Michael"
  tax_levels:        [Phylum, Order]
  top_n:             15
  multipanel_root:   "Multipanels"
  settings_file:     "domain_settings.R"
---

> **Relative-abundance workflow (bacteria and fungi).**
> Stacked bars are descriptive; inferential differential-abundance testing is
> done separately with unrarefied counts using ANCOM-BC2 (step 05).
>
> Two settings differ by domain (see `domain_settings.R`):
>
> * **Input object.** Fungi use `ps_rare_2000.rds` (same rarefied object as
>   alpha/beta diversity); bacteria use `ps_rel.rds` (relative abundance of
>   the filtered, unrarefied counts).
> * **Top-N selection.** `global`: one global Top N of identified taxa;
>   `Unassigned` is never part of the Top N nor of `Others` and is always shown
>   last. `per_crop_union`: the Top N of each crop are pooled, ordered by global
>   abundance and capped at N, so a taxon dominant in one crop is not hidden
>   in `Others`.

## Chunk 1 — Setup
```

````
```{r setup, message=FALSE, warning=FALSE}
suppressPackageStartupMessages({
  library(phyloseq)
  library(tidyverse)
  library(openxlsx)
  library(patchwork)
  library(glue)
  library(scales)
  library(stringi)
  library(Cairo)
})
theme_set(theme_bw(base_size = 12))

# ---- Domain settings (single source of truth: domain_settings.R)
settings_file <- params$settings_file
if (!file.exists(settings_file)) {
  settings_file <- file.path(dirname(knitr::current_input(dir = TRUE)), settings_file)
}
source(settings_file)

cfg <- domain_settings(params$domain, params)
domain_base_dir <- domain_path(params$project_root, cfg)

stopifnot(cfg$relabund_top_mode %in% c("global", "per_crop_union"))

message("Domain: ", cfg$domain, " — ", cfg$label_long,
        " | input: ", cfg$relabund_input,
        " | Top-N mode: ", cfg$relabund_top_mode)
```

---

## Chunk 2 — Helpers

```{r helpers, message=FALSE, warning=FALSE}

# ============================================================
# Robust UTF-8
# ============================================================
force_utf8 <- function(x){
  x <- as.character(x)
  stringi::stri_enc_toutf8(x, is_unknown_8bit = TRUE)
}

# ============================================================
# Palettes
# ============================================================
col_state <- c(
  "Diseased_Plant" = "#F8766D",
  "Healthy_Plant"  = "#00BFC4"
)

palette_sobria_11 <- c(
  "#4C566A","#5E81AC","#D08770","#A3BE8C","#88C0D0",
  "#BF616A","#EBCB8B","#81A1C1","#636E75","#8FBCBB",
  "#B0B0B0"
)

# ============================================================
# Directories
# ============================================================
make_domain_dirs <- function(dom){
  analysis_dir <- file.path(domain_base_dir, params$analysis_subdir)
  dir.create(file.path(analysis_dir, "Figures"), recursive = TRUE, showWarnings = FALSE)
  dir.create(file.path(analysis_dir, "Tables"),  recursive = TRUE, showWarnings = FALSE)
  list(base_dir = domain_base_dir, analysis_dir = analysis_dir)
}

```
````

```r
make_multipanel_dirs <- function(){
  mp_dir <- file.path(domain_base_dir, params$multipanel_root, params$analysis_subdir)
  dir.create(file.path(mp_dir, "Figures"), recursive = TRUE, showWarnings = FALSE)
  dir.create(file.path(mp_dir, "Tables"),  recursive = TRUE, showWarnings = FALSE)
  mp_dir
}

# ============================================================
# Load phyloseq and convert to sample-wise relative abundance
#
# If the input is a rarefied object (ps_rare_*), its depth and
# sample number are checked first. Converting an object that is
# already relative (ps_rel) leaves it unchanged.
# ============================================================
load_ps <- function(dom){

  ps_path <- file.path(
    domain_base_dir,
    params$filtering_subdir,
    params$filtering_rds_subdir,
    cfg$relabund_input
  )

  message("📦 Loading: ", ps_path)
  if(!file.exists(ps_path)){
    stop("❌ File not found: ", ps_path)
  }

  ps <- readRDS(ps_path)

  if(grepl("^ps_rare", basename(ps_path))){

    depths <- as.numeric(sample_sums(ps))

    if(!is.null(cfg$expected_n) && nsamples(ps) != cfg$expected_n){
      warning("⚠ ", cfg$expected_n, " samples were expected; ",
              nsamples(ps), " were observed.")
    }

    if(any(depths != cfg$rare_depth)){
      stop("❌ ", basename(ps_path), " does not contain exactly ",
           format(cfg$rare_depth, big.mark = ","), " reads/sample. ",
           "Observed range: ", min(depths), "–", max(depths), ".")
    }
  }

  # Relative abundance, calculated sample-wise
  ps_rel <- phyloseq::transform_sample_counts(
    ps,
    function(x){
      s <- sum(x)
      if(s > 0) x / s else x
    }
  )

  message("✔ ", dom, ": ", nsamples(ps_rel), " samples | ",
          ntaxa(ps_rel), " ASVs")

  ps_rel
}

# ============================================================
# QIIME2 prefixes for parsing the Taxon string column
# ============================================================
rank_prefix_map <- c(
  Domain  = "d__", Kingdom = "k__", Phylum  = "p__",
  Class   = "c__", Order   = "o__", Family  = "f__",
  Genus   = "g__", Species = "s__"
```

```
)

extract_rank_token <- function(taxon_str, prefix){
  if(is.na(taxon_str) || taxon_str == "") return(NA_character_)
  toks <- unlist(strsplit(taxon_str, ";\\s*"))
  hit  <- toks[grepl(paste0("^", prefix), toks)]
  if(!length(hit)) return(NA_character_)
  val <- trimws(sub(paste0("^", prefix), "", hit[1]))
  if(val == "" || val == "NA") return(NA_character_)
  val
}

# ============================================================
# can_derive_rank — format A (direct column) OR format B (Taxon string)
# Resolves the difference between tax_tables with parsed rank
# columns and tax_tables with a single QIIME2 Taxon column.
# ============================================================
can_derive_rank <- function(ps, rk){
  tt <- phyloseq::tax_table(ps, errorIfNULL = FALSE)
  if(is.null(tt)) return(FALSE)
  tx <- as.data.frame(tt, stringsAsFactors = FALSE, check.names = FALSE)

  # Format A: the rank already exists as a column
  if(rk %in% colnames(tx)) return(TRUE)

  # Format B: there is a Taxon column with QIIME2-style strings
  if(!"Taxon" %in% colnames(tx)) return(FALSE)
  pref <- rank_prefix_map[[rk]]
  if(is.null(pref) || is.na(pref)) return(FALSE)
  tax_str <- stringi::stri_enc_toutf8(as.character(tx$Taxon), is_unknown_8bit = TRUE)
  any(grepl(paste0("(^|;\\s*)", pref), tax_str))
}

# ============================================================
# build_rank_table — unified for both formats
# Format A: reads the rank column directly
# Format B: parses from the Taxon string (QIIME2)
# ============================================================
build_rank_table <- function(ps, dom, ranks){
  tt <- phyloseq::tax_table(ps, errorIfNULL = FALSE)
  if(is.null(tt)) stop("tax_table() is NULL in: ", dom)
  tx <- as.data.frame(tt, stringsAsFactors = FALSE, check.names = FALSE)

  out <- tibble::tibble(OTU = rownames(tx))

  for(rk in ranks){

    # --- Format A: direct column ---
    if(rk %in% colnames(tx)){
      vals <- force_utf8(tx[[rk]])
      out[[rk]] <- ifelse(is.na(vals) | vals == "" | vals == "NA", "Unassigned", vals)
      next
    }

    # --- Format B: parse from the Taxon string ---
    if("Taxon" %in% colnames(tx)){
      pref <- rank_prefix_map[[rk]]
      if(!is.null(pref) && !is.na(pref)){
        tax_str <- force_utf8(tx$Taxon)
        out[[rk]] <- vapply(tax_str, function(s){
          v <- extract_rank_token(s, pref)
          if(is.na(v)) "Unassigned" else v
        }, character(1))
        next
      }
    }

    # --- Fallback: rank not available ---
```

```
    warning(glue("Rank '{rk}' not available in {dom} — assigning 'Unassigned'"))
    out[[rk]] <- "Unassigned"
  }

  out
}

# ============================================================
# Filter for taxa with problematic labels (after taxonomic filtering)
# Complements the filters of the filtering pipeline
# ============================================================
is_weird_taxon <- function(x){
  x <- force_utf8(x)
  x2 <- tolower(trimws(x))
  has_digits      <- grepl("[0-9]", x2)
  starts_unknown  <- grepl("^(unknown|unidentified|unclassified|uncultured)\\b", x2)
  code_like       <- grepl("\\b(ccd\\d+|bacteriap\\d+)\\b", x2)
  bad_tokens      <- grepl("\\b(norank|no_rank|metagenome|ambiguous)\\b", x2)
  incertae_sedis  <- grepl("incertae.sedis", x2)   # taxa without an assigned formal name
  has_digits | starts_unknown | code_like | bad_tokens | incertae_sedis
}

# ============================================================
# phyloseq -> long table of relative abundance per sample × taxon
# ============================================================
ps_to_relabund_taxon_sample <- function(ps, tax_level, dom){

  rank_tbl <- build_rank_table(ps, dom, ranks = tax_level)

  df <- phyloseq::psmelt(ps) %>%
    as_tibble() %>%
    left_join(rank_tbl, by = c("OTU" = "OTU"))

  if(!tax_level %in% colnames(df)){
    stop(glue("Could not derive rank '{tax_level}' for {dom}."))
  }

  out <- df %>%
    transmute(
      SampleID        = force_utf8(as.character(Sample)),
      Crop            = force_utf8(.data[[params$crop_var]]),
      Plant_Condition = force_utf8(.data[[params$state_var]]),
      Taxon           = force_utf8(.data[[tax_level]]),
      Abundance       = as.numeric(Abundance),
      Domain          = dom,
      TaxLevel        = tax_level
    ) %>%
    mutate(
      Taxon = ifelse(is.na(Taxon) | Taxon == "" | Taxon == "NA", "Unassigned", Taxon),
      Plant_Condition = factor(Plant_Condition,
                               levels = c("Healthy_Plant", "Diseased_Plant")),
      Crop = factor(Crop)
    ) %>%
    filter(
      is.finite(Abundance),
      !is.na(Crop), !is.na(Plant_Condition),
      Plant_Condition %in% c("Healthy_Plant", "Diseased_Plant")
    )

  # Filter to crops of interest
  if(!is.null(params$target_crops) && length(params$target_crops) > 0){
    out <- out %>% filter(as.character(Crop) %in% params$target_crops)
  }

  # Sum per SampleID × Taxon (in case of duplicates after the join)
  out2 <- out %>%
    group_by(SampleID, Crop, Plant_Condition, Taxon, Domain, TaxLevel) %>%
    summarise(Abundance = sum(Abundance, na.rm = TRUE), .groups = "drop")
```

```r

  out2 %>% filter(!is_weird_taxon(Taxon))
}

# ============================================================
# Top N + Others (+ Unassigned)
#
# mode = "global"
#   1) Top N identified taxa (global ranking; Unassigned excluded
#      from the ranking so it never takes a Top-N position)
#   2) Others
#   3) Unassigned, always last in the legend/bar
#
# mode = "per_crop_union"
#   Top N of EACH crop, pooled (union), ordered by global abundance
#   and capped at N, so a taxon dominant in one crop is not hidden
#   in "Others". All remaining taxa -> Others.
# ============================================================
collapse_top_taxa <- function(df_tax_sample, top_n,
                              mode = cfg$relabund_top_mode,
                              by_group = "Crop"){

  df_tax_sample <- df_tax_sample %>%
    mutate(
      Taxon = as.character(Taxon),
      Taxon = case_when(
        is.na(Taxon)        ~ "Unassigned",
        trimws(Taxon) == "" ~ "Unassigned",
        TRUE                ~ Taxon
      )
    )

  if(mode == "global"){

    global_order <- df_tax_sample %>%
      filter(Taxon != "Unassigned") %>%
      group_by(Taxon) %>%
      summarise(total_global = sum(Abundance, na.rm = TRUE), .groups = "drop") %>%
      arrange(desc(total_global))

    top_taxa <- global_order %>% slice_head(n = top_n) %>% pull(Taxon)

    df2 <- df_tax_sample %>%
      mutate(Taxon2 = case_when(
        Taxon == "Unassigned" ~ "Unassigned",
        Taxon %in% top_taxa   ~ Taxon,
        TRUE                  ~ "Others"
      ))

    taxa_levels <- unique(c(top_taxa, "Others", "Unassigned"))

  } else {

    # Global ranking (total abundance): legend order and final cap
    global_order <- df_tax_sample %>%
      group_by(Taxon) %>%
      summarise(total_global = sum(Abundance, na.rm = TRUE), .groups = "drop") %>%
      arrange(desc(total_global))

    if(!is.null(by_group) && by_group %in% colnames(df_tax_sample)){
      # Step 1: Top N per crop -> union (each crop is represented)
      top_per_group <- df_tax_sample %>%
        group_by(.data[[by_group]], Taxon) %>%
        summarise(total = sum(Abundance, na.rm = TRUE), .groups = "drop") %>%
        group_by(.data[[by_group]]) %>%
        slice_max(order_by = total, n = top_n, with_ties = FALSE) %>%
        ungroup() %>%
        pull(Taxon) %>%
```

```r
        unique()

      # Step 2: order the union by global ranking and cap at top_n
      top_taxa <- global_order %>%
        filter(Taxon %in% top_per_group) %>%
        slice_head(n = top_n) %>%
        pull(Taxon)
    } else {
      # Fallback: direct global Top N
      top_taxa <- global_order %>% slice_head(n = top_n) %>% pull(Taxon)
    }

    df2 <- df_tax_sample %>%
      mutate(Taxon2 = ifelse(Taxon %in% top_taxa, Taxon, "Others"))

    taxa_levels <- c(top_taxa, "Others")
  }

  df2 <- df2 %>% mutate(Taxon2 = factor(Taxon2, levels = taxa_levels))

  list(df = df2, taxa_levels = taxa_levels, ranking = global_order)
}

# ============================================================
# Consistent color palette per Taxon2
#
# Top taxa   = palette_sobria_11 (interpolated)
# Others     = black
# Unassigned = grey (only in "global" mode, where it is a
#              separate category)
# Colors follow exactly the order of taxa_levels.
# ============================================================
build_tax_colors <- function(taxa_levels,
                             unassigned_special = (cfg$relabund_top_mode == "global")){

  taxa_levels <- as.character(taxa_levels)
  special     <- c("Others", if(unassigned_special) "Unassigned")
  taxa_main   <- setdiff(taxa_levels, special)
  n           <- length(taxa_main)

  cols_main <- if(n > 0){
    cc <- grDevices::colorRampPalette(palette_sobria_11)(n)
    names(cc) <- taxa_main
    cc
  } else setNames(character(), character())

  if("Others" %in% taxa_levels) cols_main <- c(cols_main, Others = "black")
  if(unassigned_special && "Unassigned" %in% taxa_levels){
    cols_main <- c(cols_main, Unassigned = "grey70")
  }

  cols_main[taxa_levels]
}

# ============================================================
# Correct 100% summary
#
# 1) one row per SampleID × Taxon2
# 2) absent categories completed with zero
# 3) renormalized per sample (sum = 1 after taxonomic grouping)
# 4) averaged per Crop × Plant_Condition × Taxon2
# ============================================================
summarise_groups_bars_100 <- function(df2, taxa_levels){

  # 1) sum per sample × Taxon2
  df_s <- df2 %>%
    group_by(SampleID, Crop, Plant_Condition, Taxon2) %>%
    summarise(Abundance = sum(Abundance, na.rm = TRUE), .groups = "drop")
```

```
  # 2) all sample × category combinations, zeros completed, so an
  #    absent category does not disappear from the group mean
  samp_meta <- df_s %>% distinct(SampleID, Crop, Plant_Condition)
  grid <- tidyr::expand_grid(
    samp_meta,
    Taxon2 = factor(taxa_levels, levels = taxa_levels)
  )
  df_c <- grid %>%
    left_join(df_s, by = c("SampleID", "Crop", "Plant_Condition", "Taxon2")) %>%
    mutate(
      Abundance = replace_na(Abundance, 0),
      Taxon2    = factor(as.character(Taxon2), levels = taxa_levels)
    )

  # 3) renormalize per sample
  df_c <- df_c %>%
    group_by(SampleID, Crop, Plant_Condition) %>%
    mutate(
      s         = sum(Abundance, na.rm = TRUE),
      Abundance = ifelse(s > 0, Abundance / s, 0)
    ) %>%
    ungroup() %>%
    select(-s)

  # 4) mean per Crop × Plant_Condition
  gsum <- df_c %>%
    group_by(Crop, Plant_Condition, Taxon2) %>%
    summarise(mean_ab = mean(Abundance, na.rm = TRUE), .groups = "drop") %>%
    mutate(Taxon2 = factor(as.character(Taxon2), levels = taxa_levels))

  # Audit: every Crop × Plant_Condition must sum to 1
  audit <- gsum %>%
    group_by(Crop, Plant_Condition) %>%
    summarise(sum_mean = sum(mean_ab, na.rm = TRUE), .groups = "drop") %>%
    mutate(ok = abs(sum_mean - 1) < 1e-6)

  list(gsum = gsum, audit = audit, df_sample_completed = df_c)
}

# ============================================================
# n = per group for x-axis labels
# ============================================================
make_n_labels <- function(df_c){
  df_c %>%
    distinct(SampleID, Crop, Plant_Condition) %>%
    count(Crop, Plant_Condition, name = "n") %>%
    mutate(x_label = glue("{Plant_Condition}\n(n={n})"))
}
```

Chunk 3 — Check derivable ranks

```{r tax_levels_selection, message=TRUE, warning=TRUE}
tax_levels_use <- list()

for(dom in cfg$domain){
  cat("\n==============================================\n")
  cat("🔎 Rank check —", dom, "\n")

  ps   <- load_ps(dom)
  keep <- params$tax_levels[
    vapply(params$tax_levels,
           function(rk) can_derive_rank(ps, rk),
           logical(1))
  ]
```

```
  tax_levels_use[[dom]] <- keep
  cat("Usable:", ifelse(length(keep), paste(keep, collapse = ", "), "(none)"), "\n")

  if(length(keep) == 0){
    warning("No derivable taxonomic levels for ", dom)
  }
}
```

Chunk 4 — Main loop: 100% stacked bars per taxonomic level

```{r relabund_loop_bars, message=TRUE, warning=TRUE}
mp_dir      <- make_multipanel_dirs()
bars_store  <- list()

top_label <- if(cfg$relabund_top_mode == "global"){
  glue("Top {params$top_n} global + Others + Unassigned")
} else {
  glue("Top {params$top_n} per crop (union) + Others")
}

for(dom in cfg$domain){
  cat("\n==========================================\n")
  cat("📌 Relative abundance (100% bars) —", dom, "\n")

  dirs         <- make_domain_dirs(dom)
  analysis_dir <- dirs$analysis_dir
  ps           <- load_ps(dom)
  levels_dom   <- tax_levels_use[[dom]]

  if(is.null(levels_dom) || length(levels_dom) == 0){
    warning("No valid levels for ", dom, " — skipped.")
    next
  }

  for(tax_level in levels_dom){
    cat("  ▶ Level:", tax_level, "\n")

    # ── Long table per sample ──────────────────────────
    df_tax_sample <- ps_to_relabund_taxon_sample(ps, tax_level, dom)

    if(nrow(df_tax_sample) == 0){
      warning("No data after filtering for ", dom, " / ", tax_level)
      next
    }

    # ── Top N + Others ─────────────────────────────────
    top_res     <- collapse_top_taxa(df_tax_sample, params$top_n)
    df2         <- top_res$df
    taxa_levels <- top_res$taxa_levels

    # ── 100% summary with audit ────────────────────────
    res100 <- summarise_groups_bars_100(df2, taxa_levels)
    gsum   <- res100$gsum
    audit  <- res100$audit
    df_c   <- res100$df_sample_completed

    # ── Colors ─────────────────────────────────────────
    tax_cols <- build_tax_colors(taxa_levels)

    # ── n= per group for x-axis labels ─────────────────
    n_labels <- make_n_labels(df_c)

    # Build x labels per Crop × Plant_Condition
    gsum_plot <- gsum %>%
```

```
    left_join(n_labels, by = c("Crop", "Plant_Condition")) %>%
    mutate(x_label = factor(x_label,
                            levels = unique(x_label[order(Plant_Condition)])))

  # ── Store in memory ──────────────────────────────────────
  key <- paste(dom, tax_level, sep = "__")
  bars_store[[key]] <- list(
    gsum        = gsum,
    audit       = audit,
    tax_cols    = tax_cols,
    taxa_levels = taxa_levels,
    n_labels    = n_labels,
    ranking     = top_res$ranking
  )

  # ── Excel per domain (auditable) ─────────────────────────
  out_xlsx <- file.path(
    analysis_dir, "Tables",
    glue("RelativeAbundance_BARS_{dom}_{tax_level}.xlsx")
  )
  openxlsx::write.xlsx(
    list(
      Taxon_bySample       = df_tax_sample,
      Collapsed_bySample   = df2,
      SampleCompleted_100  = df_c,
      GroupMean_100        = gsum,
      Audit_GroupSums      = audit,
      TopTaxa_Levels       = tibble::tibble(Taxon2 = taxa_levels),
      Taxon_Global_Ranking = top_res$ranking
    ),
    out_xlsx, overwrite = TRUE
  )

  if(any(!audit$ok)){
    warning("⚠️ Sums ≠ 1 in ", dom, "/", tax_level,
            ". Check Audit_GroupSums in the Excel file.")
  } else {
    message("✔ Audit OK (bars sum to 1) — ", dom, " / ", tax_level)
  }

  # ── Plot ─────────────────────────────────────────────────
  p_dom <- ggplot(gsum_plot,
                  aes(x = x_label, y = mean_ab, fill = Taxon2)) +
    geom_col(width = 0.6, color = "grey25", linewidth = 0.15) +
    facet_grid(. ~ Crop, scales = "free_x", space = "free_x") +
    scale_fill_manual(values = tax_cols, drop = FALSE) +
    scale_y_continuous(
      labels = scales::percent_format(accuracy = 1),
      limits = c(0, 1),
      expand = expansion(mult = c(0, 0.02))
    ) +
    labs(
      title    = glue("Relative abundance — {cfg$label} ({tax_level})"),
      subtitle = glue("{top_label} | Group mean (zero-filled + renormalized)"),
      x        = NULL,
      y        = "Mean relative abundance (%)",
      fill     = tax_level
    ) +
    theme(
      strip.text          = element_text(face = "bold"),
      axis.text.x         = element_text(size = 9),
      panel.grid.major.x  = element_blank(),
      panel.grid.minor    = element_blank()
    )

  # ── Export PDF/PNG per domain ────────────────────────────
  out_pdf <- file.path(analysis_dir, "Figures",
                       glue("RelAbund_BARS_{dom}_{tax_level}_facetCrop.pdf"))
```

```
    out_png <- file.path(analysis_dir, "Figures",
                         glue("RelAbund_BARS_{dom}_{tax_level}_facetCrop.png"))
    ggsave(out_pdf, p_dom, width = 11, height = 5, device = cairo_pdf)
    ggsave(out_png, p_dom, width = 11, height = 5, dpi = 300)

    # ── Export PDF/PNG multipanel ──────────────────────────
    out_pdf2 <- file.path(mp_dir, "Figures",
                          glue("Multipanel_BARS_{tax_level}_{dom}_CropsSideBySide.pdf"))
    out_png2 <- file.path(mp_dir, "Figures",
                          glue("Multipanel_BARS_{tax_level}_{dom}_CropsSideBySide.png"))
    ggsave(out_pdf2, p_dom, width = 11, height = 7, device = cairo_pdf)
    ggsave(out_png2, p_dom, width = 11, height = 7, dpi = 300)

    cat("    ✔ Figures saved —", dom, tax_level, "\n")
  }
}
```

Chunk 5 — Interpretation note

```{r interpretation_note, message=TRUE, warning=FALSE}
cat(
  "\nRelative abundance is descriptive. ",
  "No inferential significance is assigned from these stacked-bar plots.\n",
  "Differential abundance is evaluated separately using unrarefied counts ",
  "with the farm-aware ANCOM-BC2 workflow (step 05).\n"
)
```

Chunk 6 — Output verification

```{r verification, message=TRUE}
cat("\n=== Output verification ===\n\n")

mp_fig_dir <- file.path(
  domain_base_dir,
  params$multipanel_root,
  params$analysis_subdir,
  "Figures"
)

for(tl in tax_levels_use[[cfg$domain]]){
  f <- file.path(
    mp_fig_dir,
    glue("Multipanel_BARS_{tl}_{cfg$domain}_CropsSideBySide.pdf")
  )
  cat(ifelse(file.exists(f), "✔ OK  ", "✗ NO  "), f, "\n", sep = "")
}

cat("\nMultipanel directory: ", mp_fig_dir, "\n", sep = "")
cat("\n✔ Script 04 finished.\n")
```

Session information

```{r session_info}
sessionInfo()
```

05 — Differential abundance (ANCOM-BC2, genus level, farm-adjusted) | [domain]

Script: 05_DA_ANCOMBC2.Rmd · source listing (5007 lines) · code shown as written; chunks not executed

R chunks

line 52: ```{r setup, message=FALSE, warning=FALSE}
line 107: ```{r helpers, message=FALSE, warning=FALSE}
line 2090: ```{r qc_library_sizes_by_crop, message=TRUE, warning=FALSE}
line 2284: ```{r tax_level_genus_check, message=TRUE, warning=TRUE}
line 2341: ```{r qc_design_land_crop_condition, message=TRUE, warning=FALSE}
line 2492: ```{r da_ancombc2_genus, message=TRUE, warning=TRUE}
line 3834: ```{r audit_lfc_direction, message=TRUE, warning=TRUE}
line 5004: ```{r session_info}

```
---
title: "05 — Differential abundance (ANCOM-BC2, genus level, farm-adjusted) | `r params$domain`"
author: "Sergio Pardo-Díaz"
date: "`r format(Sys.Date())`"

output:
  html_document:
    toc: true
    toc_depth: 3
    number_sections: true

params:
  # === Domain: "Bacterias" (16S rRNA) or "Hongos" (ITS2 no-trunc) ===
  domain: "Hongos"
  # Parent folder; data are read from <project_root>/<domain_dir>
  project_root: "."
  # NULL = use the default from domain_settings.R
  domain_dir: null
  ancombc_prv_cut: null   # bacteria 0.01 | fungi 0.10

  filtering_subdir: "01_Filtering_ASVs"
  filtering_rds_subdir: "RDS"

  analysis_subdir: "05_DA"
  multipanel_root: "Multipanels"

  state_var: "Plant_Condition"
  crop_var: "Crop"
  land_var: "Land"

  target_crops: ["Dominico_Harton", "Banano_Gros_Michael"]

  tax_level: "Genus"
  top_n_plot: 10

  ps_counts_file: "ps_counts.rds"

  fdr_cut: 0.05
  p_adj_method: "BH"

  lib_cut: 0
  struc_zero: true
  neg_lb: true
  settings_file: "domain_settings.R"
---

> **Differential-abundance workflow (bacteria and fungi).**
> Input: `01_Filtering_ASVs/RDS/ps_counts.rds`, i.e. **unrarefied post-filtering counts**. ANCOM-BC2 is run at **Genus** level separately within each crop, using `Land + Plant_Condition` so that farm-level heterogeneity is
```

incorporated as a fixed-effect covariate. Significance is defined as BH-adjusted `q < 0.05`. The prevalence filter `prv_cut` is set per domain in `domain_settings.R` (bacteria 0.01; fungi 0.10).

1. Setup

```{r setup, message=FALSE, warning=FALSE}

suppressPackageStartupMessages({
  library(phyloseq)
  library(tidyverse)
  library(openxlsx)
  library(patchwork)
  library(glue)
  library(scales)
  library(stringi)
  library(ANCOMBC)
  library(grDevices)
})

theme_set(
  theme_bw(base_size = 12)
)

# ---- Domain settings (single source of truth: domain_settings.R)
settings_file <- params$settings_file
if (!file.exists(settings_file)) {
  settings_file <- file.path(dirname(knitr::current_input(dir = TRUE)), settings_file)
}
source(settings_file)

cfg <- domain_settings(params$domain, params)

# Domain folder (<project_root>/<domain_dir>)
project_root <- normalizePath(
  domain_path(params$project_root, cfg),
  winslash = "/",
  mustWork = FALSE
)

message("📌 Domain: ", cfg$domain, " — ", cfg$label_long,
        " | prv_cut = ", cfg$ancombc_prv_cut)

message("📌 project_root = ", project_root)
message("📌 getwd()      = ", getwd())
```

---

2. Helpers

Main features implemented in the helper functions:

- reading of the unrarefied `ps_counts.rds`;
- agglomeration at `Genus` level only;
- Top 10 genera by lowest `q-value`;
- significance labels `***`, `**`, `*`, `NS`;
- structural zeros;
- model `Land + Plant_Condition`;
- robust extraction of ANCOM-BC2 results.

```{r helpers, message=FALSE, warning=FALSE}

# ============================================================
# Helper: observed prevalence per taxon
# ============================================================

calculate_taxon_prevalence <- function(
    ps,
```

```
    state_var
) {

  # ------------------------------------------------------------
  # Extract count matrix
  # ------------------------------------------------------------

  otu <- as(
    phyloseq::otu_table(ps),
    "matrix"
  )

  # Ensure layout:
  # rows = taxa
  # columns = samples
  if (
    !phyloseq::taxa_are_rows(ps)
  ) {

    otu <- t(otu)
  }


  # ------------------------------------------------------------
  # Metadata
  # ------------------------------------------------------------

  md <- data.frame(
    phyloseq::sample_data(ps),
    stringsAsFactors = FALSE,
    check.names = FALSE
  )


  # Keep the same sample order in metadata and matrix
  md <- md[
    colnames(otu),
    ,
    drop = FALSE
  ]


  state <- as.character(
    md[[state_var]]
  )


  idx_asymptomatic <- state ==
    "Healthy_Plant"

  idx_symptomatic <- state ==
    "Diseased_Plant"


  # ------------------------------------------------------------
  # Number of samples
  # ------------------------------------------------------------

  n_overall <- ncol(
    otu
  )

  n_asymptomatic <- sum(
    idx_asymptomatic,
    na.rm = TRUE
  )

  n_symptomatic <- sum(
```

```
    idx_symptomatic,
    na.rm = TRUE
  )


  # ------------------------------------------------------------
  # Number of samples in which each taxon is present
  # Presencia = count > 0
  # ------------------------------------------------------------

  present_overall <- rowSums(
    otu > 0,
    na.rm = TRUE
  )


  present_asymptomatic <- rowSums(
    otu[
      ,
      idx_asymptomatic,
      drop = FALSE
    ] > 0,
    na.rm = TRUE
  )


  present_symptomatic <- rowSums(
    otu[
      ,
      idx_symptomatic,
      drop = FALSE
    ] > 0,
    na.rm = TRUE
  )


  # ------------------------------------------------------------
  # Build table
  # ------------------------------------------------------------

  tibble::tibble(

    TaxID = rownames(
      otu
    ),

    N_samples_overall =
      n_overall,

    N_present_overall =
      as.integer(
        present_overall
      ),

    Prevalence_overall =
      present_overall /
      n_overall,

    N_samples_asymptomatic =
      n_asymptomatic,

    N_present_asymptomatic =
      as.integer(
        present_asymptomatic
      ),

    Prevalence_asymptomatic =
      if (
```

```
        n_asymptomatic > 0
      ) {

        present_asymptomatic /
          n_asymptomatic

      } else {

        NA_real_
      },

    N_samples_symptomatic =
      n_symptomatic,

    N_present_symptomatic =
      as.integer(
        present_symptomatic
      ),

    Prevalence_symptomatic =
      if (
        n_symptomatic > 0
      ) {

        present_symptomatic /
          n_symptomatic

      } else {

        NA_real_
      }
  )
}

# ================================================================
# 1. General utilities
# ================================================================

`%||%` <- function(x, y) {
  if (
    !is.null(x) &&
    length(x) == 1 &&
    !is.na(x) &&
    nzchar(x)
  ) {
    return(x)
  }

  y
}


force_utf8 <- function(x) {

  x <- as.character(x)

  if (!length(x)) {
    return(x)
  }

  out <- tryCatch(
    stringi::stri_enc_toutf8(
      x,
      is_unknown_8bit = TRUE
    ),
    error = function(e) x
  )

```

```
  out2 <- suppressWarnings(
    iconv(
      out,
      from = "",
      to = "UTF-8",
      sub = ""
    )
  )

  bad <- is.na(out2) & !is.na(out)

  if (any(bad)) {
    out2[bad] <- suppressWarnings(
      iconv(
        out[bad],
        from = "latin1",
        to = "UTF-8",
        sub = ""
      )
    )
  }

  bad2 <- is.na(out2) & !is.na(out)

  if (any(bad2)) {
    out2[bad2] <- ""
  }

  out2
}


clean_printable <- function(x) {

  x <- force_utf8(x)

  stringi::stri_replace_all_regex(
    x,
    "[^[:print:]]",
    ""
  )
}


# ================================================================
# 2. Colour palette
# ================================================================

col_state <- c(
  "Diseased_Plant" = "#F8766D",
  "Healthy_Plant"  = "#00BFC4"
)


# ================================================================
# 3. Directories
# ================================================================

ensure_dir <- function(path) {

  stopifnot(
    is.character(path),
    length(path) == 1,
    !is.na(path),
    nzchar(path)
  )

  dir.create(
```

```
    path,
    recursive = TRUE,
    showWarnings = FALSE
  )

  path
}


make_domain_dirs <- function(domain) {

  # Outputs are written inside the domain folder (<project_root>/<domain_dir>).
  base_dir <- project_root

  analysis_dir <- file.path(
    base_dir,
    params$analysis_subdir
  )

  ensure_dir(
    file.path(
      analysis_dir,
      "Figures"
    )
  )

  ensure_dir(
    file.path(
      analysis_dir,
      "Tables"
    )
  )

  list(
    base_dir = base_dir,
    analysis_dir = analysis_dir
  )
}


make_multipanel_dirs <- function() {

  mp_dir <- file.path(
    project_root,
    params$multipanel_root,
    params$analysis_subdir
  )

  ensure_dir(
    file.path(
      mp_dir,
      "Figures"
    )
  )

  ensure_dir(
    file.path(
      mp_dir,
      "Tables"
    )
  )

  mp_dir
}


# ============================================================
# 4. Saving figures
```

```r
# ================================================================

save_plot_pdf <- function(
    filename,
    plot,
    width,
    height) {

  ggsave(
    filename = filename,
    plot = plot,
    width = width,
    height = height,
    device = cairo_pdf
  )
}


save_plot_png <- function(
    filename,
    plot,
    width,
    height,
    dpi = 300) {

  ggsave(
    filename = filename,
    plot = plot,
    width = width,
    height = height,
    dpi = dpi
  )
}


# ================================================================
# 5. Locate and load ps_counts.rds
# ================================================================

find_ps_file <- function(
    domain,
    filename) {

  dom_dir <- project_root

  expected <- file.path(
    project_root,
    params$filtering_subdir,
    params$filtering_rds_subdir,
    filename
  )

  if (file.exists(expected)) {
    return(expected)
  }

  hits <- list.files(
    dom_dir,
    pattern = paste0(
      "^",
      filename,
      "$"
    ),
    recursive = TRUE,
    full.names = TRUE
  )

  if (length(hits) == 1) {
```

```
    return(hits)
  }

  if (length(hits) > 1) {
    stop(
      "❌ Multiple files named ",
      filename,
      " were found in ",
      dom_dir,
      ". Please check the path."
    )
  }

  stop(
    glue(
      "❌ {filename} was not found in {dom_dir}."
    )
  )
}


load_ps_counts <- function(domain) {

  if (domain != cfg$domain) {
    stop("Unsupported domain for this render: ", domain)
  }

  filename <- params$ps_counts_file

  path <- find_ps_file(
    domain,
    filename
  )

  message(
    "📦 Loading phyloseq counts: ",
    path
  )

  ps <- readRDS(path)

  stopifnot(
    inherits(
      ps,
      "phyloseq"
    )
  )

  # Expected number of non-zero samples after filtering, before
  # rarefaction (domain_settings.R). Do not stop if it differs,
  # but make any discrepancy explicit.
  if (isTRUE(!is.na(cfg$counts_expected_n) && nsamples(ps) != cfg$counts_expected_n)) {
    warning(
      "⚠ ", cfg$counts_expected_n, " samples were expected in ps_counts.rds after ",
      "filtering; ", nsamples(ps), " were observed."
    )
  }

  message(
    "✔ Unrarefied counts: ",
    nsamples(ps), " samples | ",
    ntaxa(ps), " ASVs | ",
    format(sum(sample_sums(ps)), big.mark = ","), " reads"
  )

  ps
}
```

```

# ============================================================
# 6. Sanitize sample_data
# ============================================================

sanitize_sample_data <- function(ps) {

  meta <- data.frame(
    sample_data(ps),
    stringsAsFactors = FALSE,
    check.names = FALSE
  )

  colnames(meta) <- clean_printable(
    colnames(meta)
  )

  rownames(meta) <- clean_printable(
    rownames(meta)
  )

  for (nm in names(meta)) {

    value <- meta[[nm]]

    if (is.factor(value)) {
      value <- as.character(value)
    }

    if (is.character(value)) {
      value <- clean_printable(value)
    }

    meta[[nm]] <- value
  }

  sample_data(ps) <- sample_data(meta)

  ps
}


# ============================================================
# 7. Count QC
# ============================================================

qc_counts <- function(
    ps,
    label = "") {

  otu <- as(
    otu_table(ps),
    "matrix"
  )

  if (!taxa_are_rows(ps)) {
    otu <- t(otu)
  }

  is_int <- all(
    abs(
      otu - round(otu)
    ) < 1e-8
  )

  ss <- colSums(otu)

  message(
```

```
    "---- QC ",
    label,
    " ----"
  )

  message(
    "taxa x samples: ",
    nrow(otu),
    " x ",
    ncol(otu)
  )

  message(
    "counts integer: ",
    is_int
  )

  message(
    "libsize min/median/max: ",
    min(ss),
    " / ",
    median(ss),
    " / ",
    max(ss)
  )

  if (!is_int) {
    warning(
      "⚠️ The object does not contain integer counts."
    )
  }

  invisible(is_int)
}


# ================================================================
# 8. Taxonomy
# ================================================================

rank_prefix_map <- c(
  Domain  = "d__",
  Kingdom = "k__",
  Phylum  = "p__",
  Class   = "c__",
  Order   = "o__",
  Family  = "f__",
  Genus   = "g__",
  Species = "s__"
)


get_tax_table_df <- function(
    ps,
    domain = NA_character_) {

  tt <- tax_table(
    ps,
    errorIfNULL = FALSE
  )

  if (is.null(tt)) {
    stop(
      "tax_table() is NULL",
      if (!is.na(domain)) {
        paste0(" in ", domain)
      } else {
        ""
```

```
      }
    )
  }

  tx <- as.data.frame(
    tt,
    stringsAsFactors = FALSE,
    check.names = FALSE
  )

  tx[] <- lapply(
    tx,
    function(col) {

      if (is.factor(col)) {
        col <- as.character(col)
      }

      if (is.character(col)) {
        col <- clean_printable(col)
      }

      col
    }
  )

  tx
}


extract_rank_token <- function(
    taxon_str,
    prefix) {

  if (
    is.na(taxon_str) ||
    taxon_str == ""
  ) {
    return(NA_character_)
  }

  tokens <- unlist(
    strsplit(
      taxon_str,
      ";\\s*"
    )
  )

  hit <- tokens[
    grepl(
      paste0(
        "^",
        prefix
      ),
      tokens
    )
  ]

  if (!length(hit)) {
    return(NA_character_)
  }

  value <- sub(
    paste0(
      "^",
      prefix
    ),
    "",
```

```
    hit[1]
  )

  value <- trimws(value)

  if (
    value == "" ||
    value == "NA"
  ) {
    return(NA_character_)
  }

  value
}


build_rank_table <- function(
    ps,
    domain,
    ranks) {

  tx <- get_tax_table_df(
    ps,
    domain = domain
  )

  out <- tibble(
    OTU = rownames(tx)
  )

  direct_rank_hits <- intersect(
    ranks,
    colnames(tx)
  )

  for (rk in direct_rank_hits) {

    value <- as.character(
      tx[[rk]]
    )

    value <- clean_printable(
      value
    )

    value[
      is.na(value) |
      trimws(value) == "" |
      value == "NA"
    ] <- "Unassigned"

    out[[rk]] <- value
  }

  ranks_missing <- setdiff(
    ranks,
    direct_rank_hits
  )

  if (length(ranks_missing) > 0) {

    if (!"Taxon" %in% colnames(tx)) {

      for (rk in ranks_missing) {
        out[[rk]] <- "Unassigned"
      }

      return(out)
```

```
    }

    tax_str <- clean_printable(
      tx$Taxon
    )

    for (rk in ranks_missing) {

      prefix <- rank_prefix_map[[rk]]

      out[[rk]] <- vapply(
        tax_str,
        function(s) {

          value <- extract_rank_token(
            s,
            prefix
          )

          if (is.na(value)) {
            "Unassigned"
          } else {
            value
          }
        },
        character(1)
      )
    }
  }

  out
}


can_derive_rank <- function(
    ps,
    rk) {

  tx <- tryCatch(
    get_tax_table_df(ps),
    error = function(e) NULL
  )

  if (is.null(tx)) {
    return(FALSE)
  }

  if (rk %in% colnames(tx)) {

    value <- clean_printable(
      as.character(
        tx[[rk]]
      )
    )

    return(
      any(
        !is.na(value) &
        trimws(value) != "" &
        value != "NA"
      )
    )
  }

  if (!"Taxon" %in% colnames(tx)) {
    return(FALSE)
  }

```

```
  prefix <- rank_prefix_map[[rk]]

  tax_str <- clean_printable(
    tx$Taxon
  )

  any(
    grepl(
      paste0(
        "(^|;\\s*)",
        prefix
      ),
      tax_str
    )
  )
}


# ================================================================
# 9. Agglomeration: GENUS ONLY
# ================================================================

ps_glom_genus <- function(
    ps,
    domain) {

  ranks_need <- c(
    "Phylum",
    "Class",
    "Order",
    "Family",
    "Genus"
  )

  rank_tbl <- build_rank_table(
    ps,
    domain = domain,
    ranks = ranks_need
  )

  tx_new <- rank_tbl |>
    select(-OTU) |>
    as.data.frame(
      stringsAsFactors = FALSE,
      check.names = FALSE
    )

  tx_mat <- as.matrix(
    tx_new
  )

  rownames(tx_mat) <- rank_tbl$OTU

  tax_table(ps) <- tax_table(
    tx_mat
  )

  tax_glom(
    ps,
    taxrank = "Genus",
    NArm = FALSE
  )
}


# ================================================================
# 10. TAXONOMIC FILTERING AT GENUS LEVEL
# ================================================================
```

```

# ------------------------------------------------------------------
# Normalize taxonomic labels for comparisons across ranks
# ------------------------------------------------------------------

normalize_tax_label <- function(x) {

  x <- trimws(
    force_utf8(
      as.character(x)
    )
  )

  x[
    is.na(x) |
      x == "" |
      x == "NA"
  ] <- NA_character_

  x <- tolower(x)

  # Remove taxonomic prefixes, if present
  x <- sub(
    "^[dkpcofgs]__",
    "",
    x
  )

  # Remove the Candidatus prefix ONLY to compare across ranks.
  # The original name is NOT modified in the final table.
  x <- sub(
    "^candidatus[ _.-]+",
    "",
    x
  )

  trimws(x)
}


# ------------------------------------------------------------------
# Detect names clearly incompatible with a resolved genus
# ------------------------------------------------------------------

is_weird_taxon <- function(x) {

  x2 <- trimws(
    force_utf8(
      as.character(x)
    )
  )

  x_low <- tolower(x2)


  # 1. Empty / NA
  is_empty <-
    is.na(x2) |
    x2 == "" |
    x_low == "na"


  # 2. Provisional names / codes containing numbers
  #
  # Examples:
  # AD3
  # A0839
```

```
  # 0319-6G20
  # KD4-96
  has_number <- grepl(
    "[0-9]",
    x2
  )


  # 3. Clades/subgroups not resolved to genus
  is_subgroup <- grepl(
    "subgroup|subclade|clade",
    x_low
  )


  # 4. Unresolved categories
  unresolved <- grepl(
    paste0(
      "unknown|",
      "unidentified|",
      "unclassified|",
      "uncultured|",
      "unassigned|",
      "norank|",
      "no_rank|",
      "metagenome|",
      "ambiguous"
    ),
    x_low
  )


  # 5. Incertae sedis
  incertae <- grepl(
    "incertae[ _.-]*sedis",
    x_low
  )


  # 6. Unambiguous suffixes of higher ranks
  #
  # -aceae = Family
  # -ales  = Order
  higher_rank_suffix <- grepl(
    "(aceae|ales)$",
    x_low
  )


  is_empty |
    has_number |
    is_subgroup |
    unresolved |
    incertae |
    higher_rank_suffix
}


# ------------------------------------------------------------------
# Detect whether the putative Genus is actually a higher rank
# ------------------------------------------------------------------

is_partial_genus_assignment <- function(
    genus,
    family = NA_character_,
    order = NA_character_,
    class = NA_character_,
    phylum = NA_character_
```

```
) {

  g <- normalize_tax_label(genus)
  f <- normalize_tax_label(family)
  o <- normalize_tax_label(order)
  c <- normalize_tax_label(class)
  p <- normalize_tax_label(phylum)


  genus_missing <- is.na(g)


  # The putative genus literally reproduces a higher rank
  same_as_family <-
    !is.na(g) &
    !is.na(f) &
    g == f

  same_as_order <-
    !is.na(g) &
    !is.na(o) &
    g == o

  same_as_class <-
    !is.na(g) &
    !is.na(c) &
    g == c

  same_as_phylum <-
    !is.na(g) &
    !is.na(p) &
    g == p


  # Suffixes that reveal a family or order
  looks_family <-
    !is.na(g) &
    grepl(
      "aceae$",
      g
    )

  looks_order <-
    !is.na(g) &
    grepl(
      "ales$",
      g
    )


  genus_missing |
    same_as_family |
    same_as_order |
    same_as_class |
    same_as_phylum |
    looks_family |
    looks_order
}


# ------------------------------------------------------------------
# Build the complete taxonomic QC
# ------------------------------------------------------------------

build_taxonomy_qc <- function(
    tax_id,
    genus,
    family,
```

```
    order,
    class,
    phylum
) {

  genus_clean <- trimws(
    force_utf8(
      as.character(genus)
    )
  )

  family_clean <- trimws(
    force_utf8(
      as.character(family)
    )
  )

  order_clean <- trimws(
    force_utf8(
      as.character(order)
    )
  )

  class_clean <- trimws(
    force_utf8(
      as.character(class)
    )
  )

  phylum_clean <- trimws(
    force_utf8(
      as.character(phylum)
    )
  )


  weird <- is_weird_taxon(
    genus_clean
  )


  partial <- is_partial_genus_assignment(
    genus = genus_clean,
    family = family_clean,
    order = order_clean,
    class = class_clean,
    phylum = phylum_clean
  )


  unassigned <-
    is.na(genus_clean) |
    genus_clean == "" |
    genus_clean == "NA" |
    tolower(genus_clean) == "unassigned"


  # ------------------------------------------------------------
  # Specific reason for exclusion
  # ------------------------------------------------------------

  g_norm <- normalize_tax_label(
    genus_clean
  )

  f_norm <- normalize_tax_label(
    family_clean
  )
```

```
  o_norm <- normalize_tax_label(
    order_clean
  )

  c_norm <- normalize_tax_label(
    class_clean
  )

  p_norm <- normalize_tax_label(
    phylum_clean
  )


  exclusion_reason <- case_when(

    unassigned ~
      "Unassigned genus",

    grepl(
      "[0-9]",
      genus_clean
    ) ~
      "Provisional/alphanumeric taxon",

    grepl(
      "subgroup|subclade|clade",
      tolower(genus_clean)
    ) ~
      "Subgroup/clade assignment",

    grepl(
      paste0(
        "unknown|",
        "unidentified|",
        "unclassified|",
        "uncultured|",
        "norank|",
        "no_rank|",
        "metagenome|",
        "ambiguous"
      ),
      tolower(genus_clean)
    ) ~
      "Unresolved taxonomic assignment",

    grepl(
      "incertae[ _.-]*sedis",
      tolower(genus_clean)
    ) ~
      "Incertae sedis",

    grepl(
      "aceae$",
      tolower(genus_clean)
    ) ~
      "Family-level label in Genus field",

    grepl(
      "ales$",
      tolower(genus_clean)
    ) ~
      "Order-level label in Genus field",

    !is.na(g_norm) &
      !is.na(f_norm) &
      g_norm == f_norm ~
      "Genus reproduces Family",
```

```
    !is.na(g_norm) &
      !is.na(o_norm) &
      g_norm == o_norm ~
      "Genus reproduces Order",

    !is.na(g_norm) &
      !is.na(c_norm) &
      g_norm == c_norm ~
      "Genus reproduces Class",

    !is.na(g_norm) &
      !is.na(p_norm) &
      g_norm == p_norm ~
      "Genus reproduces Phylum",

    TRUE ~
      "Retained"
  )


  tibble(
    TaxID = tax_id,
    Phylum = phylum_clean,
    Class = class_clean,
    Order = order_clean,
    Family = family_clean,
    Genus = genus_clean,
    Weird_assignment = weird,
    Partial_assignment = partial,
    Exclusion_reason = exclusion_reason,
    Exclude =
      unassigned |
      weird |
      partial
  )
}
# ================================================================
# 11. Significance
# ================================================================

sig_stars <- function(q) {

  if (
    is.na(q) ||
    !is.finite(q)
  ) {
    return("NS")
  }

  if (q < 0.001) {
    return("***")
  }

  if (q < 0.01) {
    return("**")
  }

  if (q < 0.05) {
    return("*")
  }

  "NS"
}


dedup_by_taxon <- function(tbl) {

```

```
  if (
    is.null(tbl) ||
    nrow(tbl) == 0
  ) {
    return(tbl)
  }

  tbl |>
    mutate(
      Taxon = trimws(
        force_utf8(Taxon)
      ),
      Taxon = ifelse(
        is.na(Taxon) |
        Taxon == "" |
        Taxon == "NA",
        "Unassigned",
        Taxon
      ),
      abs_lfc = abs(lfc)
    ) |>
    arrange(
      Taxon,
      ifelse(
        is.finite(q_val),
        0,
        1
      ),
      q_val,
      desc(abs_lfc)
    ) |>
    group_by(Taxon) |>
    slice(1) |>
    ungroup() |>
    select(-abs_lfc)
}


# ================================================================
# 12. Top 10 by q-value
# ================================================================

select_top_q <- function(
    tbl,
    top_n = params$top_n_plot) {

  if (
    is.null(tbl) ||
    nrow(tbl) == 0
  ) {
    return(tbl)
  }

  tbl |>
    dedup_by_taxon() |>
    filter(
      is.finite(lfc),
      is.finite(q_val)
    ) |>
    arrange(
      q_val,
      p_val,
      desc(abs(lfc))
    ) |>
    slice_head(
      n = top_n
    ) |>
    mutate(
```

```
      Rank_q = row_number(),
      sig_label = vapply(
        q_val,
        sig_stars,
        character(1)
      )
    )
}


# ================================================================
# 13. Top 10 plot
# ================================================================

plot_lfc_top10 <- function(
    tbl,
    title,
    subtitle,
    top_n = params$top_n_plot) {

  df <- select_top_q(
    tbl,
    top_n = top_n
  )

  if (
    is.null(df) ||
    nrow(df) == 0
  ) {

    return(
      ggplot() +
        theme_void() +
        labs(
          title = title,
          subtitle = paste0(
            subtitle,
            " | No taxa available"
          )
        )
    )
  }

  df <- df |>
    arrange(lfc) |>
    mutate(
      Taxon = factor(
        Taxon,
        levels = Taxon
      ),
      enriched_in = factor(
        enriched_in,
        levels = c(
          "Diseased_Plant",
          "Healthy_Plant"
        )
      )
    )

  x_max <- max(
    abs(
      c(
        df$lfc,
        df$lfc - df$se,
        df$lfc + df$se
      )
    ),
    na.rm = TRUE
```

```
)

if (
  !is.finite(x_max) ||
  x_max == 0
) {
  x_max <- 1
}

offset <- 0.05 * x_max

df <- df |>
  mutate(
    label_x = ifelse(
      lfc >= 0,
      lfc + offset,
      lfc - offset
    ),
    label_hjust = ifelse(
      lfc >= 0,
      0,
      1
    )
  )

ggplot(
  df,
  aes(
    x = lfc,
    y = Taxon,
    fill = enriched_in
  )
) +

  geom_col(
    width = 0.72,
    color = "grey20",
    linewidth = 0.2
  ) +

  geom_errorbar(
    aes(
      xmin = lfc - se,
      xmax = lfc + se
    ),
    width = 0.25,
    linewidth = 0.3,
    orientation = "y",
    na.rm = TRUE
  ) +

  geom_text(
    aes(
      x = label_x,
      label = sig_label,
      hjust = label_hjust
    ),
    fontface = "bold",
    size = 3.8,
    show.legend = FALSE
  ) +

  geom_vline(
    xintercept = 0,
    linewidth = 0.4
  ) +

  scale_fill_manual(
```

```
      values = col_state,
      drop = FALSE
    ) +

    scale_x_continuous(
      expand = expansion(
        mult = c(
          0.15,
          0.15
        )
      )
    ) +

    labs(
      title = title,
      subtitle = subtitle,
      x = "Log Fold Change (Diseased vs Healthy)",
      y = NULL,
      fill = "Enriched in",
      caption = paste0(
        "Top ",
        top_n,
        " taxa ranked by BH-adjusted q-value. ",
        "*** q < 0.001; ** q < 0.01; ",
        "* q < 0.05; NS q >= 0.05. ",
        "Only q < 0.05 is considered statistically significant."
      )
    ) +

    theme(
      legend.position = "right",
      panel.grid.major.y = element_blank(),
      plot.caption = element_text(
        size = 9,
        hjust = 0
      )
    ) +

    coord_cartesian(
      clip = "off"
    )
}


# ================================================================
# 14. Structural zeros
# ================================================================

extract_structural_zeros <- function(
    fit,
    ps) {

  if (
    is.null(fit$zero_ind)
  ) {
    return(tibble())
  }

  zero_df <- as.data.frame(
    fit$zero_ind,
    stringsAsFactors = FALSE,
    check.names = FALSE
  )

  if (nrow(zero_df) == 0) {
    return(tibble())
  }

```

```
  if (
    "taxon" %in% colnames(zero_df)
  ) {

    zero_df$TaxID <- as.character(
      zero_df$taxon
    )

  } else {

    zero_df$TaxID <- rownames(
      zero_df
    )
  }

  tt <- as.data.frame(
    tax_table(ps),
    stringsAsFactors = FALSE,
    check.names = FALSE
  )

  tax_map <- tibble(
    TaxID = rownames(tt),
    Taxon = trimws(
      force_utf8(
        as.character(
          tt$Genus
        )
      )
    )
  )

  zero_df |>
    as_tibble() |>
    left_join(
      tax_map,
      by = "TaxID"
    )
}


# ================================================================
# 15. ANCOM-BC2
# ================================================================

run_ancombc2 <- function(
    ps,
    group_var,
    block_var = params$land_var) {

  md <- data.frame(
    sample_data(ps),
    stringsAsFactors = FALSE,
    check.names = FALSE
  )

  required <- c(
    group_var,
    block_var
  )

  missing <- setdiff(
    required,
    names(md)
  )

  if (length(missing) > 0) {
    stop(
```

```
      "Variables missing from metadata: ",
      paste(
        missing,
        collapse = ", "
      )
    )
  }

  keep <- rownames(md)[
    !is.na(md[[group_var]]) &
    !is.na(md[[block_var]]) &
    as.character(
      md[[group_var]]
    ) %in%
      c(
        "Healthy_Plant",
        "Diseased_Plant"
      )
  ]

  ps <- prune_samples(
    keep,
    ps
  )

  ps <- prune_taxa(
    taxa_sums(ps) > 0,
    ps
  )

  md <- data.frame(
    sample_data(ps),
    stringsAsFactors = FALSE,
    check.names = FALSE
  )

  md[[group_var]] <- factor(
    as.character(
      md[[group_var]]
    ),
    levels = c(
      "Healthy_Plant",
      "Diseased_Plant"
    )
  )

  md[[block_var]] <- droplevels(
    factor(
      as.character(
        md[[block_var]]
      )
    )
  )

  if (
    nlevels(
      md[[group_var]]
    ) < 2
  ) {
    stop(
      "Plant_Condition does not contain both conditions."
    )
  }

  if (
    nlevels(
      md[[block_var]]
    ) < 2
```

```
) {
  stop(
    "At least two Land levels (farms) are required."
  )
}

design_tab <- table(
  md[[block_var]],
  md[[group_var]]
)

message(
  "Land × Plant_Condition:"
)

print(
  design_tab
)

if (
  any(
    rowSums(
      design_tab > 0
    ) < 2
  )
) {

  warning(
    "One or more farms do not contain both conditions."
  )
}

sample_data(ps) <- sample_data(
  md
)

fix_formula_use <- paste(
  block_var,
  group_var,
  sep = " + "
)

message(
  "ANCOM-BC2 fix_formula = ",
  fix_formula_use
)

args <- list(
  data = ps,
  fix_formula = fix_formula_use,
  rand_formula = NULL,
  group = group_var,
  p_adj_method = params$p_adj_method,
  prv_cut = cfg$ancombc_prv_cut,
  lib_cut = params$lib_cut,
  struc_zero = params$struc_zero,
  neg_lb = params$neg_lb,
  global = FALSE,
  pairwise = FALSE,
  dunnet = FALSE,
  trend = FALSE,
  verbose = FALSE
)

allowed <- names(
  formals(
    ANCOMBC::ancombc2
  )
```

```
  )

  args <- args[
    names(args) %in% allowed
  ]

  res <- NULL

  suppressWarnings(
    suppressMessages(
      capture.output(
        {
          res <- do.call(
            ANCOMBC::ancombc2,
            args
          )
        }
      )
    )
  )

  res
}


# ============================================================
# 16. Extract ANCOM-BC2 columns
# ============================================================

.pick_exact_res_col <- function(
    resdf,
    prefix,
    group_var,
    lvl) {

  cn <- names(
    resdf
  )

  candidates <- c(
    paste0(
      prefix,
      "_",
      group_var,
      lvl
    ),
    paste0(
      prefix,
      "_",
      group_var,
      "_",
      lvl
    )
  )

  hit <- candidates[
    candidates %in% cn
  ]

  if (length(hit) > 0) {
    return(
      hit[1]
    )
  }

  hit2 <- cn[
    startsWith(
      cn,
```

```
      paste0(
        prefix,
        "_"
      )
    ) &
    grepl(
      group_var,
      cn,
      fixed = TRUE
    ) &
    grepl(
      lvl,
      cn,
      fixed = TRUE
    )
  ]

  if (length(hit2) > 0) {
    return(
      hit2[1]
    )
  }

  NA_character_
}


extract_ancombc2_from_resdf <- function(
    fit,
    group_var = params$state_var,
    lvl = "Diseased_Plant") {

  resdf <- fit$res

  if (!is.data.frame(resdf)) {

    return(
      list(
        ok = FALSE,
        reason = "fit$res is not a data.frame",
        resdf = NULL
      )
    )
  }

  lfc_col <- .pick_exact_res_col(
    resdf,
    "lfc",
    group_var,
    lvl
  )

  se_col <- .pick_exact_res_col(
    resdf,
    "se",
    group_var,
    lvl
  )

  p_col <- .pick_exact_res_col(
    resdf,
    "p",
    group_var,
    lvl
  )

  q_col <- .pick_exact_res_col(
    resdf,
```

```
    "q",
    group_var,
    lvl
  )

  diff_col <- .pick_exact_res_col(
    resdf,
    "diff",
    group_var,
    lvl
  )

  passed_col <- .pick_exact_res_col(
    resdf,
    "passed_ss",
    group_var,
    lvl
  )

  robust_col <- .pick_exact_res_col(
    resdf,
    "diff_robust",
    group_var,
    lvl
  )

  if (
    any(
      is.na(
        c(
          lfc_col,
          p_col,
          q_col
        )
      )
    )
  ) {

    return(
      list(
        ok = FALSE,
        reason = paste0(
          "Required columns were not detected. ",
          "lfc=", lfc_col,
          " | p=", p_col,
          " | q=", q_col
        ),
        resdf = resdf
      )
    )
  }

  list(
    ok = TRUE,
    resdf = resdf,
    cols = list(
      lfc = lfc_col,
      se = se_col,
      p = p_col,
      q = q_col,
      diff_abn = diff_col,
      passed_ss = passed_col,
      diff_robust = robust_col
    )
  )
}
```

```
## 3. Library-size QC


```{r qc_library_sizes_by_crop, message=TRUE, warning=FALSE}

summarise_sums <- function(
    ps,
    domain) {

  md <- data.frame(
    sample_data(ps),
    stringsAsFactors = FALSE,
    check.names = FALSE
  )

  ss <- sample_sums(ps)

  df <- tibble(
    SampleID = names(ss),
    LibrarySize = as.numeric(ss)
  ) |>
    left_join(
      md |>
        rownames_to_column(
          "SampleID"
        ),
      by = "SampleID"
    ) |>
    mutate(
      Domain = domain,
      Crop = as.character(
        .data[[params$crop_var]]
      ),
      State = as.character(
        .data[[params$state_var]]
      )
    )

  sum_crop <- df |>
    filter(
      Crop %in%
        params$target_crops
    ) |>
    group_by(
      Domain,
      Crop
    ) |>
    summarise(
      n_samples = n(),
      min_lib = min(
        LibrarySize
      ),
      median_lib = median(
        LibrarySize
      ),
      max_lib = max(
        LibrarySize
      ),
      .groups = "drop"
    )

  count_state <- df |>
    filter(
      Crop %in%
        params$target_crops
    ) |>
    group_by(
      Domain,
```
```

```
      Crop,
      State
    ) |>
    summarise(
      n = n(),
      .groups = "drop"
    ) |>
    pivot_wider(
      names_from = State,
      values_from = n,
      values_fill = 0
    )

  sum_crop |>
    left_join(
      count_state,
      by = c(
        "Domain",
        "Crop"
      )
    ) |>
    arrange(
      Domain,
      Crop
    )
}


out_dir <- file.path(
  project_root,
  "QC_LibrarySizes"
)

ensure_dir(
  out_dir
)

all_summary <- list()


for (domain in cfg$domain) {

  message(
    "============================================="
  )

  message(
    "📌 QC Library sizes — ",
    domain
  )

  ps0 <- load_ps_counts(
    domain
  ) |>
    sanitize_sample_data()

  md0 <- data.frame(
    sample_data(ps0),
    stringsAsFactors = FALSE
  )

  keep <- rownames(md0)[
    as.character(
      md0[[params$crop_var]]
    ) %in%
      params$target_crops &
    as.character(
      md0[[params$state_var]]
```

```
    ) %in%
      c(
        "Healthy_Plant",
        "Diseased_Plant"
      )
  ]

  ps0 <- prune_samples(
    keep,
    ps0
  )

  ps0 <- prune_taxa(
    taxa_sums(ps0) > 0,
    ps0
  )

  all_summary[[domain]] <-
    summarise_sums(
      ps0,
      domain
    )

  print(
    all_summary[[domain]]
  )
}


summary_tbl <- bind_rows(
  all_summary
)

out_xlsx <- file.path(
  out_dir,
  "QC_LibrarySizes_byCrop.xlsx"
)

openxlsx::write.xlsx(
  list(
    Summary_byCrop =
      summary_tbl
  ),
  out_xlsx,
  overwrite = TRUE
)

message(
  "✅ QC saved to: ",
  out_xlsx
)
```

---

4. Taxonomic QC

Iterating over Order/Family/Genus is not needed.

This chunk checks that `Genus` can be derived in the count object.

```{r tax_level_genus_check, message=TRUE, warning=TRUE}

for (domain in cfg$domain) {

  cat(
    "\n==========================================\n"
  )
```

```
  cat(
    "🔎 Genus check —",
    domain,
    "\n"
  )

  ps <- load_ps_counts(
    domain
  ) |>
    sanitize_sample_data()

  qc_counts(
    ps,
    label = paste(
      domain,
      "(raw counts)"
    )
  )

  genus_ok <- can_derive_rank(
    ps,
    "Genus"
  )

  cat(
    "Genus derivable:",
    genus_ok,
    "\n"
  )

  if (!genus_ok) {
    stop(
      "❌ Genus cannot be derived for ",
      domain,
      "."
    )
  }
}

message(
  "✅ Genus available for ", cfg$label, "."
)
```

5. QC of the Land × Crop × Plant_Condition design

This chunk summarises the sampling design and checks that every farm contains both plant conditions:

```{r qc_design_land_crop_condition, message=TRUE, warning=FALSE}

for (dom in cfg$domain) {

  cat(
    "\n==========================================\n"
  )

  cat(
    "📌 ANCOM-BC2 design —",
    dom,
    "\n"
  )

  ps_qc <- load_ps_counts(
    dom
  ) |>
    sanitize_sample_data()
```

```
2359
2360  md_qc <- data.frame(
2361    sample_data(ps_qc),
2362    stringsAsFactors = FALSE,
2363    check.names = FALSE
2364  )
2365
2366  required <- c(
2367    params$crop_var,
2368    params$land_var,
2369    params$state_var
2370  )
2371
2372  missing <- setdiff(
2373    required,
2374    names(md_qc)
2375  )
2376
2377  if (length(missing) > 0) {
2378
2379    stop(
2380      "Missing variables: ",
2381      paste(
2382        missing,
2383        collapse = ", "
2384      )
2385    )
2386  }
2387
2388  md_qc <- md_qc |>
2389    mutate(
2390      Crop = as.character(
2391        .data[[params$crop_var]]
2392      ),
2393      Land = as.character(
2394        .data[[params$land_var]]
2395      ),
2396      Plant_Condition = as.character(
2397        .data[[params$state_var]]
2398      )
2399    ) |>
2400    filter(
2401      Crop %in%
2402        params$target_crops,
2403      Plant_Condition %in%
2404        c(
2405          "Healthy_Plant",
2406          "Diseased_Plant"
2407        ),
2408      !is.na(Land)
2409    )
2410
2411  cat(
2412    "Total number of farms:",
2413    n_distinct(
2414      md_qc$Land
2415    ),
2416    "\n"
2417  )
2418
2419  cat(
2420    "\nFarms per crop:\n"
2421  )
2422
2423  print(
2424    md_qc |>
2425      distinct(
2426        Crop,
```

```
        Land
      ) |>
      count(
        Crop,
        name = "n_Land"
      )
  )

  design_qc <- md_qc |>
    count(
      Crop,
      Land,
      Plant_Condition,
      name = "n"
    ) |>
    pivot_wider(
      names_from = Plant_Condition,
      values_from = n,
      values_fill = 0
    ) |>
    arrange(
      Crop,
      Land
    )

  cat(
    "\nSamples per Crop × Land × Plant_Condition:\n"
  )

  print(
    design_qc
  )

  bad_design <- design_qc |>
    filter(
      Healthy_Plant == 0 |
      Diseased_Plant == 0
    )

  if (nrow(bad_design) > 0) {

    warning(
      "⚠️ Some farms do not contain both conditions."
    )

    print(
      bad_design
    )

  } else {

    message(
      "✅ All farms contain both conditions."
    )
  }
}

```

```
# 6. Main ANCOM-BC2 analysis — Genus only


```{r da_ancombc2_genus, message=TRUE, warning=TRUE}

mp_dir <- make_multipanel_dirs()
```
```

```

tax_level <- "Genus"

# ============================================================
# Store of objects for the LFC-direction audit
# ============================================================

direction_audit_store <- list()

for (dom in cfg$domain) {

  cat(
    "\n==============================================\n"
  )

  cat(
    "📌 ANCOM-BC2 —",
    dom,
    "— Genus\n"
  )


  dirs <- make_domain_dirs(
    dom
  )

  analysis_dir <- dirs$analysis_dir


  # ============================================================
  # 1. Load counts
  # ============================================================

  ps0 <- load_ps_counts(
    dom
  ) |>
    sanitize_sample_data()


  qc_counts(
    ps0,
    label = paste(
      dom,
      "pre-aggregation"
    )
  )


  # ============================================================
  # 2. Agglomerate to Genus
  # ============================================================

  ps_genus <- ps_glom_genus(
    ps0,
    domain = dom
  )


  ps_genus <- prune_taxa(
    taxa_sums(ps_genus) > 0,
    ps_genus
  )


  cat(
    "Number of genera after tax_glom:",
    ntaxa(ps_genus),
    "\n"
```

```
)


plots_crop <- list()


# ============================================================
# 3. Loop over Crop
# ============================================================

for (crop in params$target_crops) {

  cat(
    "\n-----------------------------------------------\n"
  )

  cat(
    "Crop:",
    crop,
    "\n"
  )


  md <- data.frame(
    sample_data(ps_genus),
    stringsAsFactors = FALSE,
    check.names = FALSE
  )


  # ============================================================
  # 4. Filter samples
  # ============================================================

  keep_samples <- rownames(md)[

    as.character(
      md[[params$crop_var]]
    ) == crop &

    as.character(
      md[[params$state_var]]
    ) %in%
      c(
        "Healthy_Plant",
        "Diseased_Plant"
      ) &

    !is.na(
      md[[params$land_var]]
    )
  ]


  if (
    length(keep_samples) < 4
  ) {

    warning(
      "Too few samples for ",
      dom,
      " / ",
      crop
    )

    next
  }
```

```
2631
2632      ps_crop <- prune_samples(
2633        keep_samples,
2634        ps_genus
2635      )
2636
2637
2638      ps_crop <- prune_taxa(
2639        taxa_sums(ps_crop) > 0,
2640        ps_crop
2641      )
2642
2643
2644      cat(
2645        "Samples:",
2646        nsamples(ps_crop),
2647        " | Genera:",
2648        ntaxa(ps_crop),
2649        "\n"
2650      )
2651
2652
2653      # ============================================================
2654      # 5. ANCOM-BC2
2655      #
2656      # Model:
2657      # Land + Plant_Condition
2658      # ============================================================
2659
2660      fit <- tryCatch(
2661
2662        run_ancombc2(
2663          ps_crop,
2664          group_var = params$state_var,
2665          block_var = params$land_var
2666        ),
2667
2668        error = function(e) e
2669      )
2670
2671
2672      if (
2673        inherits(
2674          fit,
2675          "error"
2676        )
2677      ) {
2678
2679        warning(
2680          "ANCOM-BC2 failed: ",
2681          dom,
2682          " / ",
2683          crop,
2684          " | ",
2685          conditionMessage(fit)
2686        )
2687
2688        next
2689      }
2690
2691
2692      # ============================================================
2693      # 6. Extract the Diseased vs Healthy coefficient
2694      # ============================================================
2695
2696      ex <- extract_ancombc2_from_resdf(
2697        fit,
2698        group_var = params$state_var,
```

```
      lvl = "Diseased_Plant"
    )

# ============================================================
# DIRECT CHECK OF THE ANCOM-BC2 COEFFICIENT
# ============================================================

cat(
  "\n=====================================================\n",
  "DIRECT COEFFICIENT CHECK\n",
  "Domain: ", dom, "\n",
  "Crop:   ", crop, "\n",
  "=====================================================\n",
  sep = ""
)

# 1. Show the levels actually used in the model
md_check <- data.frame(
  sample_data(ps_crop),
  stringsAsFactors = FALSE,
  check.names = FALSE
)

md_check[[params$state_var]] <- factor(
  as.character(md_check[[params$state_var]]),
  levels = c(
    "Healthy_Plant",
    "Diseased_Plant"
  )
)

cat("\nPlant_Condition levels:\n")
print(levels(md_check[[params$state_var]]))

cat("\nReference level:\n")
print(levels(md_check[[params$state_var]])[1])

cat("\nDesign-matrix coding:\n")
print(
  contrasts(
    md_check[[params$state_var]]
  )
)


# 2. Show ALL columns produced by ANCOM-BC2
cat("\nColumns in fit$res:\n")

print(
  names(fit$res)
)


# 3. Show the LFC columns specifically
lfc_columns <- grep(
  "^lfc_",
  names(fit$res),
  value = TRUE
)

cat("\nLFC columns detected:\n")

print(
  lfc_columns
)


# 4. Show exactly which column the extractor selected
```

```
cat("\nColumn selected by extract_ancombc2_from_resdf():\n")

print(
  ex$cols$lfc
)


# 5. Automatic check
expected_lfc <- paste0(
  "lfc_",
  params$state_var,
  "Diseased_Plant"
)

cat("\nExpected LFC column:\n")
print(expected_lfc)


if (
  identical(
    ex$cols$lfc,
    expected_lfc
  )
) {

  message(
    "✅ VERIFIED: extracted LFC corresponds to ",
    "Diseased_Plant vs Healthy_Plant."
  )

} else {

  warning(
    "⚠️ CHECK REQUIRED: extracted LFC column is '",
    ex$cols$lfc,
    "' but expected '",
    expected_lfc,
    "'."
  )
}


# 6. Explicit interpretation
cat(
  "\n==================================================\n",
  "EXPECTED INTERPRETATION\n",
  "LFC > 0 -> Diseased_Plant\n",
  "LFC < 0 -> Healthy_Plant\n",
  "==================================================\n"
)
    if (
      !isTRUE(
        ex$ok
      )
    ) {

      warning(
        "Results could not be extracted: ",
        ex$reason
      )

      next
    }


    resdf <- ex$resdf
    cols <- ex$cols

```

```
2835
2836  # ============================================================
2837  # 7. Correct taxonomic identifier
2838  # ============================================================
2839
2840  if (
2841    "taxon" %in%
2842    colnames(resdf)
2843  ) {
2844
2845    tax_id <- as.character(
2846      resdf$taxon
2847    )
2848
2849    message(
2850      "✅ Taxonomic identifier obtained from fit$res$taxon."
2851    )
2852
2853  } else {
2854
2855    tax_id <- rownames(
2856      resdf
2857    )
2858
2859    warning(
2860      "fit$res does not contain a 'taxon' column. ",
2861      "rownames(resdf) will be used."
2862    )
2863  }
2864
2865
2866  if (
2867    is.null(tax_id) ||
2868    length(tax_id) != nrow(resdf) ||
2869    anyNA(tax_id) ||
2870    any(
2871      trimws(tax_id) == ""
2872    )
2873  ) {
2874
2875    stop(
2876      "The taxonomic identifiers in fit$res are not valid."
2877    )
2878  }
2879
2880
2881  tax_id <- trimws(
2882    force_utf8(
2883      tax_id
2884    )
2885  )
2886
2887
2888  # ============================================================
2889  # 8. Observed prevalence per genus
2890  # ============================================================
2891
2892  prevalence_tbl <- calculate_taxon_prevalence(
2893    ps_crop,
2894    state_var = params$state_var
2895  )
2896
2897
2898  # ============================================================
2899  # 9. Taxonomy of the object used in ANCOM-BC2
2900  # ============================================================
2901
2902  tt <- as.data.frame(
```

```
  tax_table(ps_crop),
  stringsAsFactors = FALSE,
  check.names = FALSE
)


required_tax_ranks <- c(
  "Phylum",
  "Class",
  "Order",
  "Family",
  "Genus"
)


missing_ranks <- setdiff(
  required_tax_ranks,
  colnames(tt)
)


if (
  length(missing_ranks) > 0
) {

  stop(
    "Taxonomic ranks missing from tax_table(ps_crop): ",
    paste(
      missing_ranks,
      collapse = ", "
    )
  )
}


rownames(tt) <- trimws(
  force_utf8(
    rownames(tt)
  )
)


# ============================================================
# 9b. TaxID diagnostics
# ============================================================

matched <- tax_id %in%
  rownames(tt)


missing_tax_ids <- tax_id[
  !matched
]


cat(
  "Taxa returned by ANCOM-BC2:",
  length(tax_id),
  "\n"
)


cat(
  "TaxIDs found in tax_table:",
  sum(matched),
  "of",
  length(tax_id),
  "\n"
```

```
)


if (
  length(missing_tax_ids) > 0
) {

  warning(
    length(missing_tax_ids),
    " TaxIDs could not be mapped."
  )


  cat(
    "First unmatched TaxIDs:\n"
  )


  print(
    head(
      missing_tax_ids,
      20
    )
  )
}


# ============================================================
# 10. Local helper: map one rank
# ============================================================

map_rank <- function(
    rank_name
) {

  out <- rep(
    NA_character_,
    length(tax_id)
  )


  out[matched] <- as.character(
    tt[
      tax_id[matched],
      rank_name,
      drop = TRUE
    ]
  )


  out <- trimws(
    force_utf8(
      out
    )
  )


  out[
    is.na(out) |
      out == "" |
      out == "NA"
  ] <- "Unassigned"


  out
}
```

```
# ============================================================
# 11. Map the complete taxonomic lineage
# ============================================================

mapped_phylum <- map_rank(
  "Phylum"
)

mapped_class <- map_rank(
  "Class"
)

mapped_order <- map_rank(
  "Order"
)

mapped_family <- map_rank(
  "Family"
)

tax_label <- map_rank(
  "Genus"
)


# ============================================================
# 12. DEFINITIVE TAXONOMIC QC
# ============================================================

taxonomy_qc <- build_taxonomy_qc(

  tax_id = tax_id,

  genus = tax_label,

  family = mapped_family,

  order = mapped_order,

  class = mapped_class,

  phylum = mapped_phylum
)


cat(
  "\n=========================================================\n"
)

cat(
  "TAXONOMIC QC — ",
  dom,
  " | ",
  crop,
  "\n",
  sep = ""
)


cat(
  "Taxa evaluated: ",
  nrow(taxonomy_qc),
  "\n"
)


cat(
  "Genera retained: ",
```

```
    sum(
      !taxonomy_qc$Exclude
    ),
    "\n"
  )


  cat(
    "Assignments excluded: ",
    sum(
      taxonomy_qc$Exclude
    ),
    "\n"
  )


  cat(
    "=====================================================\n"
  )


  # ------------------------------------------------------------
  # Summary by exclusion reason
  # ------------------------------------------------------------

  taxonomy_qc_summary <- taxonomy_qc |>
    count(
      Exclusion_reason,
      Exclude,
      name = "n"
    ) |>
    arrange(
      desc(Exclude),
      desc(n)
    )


  print(
    taxonomy_qc_summary
  )


  # ------------------------------------------------------------
  # Show excluded taxa only
  # ------------------------------------------------------------

  excluded_taxa <- taxonomy_qc |>
    filter(
      Exclude
    ) |>
    select(
      TaxID,
      Phylum,
      Class,
      Order,
      Family,
      Genus,
      Exclusion_reason
    )


  if (
    nrow(excluded_taxa) > 0
  ) {

    cat(
      "\nAssignments removed:\n"
    )
```

```
    print(
      excluded_taxa,
      n = Inf
    )

  } else {

    message(
      "✅ No problematic taxonomic assignments were detected."
    )
  }


  # ============================================================
  # 13. Build the complete statistics table
  # ============================================================

  tbl_raw <- tibble(

    TaxID = tax_id,

    Domain = dom,

    TaxLevel = "Genus",

    Crop = crop,

    Phylum = mapped_phylum,

    Class = mapped_class,

    Order = mapped_order,

    Family = mapped_family,

    Taxon = tax_label,

    lfc = as.numeric(
      resdf[[cols$lfc]]
    ),

    se = if (
      !is.na(
        cols$se
      )
    ) {

      as.numeric(
        resdf[[cols$se]]
      )

    } else {

      NA_real_
    },

    p_val = as.numeric(
      resdf[[cols$p]]
    ),

    q_val = as.numeric(
      resdf[[cols$q]]
    ),

    diff_abn = if (
      !is.na(
        cols$diff_abn
```

```
        )
      ) {

        as.logical(
          resdf[[cols$diff_abn]]
        )

      } else {

        NA
      },

      passed_ss = if (
        !is.na(
          cols$passed_ss
        )
      ) {

        as.logical(
          resdf[[cols$passed_ss]]
        )

      } else {

        NA
      },

      diff_robust = if (
        !is.na(
          cols$diff_robust
        )
      ) {

        as.logical(
          resdf[[cols$diff_robust]]
        )

      } else {

        NA
      }
    )


    # ============================================================
# 14. Add taxonomic QC result and prevalence
# ============================================================

tbl_raw <- tbl_raw |>
  left_join(

    taxonomy_qc |>
      select(
        TaxID,
        Weird_assignment,
        Partial_assignment,
        Exclusion_reason,
        Exclude
      ),

    by = "TaxID"
  ) |>
  left_join(
    prevalence_tbl,
    by = "TaxID"
  )


```

```
    # ============================================================
    # 15. Final table: valid genera ONLY
    # ============================================================

    tbl <- tbl_raw |>

      filter(
        !Exclude
      ) |>

      mutate(

        significant_FDR =
          is.finite(q_val) &
          q_val < params$fdr_cut,

        sig_label = vapply(
          q_val,
          sig_stars,
          character(1)
        ),

        enriched_in = case_when(

          is.finite(lfc) &
            lfc > 0 ~
            "Diseased_Plant",

          is.finite(lfc) &
            lfc < 0 ~
            "Healthy_Plant",

          TRUE ~
            NA_character_
        )
      ) |>

      select(
        -Exclude,
        -Weird_assignment,
        -Partial_assignment,
        -Exclusion_reason
      ) |>

      arrange(
        q_val,
        p_val
      )


  # ============================================================
# 16. Remove Genus duplicates
# ============================================================

tbl <- dedup_by_taxon(
  tbl
)


# ============================================================
# 16B. Store objects for the direction audit
# ============================================================

audit_key <- paste(
  dom,
  crop,
  sep = "__"
)
```

```

direction_audit_store[[audit_key]] <- list(

  Domain = dom,

  Crop = crop,

  ps_crop = ps_crop,

  tbl = tbl
)


message(
  "✔ Stored for audit: ",
  audit_key,
  " | ",
  nsamples(ps_crop),
  " samples | ",
  nrow(tbl),
  " genera"
)


# ============================================================
# 17. Top 10 by q-value
# ============================================================

top10_tbl <- select_top_q(
  tbl,
  top_n = params$top_n_plot
)


    cat(
      "\nTop ",
      params$top_n_plot,
      " by q-value — ",
      dom,
      " / ",
      crop,
      "\n",
      sep = ""
    )


    print(
      top10_tbl |>
        select(
          Taxon,
          lfc,
          p_val,
          q_val,
          sig_label,
          passed_ss,
          diff_robust
        )
    )


    # ============================================================
    # 18. Final statistical summary
    # ============================================================

    final_summary <- tibble(

      Domain = dom,

```

```
    Crop = crop,

    Genera_retained =
      nrow(tbl),

    FDR_significant =
      sum(
        tbl$significant_FDR,
        na.rm = TRUE
      ),

    Higher_in_Diseased =
      sum(
        tbl$significant_FDR &
          tbl$lfc > 0,
        na.rm = TRUE
      ),

    Higher_in_Healthy =
      sum(
        tbl$significant_FDR &
          tbl$lfc < 0,
        na.rm = TRUE
      ),

    Passed_sensitivity =
      sum(
        tbl$passed_ss %in% TRUE,
        na.rm = TRUE
      ),

    Robust_significant =
      sum(
        tbl$diff_robust %in% TRUE,
        na.rm = TRUE
      )
  )


cat(
  "\nFinal summary:\n"
)


print(
  final_summary
)


# ============================================================
# 19. Structural zeros
# ============================================================

structural_zero_tbl <- extract_structural_zeros(
  fit = fit,
  ps = ps_crop
)


# ============================================================
# 20. Excel
# ============================================================

out_xlsx <- file.path(

  analysis_dir,

  "Tables",
```

```
      glue(
        "ANCOMBC2_{dom}_Genus_{crop}_LandAdjusted.xlsx"
      )
    )


    notes_tbl <- tibble(

      Info = c(

        "Taxonomic level: Genus only.",

        paste0(
          "ANCOM-BC2 model: ",
          params$land_var,
          " + ",
          params$state_var,
          "; analyses were performed separately for each Crop."
        ),

        "Land was included as a categorical fixed-effect covariate to account for farm-level heterogeneity.",

        "Healthy_Plant was used as the reference level for Plant_Condition.",

        "The extracted model coefficient corresponds to Plant_ConditionDiseased_Plant.",

        "Positive LFC values indicate greater estimated abundance in Diseased_Plant; negative LFC values indicate greater estimated abundance in Healthy_Plant.",

        "Taxonomic assignments were evaluated using the complete Phylum-Class-Order-Family-Genus lineage.",

        "Assignments were excluded when the Genus field was unresolved, provisional, represented a higher taxonomic rank, or reproduced Family, Order, Class, or Phylum.",

        "Candidatus designations were not excluded automatically; they were retained when they represented a distinct genus-level assignment.",

        paste0(
          "Primary statistical criterion: BH-adjusted q < ",
          params$fdr_cut,
          "."
        ),

        "*** q < 0.001; ** q < 0.01; * q < 0.05; NS q >= 0.05.",

        paste0(
          "Figures display the Top ",
          params$top_n_plot,
          " retained genera ranked by BH-adjusted q-value."
        ),

        "Genera labelled NS are shown for context and are not considered statistically significant.",

        "passed_ss and diff_robust are reported separately as ANCOM-BC2 pseudo-count sensitivity and robustness indicators.",

        paste0(
          "prv_cut = ",
          cfg$ancombc_prv_cut,
          "; lib_cut = ",
          params$lib_cut,
          "; p_adj_method = ",
          params$p_adj_method
        ),

        paste0(
          "The primary prevalence threshold was set to ",
```

```
          100 * cfg$ancombc_prv_cut,
          "% to reduce the influence of extremely sparse genera while retaining taxa recurrent across the dataset."
        ),

        paste0(
          "Differential-abundance inference was performed on unrarefied counts; the ",
          format(cfg$rare_depth, big.mark = ","),
          "-read rarefaction used for alpha/beta diversity was not applied to ANCOM-BC2."
        )
      )
    )


    openxlsx::write.xlsx(

      list(

        Results_AllTaxa =
          tbl,

        Results_Top10_by_q =
          top10_tbl,

        Results_SigOnly =
          tbl |>
            filter(
              significant_FDR
            ),

        Results_RobustOnly =
          tbl |>
            filter(
              !is.na(diff_robust),
              diff_robust
            ),

        Taxonomy_QC =
          taxonomy_qc,

        Taxonomy_Excluded =
          excluded_taxa,

        Taxonomy_QC_Summary =
          taxonomy_qc_summary,

        Analysis_Summary =
          final_summary,

        Structural_Zeros =
          structural_zero_tbl,

        Notes =
          notes_tbl
      ),

      out_xlsx,

      overwrite = TRUE
    )


    message(
      "📑 Table saved: ",
      out_xlsx
    )


    # ============================================================
```

```
# 21. Top 10 figure
# ============================================================

p <- plot_lfc_top10(

  tbl,

  title = glue(
    "Differential abundance — {dom}"
  ),

  subtitle = glue(
    "{crop} | Genus | Land + Plant_Condition"
  ),

  top_n = params$top_n_plot
)


# ============================================================
# 22. Save individual figure
# ============================================================

out_pdf <- file.path(
  analysis_dir,
  "Figures",
  glue(
    "LFC_TOP10_Q_{dom}_Genus_{crop}.pdf"
  )
)


out_png <- file.path(
  analysis_dir,
  "Figures",
  glue(
    "LFC_TOP10_Q_{dom}_Genus_{crop}.png"
  )
)


save_plot_pdf(
  out_pdf,
  p,
  width = 10,
  height = 7
)


save_plot_png(
  out_png,
  p,
  width = 10,
  height = 7,
  dpi = 300
)


# ============================================================
# 23. Copy for Multipanels
# ============================================================

save_plot_pdf(
  file.path(
    mp_dir,
    "Figures",
    basename(out_pdf)
  ),
```

```
      p,
      width = 10,
      height = 7
    )


    save_plot_png(
      file.path(
        mp_dir,
        "Figures",
        basename(out_png)
      ),
      p,
      width = 10,
      height = 7,
      dpi = 300
    )


    plots_crop[[crop]] <- p

  } # FIN Crop


  # ============================================================
  # 24. Multipanel per domain
  # ============================================================

  if (
    all(
      params$target_crops %in%
        names(plots_crop)
    )
  ) {

    p_left <- plots_crop[[
      params$target_crops[1]
    ]]


    p_right <- plots_crop[[
      params$target_crops[2]
    ]]


    mp <- (p_left | p_right) +

      patchwork::plot_annotation(

        title = glue(
          "ANCOM-BC2 — {dom} — Genus"
        ),

        subtitle = paste0(
          "Top ",
          params$top_n_plot,
          " retained genera ranked by BH-adjusted q-value in each crop. ",
          "Only q < ",
          params$fdr_cut,
          " is considered statistically significant."
        )
      )


    mp_pdf <- file.path(
      mp_dir,
      "Figures",
      glue(
```

```
        "Multipanel_DA_LFC_TOP10_Q_Genus_{dom}_CropsSideBySide.pdf"
      )
    )


    mp_png <- file.path(
      mp_dir,
      "Figures",
      glue(
        "Multipanel_DA_LFC_TOP10_Q_Genus_{dom}_CropsSideBySide.png"
      )
    )


    save_plot_pdf(
      mp_pdf,
      mp,
      width = 15,
      height = 7
    )


    save_plot_png(
      mp_png,
      mp,
      width = 15,
      height = 7,
      dpi = 300
    )


    message(
      "✅ Multipanel saved: ",
      mp_pdf
    )

  } else {

    warning(
      "Could not generate the multipanel for ",
      dom,
      " because one or both crops are missing."
    )
  }

} # FIN Domain

```

7. Independent audit of the LFC direction

```{r audit_lfc_direction, message=TRUE, warning=TRUE}

# ============================================================
# ENRICHMENT-DIRECTION AUDIT
#
# ANCOM-BC2 model:
#
#   Land + Plant_Condition
#
# Reference:
#   Healthy_Plant
#
# Coefficient:
#   Plant_ConditionDiseased_Plant
#
# Therefore:
#
```

```
#   LFC > 0  = higher estimated abundance in Diseased_Plant
#   LFC < 0  = higher estimated abundance in Healthy_Plant
#
# This audit does NOT modify the ANCOM-BC2 inference.
# It only checks that the interpretation of the sign is not
# inverted, and compares the sign with the observed data.
# ================================================================


# ----------------------------------------------------------------
# 1. Expected convention
# ----------------------------------------------------------------

reference_level <- "Healthy_Plant"

comparison_level <- "Diseased_Plant"


cat(
  "\n============================================================\n",
  "ANCOM-BC2 — LFC DIRECTION AUDIT\n",
  "============================================================\n",
  "Reference level : ", reference_level, "\n",
  "Coefficient     : ", comparison_level, "\n",
  "Contrast        : Diseased_Plant - Healthy_Plant\n",
  "LFC > 0         : Diseased_Plant\n",
  "LFC < 0         : Healthy_Plant\n",
  "============================================================\n\n",
  sep = ""
)


# ================================================================
# 2. Check that stored results exist
# ================================================================

if (
  !exists("direction_audit_store") ||
  length(direction_audit_store) == 0
) {

  stop(
    "❌ direction_audit_store is empty. ",
    "Run the main ANCOM-BC2 chunk first."
  )
}


# ================================================================
# 3. Audit function
# ================================================================

audit_ancom_direction <- function(
    ps_crop,
    tbl,
    domain,
    crop) {


  cat(
    "\n------------------------------------------------------------\n",
    domain,
    " | ",
    crop,
    "\n",
    "------------------------------------------------------------\n",
    sep = ""
  )
```

```

# ============================================================
# A. Metadata
# ============================================================

md <- data.frame(
  sample_data(ps_crop),
  stringsAsFactors = FALSE,
  check.names = FALSE
)

md$SampleID <- rownames(md)


# ============================================================
# B. Confirm Plant_Condition levels
# ============================================================

condition_observed <- unique(
  as.character(
    md[[params$state_var]]
  )
)


if (
  !all(
    c(
      reference_level,
      comparison_level
    ) %in%
      condition_observed
  )
) {

  stop(
    "❌ Both conditions are not present in ",
    crop
  )
}


# Reproduce exactly the coding used
# by run_ancombc2()

md[[params$state_var]] <- factor(
  as.character(
    md[[params$state_var]]
  ),
  levels = c(
    reference_level,
    comparison_level
  )
)


if (
  levels(
    md[[params$state_var]]
  )[1] != reference_level
) {

  stop(
    "❌ ERROR: Healthy_Plant is not the reference level."
  )
}

```

```
3987
3988  cat(
3989    "✔ Reference level confirmed: ",
3990    levels(
3991      md[[params$state_var]]
3992    )[1],
3993    "\n",
3994    sep = ""
3995  )
3996
3997
3998  cat(
3999    "✔ Expected coefficient: ",
4000    params$state_var,
4001    comparison_level,
4002    "\n",
4003    sep = ""
4004  )
4005
4006
4007  cat(
4008    "✔ Expected contrast: ",
4009    comparison_level,
4010    " - ",
4011    reference_level,
4012    "\n",
4013    sep = ""
4014  )
4015
4016
4017  # ============================================================
4018  # C. Count matrix
4019  # ============================================================
4020
4021  otu <- as(
4022    otu_table(ps_crop),
4023    "matrix"
4024  )
4025
4026
4027  if (
4028    !taxa_are_rows(ps_crop)
4029  ) {
4030
4031    otu <- t(otu)
4032  }
4033
4034
4035  # ============================================================
4036  # D. Taxonomy
4037  # ============================================================
4038
4039  tt <- as.data.frame(
4040    tax_table(ps_crop),
4041    stringsAsFactors = FALSE,
4042    check.names = FALSE
4043  )
4044
4045
4046  genus_map <- tibble(
4047    TaxID = rownames(tt),
4048
4049    Taxon = trimws(
4050      force_utf8(
4051        as.character(
4052          tt$Genus
4053        )
4054      )
```

```
  )
)


# ============================================================
# E. Counts in long format
# ============================================================

counts_long <- as.data.frame(
  otu
) |>

  rownames_to_column(
    "TaxID"
  ) |>

  pivot_longer(
    cols = -TaxID,
    names_to = "SampleID",
    values_to = "Count"
  ) |>

  left_join(
    genus_map,
    by = "TaxID"
  ) |>

  left_join(

    md |>
      select(
        SampleID,
        all_of(
          params$state_var
        ),
        all_of(
          params$land_var
        )
      ),

    by = "SampleID"
  )


# ============================================================
# F. Relative abundance per sample
#
# ONLY for the descriptive audit.
# ANCOM-BC2 used the unrarefied counts.
# ============================================================

counts_long <- counts_long |>

  group_by(
    SampleID
  ) |>

  mutate(

    LibrarySize =
      sum(
        Count,
        na.rm = TRUE
      ),

    Relative_abundance =
      ifelse(
        LibrarySize > 0,
```

```
          Count / LibrarySize,
          0
        )
    ) |>

    ungroup()


  # ============================================================
  # G. Global summary by condition
  # ============================================================

  global_summary <- counts_long |>

    group_by(
      TaxID,
      Taxon,
      .data[[params$state_var]]
    ) |>

    summarise(

      Mean_RA =
        mean(
          Relative_abundance,
          na.rm = TRUE
        ),

      Median_RA =
        median(
          Relative_abundance,
          na.rm = TRUE
        ),

      Mean_Count =
        mean(
          Count,
          na.rm = TRUE
        ),

      Prevalence =
        mean(
          Count > 0,
          na.rm = TRUE
        ),

      .groups = "drop"
    ) |>

    pivot_wider(

      names_from =
        all_of(
          params$state_var
        ),

      values_from = c(
        Mean_RA,
        Median_RA,
        Mean_Count,
        Prevalence
      ),

      names_sep = "_"
    )


  # ============================================================
```

```
# H. Global observed difference
#
# Diseased - Healthy
# ============================================================

global_summary <- global_summary |>

  mutate(

    Delta_Mean_RA =

      .data[[
        paste0(
          "Mean_RA_",
          comparison_level
        )
      ]] -

      .data[[
        paste0(
          "Mean_RA_",
          reference_level
        )
      ]],


    Observed_global_direction =
      case_when(

        Delta_Mean_RA > 0 ~
          comparison_level,

        Delta_Mean_RA < 0 ~
          reference_level,

        TRUE ~
          "Equal"
      )
  )


# ============================================================
# I. Mean per farm × condition
#
# This comparison matters because
# ANCOM-BC2 adjusted for Land.
# ============================================================

farm_condition <- counts_long |>

  group_by(
    TaxID,
    Taxon,
    .data[[params$land_var]],
    .data[[params$state_var]]
  ) |>

  summarise(

    Mean_RA_farm =
      mean(
        Relative_abundance,
        na.rm = TRUE
      ),

    .groups = "drop"
  ) |>

```

```
    pivot_wider(

      names_from =
        all_of(
          params$state_var
        ),

      values_from =
        Mean_RA_farm
    )


  # ============================================================
  # J. Keep only farms with both conditions
  # ============================================================

  farm_condition <- farm_condition |>

    filter(

      !is.na(
        .data[[reference_level]]
      ),

      !is.na(
        .data[[comparison_level]]
      )
    )


  # ============================================================
  # K. Within-farm difference
  #
  # Diseased - Healthy
  # ============================================================

  farm_condition <- farm_condition |>

    mutate(

      Delta_farm =

        .data[[comparison_level]] -
        .data[[reference_level]],


      Farm_direction =
        case_when(

          Delta_farm > 0 ~
            comparison_level,

          Delta_farm < 0 ~
            reference_level,

          TRUE ~
            "Equal"
        )
    )


  # ============================================================
  # L. Direction summary across farms
  # ============================================================

  farm_summary <- farm_condition |>

    group_by(
```

```
      TaxID,
      Taxon
    ) |>

    summarise(

      N_farms =
        n(),

      Farms_Diseased_higher =
        sum(
          Farm_direction ==
            comparison_level,
          na.rm = TRUE
        ),

      Farms_Healthy_higher =
        sum(
          Farm_direction ==
            reference_level,
          na.rm = TRUE
        ),

      Farms_equal =
        sum(
          Farm_direction ==
            "Equal",
          na.rm = TRUE
        ),

      Median_Delta_farm =
        median(
          Delta_farm,
          na.rm = TRUE
        ),

      Mean_Delta_farm =
        mean(
          Delta_farm,
          na.rm = TRUE
        ),

      Farm_majority_direction =
        case_when(

          Farms_Diseased_higher >
            Farms_Healthy_higher ~
            comparison_level,

          Farms_Healthy_higher >
            Farms_Diseased_higher ~
            reference_level,

          TRUE ~
            "Mixed/Tie"
        ),

      .groups = "drop"
    )


# ============================================================
# M. Prepare ANCOM-BC2 results
# ============================================================

ancom_tbl <- tbl |>

  select(
```

```
      TaxID,
      Taxon,
      lfc,
      se,
      p_val,
      q_val,
      significant_FDR,
      passed_ss,
      diff_robust,
      Prevalence_overall,
      Prevalence_asymptomatic,
      Prevalence_symptomatic,
      everything()
    ) |>

    mutate(

      ANCOMBC2_direction =
        case_when(

          is.finite(lfc) &
            lfc > 0 ~
            comparison_level,

          is.finite(lfc) &
            lfc < 0 ~
            reference_level,

          TRUE ~
            "No_direction"
        )
    )


  # ============================================================
  # N. Merge audit
  # ============================================================

  audit <- ancom_tbl |>

    left_join(
      global_summary |>
        select(
          TaxID,
          Delta_Mean_RA,
          Observed_global_direction,
          starts_with(
            "Mean_RA_"
          ),
          starts_with(
            "Median_RA_"
          ),
          starts_with(
            "Mean_Count_"
          ),
          starts_with(
            "Prevalence_"
          )
        ),
      by = "TaxID",
      suffix = c("", "_audit")
    ) |>

    left_join(
      farm_summary |>
        select(
          TaxID,
          N_farms,
```

```
          Farms_Diseased_higher,
          Farms_Healthy_higher,
          Farms_equal,
          Median_Delta_farm,
          Mean_Delta_farm,
          Farm_majority_direction
        ),
      by = "TaxID"
    )


# ============================================================
# 0. Concordance
# ============================================================

audit <- audit |>

  mutate(

    Global_direction_check =
      case_when(

        ANCOMBC2_direction ==
          Observed_global_direction ~
          "Concordant",

        Observed_global_direction ==
          "Equal" ~
          "Equal",

        is.na(
          Observed_global_direction
        ) ~
          "Not_evaluable",

        TRUE ~
          "Discordant"
      ),


    Farm_direction_check =
      case_when(

        ANCOMBC2_direction ==
          Farm_majority_direction ~
          "Concordant",

        Farm_majority_direction ==
          "Mixed/Tie" ~
          "Mixed/Tie",

        is.na(
          Farm_majority_direction
        ) ~
          "Not_evaluable",

        TRUE ~
          "Discordant"
      )
  )


# ============================================================
# P. Final audit
#
# IMPORTANT:
#
# This classification is NOT a new statistical test.
```

```
# It is a descriptive check only.
# ============================================================

audit <- audit |>

  mutate(

    Direction_audit =
      case_when(

        Farm_direction_check ==
          "Concordant" ~
          "SUPPORTED",

        Farm_direction_check ==
          "Mixed/Tie" &
        Global_direction_check ==
          "Concordant" ~
          "SUPPORTED_GLOBAL_ONLY",

        Farm_direction_check ==
          "Discordant" ~
          "CHECK_DIRECTION",

        Global_direction_check ==
          "Discordant" ~
          "CHECK_DIRECTION",

        TRUE ~
          "REVIEW"
      ),

    Domain = domain,

    Crop = crop,

    Reference =
      reference_level,

    Comparison =
      comparison_level,

    Contrast =
      paste0(
        comparison_level,
        " - ",
        reference_level
      ),

    .before = 1
  )


# ============================================================
# Q. Significant results
# ============================================================

audit_sig <- audit |>

  filter(
    significant_FDR
  ) |>

  arrange(
    q_val
  )
```

```
cat(
  "\nSignificant genera (q < ",
  params$fdr_cut,
  "): ",
  nrow(audit_sig),
  "\n",
  sep = ""
)


if (
  nrow(audit_sig) > 0
) {

  print(

    audit_sig |>

      select(
        Taxon,
        lfc,
        q_val,
        ANCOMBC2_direction,
        Observed_global_direction,
        Farm_majority_direction,
        Farms_Diseased_higher,
        Farms_Healthy_higher,
        Global_direction_check,
        Farm_direction_check,
        Direction_audit
      )
  )
}


# ============================================================
# R. Alerts
# ============================================================

suspicious <- audit_sig |>

  filter(
    Direction_audit ==
      "CHECK_DIRECTION"
  )


if (
  nrow(suspicious) > 0
) {

  warning(
    "⚠ ",
    nrow(suspicious),
    " significant genera require manual inspection in ",
    crop,
    "."
  )

} else {

  message(
    "✔ No evident direction inversion detected for significant ",
    "genera in ",
    crop,
    "."
  )
}
```

```
4667
4668
4669    # ============================================================
4670    # S. Return
4671    # ============================================================
4672
4673    list(
4674
4675      all =
4676        audit,
4677
4678      significant =
4679        audit_sig,
4680
4681      farm_detail =
4682        farm_condition
4683    )
4684  }
4685
4686
4687  # ============================================================
4688  # 4. Run the audit on ALL stored analyses
4689  # ============================================================
4690
4691  audit_results <- list()
4692
4693
4694  for (
4695    key in names(
4696      direction_audit_store
4697    )
4698  ) {
4699
4700    obj <- direction_audit_store[[
4701      key
4702    ]]
4703
4704
4705    audit_results[[key]] <-
4706      audit_ancom_direction(
4707
4708        ps_crop =
4709          obj$ps_crop,
4710
4711        tbl =
4712          obj$tbl,
4713
4714        domain =
4715          obj$Domain,
4716
4717        crop =
4718          obj$Crop
4719      )
4720  }
4721
4722
4723  # ============================================================
4724  # 5. Combine results
4725  # ============================================================
4726
4727  audit_all <- bind_rows(
4728
4729    lapply(
4730      audit_results,
4731      function(x) x$all
4732    )
4733  )
4734
```

```

audit_significant <- bind_rows(

  lapply(
    audit_results,
    function(x) x$significant
  )
)


farm_details <- bind_rows(

  lapply(
    names(
      audit_results
    ),

    function(key) {

      x <- audit_results[[
        key
      ]]

      meta <- direction_audit_store[[
        key
      ]]

      x$farm_detail |>

        mutate(
          Domain =
            meta$Domain,

          Crop =
            meta$Crop,

          .before = 1
        )
    }
  )
)


# ================================================================
# 6. Final summary
# ================================================================

direction_summary <- audit_significant |>

  count(
    Domain,
    Crop,
    ANCOMBC2_direction,
    Direction_audit,
    name = "N_genera"
  ) |>

  arrange(
    Domain,
    Crop,
    Direction_audit
  )


cat(
  "\n================================================================\n",
  "FINAL DIRECTION AUDIT\n",
  "================================================================\n"
```

```
)


print(
  direction_summary
)


# ================================================================
# 7. Review possible discordances
# ================================================================

direction_flags <- audit_significant |>

  filter(
    Direction_audit %in%
      c(
        "CHECK_DIRECTION",
        "REVIEW"
      )
  ) |>

  select(
    Domain,
    Crop,
    Taxon,
    lfc,
    q_val,
    ANCOMBC2_direction,
    Observed_global_direction,
    Farm_majority_direction,
    Farms_Diseased_higher,
    Farms_Healthy_higher,
    Median_Delta_farm,
    Global_direction_check,
    Farm_direction_check,
    Direction_audit
  )


if (
  nrow(
    direction_flags
  ) > 0
) {

  warning(
    "⚠ ",
    nrow(direction_flags),
    " significant genera require inspection."
  )

  print(
    direction_flags
  )

} else {

  message(
    "✅ Direction audit completed: ",
    "no evident LFC direction inversion detected."
  )
}


# ================================================================
# 8. Convention used
# ================================================================
```

```
4871
4872 convention_tbl <- tibble(
4873
4874   Parameter = c(
4875
4876     "Reference level",
4877
4878     "Model coefficient",
4879
4880     "Contrast",
4881
4882     "Positive LFC",
4883
4884     "Negative LFC",
4885
4886     "ANCOM-BC2 input",
4887
4888     "ANCOM-BC2 model",
4889
4890     "Primary significance criterion",
4891
4892     "Audit purpose"
4893   ),
4894
4895   Value = c(
4896
4897     "Healthy_Plant",
4898
4899     "Plant_ConditionDiseased_Plant",
4900
4901     "Diseased_Plant - Healthy_Plant",
4902
4903     "Higher estimated abundance in Diseased_Plant",
4904
4905     "Higher estimated abundance in Healthy_Plant",
4906
4907     "Unrarefied genus-level counts",
4908
4909     "Land + Plant_Condition",
4910
4911     paste0(
4912       "BH-adjusted q < ",
4913       params$fdr_cut
4914     ),
4915
4916     paste0(
4917       "Descriptive verification of LFC direction; ",
4918       "does not modify ANCOM-BC2 inference"
4919     )
4920   )
4921 )
4922
4923
4924 # ============================================================
4925 # 9. Export Excel
4926 # ============================================================
4927
4928 audit_dir <- file.path(
4929   project_root,
4930   params$analysis_subdir,
4931   "Tables"
4932 )
4933
4934
4935 ensure_dir(
4936   audit_dir
4937 )
4938
```

```

audit_file <- file.path(
  audit_dir,
  glue("ANCOMBC2_Direction_Audit_{cfg$domain}.xlsx")
)


openxlsx::write.xlsx(

  list(

    Significant_Audit =
      audit_significant,

    All_Audit =
      audit_all,

    Direction_Flags =
      direction_flags,

    Farm_Details =
      farm_details,

    Summary =
      direction_summary,

    Convention =
      convention_tbl
  ),

  audit_file,

  overwrite = TRUE
)


message(
  "📑 Direction audit saved: ",
  audit_file
)


# ================================================================
# 10. Final message
# ================================================================

cat(
  "\n================================================================\n",
  "AUDIT INTERPRETATION\n",
  "================================================================\n",
  "SUPPORTED             = LFC agrees with the majority of farms\n",
  "SUPPORTED_GLOBAL_ONLY = global direction agrees; farms are mixed\n",
  "CHECK_DIRECTION       = observed direction is discordant; inspect\n",
  "REVIEW                = insufficient/ambiguous descriptive support\n",
  "\n",
  "IMPORTANT:\n",
  "These categories are descriptive QC flags and are NOT additional\n",
  "statistical significance criteria. ANCOM-BC2 q-values remain the\n",
  "basis for differential-abundance inference.\n",
  "================================================================\n"
)
```

Session information

```{r session_info}
sessionInfo()
```

```
5007
```

06 — dbRDA with soil physicochemical variables and farm-restricted permutations, by domain and crop

Script: 06_dbRDA.Rmd · source listing (1820 lines) · code shown as written; chunks not executed

R chunks

line 61: ```{r setup, message=TRUE, warning=TRUE}
line 104: ```{r helpers_and_metadata, message=TRUE, warning=TRUE}
line 372: ```{r dbrda_function, message=TRUE, warning=TRUE}
line 1259: ```{r main_loop_export_tables, message=TRUE, warning=TRUE}
line 1517: ```{r multipanel_bac_fungi_and_global_tables, message=TRUE, warning=TRUE}
line 1817: ```{r session_info}

```
---
title: "06 — dbRDA with soil physicochemical variables and farm-restricted permutations, by domain and crop"
author: "Sergio Pardo-Díaz"
date: "`r format(Sys.Date())`"
output:
  html_document:
    toc: true
    toc_depth: 3
    number_sections: true
  pdf_document:
    toc: true
    number_sections: true

params:
  # Parent folder that contains both domain folders
  # (<project_root>/<domain_dir>, see domain_settings.R)
  project_root: "."
  # Optional per-domain folder overrides (NULL = domain_settings.R)
  domain_dir_bacteria: null
  domain_dir_fungi: null
  filtering_subdir: "01_Filtering_ASVs"
  filtering_rds_subdir: "RDS"
  analysis_subdir: "03_Beta-Diversity"

  state_var: "Plant_Condition"
  crop_var: "Crop"
  strata_var: "Land"

  # Full metadata (includes soil chemical/edaphic variables + Plant_Condition + Crop),
  # relative to project_root
  metadata_edaphic_path: "Metadata_Completo_plus_edaphic_R_16S_ITS.xlsx"

  # Subfolder for dbRDA exports
  rda_subdir: "06_RDA"

  # ANOVA permutations (dbRDA)
  nperm: 9999
  # Random seed for permutation tests (reproducible P-values)
  seed: 2026
  settings_file: "domain_settings.R"
---

# Reviewer-ready design notes

This version implements the farm-level permutation structure explicitly and obligatorily:

- Distance: Bray–Curtis.
- Inputs: the rarefied object of each domain (`ps_rare_<depth>.rds`; bacteria 5,000 reads/sample, fungi ITS2 no-trunc 2,000 reads/sample, set in `domain_settings.R`).
- Analyses: separately by microbial domain and crop.
- Environmental predictors: predefined soil physicochemical variables.
```

```
- Collinearity: iterative VIF filtering; the predictor with the highest VIF > 10 is removed until all retained predictors have VIF <= 10.
- Final dbRDA model: `Community ~ retained physicochemical predictors`.
- Farm structure: **9,999 permutations restricted within `Land` (farm)** for both the global test and tests by term.
- Model fit: full R² and adjusted R² are exported.
- There is no fallback to unrestricted permutations; the script stops if `Land` is absent or invalid.

---

# CHUNK 0 — setup

```{r setup, message=TRUE, warning=TRUE}
knitr::opts_chunk$set(
  fig.width  = 7,
  fig.height = 6,
  dpi        = 300,
  dev        = "cairo_pdf"
)

suppressPackageStartupMessages({
  library(phyloseq)
  library(vegan)
  library(permute)
  library(tidyverse)
  library(glue)
  library(ggrepel)
  library(patchwork)
  library(readxl)
  library(openxlsx)
  library(grid)
  library(Cairo)
  library(stringi)
})

# ---- Domain settings (single source of truth: domain_settings.R)
settings_file <- params$settings_file
if (!file.exists(settings_file)) {
  settings_file <- file.path(dirname(knitr::current_input(dir = TRUE)), settings_file)
}
source(settings_file)

# Fixed seed so that permutation P-values are reproducible between runs
set.seed(params$seed)

cfg_by_domain <- list(
  Bacterias = domain_settings("Bacterias", list(domain_dir = params$domain_dir_bacteria)),
  Hongos    = domain_settings("Hongos",    list(domain_dir = params$domain_dir_fungi))
)
```

CHUNK 1 — `helpers_and_metadata`

```{r helpers_and_metadata, message=TRUE, warning=TRUE}
# ============================================================
# Helpers generales
# ============================================================
.require_param <- function(x, name) {
  if (is.null(x) || length(x) == 0 || is.na(x) || x == "") {
    stop(glue("Param '{name}' is empty/NULL. Check your YAML."))
  }
  x
}

strip_accents <- function(x) {
  x <- as.character(x)
  iconv(x, from = "UTF-8", to = "ASCII//TRANSLIT")
```
```

```r
}

harmonize_condition <- function(x) {
  x2 <- as.character(x)
  x2 <- gsub("-", "_", x2)
  x2 <- gsub(" ", "_", x2)
  xlow <- tolower(x2)

  x2[xlow %in% c("healthy_plant", "healthy")]   <- "Healthy_Plant"
  x2[xlow %in% c("diseased_plant", "diseased")] <- "Diseased_Plant"

  factor(x2, levels = c("Healthy_Plant", "Diseased_Plant"))
}

harmonize_crop <- function(x) {
  x2 <- as.character(x)
  x2 <- strip_accents(x2)
  x2 <- gsub("-", "_", x2)
  x2 <- gsub(" ", "_", x2)
  factor(x2)
}

normalize_crop_key <- function(x) {
  x2 <- as.character(x)
  x2 <- strip_accents(x2)
  x2 <- gsub("-", "_", x2)
  x2 <- gsub(" ", "_", x2)
  x2
}

pretty_crop <- function(x) {
  x <- normalize_crop_key(x)
  dplyr::case_when(
    x == "Dominico_Harton"     ~ "Dominico Hartón",
    x == "Banano_Gros_Michael" ~ "Gros Michel",
    TRUE ~ x
  )
}

pretty_domain <- function(x) {
  dplyr::case_when(
    x == "Bacterias" ~ "Bacteria",
    x == "Hongos"    ~ "Fungi",
    TRUE ~ x
  )
}

p_to_stars <- function(p) {
  dplyr::case_when(
    is.na(p)      ~ "",
    p < 0.001     ~ "***",
    p < 0.01      ~ "**",
    p < 0.05      ~ "*",
    TRUE          ~ ""
  )
}

fmt_p <- function(p) {
  ifelse(is.na(p), NA_character_, format.pval(p, digits = 2, eps = 0.001))
}

# ============================================================
# Palette
# ============================================================
pal_condition <- c(
  "Healthy_Plant"  = "#00BFC4",
  "Diseased_Plant" = "#F8766D"
)
```

```
# ============================================================
# Rutas
# ============================================================
ps_path_for_domain <- function(domain) {
  domain <- as.character(domain)

  if (!domain %in% names(cfg_by_domain)) {
    stop(glue("Unsupported domain: '{domain}'. Use: Bacterias | Hongos"))
  }

  cfg_dom <- cfg_by_domain[[domain]]
  # Rarefied object of this domain (bacteria ps_rare_5000, fungi ps_rare_2000)
  ps_file <- paste0("ps_rare_", cfg_dom$rare_depth, ".rds")

  file.path(
    domain_path(.require_param(params$project_root, "project_root"), cfg_dom),
    .require_param(params$filtering_subdir, "filtering_subdir"),
    .require_param(params$filtering_rds_subdir, "filtering_rds_subdir"),
    .require_param(ps_file, glue("ps_file ({domain})"))
  )
}

outdir_for_domain <- function(domain) {
  file.path(
    domain_path(.require_param(params$project_root, "project_root"),
                cfg_by_domain[[as.character(domain)]]),
    .require_param(params$analysis_subdir, "analysis_subdir"),
    .require_param(params$rda_subdir, "rda_subdir")
  )
}

figdir_for_domain <- function(domain) {
  file.path(outdir_for_domain(domain), "Figures")
}

tabdir_for_domain <- function(domain) {
  file.path(outdir_for_domain(domain), "Tables")
}

outdir_global <- function() {
  file.path(
    .require_param(params$project_root, "project_root"),
    .require_param(params$analysis_subdir, "analysis_subdir"),
    .require_param(params$rda_subdir, "rda_subdir")
  )
}

figdir_global <- function() {
  file.path(outdir_global(), "Figures")
}

tabdir_global <- function() {
  file.path(outdir_global(), "Tables")
}

# ============================================================
# Export helpers
# ============================================================
ensure_dir <- function(path) {
  dir.create(path, recursive = TRUE, showWarnings = FALSE)
  invisible(path)
}

save_plot_pdf <- function(path, plot, width = 7, height = 6) {
  ggplot2::ggsave(
    filename = path, plot = plot,
    width = width, height = height,
```

```
    device = cairo_pdf
  )
  invisible(path)
}

save_plot_png <- function(path, plot, width = 7, height = 6, dpi = 300) {
  ggplot2::ggsave(
    filename = path, plot = plot,
    width = width, height = height,
    dpi = dpi, device = "png"
  )
  invisible(path)
}

ensure_dir(figdir_for_domain("Bacterias"))
ensure_dir(tabdir_for_domain("Bacterias"))
ensure_dir(figdir_for_domain("Hongos"))
ensure_dir(tabdir_for_domain("Hongos"))
ensure_dir(figdir_global())
ensure_dir(tabdir_global())

message("DEBUG figdir Bacterias: ", figdir_for_domain("Bacterias"))
message("DEBUG tabdir Bacterias: ", tabdir_for_domain("Bacterias"))
message("DEBUG figdir Hongos   : ", figdir_for_domain("Hongos"))
message("DEBUG tabdir Hongos   : ", tabdir_for_domain("Hongos"))
message("DEBUG figdir Global   : ", figdir_global())
message("DEBUG tabdir Global   : ", tabdir_global())
message("DEBUG ps Bacterias    : ", ps_path_for_domain("Bacterias"))
message("DEBUG ps Hongos       : ", ps_path_for_domain("Hongos"))

# ========================================================
# Metadata
# ========================================================
metadata_path <- file.path(params$project_root, params$metadata_edaphic_path)
if (!file.exists(metadata_path)) {
  stop(glue("metadata_edaphic_path does not exist: {metadata_path}"))
}

md_all <- readxl::read_xlsx(metadata_path) %>%
  mutate(
    `sample-id` = as.character(`sample-id`),
    Marker      = as.character(Marker)
  )

if (!"sample-id" %in% names(md_all)) {
  stop("The Excel metadata must contain the exact column: 'sample-id'.")
}
if (!"Marker" %in% names(md_all)) {
  stop("The Excel metadata must contain the column: 'Marker' (16S/ITS).")
}
if (!all(c(params$state_var, params$crop_var, params$strata_var) %in% names(md_all))) {
  stop(glue(
    "The Excel metadata must include: {params$state_var}, {params$crop_var} and {params$strata_var}."
  ))
}

md_all <- md_all %>%
  mutate(
    !!params$state_var  := harmonize_condition(.data[[params$state_var]]),
    !!params$crop_var   := harmonize_crop(.data[[params$crop_var]]),
    !!params$strata_var := factor(trimws(as.character(.data[[params$strata_var]]))),
    crop_key            = normalize_crop_key(.data[[params$crop_var]])
  )

if (any(is.na(md_all[[params$strata_var]]) | md_all[[params$strata_var]] == "")) {
  stop(glue("The farm variable '{params$strata_var}' contains empty/NA values."))
}

```

```r
message(glue("Metadata (Excel) loaded: {nrow(md_all)} rows, {ncol(md_all)} columns."))
message("Marker counts:")
print(table(md_all$Marker, useNA = "ifany"))

message("Crop levels (normalized) found in metadata:")
print(sort(unique(md_all$crop_key)))

# ============================================================
# Explicit edaphic variables
# ============================================================
vars_edaphic <- c(
  "pH", "Organic_M", "P",
  "Ca_Exchange", "Mg_Exchange", "K_Exchange",
  "CEC", "Zn", "Cu", "B",
  "N_Total", "Respirometry"
)

nice_names <- c(
  "pH"           = "pH",
  "Organic_M"    = "Organic matter",
  "P"            = "P",
  "Ca_Exchange"  = "Ca2+",
  "Mg_Exchange"  = "Mg2+",
  "K_Exchange"   = "K+",
  "CEC"          = "CEC",
  "Zn"           = "Zn2+",
  "Cu"           = "Cu2+",
  "B"            = "B",
  "N_Total"      = "Total N",
  "Respirometry" = "Respirometry"
)

# ============================================================
# Minimal normalization of ITS2 no-trunc IDs
# IMPORTANT: only the technical Illumina suffix of the ITS metadata
# is modified. 16S/bacterial IDs are not modified.
# E.g.: EE210-00-ITS-S7-L001 -> EE210-00-ITS_S7_L001
# ============================================================
normalize_its_id <- function(x) {
  gsub(
    "-S([0-9]+)-L([0-9]+)$",
    "_S\\1_L\\2",
    as.character(x)
  )
}
```

CHUNK 2 — `dbrda_function`

```{r dbrda_function, message=TRUE, warning=TRUE}

run_dbrda <- function(ps, env_tbl, title_label, domain_label, crop_label,
                      crop_key_use, nperm = params$nperm) {

  # ========================================================
  # 0) Minimal checks — Land is mandatory
  # ========================================================
  required_cols <- c(
    "sample-id",
    params$state_var,
    params$crop_var,
    params$strata_var
  )

  missing_cols <- setdiff(required_cols, names(env_tbl))
  if (length(missing_cols) > 0) {
    stop(glue(
```

```
      "Missing columns in metadata: {paste(missing_cols, collapse = ', ')}"
    ))
  }

  if (any(is.na(env_tbl[[params$strata_var]]) |
          trimws(as.character(env_tbl[[params$strata_var]])) == "")) {
    stop(glue(
      "{title_label}: '{params$strata_var}' contains NA/empty values. ",
      "The analysis stops because permutations must be restricted within farm."
    ))
  }

  # ======================================================
  # 1) OTU/ASV matrix
  # ======================================================
  otu_mat <- as(otu_table(ps), "matrix")
  if (taxa_are_rows(ps)) otu_mat <- t(otu_mat)

  keep_samp <- rowSums(otu_mat) > 0
  if (any(!keep_samp)) {
    otu_mat <- otu_mat[keep_samp, , drop = FALSE]
    env_tbl <- env_tbl[keep_samp, , drop = FALSE]
  }

  # ======================================================
  # 2) Explicit edaphic variables + complete cases
  # ======================================================
  env_num <- env_tbl %>%
    dplyr::select(any_of(vars_edaphic)) %>%
    dplyr::select(where(is.numeric))

  env_num <- env_num[, colSums(!is.na(env_num)) > 0, drop = FALSE]

  if (ncol(env_num) == 0) {
    stop("No valid numeric edaphic variables (vars_edaphic).")
  }

  cc <- stats::complete.cases(env_num)

  if (any(!cc)) {
    message(glue(
      ">> {title_label}: {sum(!cc)} samples removed due to NA in edaphic variables."
    ))
    otu_mat <- otu_mat[cc, , drop = FALSE]
    env_tbl <- env_tbl[cc, , drop = FALSE]
    env_num <- env_num[cc, , drop = FALSE]
  }

  stopifnot(nrow(otu_mat) == nrow(env_num))
  stopifnot(nrow(env_tbl) == nrow(env_num))

  # Land after ALL filters, to guarantee alignment
  land_vec <- droplevels(
    factor(as.character(env_tbl[[params$strata_var]]))
  )

  if (nlevels(land_vec) < 2) {
    stop(glue(
      "{title_label}: only one farm remained after filtering. ",
      "A farm-level permutation structure cannot be implemented."
    ))
  }

  message(glue(
    "  Farm-level permutation block: {params$strata_var} | ",
    "{nlevels(land_vec)} farms | {nrow(env_tbl)} samples."
  ))
  print(table(land_vec))
```

```r

# Standardization of environmental variables
env_scaled <- as.data.frame(scale(env_num))

# Remove zero-variance variables
keep_var <- vapply(
  env_scaled,
  function(z) is.finite(stats::sd(z, na.rm = TRUE)) &&
    stats::sd(z, na.rm = TRUE) > 0,
  logical(1)
)

env_scaled <- env_scaled[, keep_var, drop = FALSE]

if (ncol(env_scaled) == 0) {
  stop(glue(
    "{title_label}: all edaphic variables have zero variance after scaling."
  ))
}

# ======================================================
# 2b) Colinealidad — VIF iterativo
#     The predictor with the highest VIF > 10 is removed
#     and VIFs are recomputed until all VIFs are <= 10.
# ======================================================
vif_threshold <- 10
vif_log <- list()
removed_predictors <- character(0)

repeat {

  db_vif <- vegan::dbrda(
    otu_mat ~ .,
    data = env_scaled,
    distance = "bray"
  )

  vif_now <- vegan::vif.cca(db_vif)

  vif_round <- data.frame(
    Round = length(vif_log) + 1L,
    Variable = names(vif_now),
    VIF = as.numeric(vif_now),
    Removed = FALSE,
    stringsAsFactors = FALSE
  )

  worst <- names(which.max(vif_now))

  if (is.finite(vif_now[worst]) && vif_now[worst] > vif_threshold) {

    vif_round$Removed[vif_round$Variable == worst] <- TRUE
    removed_predictors <- c(removed_predictors, worst)

    message(glue(
      "  VIF > {vif_threshold}: eliminando '{worst}' ",
      "(VIF = {round(vif_now[worst], 2)})"
    ))

    vif_log[[length(vif_log) + 1L]] <- vif_round

    env_scaled <- env_scaled[
      ,
      setdiff(colnames(env_scaled), worst),
      drop = FALSE
    ]

    if (ncol(env_scaled) == 0) {
```

```
        stop(glue(
          "{title_label}: all variables were removed due to VIF > {vif_threshold}."
        ))
      }

    } else {

      vif_log[[length(vif_log) + 1L]] <- vif_round
      break
    }
  }

  vif_tbl <- dplyr::bind_rows(vif_log) %>%
    mutate(
      VIF = round(VIF, 3),
      Status = if_else(Removed, "Removed", "Retained in this round")
    )

  retained_predictors <- colnames(env_scaled)

  message(glue(
    "  Variables retenidas tras VIF: {paste(retained_predictors, collapse = ', ')}"
  ))

  if (length(removed_predictors) > 0) {
    message(glue(
      "  Variables removed due to VIF: {paste(removed_predictors, collapse = ', ')}"
    ))
  } else {
    message("  No variable was removed due to VIF.")
  }

  # ======================================================
  # 3) dbRDA final — Bray-Curtis
  # ======================================================
  db <- vegan::dbrda(
    otu_mat ~ .,
    data = env_scaled,
    distance = "bray"
  )

  final_formula <- paste(
    "Community ~",
    paste(retained_predictors, collapse = " + ")
  )

  # ======================================================
  # 3a) MANDATORY farm-level permutation design
  #
  # Permutations are restricted WITHIN Land.
  # There is no fallback to unrestricted permutations.
  # ======================================================
  perm_design <- permute::how(nperm = nperm)
  permute::setBlocks(perm_design) <- land_vec

  message(glue(
    "  Permutation scheme: {nperm} permutations restricted within ",
    "{params$strata_var} (farm)."
  ))

  # Global and by-term tests using THE SAME blocked design
  a_global <- as.data.frame(
    vegan::anova.cca(
      db,
      permutations = perm_design
    )
  )

```

```r
  p_global <- a_global$`Pr(>F)`[1]
  F_global <- a_global$F[1]

  r2_values <- vegan::RsquareAdj(db)
  r2 <- unname(r2_values$r.squared)
  r2_adj <- unname(r2_values$adj.r.squared)

  # Sequential (Type I) tests: each term is evaluated in formula order.
  # The same farm-restricted permutation design is used for every term.
  a_terms <- as.data.frame(
    vegan::anova.cca(
      db,
      by = "term",
      permutations = perm_design
    )
  ) %>%
    tibble::rownames_to_column("Variable")

  # Clean the by-term table for export and interpretation
  # IMPORTANT: anova.cca(..., by = "term") performs sequential (Type I)
  # permutation tests. Because several physicochemical predictors are tested
  # within each Domain x Crop dbRDA model, raw permutation P values are
  # adjusted within that model using Benjamini-Hochberg FDR.
  # Statistical significance of individual predictors is based on q < 0.05.
  a_terms_clean <- a_terms %>%
    filter(Variable != "Residual") %>%
    mutate(
      p_value = .data[["Pr(>F)"]],
      q_value = p.adjust(p_value, method = "BH"),
      signif = p_to_stars(q_value),
      Significant = if_else(!is.na(q_value) & q_value < 0.05, "Yes", "No"),
      Variable_nice = dplyr::recode(
        Variable,
        !!!nice_names,
        .default = Variable
      ),
      Variable_sup = if_else(
        signif == "",
        Variable_nice,
        paste0(Variable_nice, " ", signif)
      )
    )
# ======================================================
# 3b) By-term tests + BH-FDR correction
# ======================================================

# Sequential (Type I) permutation tests.
# Each physicochemical predictor is evaluated in formula order.
# The same farm-restricted permutation design is used for every term.
a_terms <- vegan::anova.cca(
  db,
  by = "term",
  permutations = perm_design
)

a_terms <- as.data.frame(a_terms) %>%
  tibble::rownames_to_column("Variable")

# Keep only the actual model predictors.
# This avoids including rows such as Residual/Model, which may differ
# between vegan versions.
a_terms_clean <- a_terms %>%
  dplyr::filter(Variable %in% retained_predictors) %>%
  dplyr::mutate(
    p_value = as.numeric(.data[["Pr(>F)"]])
  )

# Check: the table must contain exactly the predictors
```

```r
# that passed the VIF filter.
if (nrow(a_terms_clean) != length(retained_predictors)) {
  warning(glue(
    "{title_label}: ANOVA_by_term contains {nrow(a_terms_clean)} terms, ",
    "but the final model contains {length(retained_predictors)} predictors."
  ))
}

# ------------------------------------------------------
# BH-FDR
# IMPORTANT:
# applied separately within THIS
# Domain × Crop model.
# ------------------------------------------------------
a_terms_clean <- a_terms_clean %>%
  dplyr::mutate(
    q_value = stats::p.adjust(
      p_value,
      method = "BH"
    ),

    signif = dplyr::case_when(
      !is.na(q_value) & q_value < 0.001 ~ "***",
      !is.na(q_value) & q_value < 0.01  ~ "**",
      !is.na(q_value) & q_value < 0.05  ~ "*",
      TRUE                              ~ "ns"
    ),

    Significant = dplyr::if_else(
      !is.na(q_value) & q_value < 0.05,
      "Yes",
      "No"
    ),

    Variable_nice = dplyr::recode(
      Variable,
      !!!nice_names,
      .default = Variable
    ),

    # For figures:
    # only FDR-significant terms are marked with stars.
    Variable_plot = dplyr::if_else(
      Significant == "Yes",
      paste0(Variable_nice, " ", signif),
      Variable_nice
    ),

    Domain = domain_label,
    Crop = crop_label
  ) %>%

  # Reorder according to the ACTUAL model formula,
  # not by P value.
  dplyr::mutate(
    Variable = factor(
      Variable,
      levels = retained_predictors
    )
  ) %>%
  dplyr::arrange(Variable) %>%
  dplyr::mutate(
    Variable = as.character(Variable)
  )

# Explicit check
message(glue(
  "  Term tests: {nrow(a_terms_clean)} predictors; ",
```

```
  "{sum(a_terms_clean$Significant == 'Yes', na.rm = TRUE)} ",
  "significant after BH-FDR."
))

print(
  a_terms_clean %>%
    dplyr::select(
      Variable,
      p_value,
      q_value,
      signif,
      Significant
    )
)

  # ====================================================
# 4) Sample and predictor scores
# ====================================================

# ------------------------------------------------------
# Sample scores
# ------------------------------------------------------
scr_sites <- vegan::scores(
  db,
  display = "sites",
  choices = 1:2
)

df_sites <- as.data.frame(scr_sites) %>%
  tibble::rownames_to_column("SampleID")

colnames(df_sites)[2:3] <- c("dbRDA1", "dbRDA2")

df_sites <- df_sites %>%
  dplyr::mutate(
    Domain = domain_label,
    Crop = crop_label,
    Crop_key = crop_key_use,
    Farm = as.character(land_vec),
    Plant_Condition = as.character(
      env_tbl[[params$state_var]]
    )
  ) %>%
  dplyr::select(
    Domain,
    Crop,
    Crop_key,
    SampleID,
    Farm,
    Plant_Condition,
    dbRDA1,
    dbRDA2
  )

# ------------------------------------------------------
# Scores of ALL predictors retained after VIF
# ------------------------------------------------------
scr_bp <- vegan::scores(
  db,
  display = "bp",
  choices = 1:2
)

df_bp <- as.data.frame(scr_bp) %>%
  tibble::rownames_to_column("Variable")

colnames(df_bp)[2:3] <- c("dbRDA1", "dbRDA2")

```

```r
# ------------------------------------------------------
# Join scores with P, q and significance
# ------------------------------------------------------
df_bp <- df_bp %>%
  dplyr::filter(
    Variable %in% retained_predictors
  ) %>%
  dplyr::left_join(
    a_terms_clean %>%
      dplyr::select(
        Variable,
        Variable_nice,
        Variable_plot,
        p_value,
        q_value,
        signif,
        Significant
      ),
    by = "Variable"
  ) %>%
  dplyr::mutate(
    Domain = domain_label,
    Crop = crop_label,

    # Final formula order
    Variable = factor(
      Variable,
      levels = retained_predictors
    )
  ) %>%
  dplyr::arrange(Variable) %>%
  dplyr::mutate(
    Variable = as.character(Variable)
  ) %>%
  dplyr::select(
    Domain,
    Crop,
    Variable,
    Variable_nice,
    Variable_plot,
    dbRDA1,
    dbRDA2,
    p_value,
    q_value,
    signif,
    Significant
  )

# ------------------------------------------------------
# Integrity check
# ------------------------------------------------------
if (nrow(df_bp) != length(retained_predictors)) {
  warning(glue(
    "{title_label}: expected {length(retained_predictors)} ",
    "environmental vectors but obtained {nrow(df_bp)}."
  ))
}

# This object is kept only for tables
# or summaries of statistically significant variables.
df_bp_sig <- df_bp %>%
  dplyr::filter(
    Significant == "Yes"
  )

message(glue(
  "  Environmental vectors retained after VIF: {nrow(df_bp)}"
))
```

```
866
867 message(glue(
868   "  Environmental vectors significant after BH-FDR: {nrow(df_bp_sig)}"
869 ))
870   # ======================================================
871   # 5) Variance of constrained axes
872   # ======================================================
873   eig <- db$CCA$eig
874
875   var1 <- if (length(eig) >= 1) {
876     round(100 * eig[1] / sum(eig), 2)
877   } else {
878     NA_real_
879   }
880
881   var2 <- if (length(eig) >= 2) {
882     round(100 * eig[2] / sum(eig), 2)
883   } else {
884     NA_real_
885   }
886
887   # ======================================================
888   # 6) Full summary for the reviewer response
889   # ======================================================
890   model_summary <- tibble(
891     Domain = domain_label,
892     Crop = crop_label,
893     Distance = "Bray-Curtis",
894     Final_model_formula = final_formula,
895     Variable_selection = "A priori physicochemical variables; iterative VIF filtering only",
896     VIF_threshold = vif_threshold,
897     Predictors_initial = paste(colnames(env_num), collapse = "; "),
898     Predictors_removed_VIF = ifelse(
899       length(removed_predictors) == 0,
900       "None",
901       paste(removed_predictors, collapse = "; ")
902     ),
903     Predictors_retained = paste(retained_predictors, collapse = "; "),
904     n_samples = nrow(otu_mat),
905     n_taxa = ncol(otu_mat),
906     n_farms = nlevels(land_vec),
907     Farm_block = params$strata_var,
908     Permutations = nperm,
909     Permutation_scheme = glue(
910       "{nperm} permutations restricted within {params$strata_var} (farm)"
911     ),
912     Term_test = "Sequential permutation test (anova.cca, by = 'term')",
913     Multiplicity_adjustment = "Benjamini-Hochberg FDR within each Domain x Crop model",
914     Significance_threshold = "q < 0.05",
915     n_env_variables = ncol(env_scaled),
916     dbRDA1_pct = var1,
917     dbRDA2_pct = var2,
918     R2 = r2,
919     R2_adj = r2_adj,
920     F_global = F_global,
921     p_global = p_global,
922     p_global_label = fmt_p(p_global),
923     Significant_terms = sum(
924       a_terms_clean$Significant == "Yes",
925       na.rm = TRUE
926     )
927   )
928
929   # ======================================================
930 # 7) Plot
931 # ======================================================
932
933 plot_sites <- df_sites %>%
```

```
  dplyr::mutate(
    Crop_key = factor(
      Crop_key,
      levels = c(
        "Dominico_Harton",
        "Banano_Gros_Michael"
      )
    ),

    Plant_Condition = factor(
      Plant_Condition,
      levels = c(
        "Healthy_Plant",
        "Diseased_Plant"
      )
    )
  ) %>%
  dplyr::filter(
    !is.na(Crop_key),
    !is.na(Plant_Condition)
  )

# ======================================================
# Global model statistics label
# ======================================================

stats_label <- paste0(
  "R² = ", sprintf("%.3f", r2),
  "\nAdjusted R² = ", sprintf("%.3f", r2_adj),
  "\np = ", fmt_p(p_global)
)

# ACTUAL formula after VIF filtering
formula_label <- paste(
  strwrap(
    paste0(
      "Model: ",
      final_formula
    ),
    width = 95
  ),
  collapse = "\n"
)

figure_caption <- paste0(
  formula_label,
  "\nBray-Curtis distance; ",
  format(
    nperm,
    big.mark = ","
  ),
  " permutations restricted within ",
  params$strata_var,
  " (farm). ",
  "All physicochemical predictors retained after VIF filtering ",
  "(VIF ≤ 10) are shown; stars indicate significance after ",
  "BH-FDR correction (q < 0.05)."
)

# ======================================================
# Base plot
# ======================================================

p <- ggplot(
  plot_sites,
  aes(
    dbRDA1,
    dbRDA2
```

```
  )
) +

  geom_hline(
    yintercept = 0,
    linetype = "dashed",
    linewidth = 0.3
  ) +

  geom_vline(
    xintercept = 0,
    linetype = "dashed",
    linewidth = 0.3
  ) +

  geom_point(
    aes(
      color = Plant_Condition,
      shape = Crop_key
    ),
    size = 6,
    alpha = 0.95,
    stroke = 0.3
  ) +

  scale_color_manual(
    values = pal_condition,
    breaks = c(
      "Healthy_Plant",
      "Diseased_Plant"
    ),
    labels = c(
      "Non symptomatic",
      "Symptomatic"
    ),
    drop = FALSE
  ) +

  scale_shape_manual(
    values = c(
      "Dominico_Harton" = 16,
      "Banano_Gros_Michael" = 17
    ),
    breaks = c(
      "Dominico_Harton",
      "Banano_Gros_Michael"
    ),
    labels = c(
      "Dominico Hartón",
      "Gros Michel"
    ),
    drop = FALSE
  )

# ======================================================
# ALL vectors retained after VIF
# ======================================================

if (nrow(df_bp) > 0) {

  p <- p +

    geom_segment(
      data = df_bp,
      aes(
        x = 0,
        y = 0,
        xend = dbRDA1,
```

```
        yend = dbRDA2,
        alpha = Significant,
        linewidth = Significant
      ),
      inherit.aes = FALSE,
      arrow = arrow(
        length = unit(
          0.20,
          "cm"
        )
      ),
      color = "grey35"
    ) +

    scale_alpha_manual(
      values = c(
        "No" = 0.60,
        "Yes" = 1.00
      ),
      guide = "none"
    ) +

    scale_linewidth_manual(
      values = c(
        "No" = 0.45,
        "Yes" = 0.85
      ),
      guide = "none"
    ) +

    ggrepel::geom_text_repel(
  data = df_bp,
  aes(
    x = dbRDA1,
    y = dbRDA2,
    label = Variable_plot
  ),
  inherit.aes = FALSE,
  size = 3.0,
  fontface = ifelse(
    df_bp$Significant == "Yes",
    "bold",
    "plain"
  ),
  max.overlaps = Inf,
  box.padding = 0.35,
  point.padding = 0.20,
  min.segment.length = 0,
  seed = 123,
  segment.color = "grey60"
)
}

# ====================================================
# Labels, statistics and formula
# ====================================================

p <- p +

  labs(
    title = title_label,

    x = if (!is.na(var1)) {
      glue(
        "dbRDA1 ({var1}%)"
      )
    } else {
      "dbRDA1"
```

```
    },

    y = if (!is.na(var2)) {
      glue(
        "dbRDA2 ({var2}%)"
      )
    } else {
      "dbRDA2"
    },

    color = "Plant condition",
    shape = "Crop",
    caption = figure_caption
  ) +

  theme_bw(
    base_size = 12
  ) +

  theme(
    panel.grid = element_blank(),

    legend.position = "right",

    plot.title = element_text(
      face = "bold",
      hjust = 0.5
    ),

    plot.caption = element_text(
      size = 7.2,
      hjust = 0,
      face = "plain",
      margin = margin(
        t = 8
      )
    )
  )

# ====================================================
# R2, adjusted R2 and global P inside the panel
# ====================================================

p <- p +

  annotate(
    "text",
    x = Inf,
    y = -Inf,
    label = stats_label,
    hjust = 1.05,
    vjust = -0.30,
    size = 3.0,
    lineheight = 1.1
  )

  # ====================================================
# 8) Output objects
# ====================================================

# Checks before returning results
if (nrow(a_terms_clean) == 0) {
  stop(
    glue(
      "{title_label}: ANOVA_by_term is empty. ",
      "Check the extraction of anova.cca() results."
    )
  )
```

```
}

if (nrow(df_bp) == 0) {
  stop(
    glue(
      "{title_label}: biplot_all is empty. ",
      "Check the predictor scores."
    )
  )
}

message(
  glue(
    "  Export QC: ",
    "{nrow(a_terms_clean)} terms | ",
    "{nrow(df_bp)} environmental vectors | ",
    "{nrow(df_sites)} site scores."
  )
)

list(
  model = db,
  plot = p,

  model_summary = model_summary,

  anova_global = a_global,

  # ALL terms retained after VIF
  anova_terms = a_terms_clean,

  # ALL vectors retained after VIF
  biplot_all = df_bp,

  # Only q < 0.05
  biplot_sig = df_bp_sig,

  site_scores = df_sites,

  vif_table = vif_tbl,

  retained_predictors = retained_predictors,

  removed_predictors = removed_predictors,

  permutation_design = perm_design
)
}
```

CHUNK 3 — `main_loop_export_tables`

```{r main_loop_export_tables, message=TRUE, warning=TRUE}
write_dbrda_excel <- function(res, domain, crop_key_use) {

  xlsx_path <- file.path(
    tabdir_for_domain(domain),
    glue("dbRDA_{domain}_{crop_key_use}_Tables.xlsx")
  )

  wb <- openxlsx::createWorkbook()

  hs <- createStyle(
    textDecoration = "bold",
    fgFill = "#D9EAF7",
    border = "Bottom"
  )
```

```
  bold_style <- createStyle(textDecoration = "bold")
  num_style3 <- createStyle(numFmt = "0.000")
  num_style4 <- createStyle(numFmt = "0.0000")

  addWorksheet(wb, "Model_summary")
  writeData(wb, "Model_summary", res$model_summary)
  addStyle(wb, "Model_summary", hs, rows = 1, cols = 1:ncol(res$model_summary), gridExpand = TRUE)
  setColWidths(wb, "Model_summary", cols = 1:ncol(res$model_summary), widths = "auto")

  addWorksheet(wb, "ANOVA_by_term")
  writeData(wb, "ANOVA_by_term", res$anova_terms)
  addStyle(wb, "ANOVA_by_term", hs, rows = 1, cols = 1:ncol(res$anova_terms), gridExpand = TRUE)
  setColWidths(wb, "ANOVA_by_term", cols = 1:ncol(res$anova_terms), widths = "auto")

  sig_rows_terms <- which(res$anova_terms$Significant == "Yes") + 1
  if (length(sig_rows_terms) > 0) {
    addStyle(
      wb, "ANOVA_by_term", bold_style,
      rows = sig_rows_terms, cols = 1:ncol(res$anova_terms), gridExpand = TRUE, stack = TRUE
    )
  }

  addWorksheet(wb, "Supplementary_clean")
  supp_tbl <- res$biplot_all %>%
    mutate(
      Interpretation = case_when(
        Significant == "Yes" ~ "Significant vector in dbRDA after BH-FDR adjustment",
        TRUE                 ~ "Non-significant after BH-FDR adjustment"
      )
    )
  writeData(wb, "Supplementary_clean", supp_tbl)
  addStyle(wb, "Supplementary_clean", hs, rows = 1, cols = 1:ncol(supp_tbl), gridExpand = TRUE)
  setColWidths(wb, "Supplementary_clean", cols = 1:ncol(supp_tbl), widths = "auto")

  sig_rows_supp <- which(supp_tbl$Significant == "Yes") + 1
  if (length(sig_rows_supp) > 0) {
    addStyle(
      wb, "Supplementary_clean", bold_style,
      rows = sig_rows_supp, cols = 1:ncol(supp_tbl), gridExpand = TRUE, stack = TRUE
    )
  }

  addWorksheet(wb, "Site_scores")
  writeData(wb, "Site_scores", res$site_scores)
  addStyle(wb, "Site_scores", hs, rows = 1, cols = 1:ncol(res$site_scores), gridExpand = TRUE)
  setColWidths(wb, "Site_scores", cols = 1:ncol(res$site_scores), widths = "auto")

    addWorksheet(wb, "VIF_Collinearity")
  writeData(wb, "VIF_Collinearity", res$vif_table)
  addStyle(wb, "VIF_Collinearity", hs,
           rows = 1, cols = 1:ncol(res$vif_table), gridExpand = TRUE)
  setColWidths(wb, "VIF_Collinearity",
               cols = 1:ncol(res$vif_table), widths = "auto")

  addWorksheet(wb, "Inference_notes")

inference_notes <- data.frame(
  Item = c(
    "Term-specific test",
    "Permutation scheme",
    "Multiplicity adjustment",
    "Significance criterion"
  ),
  Description = c(
    "Sequential (Type I) permutation tests from vegan::anova.cca(..., by = 'term'); terms are evaluated in model-formula order.",
    glue("{params$nperm} permutations restricted within {params$strata_var} (farm)."),
```

```
    "Benjamini-Hochberg false discovery rate (BH-FDR), applied separately within each Domain x Crop model.",
    "Individual physicochemical predictors are considered statistically significant when q < 0.05."
  )
)

writeData(wb, "Inference_notes", inference_notes)

addStyle(
  wb,
  "Inference_notes",
  hs,
  rows = 1,
  cols = 1:2,
  gridExpand = TRUE
)

setColWidths(
  wb,
  "Inference_notes",
  cols = 1,
  widths = 24
)

setColWidths(
  wb,
  "Inference_notes",
  cols = 2,
  widths = 100
)
  saveWorkbook(wb, xlsx_path, overwrite = TRUE)

  invisible(xlsx_path)
}

run_dbrda_by_crop <- function(domain, crop_value, nperm = params$nperm) {

  domain <- as.character(domain)
  marker_use <- if (domain == "Bacterias") "16S" else "ITS"
  crop_key_use <- normalize_crop_key(crop_value)

  # ---------------------------------------------------------
  # 1) Load phyloseq
  # ---------------------------------------------------------
  ps_path <- ps_path_for_domain(domain)
  if (!file.exists(ps_path)) {
    stop(glue("phyloseq object not found for {domain}: {ps_path}"))
  }

  ps <- readRDS(ps_path)
  ps <- prune_taxa(taxa_sums(ps) > 0, ps)

  # ---------------------------------------------------------
  # 2) Metadata by marker + crop
  # ---------------------------------------------------------
  md_dom <- md_all %>%
    filter(Marker == marker_use) %>%
    mutate(crop_key = normalize_crop_key(.data[[params$crop_var]]))

  md_crop <- md_dom %>% filter(crop_key == crop_key_use)

  # ITS2 no-trunc uses underscores in the technical Illumina suffix,
  # whereas the edaphic Excel uses hyphens. Normalize ONLY fungal/ITS IDs.
  # Bacterial/16S IDs are left exactly as in the accepted analysis.
  if (domain == "Hongos") {
    md_crop <- md_crop %>%
      mutate(`sample-id` = normalize_its_id(`sample-id`))

    if (anyDuplicated(md_crop$`sample-id`)) {
```

```
    stop(glue("{domain}/{crop_value}: duplicated IDs after ITS normalization."))
  }
}

if (nrow(md_crop) == 0) {
  stop(glue("No metadata for {domain} (Marker={marker_use}) with Crop == {crop_value}"))
}

# ---------------------------------------------------------
# 3) Alinear IDs
# ---------------------------------------------------------
common_ids <- intersect(sample_names(ps), md_crop$`sample-id`)
if (length(common_ids) < 5) {
  stop(glue("{domain}/{crop_value}: too few samples in common ({length(common_ids)})."))
}

ps_sub <- prune_samples(common_ids, ps)

env_tbl <- md_crop %>%
  filter(`sample-id` %in% common_ids) %>%
  arrange(match(`sample-id`, sample_names(ps_sub)))

stopifnot(identical(env_tbl$`sample-id`, sample_names(ps_sub)))

# ---------------------------------------------------------
# 4) Correr dbRDA
# ---------------------------------------------------------
domain_label <- pretty_domain(domain)
crop_label   <- pretty_crop(crop_key_use)
title_label  <- glue("{domain_label} — {crop_label}")

# Explicit farm check before running
if (!params$strata_var %in% names(env_tbl)) {
  stop(glue(
    "{domain}/{crop_value}: the farm variable '{params$strata_var}' does not exist."
  ))
}

if (any(is.na(env_tbl[[params$strata_var]]) |
        trimws(as.character(env_tbl[[params$strata_var]])) == "")) {
  stop(glue(
    "{domain}/{crop_value}: '{params$strata_var}' contains NA/empty values."
  ))
}

message(glue(
  "{domain}/{crop_value}: farms included = ",
  "{paste(sort(unique(as.character(env_tbl[[params$strata_var]]))), collapse = ', ')}"
))

res <- run_dbrda(
  ps = ps_sub,
  env_tbl = env_tbl,
  title_label = title_label,
  domain_label = domain_label,
  crop_label = crop_label,
  crop_key_use = crop_key_use,
  nperm = nperm
)

# ---------------------------------------------------------
# 5) Export individual figures
# ---------------------------------------------------------
save_plot_pdf(
  file.path(figdir_for_domain(domain), glue("dbRDA_{domain}_{crop_key_use}.pdf")),
  res$plot, width = 6.8, height = 6
)
```

```
  save_plot_png(
    file.path(figdir_for_domain(domain), glue("dbRDA_{domain}_{crop_key_use}.png")),
    res$plot, width = 6.8, height = 6, dpi = 300
  )

  # ------------------------------------------------------
  # 6) Export simple TSVs
  # ------------------------------------------------------
  readr::write_tsv(
    res$model_summary,
    file.path(tabdir_for_domain(domain), glue("dbRDA_{domain}_{crop_key_use}_model_summary.tsv"))
  )

  readr::write_tsv(
    res$anova_terms,
    file.path(tabdir_for_domain(domain), glue("dbRDA_{domain}_{crop_key_use}_anova_by_term.tsv"))
  )

  readr::write_tsv(
    res$biplot_all,
    file.path(tabdir_for_domain(domain), glue("dbRDA_{domain}_{crop_key_use}_biplot_all.tsv"))
  )

  readr::write_tsv(
    res$site_scores,
    file.path(tabdir_for_domain(domain), glue("dbRDA_{domain}_{crop_key_use}_site_scores.tsv"))
  )

  # ------------------------------------------------------
  # 7) Export individual Excel
  # ------------------------------------------------------
  write_dbrda_excel(res, domain, crop_key_use)

  res
}
```

CHUNK 4 — `multipanel_bac_fungi_and_global_tables`

```{r multipanel_bac_fungi_and_global_tables, message=TRUE, warning=TRUE}
# ============================================================
# Run per domain x crop
# ============================================================
crop_levels <- c("Dominico_Harton", "Banano_Gros_Michael")

res_B_DH <- run_dbrda_by_crop("Bacterias", crop_levels[1])
res_B_GM <- run_dbrda_by_crop("Bacterias", crop_levels[2])
res_H_DH <- run_dbrda_by_crop("Hongos",    crop_levels[1])
res_H_GM <- run_dbrda_by_crop("Hongos",    crop_levels[2])

# ============================================================
# Multipanel 2x2
#   Arriba: Bacteria
#   Abajo : Fungi
#   Left  : Dominico Hartón
#   Right : Gros Michel
# ============================================================
common_theme <- theme_bw(base_size = 12) +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5),
    legend.position = "right",
    panel.grid = element_blank()
  )

p_2x2 <- (res_B_DH$plot + res_B_GM$plot) /
         (res_H_DH$plot + res_H_GM$plot) +
  patchwork::plot_layout(guides = "collect") &
```

```
  common_theme

save_plot_pdf(
  file.path(figdir_global(), "dbRDA_2x2_Bacteria_Fungi_byCrop.pdf"),
  p_2x2, width = 11, height = 9
)

save_plot_png(
  file.path(figdir_global(), "dbRDA_2x2_Bacteria_Fungi_byCrop.png"),
  p_2x2, width = 11, height = 9, dpi = 300
)

print(p_2x2)

# ============================================================
# Consolidated global tables
# ============================================================
# ============================================================
# Consolidated global tables
# ============================================================

all_model_summary <- dplyr::bind_rows(
  res_B_DH$model_summary,
  res_B_GM$model_summary,
  res_H_DH$model_summary,
  res_H_GM$model_summary
)

all_anova_terms <- dplyr::bind_rows(
  res_B_DH$anova_terms,
  res_B_GM$anova_terms,
  res_H_DH$anova_terms,
  res_H_GM$anova_terms
)

all_biplot_all <- dplyr::bind_rows(
  res_B_DH$biplot_all,
  res_B_GM$biplot_all,
  res_H_DH$biplot_all,
  res_H_GM$biplot_all
)

all_site_scores <- dplyr::bind_rows(
  res_B_DH$site_scores,
  res_B_GM$site_scores,
  res_H_DH$site_scores,
  res_H_GM$site_scores
)

all_vif <- dplyr::bind_rows(

  res_B_DH$vif_table %>%
    dplyr::mutate(
      Domain = "Bacteria",
      Crop = "Dominico Hartón",
      .before = 1
    ),

  res_B_GM$vif_table %>%
    dplyr::mutate(
      Domain = "Bacteria",
      Crop = "Gros Michel",
      .before = 1
    ),

  res_H_DH$vif_table %>%
    dplyr::mutate(
      Domain = "Fungi",
```

```
      Crop = "Dominico Hartón",
      .before = 1
    ),

  res_H_GM$vif_table %>%
    dplyr::mutate(
      Domain = "Fungi",
      Crop = "Gros Michel",
      .before = 1
    )
)

# ========================================================
# Full supplementary table
# ========================================================

supp_clean_global <- all_biplot_all %>%
  dplyr::mutate(

    Interpretation = dplyr::case_when(

      Significant == "Yes" ~
        "Significant after BH-FDR correction (q < 0.05)",

      !is.na(p_value) & p_value < 0.05 ~
        "Nominal association (p < 0.05), not significant after BH-FDR",

      TRUE ~
        "Not significant"
    )
  )

# ========================================================
# QC before export
# ========================================================

cat("\n==============================================\n")
cat("QC — GLOBAL dbRDA TABLES\n")
cat("==============================================\n")

cat(
  "Model summaries:",
  nrow(all_model_summary),
  "\n"
)

cat(
  "ANOVA terms:",
  nrow(all_anova_terms),
  "\n"
)

cat(
  "Environmental vectors:",
  nrow(all_biplot_all),
  "\n"
)

cat(
  "Site scores:",
  nrow(all_site_scores),
  "\n"
)

cat(
  "BH-FDR significant terms:",
  sum(
    all_anova_terms$Significant == "Yes",
```

```
    na.rm = TRUE
  ),
  "\n"
)

cat("============================================\n")

# Do not allow exporting an empty Excel workbook
if (nrow(all_model_summary) == 0) {
  stop("ERROR: all_model_summary is empty.")
}

if (nrow(all_anova_terms) == 0) {
  stop("ERROR: all_anova_terms is empty.")
}

if (nrow(all_biplot_all) == 0) {
  stop("ERROR: all_biplot_all is empty.")
}

if (nrow(all_site_scores) == 0) {
  stop("ERROR: all_site_scores is empty.")
}
# ========================================================
# Global TSV export
# ========================================================
readr::write_tsv(
  all_model_summary,
  file.path(tabdir_global(), "dbRDA_global_model_summary.tsv")
)

readr::write_tsv(
  all_anova_terms,
  file.path(tabdir_global(), "dbRDA_global_anova_by_term.tsv")
)

readr::write_tsv(
  supp_clean_global,
  file.path(tabdir_global(), "dbRDA_global_supplementary_clean.tsv")
)

readr::write_tsv(
  all_site_scores,
  file.path(tabdir_global(), "dbRDA_global_site_scores.tsv")
)

readr::write_tsv(
  all_vif,
  file.path(tabdir_global(), "dbRDA_global_VIF_collinearity.tsv")
)

# ========================================================
# Excel global consolidado
# ========================================================
wb_global <- createWorkbook()

hs <- createStyle(
  textDecoration = "bold",
  fgFill = "#D9EAF7",
  border = "Bottom"
)
bold_style <- createStyle(textDecoration = "bold")

addWorksheet(wb_global, "Model_summary")
writeData(wb_global, "Model_summary", all_model_summary)
addStyle(wb_global, "Model_summary", hs, rows = 1, cols = 1:ncol(all_model_summary), gridExpand = TRUE)
setColWidths(wb_global, "Model_summary", cols = 1:ncol(all_model_summary), widths = "auto")

```

```
addWorksheet(wb_global, "ANOVA_by_term")
writeData(wb_global, "ANOVA_by_term", all_anova_terms)
addStyle(wb_global, "ANOVA_by_term", hs, rows = 1, cols = 1:ncol(all_anova_terms), gridExpand = TRUE)
setColWidths(wb_global, "ANOVA_by_term", cols = 1:ncol(all_anova_terms), widths = "auto")

sig_rows_terms <- which(all_anova_terms$Significant == "Yes") + 1
if (length(sig_rows_terms) > 0) {
  addStyle(
    wb_global, "ANOVA_by_term", bold_style,
    rows = sig_rows_terms, cols = 1:ncol(all_anova_terms), gridExpand = TRUE, stack = TRUE
  )
}

addWorksheet(wb_global, "Supplementary_clean")
writeData(wb_global, "Supplementary_clean", supp_clean_global)
addStyle(wb_global, "Supplementary_clean", hs, rows = 1, cols = 1:ncol(supp_clean_global), gridExpand = TRUE)
setColWidths(wb_global, "Supplementary_clean", cols = 1:ncol(supp_clean_global), widths = "auto")

sig_rows_supp <- which(supp_clean_global$Significant == "Yes") + 1
if (length(sig_rows_supp) > 0) {
  addStyle(
    wb_global, "Supplementary_clean", bold_style,
    rows = sig_rows_supp, cols = 1:ncol(supp_clean_global), gridExpand = TRUE, stack = TRUE
  )
}

addWorksheet(wb_global, "Site_scores")
writeData(wb_global, "Site_scores", all_site_scores)
addStyle(wb_global, "Site_scores", hs, rows = 1, cols = 1:ncol(all_site_scores), gridExpand = TRUE)
setColWidths(wb_global, "Site_scores", cols = 1:ncol(all_site_scores), widths = "auto")

addWorksheet(wb_global, "VIF_Collinearity")
writeData(wb_global, "VIF_Collinearity", all_vif)
addStyle(wb_global, "VIF_Collinearity", hs, rows = 1, cols = 1:ncol(all_vif), gridExpand = TRUE)
setColWidths(wb_global, "VIF_Collinearity", cols = 1:ncol(all_vif), widths = "auto")

addWorksheet(wb_global, "Inference_notes")
inference_notes_global <- data.frame(
  Item = c(
    "Term-specific test",
    "Permutation scheme",
    "Multiplicity adjustment",
    "Significance criterion"
  ),
  Description = c(
    "Sequential (Type I) permutation tests from vegan::anova.cca(..., by = 'term'); terms are evaluated in model-formula order.",
    glue("{params$nperm} permutations restricted within {params$strata_var} (farm)."),
    "Benjamini-Hochberg false discovery rate (BH-FDR), applied separately within each Domain x Crop model before tables were combined.",
    "Individual physicochemical predictors are considered statistically significant when q < 0.05."
  )
)
writeData(wb_global, "Inference_notes", inference_notes_global)
addStyle(wb_global, "Inference_notes", hs, rows = 1, cols = 1:2, gridExpand = TRUE)
setColWidths(wb_global, "Inference_notes", cols = 1, widths = 24)
setColWidths(wb_global, "Inference_notes", cols = 2, widths = 100)

saveWorkbook(
  wb_global,
  file.path(tabdir_global(), "dbRDA_Global_Consolidated_Tables.xlsx"),
  overwrite = TRUE
)

message("dbRDA completed. Figures and Tables exported per domain and globally.")

```

```
## Session information

```{r session_info}
sessionInfo()
```
```

Annex B — Soil physicochemical analyses

Source listings of the three final soil-chemistry scripts.

B1	`Analisis_quimico.Rmd`	QC, PCA, correlations, Random Forest
B2	`PERMANOVA_Fisicoquimicos.Rmd`	Design-aware PERMANOVA, PERMDISP
B3	`07_MANOVA_Quimica_padjust_TableS3.Rmd`	MANOVA + BH-adjusted ANOVAs

06 — Soil chemistry — PCA, correlations, PERMANOVA and Random Forest by crop (elephantiasis; edaphic variables)

Supplementary analysis script

Sergio Pardo-Díaz

Contents

[OK] CHUNK 1 — setup . . . 2
CHUNK 2 — helpers + metadata + outputs . . . 2
CHUNK 3 — dataset builder . . . 5
CHUNK 3.1 — Preliminary physicochemical QC: skewness and outliers . . . 6
CHUNK 4 — PCA by crop . . . 7
CHUNK 5 — binary correlation by crop . . . 9
CHUNK 5.1 — pairwise Pearson correlations among edaphic variables by crop and condition . . . 12
CHUNK 6 — PERMANOVA by crop . . . 17
CHUNK 7 — Random Forest by crop — farm-blocked validation . . . 18
CHUNK 8 — summary table for the paper . . . 23
CHUNK 9 — final summary . . . 24
Session information . . . 24

Script: `Physicochemical/Analisis_quimico.Rmd`
Rendered: 2026-10-13 — code listing (the R Markdown source is shown as written; chunks were not executed, so no outputs are included).

YAML header / parameters

```yaml
title: "06 — Soil chemistry — PCA, correlations, PERMANOVA and Random Forest by crop (elephantiasis;
↪  edaphic variables)"
author: "Sergio Pardo-Díaz"
date: "`r format(Sys.time(), '%Y-%m-%d %H:%M')`"
output:
  html_document:
    toc: true
    toc_depth: 3
    number_sections: true
  pdf_document:
    toc: true
    number_sections: true

params:
  project_root: "."
  analysis_subdir: "03_Beta-Diversity"

  # Single edaphic metadata table (no 16S/ITS duplication)
  metadata_edaphic_path: "Metadata_Completo_plus_edaphic.xlsx"

  # Variables of interest
  state_var: "Plant_Condition"
  crop_var: "Crop"
  land_var: "Land"

  # Reviewer-ready diagnostics / Random Forest validation
  skewness_threshold: 1
  univariate_outlier_iqr_multiplier: 1.5
  rf_ntree: 1000
  rf_perm: 499
  rf_seed: 2026
```

```yaml
  # Target crops
  crop_levels: ["Dominico_Harton", "Banano_Gros_Michael"]

  # Edaphic variables
  vars_edaphic:
    - pH
    - Organic_M
    - P
    - Ca_Exchange
    - Mg_Exchange
    - K_Exchange
    - CEC
    - Zn
    - Cu
    - B
    - N_Total
    - Respirometry

  # Output subfolder
  out_subdir: "06_Quimica_PCA"
```

Reviewer-ready changes

This version explicitly inspects physicochemical variables for skewness and Tukey-IQR univariate outliers, documents that no automatic transformations are applied, retains centered/scaled PCA, and replaces sample-level Random Forest validation with leave-one-farm-out cross-validation. Random Forest condition-label permutations are restricted within `Land`. The binary condition coding remains Healthy = 0 and Diseased = 1 for point-biserial correlations, with Benjamini-Hochberg correction within crop.

[OK] CHUNK 1 — setup

r chunk: setup — options: message=TRUE, warning=TRUE

```r
knitr::opts_chunk$set(
  fig.width  = 7,
  fig.height = 6,
  dpi        = 300,
  dev        = "cairo_pdf"
)

suppressPackageStartupMessages({
  library(conflicted)
  library(tidyverse)
  library(glue)
  library(patchwork)
  library(ggrepel)
  library(readxl)
  library(vegan)
  library(randomForest)
  library(openxlsx)
  library(ggnewscale)
  library(grid)
  library(scales)
  library(readr)
})

conflicted::conflicts_prefer(dplyr::select)
conflicted::conflicts_prefer(dplyr::filter)
conflicted::conflicts_prefer(dplyr::mutate)
```

CHUNK 2 — helpers + metadata + outputs

r chunk: helpers_and_metadata — options: message=TRUE, warning=TRUE

```r
# ============================================================
# Helpers
# ============================================================
.require_param <- function(x, name) {
```

```
  if (is.null(x) || length(x) == 0 || is.na(x) || x == "") {
    stop(glue("Param '{name}' is empty/NULL. Check the YAML header."))
  }
  x
}

strip_accents <- function(x) {
  x <- as.character(x)
  iconv(x, from = "UTF-8", to = "ASCII//TRANSLIT")
}

harmonize_condition <- function(x) {
  x2 <- as.character(x)
  x2 <- trimws(x2)
  x2 <- gsub("-", "_", x2)
  x2 <- gsub(" ", "_", x2)
  xlow <- tolower(x2)

  x2[xlow %in% c("healthy_plant", "healthy")]   <- "Healthy_Plant"
  x2[xlow %in% c("diseased_plant", "diseased")] <- "Diseased_Plant"

  factor(x2, levels = c("Healthy_Plant", "Diseased_Plant"))
}

normalize_crop_key <- function(x) {
  x2 <- as.character(x)
  x2 <- trimws(x2)
  x2 <- strip_accents(x2)
  x2 <- gsub("-", "_", x2)
  x2 <- gsub(" ", "_", x2)

  xlow <- tolower(x2)
  x2[xlow %in% c("gros_michael", "banano_gros_michael", "banano_gros_michel")] <- "Banano_Gros_Michael"
  x2[xlow %in% c("dominico_harton", "dominico")]                               <- "Dominico_Harton"

  x2
}

pretty_crop <- function(x) {
  x <- normalize_crop_key(x)
  dplyr::case_when(
    x == "Dominico_Harton"     ~ "Dominico Hartón",
    x == "Banano_Gros_Michael" ~ "Gros Michel",
    TRUE ~ x
  )
}

fmt_p <- function(p) {
  ifelse(is.na(p), NA_character_, format.pval(p, digits = 2, eps = 0.001))
}

p_to_stars <- function(p) {
  dplyr::case_when(
    is.na(p)  ~ "",
    p < 0.001 ~ "***",
    p < 0.01  ~ "**",
    p < 0.05  ~ "*",
    TRUE      ~ ""
  )
}

# ============================================================
# Palettes
# ============================================================
pal_condition <- c(
  "Healthy_Plant"  = "#00BFC4",
  "Diseased_Plant" = "#F8766D"
)

grad_contrib <- c("#2166AC", "#67A9CF", "#FDDBC7", "#EF8A62", "#B2182B")
grad_corr    <- c("#053061", "#4393C3", "#F7F7F7", "#D6604D", "#67001F")

# ============================================================
# Edaphic variables from YAML
# ============================================================
vars_edaphic <- unlist(params$vars_edaphic)
```

```
nice_names <- c(
  "pH"           = "pH",
  "Organic_M"    = "Organic matter",
  "P"            = "P",
  "Ca_Exchange"  = "Ca2+",
  "Mg_Exchange"  = "Mg2+",
  "K_Exchange"   = "K+",
  "CEC"          = "CEC",
  "Zn"           = "Zn2+",
  "Cu"           = "Cu2+",
  "B"            = "B",
  "N_Total"      = "Total N",
  "Respirometry" = "Respirometry"
)

# ============================================================
# Outputs
# ============================================================
outdir_global <- function() {
  file.path(
    .require_param(params$project_root, "project_root"),
    .require_param(params$analysis_subdir, "analysis_subdir"),
    .require_param(params$out_subdir, "out_subdir")
  )
}

dir.create(outdir_global(), recursive = TRUE, showWarnings = FALSE)

subdirs_global <- c("PCA", "Pearson", "PERMANOVA", "RandomForest", "Tables")
for (s in subdirs_global) {
  dir.create(file.path(outdir_global(), s), recursive = TRUE, showWarnings = FALSE)
}

# ============================================================
# Read metadata
# ============================================================
metadata_path <- file.path(params$project_root, params$metadata_edaphic_path)
if (!file.exists(metadata_path)) stop(glue("metadata_edaphic_path not found: {metadata_path}"))

md_all <- readxl::read_xlsx(metadata_path)

if (!"sample-id" %in% names(md_all)) {
  stop("The Excel file must contain the EXACT column: 'sample-id'")
}

required_metadata_cols <- c("sample-id", params$state_var, params$crop_var, params$land_var)
missing_metadata_cols <- setdiff(required_metadata_cols, names(md_all))
if (length(missing_metadata_cols) > 0) {
  stop(glue("Required columns missing from metadata: {paste(missing_metadata_cols, collapse = ', ')}"))
}

md_all <- md_all %>%
  mutate(
    `sample-id` = as.character(`sample-id`),
    !!params$state_var := harmonize_condition(.data[[params$state_var]]),
    !!params$crop_var  := normalize_crop_key(.data[[params$crop_var]]),
    !!params$land_var  := factor(trimws(as.character(.data[[params$land_var]])))
  ) %>%
  filter(
    !is.na(.data[[params$state_var]]),
    !is.na(.data[[params$crop_var]]),
    !is.na(.data[[params$land_var]]),
    trimws(as.character(.data[[params$land_var]])) != ""
  )

stopifnot(all(c(params$state_var, params$crop_var, params$land_var) %in% names(md_all)))

message(glue("Metadata loaded: {nrow(md_all)} rows, {ncol(md_all)} columns."))
message("Plant condition counts:")
print(table(md_all[[params$state_var]], useNA = "ifany"))
message("Crop counts:")
print(table(md_all[[params$crop_var]], useNA = "ifany"))

vars_present <- intersect(vars_edaphic, names(md_all))
vars_missing <- setdiff(vars_edaphic, names(md_all))
```

```r
message("\nExpected edaphic variables: ", length(vars_edaphic))
message("[OK] Edaphic variables present: ", length(vars_present))
if (length(vars_missing) > 0) {
  message("[!] Edaphic variables missing from Excel: ", paste(vars_missing, collapse = ", "))
}

crop_levels <- normalize_crop_key(unlist(params$crop_levels))
message("\nTarget crops (normalized): ", paste(crop_levels, collapse = " | "))
```

CHUNK 3 — dataset builder

r chunk: build_datasets — options: message=TRUE, warning=TRUE

```r
build_edaphic_dataset <- function(crop_key) {

  crop_key <- normalize_crop_key(crop_key)

  md_sub <- md_all %>%
    dplyr::mutate(Crop_key = normalize_crop_key(.data[[params$crop_var]])) %>%
    dplyr::filter(Crop_key == crop_key) %>%
    dplyr::select(
      `sample-id`,
      dplyr::all_of(params$state_var),
      dplyr::all_of(params$land_var),
      Crop_key,
      dplyr::any_of(vars_present)
    ) %>%
    dplyr::mutate(
      dplyr::across(dplyr::any_of(vars_present), ~ suppressWarnings(as.numeric(.x)))
    )

  if (nrow(md_sub) == 0) {
    stop(glue("{crop_key}: no rows in metadata."))
  }

  ed_cols <- intersect(vars_present, names(md_sub))
  ed_keep <- ed_cols[colSums(!is.na(md_sub[ed_cols])) > 0]

  if (length(ed_keep) < 2) {
    stop(glue("{crop_key}: <2 valid edaphic variables after removing NA columns."))
  }

  md_sub2 <- md_sub %>%
    dplyr::select(
      `sample-id`,
      dplyr::all_of(params$state_var),
      dplyr::all_of(params$land_var),
      Crop_key,
      dplyr::all_of(ed_keep)
    )

  cc <- stats::complete.cases(md_sub2[, ed_keep, drop = FALSE])
  if (any(!cc)) {
    message(glue(">> {crop_key}: removed {sum(!cc)} samples with NA in edaphic variables."))
    md_sub2 <- md_sub2[cc, , drop = FALSE]
  }

  md_sub2 <- md_sub2 %>%
    dplyr::mutate(
      !!params$state_var := droplevels(.data[[params$state_var]]),
      !!params$land_var := droplevels(factor(.data[[params$land_var]])),
      Crop_key = factor(Crop_key, levels = crop_levels)
    )

  n_h <- sum(md_sub2[[params$state_var]] == "Healthy_Plant", na.rm = TRUE)
  n_d <- sum(md_sub2[[params$state_var]] == "Diseased_Plant", na.rm = TRUE)

  if (nrow(md_sub2) < 5) {
    stop(glue("{crop_key}: too few samples after NA filtering ({nrow(md_sub2)})."))
  }

  list(
```

```
    md          = md_sub2,
    ed_cols     = ed_keep,
    crop        = crop_key,
    n_total     = nrow(md_sub2),
    n_healthy   = n_h,
    n_diseased  = n_d
  )
}

# base datasets
ds_DH <- build_edaphic_dataset(crop_levels[1])
ds_GM <- build_edaphic_dataset(crop_levels[2])
```

CHUNK 3.1 — Preliminary physicochemical QC: skewness and outliers

r chunk: physicochemical_qc — options: message=TRUE, warning=TRUE

```
# ============================================================
# Reviewer-ready QC of physicochemical variables
# - sample skewness (moment coefficient)
# - univariate outliers by Tukey IQR rule
# - transformations are NOT applied automatically
# - PCA/MANOVA/dbRDA scaling is handled in their own analyses
# ============================================================

sample_skewness <- function(x) {
  x <- x[is.finite(x)]
  n <- length(x)
  if (n < 3 || stats::sd(x) == 0) return(NA_real_)
  z <- (x - mean(x)) / stats::sd(x)
  (n / ((n - 1) * (n - 2))) * sum(z^3)
}

qc_physicochemical <- function(ds) {
  md <- ds$md
  vars <- ds$ed_cols
  crop_lab <- pretty_crop(ds$crop)
  k <- params$univariate_outlier_iqr_multiplier
  skew_thr <- params$skewness_threshold

  purrr::map_dfr(vars, function(v) {
    x <- suppressWarnings(as.numeric(md[[v]]))
    x_ok <- x[is.finite(x)]

    q1 <- stats::quantile(x_ok, 0.25, na.rm = TRUE, names = FALSE)
    q3 <- stats::quantile(x_ok, 0.75, na.rm = TRUE, names = FALSE)
    iqr <- q3 - q1
    lower <- q1 - k * iqr
    upper <- q3 + k * iqr
    out_idx <- which(is.finite(x) & (x < lower | x > upper))
    sk <- sample_skewness(x_ok)

    tibble(
      Crop = crop_lab,
      Variable = v,
      Variable_label = dplyr::coalesce(unname(nice_names[v]), v),
      N = length(x_ok),
      Mean = mean(x_ok),
      SD = stats::sd(x_ok),
      Median = stats::median(x_ok),
      Min = min(x_ok),
      Max = max(x_ok),
      Skewness = sk,
      Abs_skewness = abs(sk),
      Skewness_flag = if_else(!is.na(sk) & abs(sk) >= skew_thr, "Yes", "No"),
      IQR_lower = lower,
      IQR_upper = upper,
      N_IQR_outliers = length(out_idx),
      Outlier_sample_ids = if (length(out_idx) == 0) "None" else paste(md$`sample-id`[out_idx], collapse =
      ↪  "; "),
      Transformation_applied = "None"
    )
  })
```

```
}

qc_DH <- qc_physicochemical(ds_DH)
qc_GM <- qc_physicochemical(ds_GM)
qc_physchem <- bind_rows(qc_DH, qc_GM)

qc_dir <- file.path(outdir_global(), "Tables")
readr::write_tsv(qc_physchem, file.path(qc_dir, "Physicochemical_QC_skewness_outliers.tsv"))

wb_qc <- openxlsx::createWorkbook()
openxlsx::addWorksheet(wb_qc, "QC_skewness_outliers")
openxlsx::writeData(wb_qc, "QC_skewness_outliers", qc_physchem)
openxlsx::saveWorkbook(
  wb_qc,
  file.path(qc_dir, "Physicochemical_QC_skewness_outliers.xlsx"),
  overwrite = TRUE
)

message("Physicochemical QC exported. No automatic transformations were applied.")
qc_physchem
```

CHUNK 4 — PCA by crop

r chunk: pca_2panels — options: message=TRUE, warning=TRUE

```
# ============================================================
# PCA by crop
# - no ellipses
# - shapes = crop
# - vectors colored by contribution ("factoextra-like")
# ============================================================
plot_pca_panel <- function(ds, show_legend_cond = FALSE, show_legend_contrib = FALSE, show_legend_crop =
↪  FALSE) {

  md   <- ds$md
  cols <- ds$ed_cols

  md$Crop_key <- factor(as.character(md$Crop_key), levels = crop_levels)

  mat <- as.matrix(md[, cols, drop = FALSE])
  rownames(mat) <- md$`sample-id`

  pca <- prcomp(mat, center = TRUE, scale. = TRUE)

  var_exp <- (pca$sdev^2) / sum(pca$sdev^2)
  pc1 <- round(100 * var_exp[1], 1)
  pc2 <- round(100 * var_exp[2], 1)

  ind_df <- as.data.frame(pca$x[, 1:2, drop = FALSE]) %>%
    tibble::rownames_to_column("sample-id") %>%
    dplyr::mutate(
      Plant_Condition = md[[params$state_var]],
      Crop_key        = md$Crop_key
    )
  colnames(ind_df)[2:3] <- c("PC1", "PC2")

  var_coord <- sweep(pca$rotation[, 1:2, drop = FALSE], 2, pca$sdev[1:2], FUN = "*")
  var_df <- as.data.frame(var_coord) %>%
    tibble::rownames_to_column("Variable")
  colnames(var_df)[2:3] <- c("PC1", "PC2")

  contrib_pc1 <- 100 * (var_df$PC1^2) / sum(var_df$PC1^2)
  contrib_pc2 <- 100 * (var_df$PC2^2) / sum(var_df$PC2^2)
  var_df$Contribution <- contrib_pc1 + contrib_pc2
  var_df$Variable_lab <- dplyr::coalesce(unname(nice_names[var_df$Variable]), var_df$Variable)

  max_ind <- max(abs(ind_df$PC1), abs(ind_df$PC2), na.rm = TRUE)
  max_var <- max(abs(var_df$PC1), abs(var_df$PC2), na.rm = TRUE)
  vec_scale   <- if (max_var > 0) (max_ind * 0.82) / max_var else 1
  label_scale <- vec_scale * 1.10

  title_lab <- pretty_crop(ds$crop)
```

```r
g <- ggplot() +
  geom_hline(yintercept = 0, linetype = "dashed", linewidth = 0.3) +
  geom_vline(xintercept = 0, linetype = "dashed", linewidth = 0.3) +
  geom_point(
    data = ind_df,
    mapping = aes(x = PC1, y = PC2, color = Plant_Condition, shape = Crop_key),
    size = 6, alpha = 0.92
  ) +
  scale_color_manual(
    values = pal_condition,
    labels = c("Healthy", "Diseased"),
    drop = FALSE,
    name = "Plant condition"
  ) +
  scale_shape_manual(
    values = c(
      "Dominico_Harton"     = 16,
      "Banano_Gros_Michael" = 17
    ),
    labels = c("Dominico Hartón", "Gros Michel"),
    drop = FALSE,
    name = "Crop"
  ) +
  ggnewscale::new_scale_color() +
  geom_segment(
    data = var_df,
    mapping = aes(
      x = 0, y = 0,
      xend = PC1 * vec_scale,
      yend = PC2 * vec_scale,
      color = Contribution
    ),
    arrow = arrow(length = unit(0.22, "cm")),
    linewidth = 0.85
  ) +
  ggrepel::geom_text_repel(
    data = var_df,
    mapping = aes(
      x = PC1 * label_scale,
      y = PC2 * label_scale,
      label = Variable_lab,
      color = Contribution
    ),
    size = 3.1,
    fontface = "bold",
    max.overlaps = Inf,
    show.legend = FALSE
  ) +
  scale_color_gradientn(
    colours = grad_contrib,
    name = "Contribution\n(PC1 + PC2)"
  ) +
  labs(
    title = title_lab,
    x = glue("PC1 ({pc1}%)"),
    y = glue("PC2 ({pc2}%)")
  ) +
  theme_bw(base_size = 12, base_family = "Helvetica") +
  theme(
    panel.grid = element_blank(),
    plot.title = element_text(hjust = 0.5, face = "bold")
  )

show_any <- (show_legend_cond || show_legend_contrib || show_legend_crop)
g <- g + theme(legend.position = if (show_any) "right" else "none")

g <- g + guides(
  shape = if (show_legend_crop) guide_legend(order = 2) else "none",
  color = if (show_legend_contrib) guide_colorbar(order = 3) else "none"
)

eig_tbl <- tibble(
  Crop   = pretty_crop(ds$crop),
  PC     = c("PC1", "PC2"),
  Variance_pct = c(pc1, pc2)
)
```

```
  list(plot = g, pca = pca, ind = ind_df, var = var_df, eig = eig_tbl)
}

p_DH <- plot_pca_panel(ds_DH, FALSE, FALSE, FALSE)
p_GM <- plot_pca_panel(ds_GM, TRUE, TRUE, TRUE)

p_pca_2 <- (p_DH$plot | p_GM$plot) +
  patchwork::plot_layout(guides = "collect") &
  theme(legend.position = "right")

print(p_pca_2)

pca_dir_g <- file.path(outdir_global(), "PCA")
ggsave(file.path(pca_dir_g, "PCA_edaphic_byCrop.pdf"),
       p_pca_2, width = 12, height = 6, dpi = 300)
ggsave(file.path(pca_dir_g, "PCA_edaphic_byCrop.png"),
       p_pca_2, width = 12, height = 6, dpi = 300)

eig_all <- bind_rows(p_DH$eig, p_GM$eig)
write_tsv(eig_all, file.path(pca_dir_g, "PCA_axis_variance.tsv"))

# Export variable contributions to the PC1-PC2 plane
contrib_all <- bind_rows(
  p_DH$var %>% mutate(Crop = pretty_crop(ds_DH$crop)),
  p_GM$var %>% mutate(Crop = pretty_crop(ds_GM$crop))
) %>%
  select(Crop, Variable, Variable_lab, PC1, PC2, Contribution) %>%
  arrange(Crop, desc(Contribution))

write_tsv(
  contrib_all,
  file.path(pca_dir_g, "PCA_variable_contributions_PC1_PC2.tsv")
)

message("[OK] PCA by crop exported to: ", pca_dir_g)
```

CHUNK 5 — binary correlation by crop

r chunk: disease_binary_correlation — options: message=TRUE, warning=TRUE, fig.width=12, fig.height=6

```
# ============================================================
# Correlation of each physicochemical variable with binary disease status
# Healthy = 0 | Diseased = 1
# Pearson = point-biserial correlation
# By Crop
# Figure:
#   - stars = raw p
#   - black outline = q < 0.05
# ============================================================

sig_star <- function(x){
  dplyr::case_when(
    is.na(x)  ~ "",
    x < 0.001 ~ "***",
    x < 0.01  ~ "**",
    x < 0.05  ~ "*",
    TRUE      ~ ""
  )
}

run_binary_disease_cor_panel <- function(ds, show_y = TRUE) {

  md      <- ds$md
  ed_cols <- ds$ed_cols

  keep <- as.character(md[[params$state_var]]) %in% c("Healthy_Plant", "Diseased_Plant")
  md2  <- md[keep, , drop = FALSE]

  if (nrow(md2) < 6) {
    panel_title <- pretty_crop(ds$crop)
    empty <- ggplot() + theme_void() + ggtitle(panel_title)
    return(list(
```

```
    plot = empty,
    stats = tibble(
      Crop = pretty_crop(ds$crop),
      Variable = character(),
      Variable_lab = character(),
      r = numeric(),
      p = numeric(),
      q = numeric(),
      n = numeric(),
      abs_r = numeric(),
      Direction = character(),
      sig_p = character(),
      sig_q = character(),
      q_sig = logical()
    )
  ))
}

md2 <- md2 %>%
  mutate(
    Disease_binary = ifelse(.data[[params$state_var]] == "Diseased_Plant", 1, 0)
  )

df_num <- md2[, ed_cols, drop = FALSE] %>%
  mutate(across(everything(), ~ suppressWarnings(as.numeric(.x))))

ed_keep <- ed_cols[colSums(!is.na(df_num)) > 0]
df_num  <- df_num[, ed_keep, drop = FALSE]
vars    <- names(df_num)

cor_one <- function(v) {
  x <- df_num[[v]]
  y <- md2$Disease_binary
  ok <- complete.cases(x, y)
  n  <- sum(ok)

  if (n < 6 || length(unique(x[ok])) < 2 || length(unique(y[ok])) < 2) {
    return(tibble(Variable = v, r = NA_real_, p = NA_real_, n = n))
  }

  ct <- suppressWarnings(cor.test(x[ok], y[ok], method = "pearson"))

  tibble(
    Variable = v,
    r = unname(ct$estimate),
    p = ct$p.value,
    n = n
  )
}

stats_tbl <- bind_rows(lapply(vars, cor_one)) %>%
  mutate(
    Crop   = pretty_crop(ds$crop),
    q      = p.adjust(p, method = "BH"),
    sig_p  = sig_star(p),
    sig_q  = sig_star(q),
    q_sig  = !is.na(q) & q < 0.05,
    abs_r  = abs(r),
    Direction = case_when(
      is.na(r) ~ NA_character_,
      r > 0    ~ "Positive with disease",
      r < 0    ~ "Negative with disease",
      TRUE     ~ "Neutral"
    ),
    Variable_lab = dplyr::coalesce(unname(nice_names[Variable]), Variable)
  ) %>%
  select(Crop, Variable, Variable_lab, r, p, q, n, abs_r, Direction, sig_p, sig_q, q_sig) %>%
  arrange(desc(abs_r))

var_order <- stats_tbl %>%
  arrange(r) %>%
  pull(Variable_lab)

plot_df <- stats_tbl %>%
  mutate(
    Variable_lab = factor(Variable_lab, levels = var_order),
```

```
      Effect = "Disease\n(Healthy=0; Diseased=1)"
    )

  panel_title <- pretty_crop(ds$crop)

  g <- ggplot(plot_df, aes(x = Variable_lab, y = Effect)) +
    geom_tile(fill = "white", color = "grey80", width = 0.92, height = 0.92) +
    geom_point(
      aes(fill = r, size = abs_r),
      shape = 21,
      color = "grey30",
      stroke = 0.5,
      alpha = 0.95
    ) +
    geom_point(
      data = subset(plot_df, q_sig),
      aes(size = abs_r),
      shape = 21,
      fill = NA,
      color = "black",
      stroke = 1.4
    ) +
    geom_text(
      data = subset(plot_df, sig_p != ""),
      aes(label = sig_p),
      color = "black",
      size = 4.2,
      fontface = "bold",
      vjust = 0.35
    ) +
    scale_size_continuous(range = c(3.0, 11.0), guide = "none") +
    scale_fill_gradientn(
      colours = grad_corr,
      limits  = c(-1, 1),
      oob     = scales::squish,
      name    = "Pearson r\nwith disease"
    ) +
    guides(
      fill = guide_colorbar(
        title.position = "top",
        title.hjust = 0.5,
        barwidth = unit(8, "cm"),
        barheight = unit(0.55, "cm")
      )
    ) +
    scale_x_discrete(expand = expansion(add = 0.9)) +
    scale_y_discrete(expand = expansion(add = 0.6)) +
    coord_fixed(clip = "off") +
    labs(title = panel_title, x = NULL, y = NULL) +
    theme_bw(base_size = 12, base_family = "Helvetica") +
    theme(
      panel.grid = element_blank(),
      axis.text.x = element_text(angle = 45, hjust = 1),
      axis.text.y = if (show_y) element_text() else element_blank(),
      axis.ticks.y = if (show_y) element_line() else element_blank(),
      plot.title = element_text(hjust = 0.5, face = "bold"),
      plot.margin = ggplot2::margin(10, 20, 10, 10)
    )

  list(plot = g, stats = stats_tbl)
}

cor_DH <- run_binary_disease_cor_panel(ds_DH, show_y = TRUE)
cor_GM <- run_binary_disease_cor_panel(ds_GM, show_y = TRUE)

p_cor_2 <- (cor_DH$plot | cor_GM$plot) +
  plot_layout(guides = "collect") &
  theme(legend.position = "bottom", legend.direction = "horizontal")

print(p_cor_2)

cor_dir_g <- file.path(outdir_global(), "Pearson")
ggsave(
  file.path(cor_dir_g, "Correlation_DiseaseBinary_vs_Edaphic_byCrop.pdf"),
  p_cor_2, width = 13, height = 6, dpi = 300
)
```

```r
ggsave(
  file.path(cor_dir_g, "Correlation_DiseaseBinary_vs_Edaphic_byCrop.png"),
  p_cor_2, width = 13, height = 6, dpi = 300
)

cor_all <- bind_rows(cor_DH$stats, cor_GM$stats)

cor_paper_tbl <- cor_all %>%
  transmute(
    Crop,
    Variable = Variable_lab,
    r = round(r, 3),
    p_raw = fmt_p(p),
    q_fdr = fmt_p(q),
    n,
    Direction,
    sig_p = ifelse(sig_p == "", "ns", sig_p),
    sig_q = ifelse(sig_q == "", "ns", sig_q),
    q_sig = ifelse(q_sig, "Yes", "No")
  )

readr::write_tsv(cor_all,       file.path(cor_dir_g, "Correlation_DiseaseBinary_vs_Edaphic_full.tsv"))
readr::write_tsv(cor_paper_tbl, file.path(cor_dir_g, "Correlation_DiseaseBinary_vs_Edaphic_paper.tsv"))

wb_cor <- createWorkbook()
addWorksheet(wb_cor, "Full_results")
writeData(wb_cor, "Full_results", cor_all)
addWorksheet(wb_cor, "Paper_table")
writeData(wb_cor, "Paper_table", cor_paper_tbl)
saveWorkbook(
  wb_cor,
  file.path(cor_dir_g, "Correlation_DiseaseBinary_vs_Edaphic.xlsx"),
  overwrite = TRUE
)

message("[OK] Variable vs binary disease correlation exported to: ", cor_dir_g)
```

CHUNK 5.1 — pairwise Pearson correlations among edaphic variables by crop and condition

r chunk: pearson_individual_ovals — options: message=TRUE, warning=TRUE, fig.width=10, fig.height=8

```r
# ============================================================
# Individual pairwise Pearson correlations
# - by crop
# - by condition
# - ellipse (oval) plot + significance
# - simplified version: upper triangle only
# - 2 x 2 multipanel:
#     row 1 = Gros Michel   (Diseased | Healthy)
#     row 2 = Dominico Hartón (Diseased | Healthy)
# ============================================================

suppressPackageStartupMessages({
  library(dplyr)
  library(tidyr)
  library(glue)
  library(openxlsx)
  library(corrplot)
  library(readr)
})

# ------------------------------------------------------------
# helper: p-matrix
# ------------------------------------------------------------
cor_pmat <- function(mat) {
  mat <- as.matrix(mat)
  pmat <- matrix(NA_real_, ncol(mat), ncol(mat))
  colnames(pmat) <- colnames(mat)
  rownames(pmat) <- colnames(mat)

  for (i in seq_len(ncol(mat))) {
    for (j in seq_len(ncol(mat))) {
      xi <- mat[, i]
      xj <- mat[, j]
```

```
      ok <- complete.cases(xi, xj)
      if (sum(ok) >= 4) {
        ct <- suppressWarnings(cor.test(xi[ok], xj[ok], method = "pearson"))
        pmat[i, j] <- ct$p.value
      }
    }
  }
  pmat
}

# ------------------------------------------------------------
# helper: short, unique sheet names for Excel
# ------------------------------------------------------------
sheet_tag <- function(crop_key, condition_label, type = c("r", "p")) {
  type <- match.arg(type)

  crop_tag <- ifelse(crop_key == "Dominico_Harton", "DH", "GM")
  cond_tag <- ifelse(condition_label == "Healthy_Plant", "H", "D")

  paste(crop_tag, cond_tag, type, sep = "_")
}

# ------------------------------------------------------------
# helper: significance stars for tables
# ------------------------------------------------------------
sig_star <- function(p){
  dplyr::case_when(
    is.na(p)  ~ "",
    p < 0.001 ~ "***",
    p < 0.01  ~ "**",
    p < 0.05  ~ "*",
    TRUE      ~ ""
  )
}

# ------------------------------------------------------------
# helper: short panel title
# ------------------------------------------------------------
panel_short_title <- function(crop_key, condition_label) {
  crop_tag <- ifelse(crop_key == "Banano_Gros_Michael", "Gros Michel", "Dominico Hartón")
  cond_tag <- ifelse(condition_label == "Healthy_Plant", "Healthy", "Diseased")
  glue("{crop_tag} — {cond_tag}")
}

# ------------------------------------------------------------
# main function: computes matrices + individual export
# ------------------------------------------------------------
plot_pearson_oval <- function(md, ed_cols, crop_key, condition_label, out_pdf, out_png) {

  md_sub <- md %>%
    dplyr::filter(.data[[params$state_var]] == condition_label)

  panel_title <- panel_short_title(crop_key, condition_label)

  if (nrow(md_sub) < 5) {
    grDevices::pdf(out_pdf, width = 8, height = 7, family = "Helvetica")
    plot.new()
    title(panel_title)
    text(0.5, 0.5, "insufficient n")
    dev.off()

    png(out_png, width = 8, height = 7, units = "in", res = 300)
    plot.new()
    title(panel_title)
    text(0.5, 0.5, "insufficient n")
    dev.off()

    return(list(
      cor_tbl = tibble(),
      p_tbl   = tibble(),
      cor_mat = NULL,
      p_mat   = NULL,
      title   = panel_title
    ))
  }
```

```
mat <- md_sub[, ed_cols, drop = FALSE] %>%
  dplyr::mutate(dplyr::across(everything(), ~ suppressWarnings(as.numeric(.x)))) %>%
  as.data.frame()

mat <- mat[, colSums(!is.na(mat)) > 0, drop = FALSE]
cc  <- complete.cases(mat)
mat <- mat[cc, , drop = FALSE]

if (nrow(mat) < 5 || ncol(mat) < 2) {
  grDevices::pdf(out_pdf, width = 8, height = 7, family = "Helvetica")
  plot.new()
  title(panel_title)
  text(0.5, 0.5, "insufficient data")
  dev.off()

  png(out_png, width = 8, height = 7, units = "in", res = 300)
  plot.new()
  title(panel_title)
  text(0.5, 0.5, "insufficient data")
  dev.off()

  return(list(
    cor_tbl = tibble(),
    p_tbl   = tibble(),
    cor_mat = NULL,
    p_mat   = NULL,
    title   = panel_title
  ))
}

cor_mat <- cor(mat, method = "pearson", use = "pairwise.complete.obs")
p_mat   <- cor_pmat(mat)

pretty_names <- dplyr::coalesce(unname(nice_names[colnames(cor_mat)]), colnames(cor_mat))
colnames(cor_mat) <- pretty_names
rownames(cor_mat) <- pretty_names
colnames(p_mat)   <- pretty_names
rownames(p_mat)   <- pretty_names

# ---- Individual PDF
grDevices::pdf(out_pdf, width = 8, height = 7, family = "Helvetica")
corrplot::corrplot(
  cor_mat,
  method = "ellipse",
  type = "upper",
  diag = FALSE,
  col = colorRampPalette(grad_corr)(200),
  tl.col = "black",
  tl.srt = 45,
  tl.cex = 0.78,
  cl.cex = 0.75,
  mar = c(0, 0, 2, 0),
  title = panel_title,
  p.mat = p_mat,
  sig.level = c(0.001, 0.01, 0.05),
  insig = "label_sig",
  pch.cex = 0.65,
  pch.col = "black"
)
dev.off()

# ---- Individual PNG
png(out_png, width = 8, height = 7, units = "in", res = 300)
corrplot::corrplot(
  cor_mat,
  method = "ellipse",
  type = "upper",
  diag = FALSE,
  col = colorRampPalette(grad_corr)(200),
  tl.col = "black",
  tl.srt = 45,
  tl.cex = 0.78,
  cl.cex = 0.75,
  mar = c(0, 0, 2, 0),
  title = panel_title,
  p.mat = p_mat,
```

```
    sig.level = c(0.001, 0.01, 0.05),
    insig = "label_sig",
    pch.cex = 0.65,
    pch.col = "black"
  )
  dev.off()

  cor_long <- as.data.frame(as.table(cor_mat)) %>%
    dplyr::rename(Var1 = Var1, Var2 = Var2, r = Freq) %>%
    dplyr::mutate(
      Crop = pretty_crop(crop_key),
      Condition = condition_label
    ) %>%
    dplyr::select(Crop, Condition, Var1, Var2, r)

  p_long <- as.data.frame(as.table(p_mat)) %>%
    dplyr::rename(Var1 = Var1, Var2 = Var2, p_value = Freq) %>%
    dplyr::mutate(
      Crop = pretty_crop(crop_key),
      Condition = condition_label,
      signif = sig_star(p_value)
    ) %>%
    dplyr::select(Crop, Condition, Var1, Var2, p_value, signif)

  list(
    cor_tbl = cor_long,
    p_tbl   = p_long,
    cor_mat = cor_mat,
    p_mat   = p_mat,
    title   = panel_title
  )
}

# ------------------------------------------------------------
# jobs in 2x2 multipanel order
# row 1 = GM Diseased | GM Healthy
# row 2 = DH Diseased | DH Healthy
# ------------------------------------------------------------
pearson_jobs <- list(
  list(ds = ds_GM, crop = "Banano_Gros_Michael", cond = "Diseased_Plant"),
  list(ds = ds_GM, crop = "Banano_Gros_Michael", cond = "Healthy_Plant"),
  list(ds = ds_DH, crop = "Dominico_Harton",     cond = "Diseased_Plant"),
  list(ds = ds_DH, crop = "Dominico_Harton",     cond = "Healthy_Plant")
)

pearson_dir_g <- file.path(outdir_global(), "Pearson")
dir.create(pearson_dir_g, recursive = TRUE, showWarnings = FALSE)

wb_pearson <- createWorkbook()
all_cor_tbl <- list()
all_p_tbl   <- list()
all_panels  <- list()

for (i in seq_along(pearson_jobs)) {

  job <- pearson_jobs[[i]]

  fname_base <- glue("PearsonOval_{job$crop}_{job$cond}")
  out_pdf <- file.path(pearson_dir_g, paste0(fname_base, ".pdf"))
  out_png <- file.path(pearson_dir_g, paste0(fname_base, ".png"))

  res_cor <- plot_pearson_oval(
    md = job$ds$md,
    ed_cols = job$ds$ed_cols,
    crop_key = job$crop,
    condition_label = job$cond,
    out_pdf = out_pdf,
    out_png = out_png
  )

  sheet_cor <- sheet_tag(job$crop, job$cond, "r")
  sheet_p   <- sheet_tag(job$crop, job$cond, "p")

  addWorksheet(wb_pearson, sheet_cor)
  writeData(wb_pearson, sheet_cor, res_cor$cor_tbl)
```

```
  addWorksheet(wb_pearson, sheet_p)
  writeData(wb_pearson, sheet_p, res_cor$p_tbl)

  all_cor_tbl[[fname_base]] <- res_cor$cor_tbl
  all_p_tbl[[fname_base]]   <- res_cor$p_tbl
  all_panels[[fname_base]]  <- res_cor
}

saveWorkbook(
  wb_pearson,
  file.path(pearson_dir_g, "Pearson_Ovals_individual_by_Crop_Condition.xlsx"),
  overwrite = TRUE
)

cor_all_df <- bind_rows(all_cor_tbl)
p_all_df   <- bind_rows(all_p_tbl)

write_tsv(cor_all_df, file.path(pearson_dir_g, "Pearson_Ovals_all_correlations.tsv"))
write_tsv(p_all_df,   file.path(pearson_dir_g, "Pearson_Ovals_all_pvalues.tsv"))

# ------------------------------------------------------------
# Multipanel 2 x 2
# ------------------------------------------------------------
multi_pdf <- file.path(pearson_dir_g, "Pearson_Ovals_multipanel_2x2.pdf")
multi_png <- file.path(pearson_dir_g, "Pearson_Ovals_multipanel_2x2.png")

# ---- PDF multipanel
grDevices::pdf(multi_pdf, width = 10, height = 8, family = "Helvetica")
par(mfrow = c(2, 2), mar = c(1, 1, 2, 1), oma = c(0, 0, 1, 0))

for (nm in names(all_panels)) {
  obj <- all_panels[[nm]]

  if (is.null(obj$cor_mat) || is.null(obj$p_mat)) {
    plot.new()
    title(obj$title, cex.main = 0.9, font.main = 2)
    text(0.5, 0.5, "insufficient data", cex = 0.9)
  } else {
    corrplot::corrplot(
      obj$cor_mat,
      method = "ellipse",
      type = "upper",
      diag = FALSE,
      col = colorRampPalette(grad_corr)(200),
      tl.col = "black",
      tl.srt = 45,
      tl.cex = 0.62,
      cl.cex = 0.60,
      mar = c(0, 0, 1.5, 0),
      title = obj$title,
      p.mat = obj$p_mat,
      sig.level = c(0.001, 0.01, 0.05),
      insig = "label_sig",
      pch.cex = 0.50,
      pch.col = "black"
    )
  }
}
dev.off()

# ---- PNG multipanel
png(multi_png, width = 10, height = 8, units = "in", res = 300)
par(mfrow = c(2, 2), mar = c(1, 1, 2, 1), oma = c(0, 0, 1, 0))

for (nm in names(all_panels)) {
  obj <- all_panels[[nm]]

  if (is.null(obj$cor_mat) || is.null(obj$p_mat)) {
    plot.new()
    title(obj$title, cex.main = 0.9, font.main = 2)
    text(0.5, 0.5, "insufficient data", cex = 0.9)
  } else {
    corrplot::corrplot(
      obj$cor_mat,
      method = "ellipse",
      type = "upper",
```

```
      diag = FALSE,
      col = colorRampPalette(grad_corr)(200),
      tl.col = "black",
      tl.srt = 45,
      tl.cex = 0.62,
      cl.cex = 0.60,
      mar = c(0, 0, 1.5, 0),
      title = obj$title,
      p.mat = obj$p_mat,
      sig.level = c(0.001, 0.01, 0.05),
      insig = "label_sig",
      pch.cex = 0.50,
      pch.col = "black"
    )
  }
}
dev.off()

message("[OK] Pearson ellipse plots exported to: ", pearson_dir_g)
message("[OK] Multipanel 2x2: ", multi_pdf)
```

CHUNK 6 — PERMANOVA by crop

r chunk: permanova_euclidean_byCrop — options: message=TRUE, warning=TRUE

```
# ============================================================
# PERMANOVA on scaled edaphic variables
# Distance: Euclidean
# By Crop
# ============================================================
run_permanova_panel <- function(ds, nperm = 9999) {

  md <- ds$md
  ed_cols <- ds$ed_cols

  df_num <- md[, ed_cols, drop = FALSE] %>%
    mutate(across(everything(), ~ suppressWarnings(as.numeric(.x))))

  cond <- md[[params$state_var]]
  keep <- cond %in% c("Healthy_Plant", "Diseased_Plant")

  df <- data.frame(
    Plant_Condition = droplevels(factor(as.character(cond[keep]), levels = c("Healthy_Plant",
    ↪  "Diseased_Plant"))),
    df_num[keep, , drop = FALSE]
  ) %>%
    drop_na()

  n_h <- sum(df$Plant_Condition == "Healthy_Plant")
  n_d <- sum(df$Plant_Condition == "Diseased_Plant")

  if (nrow(df) < 8 || nlevels(df$Plant_Condition) < 2 || n_h < 2 || n_d < 2) {
    return(tibble(
      Crop = pretty_crop(ds$crop),
      Healthy_n = n_h,
      Diseased_n = n_d,
      N = nrow(df),
      Distance = "Euclidean",
      F_value = NA_real_,
      R2 = NA_real_,
      p_value = NA_real_,
      signif = "",
      Interpretation = "Insufficient data"
    ))
  }

  mat_scaled <- scale(df[, -1, drop = FALSE])
  dist_euc   <- dist(mat_scaled, method = "euclidean")

  ad <- vegan::adonis2(
    dist_euc ~ Plant_Condition,
    data = df,
    permutations = nperm
```

```
  )

  ad_df <- as.data.frame(ad)
  model_row <- ad_df[1, , drop = FALSE]
  pval <- as.numeric(model_row$`Pr(>F)`)

  tibble(
    Crop = pretty_crop(ds$crop),
    Healthy_n = n_h,
    Diseased_n = n_d,
    N = nrow(df),
    Distance = "Euclidean",
    F_value = as.numeric(model_row$F),
    R2      = as.numeric(model_row$R2),
    p_value = pval,
    signif  = p_to_stars(pval),
    Interpretation = ifelse(!is.na(pval) & pval < 0.05, "Significant edaphic separation", "Non-significant
    ↪  edaphic separation")
  )
}

perm_tbl <- bind_rows(
  run_permanova_panel(ds_DH),
  run_permanova_panel(ds_GM)
)

perm_tbl_paper <- perm_tbl %>%
  transmute(
    Crop,
    `Healthy (n)`  = Healthy_n,
    `Diseased (n)` = Diseased_n,
    N,
    Distance,
    F = round(F_value, 3),
    R2 = round(R2, 3),
    p = fmt_p(p_value),
    sig = signif
  )

perm_dir_g <- file.path(outdir_global(), "PERMANOVA")

write_tsv(perm_tbl, file.path(perm_dir_g, "PERMANOVA_edaphic_full.tsv"))
write_tsv(perm_tbl_paper, file.path(perm_dir_g, "PERMANOVA_edaphic_paper_table.tsv"))

wb_perm <- createWorkbook()
addWorksheet(wb_perm, "PERMANOVA_full")
writeData(wb_perm, "PERMANOVA_full", perm_tbl)
addWorksheet(wb_perm, "PERMANOVA_paper")
writeData(wb_perm, "PERMANOVA_paper", perm_tbl_paper)
saveWorkbook(wb_perm, file.path(perm_dir_g, "PERMANOVA_edaphic.xlsx"), overwrite = TRUE)

write_tsv(perm_tbl_paper, file.path(outdir_global(), "Tables", "Table_Paper_PERMANOVA_edaphic.tsv"))

message("[OK] PERMANOVA exported to: ", perm_dir_g)
perm_tbl_paper
```

CHUNK 7 — Random Forest by crop — farm-blocked validation

r chunk: RF_byCrop — options: message=TRUE, warning=TRUE, fig.width=12, fig.height=6

```
# ============================================================
# Random Forest by Crop — Reviewer-ready
# - ntree explicitly reported
# - mtry explicitly set to floor(sqrt(p)) for classification
# - class balance reported
# - OOB accuracy retained as internal diagnostic
# - predictive validation = Leave-One-Farm-Out CV (LOFO-CV)
# - accuracy, balanced accuracy, sensitivity, specificity
# - permutation test preserves farm structure by shuffling labels WITHIN Land
# - variable importance = MeanDecreaseGini from final model on all samples
# ============================================================

rf_metrics <- function(obs, pred, positive = "Diseased_Plant") {
```

```
  lev <- c("Healthy_Plant", "Diseased_Plant")
  obs <- factor(obs, levels = lev)
  pred <- factor(pred, levels = lev)
  cm <- table(Observed = obs, Predicted = pred)

  tn <- cm["Healthy_Plant", "Healthy_Plant"]
  fp <- cm["Healthy_Plant", "Diseased_Plant"]
  fn <- cm["Diseased_Plant", "Healthy_Plant"]
  tp <- cm["Diseased_Plant", "Diseased_Plant"]

  sensitivity <- if ((tp + fn) > 0) tp / (tp + fn) else NA_real_
  specificity <- if ((tn + fp) > 0) tn / (tn + fp) else NA_real_
  accuracy <- sum(diag(cm)) / sum(cm)
  balanced_accuracy <- mean(c(sensitivity, specificity), na.rm = TRUE)

  list(
    confusion = cm,
    accuracy = accuracy,
    balanced_accuracy = balanced_accuracy,
    sensitivity = sensitivity,
    specificity = specificity
  )
}

lofo_rf_predict <- function(df, predictor_cols, ntree, mtry, seed = 1) {
  farms <- sort(unique(as.character(df$Land)))
  pred_all <- rep(NA_character_, nrow(df))

  for (i in seq_along(farms)) {
    farm_test <- farms[i]
    train <- df[df$Land != farm_test, , drop = FALSE]
    test  <- df[df$Land == farm_test, , drop = FALSE]

    if (nlevels(droplevels(train$Plant_Condition)) < 2) {
      stop(glue("LOFO-CV: training set without both classes when holding out farm '{farm_test}'."))
    }

    set.seed(seed + i)
    fit_i <- randomForest::randomForest(
      x = train[, predictor_cols, drop = FALSE],
      y = train$Plant_Condition,
      ntree = ntree,
      mtry = mtry,
      importance = FALSE
    )

    pred_all[df$Land == farm_test] <- as.character(
      predict(fit_i, newdata = test[, predictor_cols, drop = FALSE])
    )
  }

  factor(pred_all, levels = levels(df$Plant_Condition))
}

perm_test_lofo_blocked <- function(df, predictor_cols, ntree, mtry,
                                   nperm = 499, seed = 1) {
  set.seed(seed)

  pred_obs <- lofo_rf_predict(
    df, predictor_cols = predictor_cols,
    ntree = ntree, mtry = mtry, seed = seed + 10
  )
  obs_metrics <- rf_metrics(df$Plant_Condition, pred_obs)
  obs_bal_acc <- obs_metrics$balanced_accuracy

  null_bal_acc <- numeric(nperm)

  for (b in seq_len(nperm)) {
    dfp <- df

    # Permute condition labels within each farm, preserving farm-level structure
    dfp$Plant_Condition <- ave(
      as.character(dfp$Plant_Condition),
      dfp$Land,
      FUN = function(z) sample(z, length(z), replace = FALSE)
    )
```

```
    dfp$Plant_Condition <- factor(
      dfp$Plant_Condition,
      levels = c("Healthy_Plant", "Diseased_Plant")
    )

    pred_b <- lofo_rf_predict(
      dfp, predictor_cols = predictor_cols,
      ntree = ntree, mtry = mtry,
      seed = seed + 1000 + b
    )

    null_bal_acc[b] <- rf_metrics(
      dfp$Plant_Condition, pred_b
    )$balanced_accuracy
  }

  p_perm <- (sum(null_bal_acc >= obs_bal_acc, na.rm = TRUE) + 1) / (nperm + 1)

  list(
    predictions = pred_obs,
    metrics = obs_metrics,
    null_balanced_accuracy = null_bal_acc,
    p_perm = p_perm
  )
}

run_rf_panel <- function(ds, show_legend = FALSE,
                         ntree = params$rf_ntree,
                         nperm_model = params$rf_perm,
                         seed = params$rf_seed) {

  md <- ds$md
  ed_cols <- ds$ed_cols
  panel_title <- pretty_crop(ds$crop)

  df_num <- md[, ed_cols, drop = FALSE] %>%
    mutate(across(everything(), ~ suppressWarnings(as.numeric(.x))))

  df <- data.frame(
    Sample_ID = md$`sample-id`,
    Land = factor(as.character(md[[params$land_var]])),
    Plant_Condition = droplevels(
      factor(
        as.character(md[[params$state_var]]),
        levels = c("Healthy_Plant", "Diseased_Plant")
      )
    ),
    df_num,
    check.names = FALSE
  ) %>%
    tidyr::drop_na()

  predictor_cols <- intersect(ed_cols, names(df))
  p_predictors <- length(predictor_cols)
  mtry_used <- max(1L, floor(sqrt(p_predictors)))

  class_counts <- table(df$Plant_Condition)
  n_healthy <- unname(class_counts["Healthy_Plant"])
  n_diseased <- unname(class_counts["Diseased_Plant"])
  n_farms <- nlevels(droplevels(df$Land))

  if (nrow(df) < 15 || nlevels(df$Plant_Condition) < 2 || n_farms < 2) {
    return(list(
      plot = ggplot() + theme_void() + ggtitle(panel_title),
      imp = tibble(),
      perf = tibble(
        Panel = panel_title, Crop = panel_title, N = nrow(df),
        Healthy_n = n_healthy, Diseased_n = n_diseased, Farms = n_farms,
        ntree = ntree, mtry = mtry_used,
        OOB_Acc = NA_real_, LOFO_Acc = NA_real_, LOFO_Balanced_Acc = NA_real_,
        Sensitivity = NA_real_, Specificity = NA_real_,
        Blocked_perm_p = NA_real_, Perm_sig = ""
      ),
      predictions = tibble(),
      confusion = tibble()
    ))
```

```
}

# Final model on all samples: descriptive importance + internal OOB diagnostic
set.seed(seed)
rf_fit <- randomForest::randomForest(
  x = df[, predictor_cols, drop = FALSE],
  y = df$Plant_Condition,
  importance = TRUE,
  ntree = ntree,
  mtry = mtry_used
)

oob_acc <- 1 - rf_fit$err.rate[ntree, "OOB"]

# Farm-blocked predictive validation
lofo <- perm_test_lofo_blocked(
  df = df,
  predictor_cols = predictor_cols,
  ntree = ntree,
  mtry = mtry_used,
  nperm = nperm_model,
  seed = seed + 100
)

m <- lofo$metrics
p_perm_model <- lofo$p_perm
sig_model <- p_to_stars(p_perm_model)

imp <- randomForest::importance(rf_fit, type = 2)
imp_df <- tibble(
  Variable = rownames(imp),
  MeanDecreaseGini = as.numeric(imp[, "MeanDecreaseGini"]),
  Panel = panel_title
) %>%
  arrange(desc(MeanDecreaseGini))

means_by_cond <- df %>%
  dplyr::select(Plant_Condition, dplyr::all_of(predictor_cols)) %>%
  pivot_longer(cols = -Plant_Condition, names_to = "Variable", values_to = "Value") %>%
  group_by(Variable, Plant_Condition) %>%
  summarise(mean_val = mean(Value, na.rm = TRUE), .groups = "drop") %>%
  group_by(Variable) %>%
  slice_max(order_by = mean_val, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  transmute(Variable, Cond_max = as.character(Plant_Condition))

imp_df3 <- imp_df %>%
  left_join(means_by_cond, by = "Variable") %>%
  mutate(
    Cond_max = factor(Cond_max, levels = c("Healthy_Plant", "Diseased_Plant")),
    Variable_lab = dplyr::coalesce(unname(nice_names[Variable]), Variable)
  )

subtitle_txt <- glue(
  "LOFO balanced accuracy = {round(100 * m$balanced_accuracy, 1)}% | ",
  "blocked perm p = {format.pval(p_perm_model, digits = 2, eps = 0.001)} {sig_model}"
)

p <- imp_df3 %>%
  slice_head(n = 20) %>%
  ggplot(aes(x = reorder(Variable_lab, MeanDecreaseGini), y = MeanDecreaseGini, fill = Cond_max)) +
  geom_col(width = 0.8) +
  coord_flip() +
  scale_fill_manual(
    values = pal_condition,
    labels = c("Healthy", "Diseased"),
    drop = FALSE,
    name = "Higher mean in"
  ) +
  theme_bw(base_size = 12, base_family = "Helvetica") +
  labs(
    title = panel_title,
    subtitle = subtitle_txt,
    x = NULL,
    y = "Mean Decrease Gini"
  ) +
```

```
    theme(
      panel.grid = element_blank(),
      plot.title = element_text(hjust = 0.5, face = "bold"),
      plot.subtitle = element_text(hjust = 0.5, size = 9),
      legend.position = if (show_legend) "right" else "none"
    )

  perf_df <- tibble(
    Panel = panel_title,
    Crop = pretty_crop(ds$crop),
    N = nrow(df),
    Healthy_n = n_healthy,
    Diseased_n = n_diseased,
    Farms = n_farms,
    ntree = ntree,
    mtry = mtry_used,
    OOB_Acc = oob_acc,
    LOFO_Acc = m$accuracy,
    LOFO_Balanced_Acc = m$balanced_accuracy,
    Sensitivity = m$sensitivity,
    Specificity = m$specificity,
    Blocked_perm_p = p_perm_model,
    Perm_sig = sig_model,
    Validation = "Leave-one-farm-out cross-validation",
    Permutation_scheme = "Condition labels permuted within Land"
  )

  pred_export <- tibble(
    Sample_ID = df$Sample_ID,
    Crop = pretty_crop(ds$crop),
    Land = as.character(df$Land),
    Observed = as.character(df$Plant_Condition),
    Predicted_LOFO = as.character(lofo$predictions),
    Correct = Observed == Predicted_LOFO
  )

  conf_export <- as.data.frame(m$confusion) %>%
    as_tibble() %>%
    mutate(Crop = pretty_crop(ds$crop), .before = 1)

  imp_export <- imp_df3 %>%
    transmute(
      Panel,
      Crop = pretty_crop(ds$crop),
      Variable,
      Variable_label = Variable_lab,
      MeanDecreaseGini,
      Cond_max = as.character(Cond_max)
    )

  list(
    plot = p,
    imp = imp_export,
    perf = perf_df,
    predictions = pred_export,
    confusion = conf_export
  )
}

rf_DH <- run_rf_panel(ds_DH, show_legend = FALSE, seed = params$rf_seed + 11)
rf_GM <- run_rf_panel(ds_GM, show_legend = TRUE,  seed = params$rf_seed + 12)

p_rf_2 <- (rf_DH$plot | rf_GM$plot) +
  plot_layout(guides = "collect") &
  theme(legend.position = "right")

print(p_rf_2)

rf_dir_g <- file.path(outdir_global(), "RandomForest")
ggsave(file.path(rf_dir_g, "RF_VarImportance_top20_byCrop_FarmBlocked.pdf"),
       p_rf_2, width = 12, height = 6, dpi = 300)
ggsave(file.path(rf_dir_g, "RF_VarImportance_top20_byCrop_FarmBlocked.png"),
       p_rf_2, width = 12, height = 6, dpi = 300)

imp_all  <- bind_rows(rf_DH$imp, rf_GM$imp)
perf_all <- bind_rows(rf_DH$perf, rf_GM$perf)
```

```r
pred_all <- bind_rows(rf_DH$predictions, rf_GM$predictions)
conf_all <- bind_rows(rf_DH$confusion, rf_GM$confusion)

wb_rf <- createWorkbook()
addWorksheet(wb_rf, "Importance_all")
writeData(wb_rf, "Importance_all", imp_all)
addWorksheet(wb_rf, "Model_performance")
writeData(wb_rf, "Model_performance", perf_all)
addWorksheet(wb_rf, "LOFO_predictions")
writeData(wb_rf, "LOFO_predictions", pred_all)
addWorksheet(wb_rf, "Confusion_matrices")
writeData(wb_rf, "Confusion_matrices", conf_all)
saveWorkbook(
  wb_rf,
  file.path(rf_dir_g, "RF_outputs_byCrop_FarmBlocked.xlsx"),
  overwrite = TRUE
)

message("Reviewer-ready farm-blocked RF outputs exported in: ", rf_dir_g)
```

CHUNK 8 — summary table for the paper

r chunk: paper_tables — options: message=TRUE, warning=TRUE

```r
# ========================================================
# Paper summary table
# ========================================================
pca_summary_tbl <- bind_rows(
  p_DH$eig %>% pivot_wider(names_from = PC, values_from = Variance_pct),
  p_GM$eig %>% pivot_wider(names_from = PC, values_from = Variance_pct)
)

rf_summary_tbl <- perf_all %>%
  transmute(
    Crop,
    RF_ntree = ntree,
    RF_mtry = mtry,
    RF_OOB_Acc = round(OOB_Acc, 3),
    RF_LOFO_Acc = round(LOFO_Acc, 3),
    RF_LOFO_Balanced_Acc = round(LOFO_Balanced_Acc, 3),
    RF_Sensitivity = round(Sensitivity, 3),
    RF_Specificity = round(Specificity, 3),
    RF_blocked_perm_p = fmt_p(Blocked_perm_p),
    RF_sig = Perm_sig
  )

paper_summary_tbl <- perm_tbl %>%
  select(Crop, Healthy_n, Diseased_n, N, F_value, R2, p_value, signif) %>%
  left_join(pca_summary_tbl, by = "Crop") %>%
  left_join(rf_summary_tbl, by = "Crop") %>%
  transmute(
    Crop,
    `Healthy (n)` = Healthy_n,
    `Diseased (n)` = Diseased_n,
    N,
    `PCA PC1 (%)` = round(PC1, 1),
    `PCA PC2 (%)` = round(PC2, 1),
    `PERMANOVA F` = round(F_value, 3),
    `PERMANOVA R2` = round(R2, 3),
    `PERMANOVA p` = fmt_p(p_value),
    `PERMANOVA sig` = signif,
    `RF ntree` = RF_ntree,
    `RF mtry` = RF_mtry,
    `RF OOB Acc` = RF_OOB_Acc,
    `RF LOFO Acc` = RF_LOFO_Acc,
    `RF LOFO Balanced Acc` = RF_LOFO_Balanced_Acc,
    `RF Sensitivity` = RF_Sensitivity,
    `RF Specificity` = RF_Specificity,
    `RF blocked perm p` = RF_blocked_perm_p,
    `RF sig` = RF_sig
  )

tables_dir_g <- file.path(outdir_global(), "Tables")
```

```r
write_tsv(paper_summary_tbl, file.path(tables_dir_g, "Table_Paper_PCA_PERMANOVA_RF.tsv"))

wb_paper <- createWorkbook()
addWorksheet(wb_paper, "Paper_summary")
writeData(wb_paper, "Paper_summary", paper_summary_tbl)
addWorksheet(wb_paper, "PERMANOVA_full")
writeData(wb_paper, "PERMANOVA_full", perm_tbl)
addWorksheet(wb_paper, "RF_performance")
writeData(wb_paper, "RF_performance", perf_all)
addWorksheet(wb_paper, "Correlation_paper")
writeData(wb_paper, "Correlation_paper", cor_paper_tbl)
saveWorkbook(wb_paper, file.path(tables_dir_g, "Paper_Tables_Quimica.xlsx"), overwrite = TRUE)

paper_summary_tbl
```

CHUNK 9 — final summary

r chunk: resumen — options: echo=FALSE

```r
cat("
This R Markdown uses the single edaphic table (Metadata_Completo_plus_edaphic.xlsx) and runs
 ↪  chemical/edaphic analyses by crop:

A) Edaphic PCA (prcomp, scaled), multipanel by crop:
   - Color = Plant_Condition
   - Shape = Crop
   - Vectors showing magnitude/contribution on PC1 + PC2
   - No ellipses

B) Pearson correlation with binary disease status:
   - Healthy = 0, Diseased = 1
   - By crop
   - Stars = raw p
   - Black outline = q < 0.05

C) PERMANOVA by crop:
   - Euclidean distance on scaled variables
   - Full table and clean table for the paper

D) Random Forest by crop:
   - Top 20 variables by MeanDecreaseGini
   - Explicit ntree and mtry
   - Class balance reported
   - OOB accuracy as internal diagnostic
   - Leave-one-farm-out predictive validation (LOFO-CV)
   - Accuracy, balanced accuracy, sensitivity and specificity
   - Permutation test with labels permuted within Land
   - Export to Excel

E) Summary table for the paper:
   - PCA (PC1, PC2)
   - PERMANOVA (F, R2, p)
   - Random Forest (OOB accuracy, permutation p)
   - Correlation with binary disease status

All outputs are saved in:
  ./03_Beta-Diversity/06_Quimica_PCA/
")
```

Session information

r chunk: session_info

```r
sessionInfo()
```

Soil physicochemical profile — Design-aware PERMANOVA, PERMDISP and univariate analyses (elephantiasis; standardized metadata terminology)

Supplementary analysis script

Sergio Pardo-Díaz

Contents

Objective 2

1. Setup 2

2. Helpers 2

3. Import and design audit 3

4. Symptom status — sample level, permutations restricted within farm 7

5. Symptom status within each crop 7

6. Crop type — independent farm level 7

7. Crop_type × symptom-status interaction — farm-level multivariate contrasts 8

8. PERMDISP diagnostics 9

9. Definitive PERMANOVA/PERMDISP table 9

10. Design-aware univariate analyses 10

11. QC: R² and sums of squares 12

12. Export definitive workbook 12

Session information 14

Script: `Physicochemical/PERMANOVA_Fisicoquimicos.Rmd`
Rendered: 2026-10-13 — code listing (the R Markdown source is shown as written; chunks were not executed, so no outputs are included).

YAML header / parameters

```yaml
title: "Soil physicochemical profile — Design-aware PERMANOVA, PERMDISP and univariate analyses
↪  (elephantiasis; standardized metadata terminology)"
author: "Sergio Pardo-Díaz"
date: "`r format(Sys.time(), '%Y-%m-%d %H:%M')`"
output:
  html_document:
    toc: true
    toc_depth: 3
    number_sections: true
params:
  project_root: "."
  analysis_root: "03_Beta-Diversity/06_Quimica_PCA"
  out_subdir: "PERMANOVA_Fisicoquimicos"
  metadata_edaphic_path: "Metadata_Completo_plus_edaphic.xlsx"
  state_var: "Symptom_status"
  crop_var: "Crop_type"
  farm_var: "Farm"
```

```yaml
  department_var: "Department"
  nperm: 9999
  seed: 2026
  p_adjust_method: "BH"
  redundancy_cor_threshold: 0.90
  vars_edaphic:
    - pH
    - Organic_M
    - P
    - Ca_Exchange
    - Mg_Exchange
    - K_Exchange
    - CEC
    - Zn
    - Cu
    - B
    - N_Total
    - Respirometry
```

Objective

This analysis replaces the sample-level factorial MANOVA with a design-aware permutational analysis of the multivariate soil physicochemical profile. Continuous physicochemical variables are centered and scaled and Euclidean distances are used. Symptom status is tested at sample level with permutations restricted within farm. Crop type, which is constant within farm, is tested separately at the independent farm level using one balanced centroid per farm. The crop × symptom-status interaction is evaluated as a farm-level contrast: for each farm, the multivariate Symptomatic − Asymptomatic difference is calculated and tested between crops. This prevents the 24 vs. 72 sample imbalance from being interpreted as independent replication of crop type.

1. Setup

r chunk: setup — options: message=TRUE, warning=TRUE

```r
knitr::opts_chunk$set(echo=TRUE, message=FALSE, warning=FALSE)

suppressPackageStartupMessages({
  library(tidyverse)
  library(readxl)
  library(openxlsx)
  library(vegan)
  library(permute)
  library(glue)
})

set.seed(params$seed)
nperm <- params$nperm
p_adjust_method <- params$p_adjust_method

project_root <- normalizePath(params$project_root, winslash="/", mustWork=FALSE)
out_root <- file.path(project_root, params$analysis_root, params$out_subdir)
tables_dir <- file.path(out_root, "Tables")
rds_dir <- file.path(out_root, "RDS")
dir.create(tables_dir, recursive=TRUE, showWarnings=FALSE)
dir.create(rds_dir, recursive=TRUE, showWarnings=FALSE)

vars_candidates <- unlist(params$vars_edaphic)
redundancy_cor_threshold <- params$redundancy_cor_threshold
vars_edaphic <- vars_candidates  # replaced after the redundancy diagnostic
```

2. Helpers

r chunk: helpers

```r
fmt_p <- function(p) {
  ifelse(is.na(p), "", ifelse(p < 0.001, "<0.001", sprintf("%.4f", p)))
}
```

```r
sig_code <- function(p) {
  dplyr::case_when(
    is.na(p) ~ "—",
    p < 0.001 ~ "***",
    p < 0.01 ~ "**",
    p < 0.05 ~ "*",
    TRUE ~ "ns"
  )
}

first_col <- function(x, candidates) {
  hit <- intersect(candidates, colnames(x))
  if (!length(hit)) return(NULL)
  hit[1]
}

tidy_adonis <- function(x, model, comparison) {
  z <- as.data.frame(x, check.names=FALSE)
  z$Term <- rownames(z)
  rownames(z) <- NULL
  ss <- first_col(z, c("SumOfSqs", "Sum Sq", "SS"))
  pp <- first_col(z, c("Pr(>F)", "Pr(>F.)"))
  tibble(
    Analysis="PERMANOVA", Comparison=comparison, Model=model, Term=z$Term,
    Df=as.numeric(z$Df),
    SS=if (is.null(ss)) NA_real_ else as.numeric(z[[ss]]),
    R2=if ("R2" %in% names(z)) as.numeric(z$R2) else NA_real_,
    F=if ("F" %in% names(z)) as.numeric(z$F) else NA_real_,
    p_raw=if (is.null(pp)) NA_real_ else as.numeric(z[[pp]])
  )
}

tidy_permdisp <- function(bd, comparison, model, permutations) {
  pa <- vegan::permutest(bd, permutations=permutations)
  z <- as.data.frame(pa$tab, check.names=FALSE)
  z$Term <- rownames(z); rownames(z) <- NULL
  ss <- first_col(z, c("Sum Sq", "SumOfSqs", "SS"))
  pp <- first_col(z, c("Pr(>F)", "Pr(>F.)"))
  tibble(
    Analysis="PERMDISP", Comparison=comparison, Model=model, Term=z$Term,
    Df=as.numeric(z$Df),
    SS=if (is.null(ss)) NA_real_ else as.numeric(z[[ss]]),
    R2=NA_real_,
    F=if ("F" %in% names(z)) as.numeric(z$F) else NA_real_,
    p_raw=if (is.null(pp)) NA_real_ else as.numeric(z[[pp]])
  )
}

qc_block <- function(tab, label) {
  total_ss <- tab$SS[tab$Term == "Total"]
  if (!length(total_ss)) total_ss <- sum(tab$SS[tab$Term != "Total"], na.rm=TRUE)
  tibble(
    Block=label,
    SS_sum=sum(tab$SS[tab$Term != "Total"], na.rm=TRUE),
    SS_total=total_ss[1],
    R2_sum=sum(tab$R2[tab$Term != "Total"], na.rm=TRUE),
    OK_SS=abs(SS_sum-SS_total) < 1e-6,
    OK_R2=abs(R2_sum-1) < 1e-6
  )
}

pretty_crop <- function(x) as.character(x)
```

3. Import and design audit

r chunk: import_design

```r
metadata_path <- file.path(project_root, params$metadata_edaphic_path)
if (!file.exists(metadata_path)) stop("Metadata not found: ", metadata_path)

dat <- readxl::read_excel(metadata_path) %>% as.data.frame()

# ------------------------------------------------------------------------
```

```r
# Standardize metadata column names ONCE at import.
# The source workbook may contain legacy names; all downstream analyses use
# the definitive manuscript terminology only.
# ------------------------------------------------------------------------
rename_if_present <- function(data, new, candidates) {
  if (new %in% names(data)) return(data)
  hit <- intersect(candidates, names(data))
  if (!length(hit)) return(data)
  names(data)[match(hit[1], names(data))] <- new
  data
}

dat <- dat %>%
  rename_if_present("Sample_ID", c("sample-id", "SampleID", "Sample_ID")) %>%
  rename_if_present("Symptom_status", c("Plant_Condition", "Plant Condition", "Condition",
↪  "Symptom_status")) %>%
  rename_if_present("Crop_type", c("Crop", "Crop_key", "Crop type", "Crop_type")) %>%
  rename_if_present("Farm", c("Land", "Farm", "Farm_ID")) %>%
  rename_if_present("Department", c("Department", "Departamento"))

required <- c("Sample_ID", "Symptom_status", "Crop_type", "Farm",
              "Department", vars_candidates)
missing_cols <- setdiff(required, names(dat))
if (length(missing_cols)) {
  stop("Missing columns after metadata standardization: ",
       paste(missing_cols, collapse=", "),
       "\nAvailable columns: ", paste(names(dat), collapse=", "))
}

# Standardize factor LEVELS to the definitive terminology.
dat <- dat %>%
  mutate(
    Symptom_status = recode(as.character(Symptom_status),
      "Healthy_Plant"="Asymptomatic",
      "Healthy"="Asymptomatic",
      "Asymptomatic"="Asymptomatic",
      "Diseased_Plant"="Symptomatic",
      "Diseased"="Symptomatic",
      "Symptomatic"="Symptomatic",
      .default=as.character(Symptom_status)),
    Crop_type = recode(as.character(Crop_type),
      "Banano_Gros_Michael"="Banana",
      "Gros Michel"="Banana",
      "Banana"="Banana",
      "Dominico_Harton"="Plantain",
      "Dominico Harton"="Plantain",
      "Dominico Hartón"="Plantain",
      "Plantain"="Plantain",
      .default=as.character(Crop_type))
  )

# ------------------------------------------------------------------------
# Redundancy diagnostic BEFORE the definitive PERMANOVA
#
# PERMANOVA does not require absence of multicollinearity as a formal
# assumption. However, strongly redundant response variables can overweight
# the same physicochemical dimension in Euclidean distance. Therefore all
# candidate soil variables, including CEC, are screened before distances are
# calculated.
#
# Rule:
#   1) flag pairs with |Pearson r| >= the prespecified threshold (0.90);
#   2) within a flagged pair, remove the variable with the larger mean
#      absolute correlation with all remaining candidate variables;
#   3) repeat until no flagged pair remains.
# The correlation-matrix condition number is reported before and after
# filtering as a diagnostic, not as a PERMANOVA assumption.
# ------------------------------------------------------------------------

dat_redundancy <- dat %>%
  filter(if_all(all_of(vars_candidates), ~ !is.na(.x)))

if (nrow(dat_redundancy) < 3) {
  stop("Insufficient complete observations for the redundancy diagnostic.")
}
```

```r
condition_number_cor <- function(M) {
  ev <- eigen(M, symmetric=TRUE, only.values=TRUE)$values
  ev <- ev[ev > .Machine$double.eps]
  if (length(ev) < 2) return(Inf)
  sqrt(max(ev) / min(ev))
}

cor_before <- cor(
  dat_redundancy[, vars_candidates, drop=FALSE],
  use="pairwise.complete.obs",
  method="pearson"
)
condition_number_before <- condition_number_cor(cor_before)

selected_vars <- vars_candidates
removed_log <- tibble(
  Step=integer(),
  Variable_removed=character(),
  Partner=character(),
  Pearson_r=double(),
  Abs_r=double(),
  Mean_abs_correlation_removed=double(),
  Mean_abs_correlation_partner=double(),
  Decision=character()
)

step_i <- 0L
repeat {
  C <- cor(
    dat_redundancy[, selected_vars, drop=FALSE],
    use="pairwise.complete.obs",
    method="pearson"
  )
  diag(C) <- NA_real_

  idx <- which(abs(C) >= redundancy_cor_threshold, arr.ind=TRUE)
  idx <- idx[idx[,1] < idx[,2], , drop=FALSE]

  if (!nrow(idx)) break

  # Resolve the strongest remaining redundant pair first.
  abs_vals <- abs(C[idx])
  k <- which.max(abs_vals)
  i <- idx[k,1]
  j <- idx[k,2]
  v1 <- rownames(C)[i]
  v2 <- colnames(C)[j]

  mean_abs <- colMeans(abs(C), na.rm=TRUE)
  remove_v <- if (mean_abs[v1] >= mean_abs[v2]) v1 else v2
  keep_v   <- if (remove_v == v1) v2 else v1

  step_i <- step_i + 1L
  removed_log <- bind_rows(
    removed_log,
    tibble(
      Step=step_i,
      Variable_removed=remove_v,
      Partner=keep_v,
      Pearson_r=unname(C[v1,v2]),
      Abs_r=abs(unname(C[v1,v2])),
      Mean_abs_correlation_removed=unname(mean_abs[remove_v]),
      Mean_abs_correlation_partner=unname(mean_abs[keep_v]),
      Decision=paste0(
        "Removed ", remove_v,
        " because it had the greater mean absolute correlation across the candidate set."
      )
    )
  )

  selected_vars <- setdiff(selected_vars, remove_v)

  if (length(selected_vars) < 2) {
    stop("Redundancy filtering left fewer than two physicochemical variables.")
  }
}
```

```r
cor_after <- cor(
  dat_redundancy[, selected_vars, drop=FALSE],
  use="pairwise.complete.obs",
  method="pearson"
)
condition_number_after <- condition_number_cor(cor_after)

# Pairwise correlation table for transparent reporting
cor_pairs <- as.data.frame(as.table(cor_before), stringsAsFactors=FALSE) %>%
  as_tibble() %>%
  rename(Variable_1=Var1, Variable_2=Var2, Pearson_r=Freq) %>%
  filter(Variable_1 < Variable_2) %>%
  mutate(
    Abs_r=abs(Pearson_r),
    Flag_redundant=Abs_r >= redundancy_cor_threshold
  ) %>%
  arrange(desc(Abs_r))

redundancy_summary <- tibble(
  Metric=c(
    "Correlation threshold (absolute Pearson r)",
    "Number of candidate variables",
    "Number of retained variables",
    "Number of excluded variables",
    "Condition number before filtering",
    "Condition number after filtering",
    "Variables excluded",
    "Variables retained"
  ),
  Value=c(
    as.character(redundancy_cor_threshold),
    as.character(length(vars_candidates)),
    as.character(length(selected_vars)),
    as.character(length(setdiff(vars_candidates, selected_vars))),
    sprintf("%.4f", condition_number_before),
    sprintf("%.4f", condition_number_after),
    ifelse(length(setdiff(vars_candidates, selected_vars)) == 0,
           "None", paste(setdiff(vars_candidates, selected_vars), collapse=", ")),
    paste(selected_vars, collapse=", ")
  )
)

message("Redundancy diagnostic:")
message("  Candidate variables: ", paste(vars_candidates, collapse=", "))
message("  Excluded variables: ",
        ifelse(length(setdiff(vars_candidates, selected_vars)) == 0,
               "None", paste(setdiff(vars_candidates, selected_vars), collapse=", ")))
message("  Retained variables: ", paste(selected_vars, collapse=", "))
message("  Condition number: ",
        sprintf("%.3f", condition_number_before), " -> ",
        sprintf("%.3f", condition_number_after))

# From this point onward, ALL multivariate and univariate analyses use only
# the empirically retained variables.
vars_edaphic <- selected_vars

# Complete cases for the definitive analysis after redundancy filtering
dat <- dat %>%
  filter(if_all(all_of(vars_edaphic), ~ !is.na(.x))) %>%
  mutate(
    Symptom_status=factor(Symptom_status, levels=c("Asymptomatic","Symptomatic")),
    Crop_type=factor(Crop_type, levels=c("Banana","Plantain")),
    Farm=factor(Farm),
    Department=factor(Department)
  )

# Strict design checks
farm_audit <- dat %>% group_by(Farm) %>% summarise(
  Crop_type=first(as.character(Crop_type)), Department=first(as.character(Department)),
  n=n(), n_crop=n_distinct(Crop_type), n_department=n_distinct(Department),
  n_asymptomatic=sum(Symptom_status=="Asymptomatic"),
  n_symptomatic=sum(Symptom_status=="Symptomatic"), .groups="drop")

if (any(farm_audit$n_crop != 1)) stop("Crop_type is not constant within at least one farm.")
if (any(farm_audit$n_department != 1)) stop("Department is not constant within at least one farm.")
```

```
if (any(farm_audit$n_asymptomatic == 0 | farm_audit$n_symptomatic == 0)) stop("A farm lacks one
↪  condition.")

print(farm_audit)
write.csv(farm_audit, file.path(tables_dir, "QC_Sampling_Design.csv"), row.names=FALSE)

# Standardize globally once; all analyses use the same standardized variable space
X <- scale(dat[, vars_edaphic, drop=FALSE])
rownames(X) <- dat$Sample_ID
dist_sample <- dist(X, method="euclidean")
```

4. Symptom status — sample level, permutations restricted within farm

r chunk: condition_global

```
ctrl_condition <- permute::how(nperm=nperm, blocks=dat$Farm)
ad_condition <- vegan::adonis2(
  dist_sample ~ Symptom_status,
  data=dat,
  permutations=ctrl_condition,
  by="terms"
)
condition_global <- tidy_adonis(
  ad_condition,
  "Euclidean ~ Symptom status | permutations restricted within Farm",
  "All soil samples"
)
print(condition_global)
```

5. Symptom status within each crop

r chunk: condition_within_crop

```
within_crop <- list()
within_crop_disp <- list()

for (cr in levels(dat$Crop_type)) {
  idx <- which(dat$Crop_type == cr)
  dcr <- dat[idx, , drop=FALSE]
  Xcr <- X[idx, , drop=FALSE]
  dist_cr <- dist(Xcr, method="euclidean")
  ctrl_cr <- permute::how(nperm=nperm, blocks=dcr$Farm)

  ad_cr <- vegan::adonis2(
    dist_cr ~ Symptom_status,
    data=dcr,
    permutations=ctrl_cr,
    by="terms"
  )
  within_crop[[cr]] <- tidy_adonis(
    ad_cr,
    "Euclidean ~ Symptom status | permutations restricted within Farm",
    pretty_crop(cr)
  )

  # Diagnostic dispersion test. Restricted permutations use the same farm blocks.
  bd_cr <- vegan::betadisper(dist_cr, dcr$Symptom_status, type="centroid")
  within_crop_disp[[cr]] <- tidy_permdisp(
    bd_cr, pretty_crop(cr), "Dispersion by Symptom status | permutations restricted within Farm", ctrl_cr
  )
}
condition_by_crop <- bind_rows(within_crop)
permdisp_by_crop <- bind_rows(within_crop_disp)
```

6. Crop type — independent farm level

r chunk: crop_farm_level

```r
# Balanced cell centroids: each farm-condition cell contributes equally.
cell <- as.data.frame(X) %>%
  mutate(Farm=dat$Farm, Symptom_status=dat$Symptom_status) %>%
  group_by(Farm, Symptom_status) %>%
  summarise(across(all_of(vars_edaphic), mean), .groups="drop")

farm_centroids <- cell %>%
  group_by(Farm) %>%
  summarise(across(all_of(vars_edaphic), mean), .groups="drop")

farm_meta <- farm_audit %>% select(Farm, Crop_type, Department)
farm_df <- left_join(farm_centroids, farm_meta, by="Farm") %>%
  mutate(Crop_type=factor(Crop_type), Department=factor(Department))

Xfarm <- as.matrix(farm_df[, vars_edaphic, drop=FALSE])
rownames(Xfarm) <- farm_df$Farm
dist_farm <- dist(Xfarm, method="euclidean")

tab_dc <- with(farm_df, table(Department, Crop_type))
n_assign <- prod(apply(tab_dc, 1, function(x) {
  if (sum(x)==0) return(1)
  choose(sum(x), x[1])
}))
min_p_crop <- 1/n_assign

ctrl_farm <- permute::how(nperm=nperm, blocks=farm_df$Department)
ad_crop <- vegan::adonis2(
  dist_farm ~ Department + Crop_type,
  data=farm_df,
  permutations=ctrl_farm,
  by="terms"
)
crop_farm <- tidy_adonis(
  ad_crop,
  "Farm-level Euclidean ~ Department + Crop_type | farms permuted within Department",
  paste0("Farm level (n=", nrow(farm_df), ")")
)
# Department is a blocking/adjustment term and is not inferentially tested here.
crop_farm$p_raw[crop_farm$Term == "Department"] <- NA_real_
print(crop_farm)
```

7. Crop_type × symptom-status interaction — farm-level multivariate contrasts

r chunk: interaction_farm_level

```r
# For each farm: Symptomatic - Asymptomatic centroid in the SAME globally standardized space.
wide_cell <- cell %>%
  pivot_wider(names_from=Symptom_status, values_from=all_of(vars_edaphic), names_sep="__")

delta <- tibble(Farm=wide_cell$Farm)
for (v in vars_edaphic) {
  delta[[v]] <- wide_cell[[paste0(v, "__Symptomatic")]] -
                wide_cell[[paste0(v, "__Asymptomatic")]]
}
delta <- left_join(delta, farm_meta, by="Farm") %>%
  mutate(Crop_type=factor(Crop_type), Department=factor(Department))

Xdelta <- as.matrix(delta[, vars_edaphic, drop=FALSE])
rownames(Xdelta) <- delta$Farm
dist_delta <- dist(Xdelta, method="euclidean")

ctrl_delta <- permute::how(nperm=nperm, blocks=delta$Department)
ad_interaction <- vegan::adonis2(
  dist_delta ~ Department + Crop_type,
  data=delta,
  permutations=ctrl_delta,
  by="terms"
)
interaction_farm <- tidy_adonis(
  ad_interaction,
  "Farm-level Symptomatic − Asymptomatic multivariate contrast ~ Department + Crop_type",
  "Crop type × Symptom status (farm-level contrast)"
```

```
)
interaction_farm$p_raw[interaction_farm$Term == "Department"] <- NA_real_
print(interaction_farm)
```

8. PERMDISP diagnostics

r chunk: permdisp

```
# Global symptom-status dispersion, using farm-restricted permutations.
bd_global <- vegan::betadisper(dist_sample, dat$Symptom_status, type="centroid")
permdisp_global <- tidy_permdisp(
  bd_global, "All soil samples", "Dispersion by Symptom status | permutations restricted within Farm",
  ctrl_condition
)

# Farm-level crop dispersion (diagnostic only). With n=8 this is low-power.
bd_crop <- vegan::betadisper(dist_farm, farm_df$Crop_type, type="centroid")
permdisp_crop <- tidy_permdisp(
  bd_crop, paste0("Farm level (n=", nrow(farm_df), ")"),
  "Dispersion by Crop type at farm level | farms permuted within Department", ctrl_farm
)

# Interaction-contrast dispersion at farm level (diagnostic only).
bd_delta <- vegan::betadisper(dist_delta, delta$Crop_type, type="centroid")
permdisp_interaction <- tidy_permdisp(
  bd_delta, "Crop type × Symptom status (farm-level contrast)",
  "Dispersion of farm-level Symptomatic − Asymptomatic contrasts by Crop type", ctrl_delta
)
```

9. Definitive PERMANOVA/PERMDISP table

As in the beta-diversity analyses, PERMANOVA and PERMDISP are reported using the raw permutation P value. R^2 is reported for PERMANOVA as the proportion of multivariate variation explained by each tested term. Multiple-testing correction is reserved for the families of univariate physicochemical tests below.

r chunk: definitive_table

```
permanova_all <- bind_rows(
  condition_global,
  condition_by_crop,
  crop_farm,
  interaction_farm
)

permdisp_all <- bind_rows(
  permdisp_global,
  permdisp_by_crop,
  permdisp_crop,
  permdisp_interaction
)

raw_table <- bind_rows(permanova_all, permdisp_all) %>%
  mutate(
    sig = sig_code(p_raw),
    p_display = fmt_p(p_raw)
  )

# Manuscript-facing rows only
main_table <- raw_table %>%
  filter(
    (Analysis=="PERMANOVA" & !Term %in% c("Residual","Residuals","Total","Department")) |
    (Analysis=="PERMDISP" & tolower(Term) %in% c("groups","group"))
  ) %>%
  mutate(
    Term = case_when(
      Analysis=="PERMANOVA" & Comparison=="All soil samples" ~ "Symptom status",
      Analysis=="PERMANOVA" & str_detect(Comparison, "Banana|Plantain") ~ "Symptom status",
      Analysis=="PERMANOVA" & str_detect(Comparison, "Farm level") ~ "Crop type",
      Analysis=="PERMANOVA" & str_detect(Comparison, "Crop type ×") ~ "Crop type × Symptom status",
      Analysis=="PERMDISP" ~ "Dispersion",
      TRUE ~ Term
```

```
    )
  ) %>%
  select(Analysis, Comparison, Term, Df, SS, R2, F, p=p_display, sig)

print(main_table)
```

10. Design-aware univariate analyses

The individual physicochemical variables are analysed with the same inferential structure used for the multivariate analysis. Plant condition is tested with farm as a fixed blocking factor (`Variable ~ Farm + Symptom_status`), globally and within each crop. Crop_type is tested only at the independent farm level using the balanced farm centroids constructed above (`Variable ~ Department + Crop_type` when Department is estimable). The crop × condition question is tested on the farm-level Symptomatic − Asymptomatic contrasts. Benjamini–Hochberg correction is applied separately across the physicochemical variables retained after redundancy screening within each pre-specified comparison family.

r chunk: univariate

```
# Helpers ----------------------------------------------------------------
get_anova_term <- function(fit, term) {
  a <- as.data.frame(anova(fit))
  a$Term <- rownames(a)
  rownames(a) <- NULL
  z <- a[a$Term == term, , drop=FALSE]
  if (!nrow(z)) return(tibble(Df=NA_real_, F=NA_real_, p_raw=NA_real_))
  fcol <- intersect(c("F value", "F.value", "F"), names(z))[1]
  pcol <- intersect(c("Pr(>F)", "Pr(>F.)"), names(z))[1]
  tibble(
    Df = if ("Df" %in% names(z)) as.numeric(z$Df[1]) else NA_real_,
    F = if (!is.na(fcol)) as.numeric(z[[fcol]][1]) else NA_real_,
    p_raw = if (!is.na(pcol)) as.numeric(z[[pcol]][1]) else NA_real_
  )
}

condition_effect <- function(d, v) {
  # Mean farm-level contrast in ORIGINAL units; each farm contributes once.
  fc <- d %>%
    group_by(Farm, Symptom_status) %>%
    summarise(value=mean(.data[[v]], na.rm=TRUE), .groups="drop") %>%
    pivot_wider(names_from=Symptom_status, values_from=value)
  if (!all(c("Symptomatic","Asymptomatic") %in% names(fc))) return(NA_real_)
  mean(fc$Symptomatic - fc$Asymptomatic, na.rm=TRUE)
}

interpret_condition <- function(x) {
  case_when(is.na(x) ~ NA_character_, x > 0 ~ "Higher in Symptomatic",
            x < 0 ~ "Higher in Asymptomatic", TRUE ~ "No directional difference")
}

run_condition_uni <- function(d, v, comparison) {
  # Farm is a block because condition varies within every farm.
  f <- reformulate(c("Farm", "Symptom_status"), response=v)
  fit <- lm(f, data=d)
  st <- get_anova_term(fit, "Symptom_status")
  tibble(
    Variable=v, Comparison=comparison, Effect="Symptomatic - Asymptomatic",
    Effect_size=condition_effect(d, v), Df=st$Df, F=st$F, p_raw=st$p_raw,
    Model=paste0(v, " ~ Farm + Symptom_status")
  )
}

# Condition: global and within crop ---------------------------------------
uni_condition_global <- map_dfr(
  vars_edaphic, ~run_condition_uni(dat, .x, "Symptom status — all soil samples")
)

uni_condition_crop <- map_dfr(unique(dat$Crop_type), function(cr) {
  dd <- dat %>% filter(Crop_type == cr) %>% droplevels()
  map_dfr(vars_edaphic, ~run_condition_uni(dd, .x, paste0("Symptom status — ", pretty_crop(cr))))
})

# Raw-unit farm summaries for interpretable effect sizes ----------------
```

```
cell_raw <- dat %>%
  group_by(Farm, Symptom_status) %>%
  summarise(across(all_of(vars_edaphic), ~mean(.x, na.rm=TRUE)), .groups="drop")

farm_raw <- cell_raw %>%
  group_by(Farm) %>%
  summarise(across(all_of(vars_edaphic), mean), .groups="drop") %>%
  left_join(farm_meta, by="Farm") %>%
  mutate(Crop_type=factor(Crop_type), Department=factor(Department))

wide_raw <- cell_raw %>%
  pivot_wider(names_from=Symptom_status, values_from=all_of(vars_edaphic), names_sep="__")
delta_raw <- tibble(Farm=wide_raw$Farm)
for (v in vars_edaphic) {
  delta_raw[[v]] <- wide_raw[[paste0(v, "__Symptomatic")]] -
                    wide_raw[[paste0(v, "__Asymptomatic")]]
}
delta_raw <- left_join(delta_raw, farm_meta, by="Farm") %>%
  mutate(Crop_type=factor(Crop_type), Department=factor(Department))

# Crop_type: independent farm level ----------------------------------------
run_crop_uni <- function(v) {
  d <- farm_df
  # Mirror the farm-level PERMANOVA: adjust for Department only when the
  # term is estimable; Crop_type is never tested using the 96 sample rows.
  fit_full <- try(lm(reformulate(c("Department","Crop_type"), response=v), data=d), silent=TRUE)
  use_dept <- !inherits(fit_full, "try-error") &&
    "Crop_type" %in% rownames(as.data.frame(anova(fit_full)))
  fit <- if (use_dept) fit_full else lm(reformulate("Crop_type", response=v), data=d)
  st <- get_anova_term(fit, "Crop_type")
  eff <- mean(farm_raw[[v]][farm_raw$Crop_type=="Plantain"], na.rm=TRUE) -
         mean(farm_raw[[v]][farm_raw$Crop_type=="Banana"], na.rm=TRUE)
  tibble(
    Variable=v, Comparison="Crop type — farm level",
    Effect="Plantain - Banana",
    Effect_size=eff, Df=st$Df, F=st$F, p_raw=st$p_raw,
    Model=deparse(formula(fit))
  )
}
uni_crop <- map_dfr(vars_edaphic, run_crop_uni)

# Crop_type × condition: farm-level change ---------------------------------
run_interaction_uni <- function(v) {
  d <- delta
  fit_full <- try(lm(reformulate(c("Department","Crop_type"), response=v), data=d), silent=TRUE)
  use_dept <- !inherits(fit_full, "try-error") &&
    "Crop_type" %in% rownames(as.data.frame(anova(fit_full)))
  fit <- if (use_dept) fit_full else lm(reformulate("Crop_type", response=v), data=d)
  st <- get_anova_term(fit, "Crop_type")
  eff <- mean(delta_raw[[v]][delta_raw$Crop_type=="Plantain"], na.rm=TRUE) -
         mean(delta_raw[[v]][delta_raw$Crop_type=="Banana"], na.rm=TRUE)
  tibble(
    Variable=v, Comparison="Crop type × Symptom status — farm-level contrast",
    Effect="Difference in (Symptomatic - Asymptomatic): Plantain - Banana",
    Effect_size=eff, Df=st$Df, F=st$F, p_raw=st$p_raw,
    Model=deparse(formula(fit))
  )
}
uni_interaction <- map_dfr(vars_edaphic, run_interaction_uni)

# Combine and adjust within each pre-specified comparison family --------
uni_all <- bind_rows(
  uni_condition_global, uni_condition_crop, uni_crop, uni_interaction
) %>%
  group_by(Comparison) %>%
  mutate(
    p_adj = p.adjust(p_raw, method=p_adjust_method),
    sig = sig_code(p_adj),
    p_display = fmt_p(p_raw),
    p_adj_display = fmt_p(p_adj)
  ) %>%
  ungroup() %>%
  mutate(
    Interpretation = case_when(
      str_detect(Comparison, "Crop type × Symptom status") & Effect_size > 0 ~ "Larger Symptomatic -
      ↪  Asymptomatic contrast in Plantain",
```

```
      str_detect(Comparison, "Crop type × Symptom status") & Effect_size < 0 ~ "Larger Symptomatic -
      ↪  Asymptomatic contrast in Banana",
      Comparison == "Crop type — farm level" & Effect_size > 0 ~ "Higher in Plantain",
      Comparison == "Crop type — farm level" & Effect_size < 0 ~ "Higher in Banana",
      str_detect(Comparison, "Symptom status") ~ interpret_condition(Effect_size),
      TRUE ~ "No directional difference"
    )
  )

# Friendly variable labels for manuscript-facing table
var_labels <- c(
  pH="pH", Organic_M="Organic matter", P="P", Ca_Exchange="Ca",
  Mg_Exchange="Mg", K_Exchange="K", Zn="Zn", Cu="Cu", B="B",
  N_Total="Total N", Respirometry="Respirometry"
)

uni_table <- uni_all %>%
  mutate(Variable = recode(Variable, !!!var_labels)) %>%
  select(Comparison, Variable, Effect, Effect_size, Interpretation, Df, F,
         p=p_display, `adjusted p (BH)`=p_adj_display, sig, Model)

print(uni_table)
```

11. QC: R² and sums of squares

r chunk: qc

```
qc <- bind_rows(
  qc_block(condition_global, "PERMANOVA — Symptom status, all samples"),
  lapply(names(within_crop), function(x) qc_block(within_crop[[x]], paste0("PERMANOVA — Symptom status
  ↪  within ", x))) %>% bind_rows(),
  qc_block(crop_farm, "PERMANOVA — Crop_type, farm level"),
  qc_block(interaction_farm, "PERMANOVA — Crop_type × Symptom status, farm-level contrast")
)
print(qc)
if (!all(qc$OK_SS & qc$OK_R2)) stop("PERMANOVA QC failed: SS or R² does not sum correctly.")
```

12. Export definitive workbook

r chunk: export

```
wb <- createWorkbook()
addWorksheet(wb, "Main_Table")
addWorksheet(wb, "Raw_Results")
addWorksheet(wb, "Farm_Level_Crop")
addWorksheet(wb, "Farm_Interaction")
addWorksheet(wb, "Sampling_QC")
addWorksheet(wb, "PERMANOVA_QC")
addWorksheet(wb, "Redundancy_Summary")
addWorksheet(wb, "Redundancy_Pairs")
addWorksheet(wb, "Redundancy_Decisions")
addWorksheet(wb, "Univariate_Table")
addWorksheet(wb, "Univariate_Raw")

# Styles aligned with the existing reviewer-ready beta-diversity tables.
title_style <- createStyle(fontName="Times New Roman", fontSize=11, textDecoration="bold",
                           halign="center", valign="center", wrapText=TRUE)
header_style <- createStyle(fontName="Times New Roman", fontSize=10, textDecoration="bold",
                            fgFill="#D9D9D9", border="TopBottomLeftRight", borderStyle="thin",
                            halign="center", valign="center", wrapText=TRUE)
body_left <- createStyle(fontName="Times New Roman", fontSize=10,
                         border="TopBottomLeftRight", borderStyle="thin", valign="center")
body_center <- createStyle(fontName="Times New Roman", fontSize=10,
                           border="TopBottomLeftRight", borderStyle="thin", halign="center", valign="center")
note_style <- createStyle(fontName="Times New Roman", fontSize=9, textDecoration="italic",
                          halign="left", valign="top", wrapText=TRUE)

title <- "Table. Design-aware PERMANOVA and PERMDISP results for the multivariate soil physicochemical
↪  profile."
note <- paste0(
```

```
  paste0(length(vars_edaphic), " soil physicochemical variables retained after a prespecified redundancy
   ↪  screen (|Pearson r| ≥ ", redundancy_cor_threshold, ") were centered and scaled before Euclidean
   ↪  distances were calculated. Candidate variables included CEC. Variables retained: ",
   ↪  paste(vars_edaphic, collapse=", "), ". "),
  "Symptom status was tested using 9,999 permutations restricted within farm. Crop type was constant
   ↪  within farm and was therefore tested separately at the independent farm level (n=", nrow(farm_df),
   ↪  ") using one balanced centroid per farm and farms permuted within department. ",
  "The Crop type × Symptom status term was evaluated as the multivariate Symptomatic − Asymptomatic
   ↪  contrast for each farm and tested between crops at farm level. ",
  "PERMDISP is reported as a diagnostic of multivariate dispersion. PERMANOVA and PERMDISP P values are
   ↪  raw permutation P values, consistent with the beta-diversity analyses; no multiple-testing
   ↪  adjustment is applied to these multivariate tests. ",
  "R² is the proportion of multivariate variation explained by the corresponding PERMANOVA term.
   ↪  Significance: *** p < 0.001; ** < 0.01; * < 0.05; ns, not significant; —, not tested."
)

writeData(wb, "Main_Table", title, startRow=1, startCol=1, colNames=FALSE)
mergeCells(wb, "Main_Table", cols=1:ncol(main_table), rows=1:2)
addStyle(wb, "Main_Table", title_style, rows=1:2, cols=1:ncol(main_table), gridExpand=TRUE)
writeData(wb, "Main_Table", main_table, startRow=4, headerStyle=header_style)
addStyle(wb, "Main_Table", body_left, rows=5:(4+nrow(main_table)), cols=1:5, gridExpand=TRUE)
addStyle(wb, "Main_Table", body_center, rows=5:(4+nrow(main_table)), cols=4:ncol(main_table),
 ↪  gridExpand=TRUE)
note_row <- 6+nrow(main_table)
writeData(wb, "Main_Table", note, startRow=note_row, startCol=1, colNames=FALSE)
mergeCells(wb, "Main_Table", cols=1:ncol(main_table), rows=note_row:(note_row+7))
addStyle(wb, "Main_Table", note_style, rows=note_row:(note_row+7), cols=1:ncol(main_table),
 ↪  gridExpand=TRUE)
setColWidths(wb, "Main_Table", cols=1, widths=14)
setColWidths(wb, "Main_Table", cols=2, widths=32)
setColWidths(wb, "Main_Table", cols=3, widths=28)
setColWidths(wb, "Main_Table", cols=4:10, widths=12)
freezePane(wb, "Main_Table", firstActiveRow=5)

writeData(wb, "Raw_Results", raw_table)
writeData(wb, "Farm_Level_Crop", farm_df)
writeData(wb, "Farm_Interaction", delta)
writeData(wb, "Sampling_QC", farm_audit)
writeData(wb, "PERMANOVA_QC", qc)

# Redundancy diagnostics
writeData(wb, "Redundancy_Summary", redundancy_summary)
writeData(wb, "Redundancy_Pairs", cor_pairs)
writeData(wb, "Redundancy_Decisions", removed_log)

setColWidths(wb, "Redundancy_Summary", cols=1:2, widths=c(42, 70))
setColWidths(wb, "Redundancy_Pairs", cols=1:5, widths=c(22, 22, 14, 14, 18))
setColWidths(wb, "Redundancy_Decisions", cols=1:8,
             widths=c(8, 24, 24, 14, 14, 24, 24, 70))
freezePane(wb, "Redundancy_Pairs", firstActiveRow=2)
freezePane(wb, "Redundancy_Decisions", firstActiveRow=2)

# Univariate manuscript-facing table
writeData(wb, "Univariate_Table", "Table. Design-aware univariate analyses of individual soil
 ↪  physicochemical properties.", startRow=1, startCol=1, colNames=FALSE)
mergeCells(wb, "Univariate_Table", cols=1:ncol(uni_table), rows=1:2)
addStyle(wb, "Univariate_Table", title_style, rows=1:2, cols=1:ncol(uni_table), gridExpand=TRUE)
writeData(wb, "Univariate_Table", uni_table, startRow=4, headerStyle=header_style)
addStyle(wb, "Univariate_Table", body_left, rows=5:(4+nrow(uni_table)), cols=1:5, gridExpand=TRUE)
addStyle(wb, "Univariate_Table", body_center, rows=5:(4+nrow(uni_table)), cols=6:ncol(uni_table),
 ↪  gridExpand=TRUE)
setColWidths(wb, "Univariate_Table", cols=1, widths=42)
setColWidths(wb, "Univariate_Table", cols=2, widths=18)
setColWidths(wb, "Univariate_Table", cols=3, widths=44)
setColWidths(wb, "Univariate_Table", cols=4, widths=16)
setColWidths(wb, "Univariate_Table", cols=5, widths=34)
setColWidths(wb, "Univariate_Table", cols=6:ncol(uni_table), widths=14)
freezePane(wb, "Univariate_Table", firstActiveRow=5)
writeData(wb, "Univariate_Raw", uni_all)

xlsx_path <- file.path(tables_dir, "Table_PERMANOVA_PERMDISP_Physicochemical.xlsx")
saveWorkbook(wb, xlsx_path, overwrite=TRUE)
write.csv(raw_table, file.path(tables_dir, "Table_PERMANOVA_PERMDISP_Physicochemical_raw.csv"),
 ↪  row.names=FALSE)
write.csv(main_table, file.path(tables_dir, "Table_PERMANOVA_PERMDISP_Physicochemical.csv"),
 ↪  row.names=FALSE)
```

```r
write.csv(uni_all, file.path(tables_dir, "Table_Univariate_Physicochemical_raw.csv"), row.names=FALSE)
write.csv(uni_table, file.path(tables_dir, "Table_Univariate_Physicochemical.csv"), row.names=FALSE)
write.csv(redundancy_summary, file.path(tables_dir, "Redundancy_Summary.csv"), row.names=FALSE)
write.csv(cor_pairs, file.path(tables_dir, "Redundancy_Pairs.csv"), row.names=FALSE)
write.csv(removed_log, file.path(tables_dir, "Redundancy_Decisions.csv"), row.names=FALSE)
saveRDS(list(
  data=dat, X=X, farm_audit=farm_audit,
  condition_global=condition_global, condition_by_crop=condition_by_crop,
  crop_farm=crop_farm, interaction_farm=interaction_farm,
  permdisp=permdisp_all, main_table=main_table, raw_table=raw_table,
  min_p_crop=min_p_crop, n_assign=n_assign,
  univariate=uni_all, univariate_table=uni_table
), file.path(rds_dir, "PERMANOVA_Fisicoquimicos_results.rds"))

message("Definitive table: ", xlsx_path)
message("Output root: ", out_root)
```

Session information

r chunk: session_info

```r
sessionInfo()
```

07 — MANOVA of soil physicochemical variables with BH-adjusted univariate ANOVAs (Tables 4 and S3)

Supplementary analysis script

Sergio Pardo-Díaz

Contents

CHUNK 1 — setup 2
CHUNK 2 — helpers + metadata + outputs 2
CHUNK 3 — single dataset and checks 7
CHUNK 3.1 — MANOVA assumption diagnostics 9
CHUNK 4 — MANOVA helpers + p-adjust 11
CHUNK 5 — Global MANOVA 15
CHUNK 6 — MANOVA by crop 15
CHUNK 7 — manuscript tables 16
CHUNK 8 — build Table S3 in the layout of the previous file 17
CHUNK 8.1 — Table 4 MANOVA 19
CHUNK 9 — export TSV + consolidated Excel + Table S3 22
CHUNK 10 — summary 30
CHUNK 11 — sessionInfo 31

Script: `Physicochemical/07_MANOVA_Quimica_padjust_TableS3.Rmd`
Rendered: 2026-10-13 — code listing (the R Markdown source is shown as written; chunks were not executed, so no outputs are included).

YAML header / parameters

```yaml
title: "07 — MANOVA of soil physicochemical variables with BH-adjusted univariate ANOVAs (Tables 4 and S3)"
author: "Sergio Pardo-Díaz"
date: "`r format(Sys.time(), '%Y-%m-%d %H:%M')`"
output:
  html_document:
    toc: true
    toc_depth: 3
    number_sections: true
  pdf_document:
    toc: true
    number_sections: true

params:
  project_root: "."
  analysis_root: "03_Beta-Diversity/06_Quimica_PCA"
  out_subdir: "MANOVA"

  metadata_edaphic_path: "Metadata_Completo_plus_edaphic.xlsx"

  state_var: "Plant_Condition"
  crop_var: "Crop"

  crop_levels: ["Dominico_Harton", "Banano_Gros_Michael"]

  # Multiple-testing correction for follow-up univariate ANOVAs.
  # Recommended: "BH" (Benjamini-Hochberg/FDR).
  # To use Bonferroni instead, change to: "bonferroni"
  p_adjust_method: "BH"

  # Reviewer-ready MANOVA assumption diagnostics
  multivariate_outlier_alpha: 0.001
  multicollinearity_r_threshold: 0.90
```

```
    # Soil physicochemical variables initially considered
  vars_edaphic:
    - pH
    - Organic_M
    - P
    - Ca_Exchange
    - Mg_Exchange
    - K_Exchange
    - CEC
    - Zn
    - Cu
    - B
    - N_Total
    - Respirometry

  # Variables excluded a priori from MANOVA because of severe
  # multicollinearity identified in the initial diagnostic analysis.
  # CEC was strongly correlated with exchangeable Ca and, in some
  # datasets, Mg, and was also removed by VIF filtering in the dbRDA.
  excluded_collinear_vars:
    - CEC
```

Reviewer-ready changes

This version adds formal MANOVA assumption diagnostics: Mahalanobis-distance screening for multivariate outliers, Box's M tests for homogeneity of covariance matrices, and multicollinearity diagnostics among standardized response variables (correlation matrix, high-correlation pairs, and correlation-matrix condition number). The MANOVA response variables continue to be centered and scaled before analysis, and Pillai's trace remains the primary multivariate statistic.

CHUNK 1 — setup

r chunk: setup — options: message=TRUE, warning=TRUE

```r
knitr::opts_chunk$set(
  fig.width  = 8,
  fig.height = 6,
  dpi        = 300,
  dev        = "cairo_pdf"
)

suppressPackageStartupMessages({
  library(conflicted)
  library(tidyverse)
  library(glue)
  library(readxl)
  library(openxlsx)
  library(readr)
})

conflicted::conflicts_prefer(dplyr::select)
conflicted::conflicts_prefer(dplyr::filter)
conflicted::conflicts_prefer(dplyr::mutate)
```

CHUNK 2 — helpers + metadata + outputs

r chunk: helpers_and_metadata — options: message=TRUE, warning=TRUE

```r
.require_param <- function(x, name) {
  if (is.null(x) || length(x) == 0 || is.na(x) || x == "") {
    stop(glue("Param '{name}' is empty/NULL. Check the YAML header."))
  }
  x
}

strip_accents <- function(x) {
  x <- as.character(x)
  iconv(x, from = "UTF-8", to = "ASCII//TRANSLIT")
}

harmonize_condition <- function(x) {
  x2 <- as.character(x)
```

```
  x2 <- trimws(x2)
  x2 <- gsub("-", "_", x2)
  x2 <- gsub(" ", "_", x2)
  xlow <- tolower(x2)

  x2[xlow %in% c("healthy_plant", "healthy")]    <- "Healthy_Plant"
  x2[xlow %in% c("diseased_plant", "diseased")] <- "Diseased_Plant"

  factor(
    x2,
    levels = c(
      "Healthy_Plant",
      "Diseased_Plant"
    )
  )
}

normalize_crop_key <- function(x) {
  x2 <- as.character(x)
  x2 <- trimws(x2)
  x2 <- strip_accents(x2)
  x2 <- gsub("-", "_", x2)
  x2 <- gsub(" ", "_", x2)

  xlow <- tolower(x2)

  x2[
    xlow %in% c(
      "gros_michael",
      "banano_gros_michael",
      "banano_gros_michel"
    )
  ] <- "Banano_Gros_Michael"

  x2[
    xlow %in% c(
      "dominico_harton",
      "dominico"
    )
  ] <- "Dominico_Harton"

  x2
}

pretty_crop <- function(x) {
  x <- normalize_crop_key(x)

  dplyr::case_when(
    x == "Dominico_Harton"     ~ "Dominico Hartón",
    x == "Banano_Gros_Michael" ~ "Gros Michel",
    TRUE ~ x
  )
}

fmt_p <- function(p) {
  ifelse(
    is.na(p),
    NA_character_,
    ifelse(
      p < 0.001,
      "< 0.001",
      sprintf("%.3f", p)
    )
  )
}

p_to_stars <- function(p) {
  dplyr::case_when(
    is.na(p)  ~ NA_character_,
    p < 0.001 ~ "***",
    p < 0.01  ~ "**",
    p < 0.05  ~ "*",
    TRUE      ~ "ns"
  )
}
```

```
pretty_term <- function(x) {
  dplyr::case_when(
    x == "Crop_key" ~ "Crop",
    x == params$state_var ~ "Plant Condition",
    x == glue("Crop_key:{params$state_var}") ~
      "Crop × Plant Condition",
    x == glue("{params$state_var}:Crop_key") ~
      "Crop × Plant Condition",
    x == "Residuals" ~ "Residuals",
    TRUE ~ x
  )
}

# ============================================================
# Physicochemical variables
# ============================================================

# Complete set of measured variables initially considered
vars_edaphic_all <- unlist(
  params$vars_edaphic
)

# Variables excluded because of severe multicollinearity
excluded_collinear_vars <- unlist(
  params$excluded_collinear_vars
)

if (is.null(excluded_collinear_vars)) {
  excluded_collinear_vars <- character(0)
}

# Final response-variable set used in MANOVA
vars_edaphic <- setdiff(
  vars_edaphic_all,
  excluded_collinear_vars
)

# Safety check
if ("CEC" %in% vars_edaphic) {
  stop(
    paste0(
      "CEC remains in the MANOVA response-variable set. ",
      "Check params$excluded_collinear_vars."
    )
  )
}

p_adjust_method <- .require_param(
  params$p_adjust_method,
  "p_adjust_method"
)

nice_names <- c(
  "pH"           = "pH",
  "Organic_M"    = "Organic matter",
  "P"            = "P",
  "Ca_Exchange"  = "Ca²⁺",
  "Mg_Exchange"  = "Mg²⁺",
  "K_Exchange"   = "K⁺",
  "CEC"          = "CEC",
  "Zn"           = "Zn²⁺",
  "Cu"           = "Cu²⁺",
  "B"            = "B",
  "N_Total"      = "Total N",
  "Respirometry" = "Respirometry"
)

# ============================================================
# Output directory
# ============================================================

outdir_global <- function() {
  file.path(
    .require_param(
      params$project_root,
      "project_root"
```

```
    ),
    .require_param(
      params$analysis_root,
      "analysis_root"
    ),
    .require_param(
      params$out_subdir,
      "out_subdir"
    )
  )
}

dir.create(
  outdir_global(),
  recursive = TRUE,
  showWarnings = FALSE
)

# ============================================================
# Metadata
# ============================================================

metadata_path <- file.path(
  params$project_root,
  params$metadata_edaphic_path
)

if (!file.exists(metadata_path)) {
  stop(
    glue(
      "metadata_edaphic_path does not exist: {metadata_path}"
    )
  )
}

md_all <- readxl::read_xlsx(
  metadata_path
)

if (!"sample-id" %in% names(md_all)) {
  stop(
    "The Excel file must contain the EXACT column: 'sample-id'"
  )
}

if (!params$state_var %in% names(md_all)) {
  stop(
    glue(
      "Plant condition column not found: {params$state_var}"
    )
  )
}

if (!params$crop_var %in% names(md_all)) {
  stop(
    glue(
      "Crop column not found: {params$crop_var}"
    )
  )
}

md_all <- md_all %>%
  mutate(
    `sample-id` = as.character(`sample-id`),

    !!params$state_var :=
      harmonize_condition(
        .data[[params$state_var]]
      ),

    !!params$crop_var :=
      normalize_crop_key(
        .data[[params$crop_var]]
      )
  ) %>%
  filter(
```

```
    !is.na(.data[[params$state_var]]),
    !is.na(.data[[params$crop_var]])
  )

# ========================================================
# Check availability of FINAL MANOVA variables
# ========================================================

vars_present <- intersect(
  vars_edaphic,
  names(md_all)
)

vars_missing <- setdiff(
  vars_edaphic,
  names(md_all)
)

message(
  glue(
    "Metadata loaded: {nrow(md_all)} rows, ",
    "{ncol(md_all)} columns."
  )
)

message("\nPlant condition counts:")
print(
  table(
    md_all[[params$state_var]],
    useNA = "ifany"
  )
)

message("\nCrop counts:")
print(
  table(
    md_all[[params$crop_var]],
    useNA = "ifany"
  )
)

cat("\n==========================================\n")
cat("MANOVA RESPONSE-VARIABLE SELECTION\n")
cat("==========================================\n")

message(
  "Variables measured/initially considered: ",
  length(vars_edaphic_all)
)

message(
  "Excluded because of multicollinearity: ",
  ifelse(
    length(excluded_collinear_vars) == 0,
    "None",
    paste(
      excluded_collinear_vars,
      collapse = ", "
    )
  )
)

message(
  "Final variables entering MANOVA: ",
  length(vars_edaphic)
)

message(
  "Final MANOVA variables: ",
  paste(
    vars_edaphic,
    collapse = ", "
  )
)

message(
```

```r
  "Variables present in metadata: ",
  length(vars_present)
)

if (length(vars_missing) > 0) {
  message(
    "[!] Edaphic variables missing from Excel: ",
    paste(
      vars_missing,
      collapse = ", "
    )
  )
}

cat("============================================\n")

crop_levels <- normalize_crop_key(
  unlist(
    params$crop_levels
  )
)

message(
  "\nTarget crops (normalized): ",
  paste(
    crop_levels,
    collapse = " | "
  )
)

message(
  "p-adjust method for univariate ANOVAs: ",
  p_adjust_method
)
```

CHUNK 3 — single dataset and checks

r chunk: build_dataset — options: message=TRUE, warning=TRUE

```r
build_edaphic_dataset <- function(md, crop_key = NULL) {

  dat <- md %>%
    dplyr::mutate(Crop_key = normalize_crop_key(.data[[params$crop_var]])) %>%
    # The analysis is explicitly restricted to the two target crops.
    dplyr::filter(Crop_key %in% crop_levels)

  if (!is.null(crop_key)) {
    crop_key <- normalize_crop_key(crop_key)
    dat <- dat %>%
      dplyr::filter(Crop_key == crop_key)
  }

  dat <- dat %>%
    dplyr::select(
      `sample-id`,
      dplyr::all_of(params$state_var),
      Crop_key,
      dplyr::any_of(vars_present)
    ) %>%
    dplyr::mutate(
      dplyr::across(
        dplyr::any_of(vars_present),
        ~ suppressWarnings(as.numeric(.x))
      )
    )

  ed_cols <- intersect(vars_present, names(dat))
  ed_keep <- ed_cols[colSums(!is.na(dat[ed_cols])) > 0]

  if (length(ed_keep) < 2) {
    stop("Fewer than 2 edaphic variables available for MANOVA.")
  }

  dat2 <- dat %>%
    dplyr::select(
```

```
      `sample-id`,
      dplyr::all_of(params$state_var),
      Crop_key,
      dplyr::all_of(ed_keep)
    )

  cc <- stats::complete.cases(dat2[, ed_keep, drop = FALSE])

  if (any(!cc)) {
    message(glue("Removed {sum(!cc)} samples with NA in edaphic variables."))
    dat2 <- dat2[cc, , drop = FALSE]
  }

  # Exclude variables with no variation after filtering.
  variable_sd <- vapply(
    dat2[, ed_keep, drop = FALSE],
    stats::sd,
    numeric(1),
    na.rm = TRUE
  )

  zero_var <- names(variable_sd)[is.na(variable_sd) | variable_sd == 0]

  if (length(zero_var) > 0) {
    message(
      "[!] Variables excluded due to zero variance: ",
      paste(zero_var, collapse = ", ")
    )
  }

  ed_keep <- setdiff(ed_keep, zero_var)

  if (length(ed_keep) < 2) {
    stop("Fewer than 2 edaphic variables with variance > 0 for MANOVA.")
  }

  dat2 <- dat2 %>%
    dplyr::select(
      `sample-id`,
      dplyr::all_of(params$state_var),
      Crop_key,
      dplyr::all_of(ed_keep)
    ) %>%
    dplyr::mutate(
      !!params$state_var := droplevels(.data[[params$state_var]]),
      Crop_key = droplevels(factor(Crop_key, levels = crop_levels))
    )

  if (nlevels(dat2[[params$state_var]]) < 2) {
    stop("Plant_Condition has fewer than 2 levels after filtering.")
  }

  list(
    md = dat2,
    ed_cols = ed_keep
  )
}

ds_all <- build_edaphic_dataset(md_all, crop_key = NULL)
ds_DH  <- build_edaphic_dataset(md_all, crop_key = crop_levels[1])
ds_GM  <- build_edaphic_dataset(md_all, crop_key = crop_levels[2])

message("Global n = ", nrow(ds_all$md))
message("DH n = ", nrow(ds_DH$md))
message("GM n = ", nrow(ds_GM$md))

cat("\nGlobal Crop × Plant_Condition design:\n")

design_table <- table(
  Crop = ds_all$md$Crop_key,
  Plant_Condition = ds_all$md[[params$state_var]]
)

print(design_table)
```

CHUNK 3.1 — MANOVA assumption diagnostics

r chunk: manova_assumptions — options: message=TRUE, warning=TRUE

```r
# ============================================================
# Reviewer-ready MANOVA assumptions
# 1) multivariate outliers: Mahalanobis distance on standardized responses
# 2) homogeneity of covariance matrices: Box's M
# 3) multicollinearity among responses: correlation matrix, |r| threshold,
#    and condition number of the correlation matrix
# ============================================================

box_m_test <- function(Y, group) {
  Y <- as.matrix(Y)
  group <- droplevels(factor(group))
  keep <- complete.cases(Y) & !is.na(group)
  Y <- Y[keep, , drop = FALSE]
  group <- droplevels(group[keep])

  p <- ncol(Y)
  N <- nrow(Y)
  g <- nlevels(group)

  if (g < 2) stop("Box's M requires at least two groups.")

  split_idx <- split(seq_len(N), group)
  n_i <- vapply(split_idx, length, integer(1))

  if (any(n_i <= p)) {
    warning("At least one group has n <= number of response variables; Box's M may be unstable.")
  }

  S_i <- lapply(split_idx, function(ii) stats::cov(Y[ii, , drop = FALSE]))

  # Protect against singular/non-positive covariance matrices
  logdet <- function(S) {
    d <- determinant(S, logarithm = TRUE)
    if (d$sign <= 0) return(NA_real_)
    as.numeric(d$modulus)
  }

  pooled_num <- Reduce(`+`, Map(function(S, ni) (ni - 1) * S, S_i, n_i))
  S_pooled <- pooled_num / (N - g)

  ld_pool <- logdet(S_pooled)
  ld_i <- vapply(S_i, logdet, numeric(1))

  if (!is.finite(ld_pool) || any(!is.finite(ld_i))) {
    return(tibble(
      Chi_square = NA_real_, df = NA_real_, p_value = NA_real_,
      Note = "Singular/non-positive covariance matrix; Box's M not estimable reliably"
    ))
  }

  M <- (N - g) * ld_pool - sum((n_i - 1) * ld_i)
  C <- ((2 * p^2 + 3 * p - 1) / (6 * (p + 1) * (g - 1))) *
    (sum(1 / (n_i - 1)) - 1 / (N - g))

  chi_sq <- (1 - C) * M
  df_box <- (g - 1) * p * (p + 1) / 2
  p_val <- stats::pchisq(chi_sq, df = df_box, lower.tail = FALSE)

  tibble(
    Chi_square = chi_sq,
    df = df_box,
    p_value = p_val,
    Note = "Approximate chi-square Box's M test"
  )
}

manova_assumption_diagnostics <- function(ds, grouping, model_label) {
  dat <- ds$md
  vars <- ds$ed_cols

  # Standardized responses, identical scaling principle to MANOVA model
  Y <- scale(dat[, vars, drop = FALSE])
  Y <- as.matrix(Y)
```

```r
  # ---------- Multivariate outliers ----------
  center <- colMeans(Y)
  cov_mat <- stats::cov(Y)

  mahal <- tryCatch(
    stats::mahalanobis(Y, center = center, cov = cov_mat),
    error = function(e) rep(NA_real_, nrow(Y))
  )

  p_dim <- ncol(Y)
  alpha <- params$multivariate_outlier_alpha
  cutoff <- stats::qchisq(1 - alpha, df = p_dim)
  mahal_p <- stats::pchisq(mahal, df = p_dim, lower.tail = FALSE)

  outliers <- tibble(
    Model = model_label,
    Sample_ID = dat$`sample-id`,
    Mahalanobis_D2 = mahal,
    df = p_dim,
    p_value = mahal_p,
    Cutoff_alpha = alpha,
    D2_cutoff = cutoff,
    Multivariate_outlier = !is.na(mahal) & mahal > cutoff
  )

  # ---------- Box's M ----------
  group <- grouping(dat)
  boxm <- box_m_test(Y, group) %>%
    mutate(
      Model = model_label,
      Grouping = deparse(substitute(grouping)),
      N = nrow(Y),
      Groups = nlevels(droplevels(factor(group))),
      Responses = p_dim,
      .before = 1
    )

  # ---------- Multicollinearity ----------
  cor_mat <- stats::cor(Y, use = "pairwise.complete.obs")
  kappa_cor <- tryCatch(kappa(cor_mat, exact = TRUE), error = function(e) NA_real_)

  cor_long <- as.data.frame(as.table(cor_mat), stringsAsFactors = FALSE) %>%
    as_tibble() %>%
    rename(Response1 = Var1, Response2 = Var2, r = Freq) %>%
    mutate(Model = model_label, .before = 1)

  idx <- which(upper.tri(cor_mat), arr.ind = TRUE)
  high_pairs <- tibble(
    Model = model_label,
    Response1 = rownames(cor_mat)[idx[, 1]],
    Response2 = colnames(cor_mat)[idx[, 2]],
    r = cor_mat[idx]
  ) %>%
    mutate(abs_r = abs(r)) %>%
    filter(abs_r >= params$multicollinearity_r_threshold) %>%
    arrange(desc(abs_r))

  coll_summary <- tibble(
    Model = model_label,
    N = nrow(Y),
    Responses = p_dim,
    Max_abs_pairwise_r = if (length(idx) > 0) max(abs(cor_mat[idx]), na.rm = TRUE) else NA_real_,
    High_correlation_threshold = params$multicollinearity_r_threshold,
    N_high_correlation_pairs = nrow(high_pairs),
    Correlation_matrix_condition_number = kappa_cor
  )

  list(
    outliers = outliers,
    boxm = boxm,
    cor_matrix = cor_long,
    high_pairs = high_pairs,
    collinearity_summary = coll_summary
  )
}
```

```r
assump_global <- manova_assumption_diagnostics(
  ds_all,
  grouping = function(d) interaction(d$Crop_key, d[[params$state_var]], drop = TRUE),
  model_label = "Global: Crop × Plant_Condition"
)

assump_DH <- manova_assumption_diagnostics(
  ds_DH,
  grouping = function(d) d[[params$state_var]],
  model_label = "Dominico Hartón: Plant_Condition"
)

assump_GM <- manova_assumption_diagnostics(
  ds_GM,
  grouping = function(d) d[[params$state_var]],
  model_label = "Gros Michel: Plant_Condition"
)

manova_outliers <- bind_rows(assump_global$outliers, assump_DH$outliers, assump_GM$outliers)
manova_boxm <- bind_rows(assump_global$boxm, assump_DH$boxm, assump_GM$boxm)
manova_correlations <- bind_rows(assump_global$cor_matrix, assump_DH$cor_matrix, assump_GM$cor_matrix)
manova_high_cor_pairs <- bind_rows(assump_global$high_pairs, assump_DH$high_pairs, assump_GM$high_pairs)
manova_collinearity_summary <- bind_rows(
  assump_global$collinearity_summary,
  assump_DH$collinearity_summary,
  assump_GM$collinearity_summary
)

cat("\nMultivariate outlier counts:\n")
print(manova_outliers %>% count(Model, Multivariate_outlier))
cat("\nBox's M results:\n")
print(manova_boxm)
cat("\nMulticollinearity summary:\n")
print(manova_collinearity_summary)
```

CHUNK 4 — MANOVA helpers + p-adjust

r chunk: manova_helpers — options: message=TRUE, warning=TRUE

```r
extract_manova_test <- function(summary_obj, test_name, model_label) {

  # ============================================================
  # Extract original MANOVA statistics matrix
  # ============================================================

  stats_mat <- summary_obj$stats

  if (is.null(stats_mat)) {
    stop(
      paste0(
        "summary_obj$stats is NULL for ",
        model_label,
        " / ",
        test_name
      )
    )
  }

  # ------------------------------------------------------------
  # IMPORTANT:
  # Extract term names BEFORE conversion to data.frame
  # ------------------------------------------------------------

  term_names <- rownames(stats_mat)

  if (is.null(term_names) || length(term_names) != nrow(stats_mat)) {

    stop(
      paste0(
        "Could not recover MANOVA term names for ",
        model_label,
        " / ",
        test_name,
        ". Number of rows in stats = ",
```

```
      nrow(stats_mat),
      "; number of term names = ",
      length(term_names),
      "."
    )
  )
}

# Convert statistics matrix to data.frame
tb <- as.data.frame(
  stats_mat,
  check.names = FALSE
)

# Add terms explicitly
tb$Term <- term_names

# ============================================================
# Identify columns
# ============================================================

test_stat_col <- intersect(
  c(
    "Pillai",
    "Wilks",
    "Hotelling-Lawley",
    "Roy"
  ),
  colnames(tb)
)[1]

approx_f_col <- intersect(
  c(
    "approx F",
    "approx. F",
    "F approx",
    "F"
  ),
  colnames(tb)
)[1]

num_df_col <- intersect(
  c(
    "num Df",
    "num.Df",
    "Num Df",
    "NumDf"
  ),
  colnames(tb)
)[1]

den_df_col <- intersect(
  c(
    "den Df",
    "den.Df",
    "Den Df",
    "DenDf"
  ),
  colnames(tb)
)[1]

p_col <- intersect(
  c(
    "Pr(>F)",
    "Pr(>F) "
  ),
  colnames(tb)
)[1]

# ============================================================
# QC
# ============================================================

detected <- c(
  test_stat_col,
  approx_f_col,
```

```
    num_df_col,
    den_df_col,
    p_col
  )

  if (any(is.na(detected))) {

    cat("\nAvailable MANOVA columns:\n")
    print(colnames(tb))

    stop(
      paste0(
        "Could not identify all MANOVA columns for ",
        model_label,
        " / ",
        test_name
      )
    )
  }

  # ============================================================
  # Output
  # ============================================================

  tibble(
    Model = model_label,
    Test = test_name,
    Term = trimws(tb$Term),

    Df = as.numeric(tb$Df),

    Test_statistic = as.numeric(
      tb[[test_stat_col]]
    ),

    Approx_F = as.numeric(
      tb[[approx_f_col]]
    ),

    Num_Df = as.numeric(
      tb[[num_df_col]]
    ),

    Den_Df = as.numeric(
      tb[[den_df_col]]
    ),

    p_value = as.numeric(
      tb[[p_col]]
    ),

    sig = p_to_stars(
      as.numeric(tb[[p_col]])
    )
  )
}
extract_univariate_from_manova <- function(fit, model_label) {

  sm <- summary.aov(fit)

  purrr::imap_dfr(sm, function(tab, response_name) {

    tb <- as.data.frame(tab)
    tb$Term <- rownames(tb)
    rownames(tb) <- NULL

    p_col <- grep("^Pr\\(", colnames(tb), value = TRUE)[1]

    response_name <- trimws(response_name)
response_name <- sub("^Response\\s+", "", response_name)
    tb %>%
      as_tibble() %>%
      transmute(
        Model = model_label,
        Response = dplyr::coalesce(
          unname(nice_names[response_name]),
```

```
          response_name
        ),
        Term = Term,
        Df = Df,
        `Sum Sq` = `Sum Sq`,
        `Mean Sq` = `Mean Sq`,
        `F value` = `F value`,
        p_value = .data[[p_col]]
      )
  })
}

# Multiple-comparison correction:
# Each Model × Term combination constitutes one family of hypotheses.
#
# CEC was excluded from the MANOVA response set because of severe
# multicollinearity identified in the initial diagnostics.
# Therefore, the current analyses contain 11 physicochemical
# response variables.
#
# Examples:
# - Global / Crop: 11 physicochemical variables
# - Global / Plant Condition: 11 physicochemical variables
# - Global / Crop × Plant Condition: 11 physicochemical variables
# - Dominico Hartón / Plant Condition: 11 variables
# - Gros Michel / Plant Condition: 11 variables
#
# Residuals are not adjusted.
add_p_adjust <- function(df, method = p_adjust_method) {

  df %>%
    mutate(
      Term_pretty = pretty_term(Term)
    ) %>%
    group_by(Model, Term_pretty) %>%
    mutate(
      p_adjust = if_else(
        Term_pretty == "Residuals",
        NA_real_,
        stats::p.adjust(p_value, method = method)
      ),
      sig_raw = p_to_stars(p_value),
      sig_adjust = p_to_stars(p_adjust)
    ) %>%
    ungroup()
}

fit_manova_model <- function(ds, rhs, model_label) {

  dat <- ds$md
  ed_cols <- ds$ed_cols

  Y <- scale(dat[, ed_cols, drop = FALSE])
  Y_df <- as.data.frame(Y)

  dat_model <- bind_cols(
    dat %>%
      dplyr::select(
        `sample-id`,
        dplyr::all_of(params$state_var),
        Crop_key
      ),
    Y_df
  )

  fit <- stats::manova(
    as.formula(
      glue(
        "cbind({paste(colnames(Y_df), collapse=', ')}) ~ {rhs}"
      )
    ),
    data = dat_model
  )

  wilks <- extract_manova_test(
    summary(fit, test = "Wilks"),
```

```
    "Wilks",
    model_label
  )

  pillai <- extract_manova_test(
    summary(fit, test = "Pillai"),
    "Pillai",
    model_label
  )

  hl <- extract_manova_test(
    summary(fit, test = "Hotelling-Lawley"),
    "Hotelling-Lawley",
    model_label
  )

  roy <- extract_manova_test(
    summary(fit, test = "Roy"),
    "Roy",
    model_label
  )

  uni <- extract_univariate_from_manova(
    fit,
    model_label
  ) %>%
    add_p_adjust(method = p_adjust_method)

  list(
    fit = fit,
    multivariate = bind_rows(
      wilks,
      pillai,
      hl,
      roy
    ),
    univariate = uni
  )
}
```

CHUNK 5 — Global MANOVA

r chunk: manova_global — options: message=TRUE, warning=TRUE

```
res_global <- fit_manova_model(
  ds = ds_all,
  rhs = glue("Crop_key * `{params$state_var}`"),
  model_label = "Global: Crop * Plant_Condition"
)

manova_global_mv <- res_global$multivariate
manova_global_uv <- res_global$univariate

manova_global_mv
```

CHUNK 6 — MANOVA by crop

r chunk: manova_by_crop — options: message=TRUE, warning=TRUE

```
res_DH <- fit_manova_model(
  ds = ds_DH,
  rhs = glue("`{params$state_var}`"),
  model_label = "Dominico Hartón: Plant_Condition"
)

res_GM <- fit_manova_model(
  ds = ds_GM,
  rhs = glue("`{params$state_var}`"),
  model_label = "Gros Michel: Plant_Condition"
)

manova_bycrop_mv <- bind_rows(
  res_DH$multivariate,
  res_GM$multivariate
```

```
)

manova_bycrop_uv <- bind_rows(
  res_DH$univariate,
  res_GM$univariate
)

manova_bycrop_mv
```

CHUNK 7 — manuscript tables

r chunk: paper_tables — options: message=TRUE, warning=TRUE

```
out_dir <- outdir_global()

# ============================================================
# MANOVA table for manuscript
# ============================================================
# Pillai's trace is the primary multivariate statistic.
#
# Report separately:
# - Pillai's trace
# - Approximate F
# - numerator df
# - denominator df
# - p-value
#
# This directly addresses Reviewer Comment 22.
# ============================================================

paper_mv <- bind_rows(
  manova_global_mv,
  manova_bycrop_mv
) %>%

  filter(
    Test == "Pillai"
  ) %>%

  mutate(
    Term = pretty_term(Term)
  ) %>%

  # Residuals do not have a Pillai test result and are not required
  # in the publication table.
  filter(
    Term != "Residuals"
  ) %>%

  transmute(
    Model,
    Term,
    Test,

    `Pillai's trace` = round(
      Test_statistic,
      3
    ),

    `Approx. F` = round(
      Approx_F,
      3
    ),

    `Num Df` = Num_Df,

    `Den Df` = Den_Df,

    `p-value` = fmt_p(
      p_value
    ),

    sig = p_to_stars(
      p_value
```

```
    )
  )


# ============================================================
# Follow-up univariate ANOVA
# ============================================================

paper_uv <- bind_rows(
  manova_global_uv,
  manova_bycrop_uv
) %>%

  filter(
    Term_pretty != "Residuals"
  ) %>%

  transmute(
    Model,
    `Response variable` = Response,
    Term = Term_pretty,
    Df,
    `F value` = round(
      `F value`,
      3
    ),
    `p-value` = fmt_p(
      p_value
    ),
    `p-adjust` = fmt_p(
      p_adjust
    ),
    sig = sig_adjust
  )


# ============================================================
# Inspect
# ============================================================

cat("\n============================================\n")
cat("MANOVA TABLE — PILLAI TEST\n")
cat("============================================\n")

print(
  paper_mv
)

cat("============================================\n")

paper_uv
```

CHUNK 8 — build Table S3 in the layout of the previous file

r chunk: build_table_s3 — options: message=TRUE, warning=TRUE

```
# This table reproduces the layout of TableS3_MAnovaUniva.xlsx:
# - title on top
# - header
# - separator rows per model
# - variables in rows
# - term, Df, F, p-value, p-adjust and adjusted significance
# - footnote

all_uv_table <- bind_rows(
  manova_global_uv,
  manova_bycrop_uv
) %>%
  mutate(
    Term = Term_pretty,
    `F value display` = if_else(
      Term == "Residuals",
      NA_character_,
      sprintf("%.3f", `F value`)
    ),
```

```
    `p-value display` = if_else(
      Term == "Residuals",
      NA_character_,
      fmt_p(p_value)
    ),
    `p-adjust display` = if_else(
      Term == "Residuals",
      NA_character_,
      fmt_p(p_adjust)
    ),
    `sig display` = if_else(
      Term == "Residuals",
      NA_character_,
      sig_adjust
    )
  ) %>%
  select(
    Model,
    Response,
    Term,
    Df,
    `F value display`,
    `p-value display`,
    `p-adjust display`,
    `sig display`
  )

model_order <- c(
  "Global: Crop * Plant_Condition",
  "Dominico Hartón: Plant_Condition",
  "Gros Michel: Plant_Condition"
)

table_s3_rows <- purrr::map_dfr(
  model_order,
  function(mod) {

    dat_mod <- all_uv_table %>%
      filter(Model == mod)

    if (nrow(dat_mod) == 0) {
      return(tibble())
    }

    section_row <- tibble(
      `Response variable` = mod,
      Term = NA_character_,
      Df = NA_character_,
      `F value` = NA_character_,
      `p-value` = NA_character_,
      `p-adjust` = NA_character_,
      sig = NA_character_
    )

    data_rows <- dat_mod %>%
      transmute(
        `Response variable` = Response,
        Term,
        Df = as.character(Df),
        `F value` = `F value display`,
        `p-value` = `p-value display`,
        `p-adjust` = `p-adjust display`,
        sig = `sig display`
      )

    bind_rows(
      section_row,
      data_rows
    )
  }
)

table_s3_title <- glue(
  "Table S3. Univariate ANOVA results for individual soil physicochemical ",
  "variables within the global and crop-specific MANOVA models, including ",
  "{p_adjust_method}-adjusted p-values."
```

```
)

table_s3_note <- glue(
  "sig: *** adjusted p < 0.001; ** adjusted p < 0.01; * adjusted p < 0.05; ",
  "ns, not significant. p-adjust values were calculated using the ",
  "{p_adjust_method} method within each Model × Term family of soil ",
  "physicochemical variables. Response variables were standardized prior to analysis."
)

table_s3_rows
```

CHUNK 8.1 — Table 4 MANOVA

r chunk: build_table4_manova — options: message=TRUE, warning=TRUE

```
# ============================================================
# TABLE 4 — MANOVA
# Reviewer Comment 22
# ============================================================

# ------------------------------------------------------------
# Model order
# ------------------------------------------------------------

table4_raw <- paper_mv %>%
  mutate(

    Model_group = case_when(

      Model == "Global: Crop * Plant_Condition" ~
        "GLOBAL MODEL: Crop × Plant Condition",

      Model == "Dominico Hartón: Plant_Condition" ~
        "Dominico Harton",

      Model == "Gros Michel: Plant_Condition" ~
        "Gros Michel",

      TRUE ~ Model
    ),

    Model_order = case_when(

      Model == "Global: Crop * Plant_Condition" ~ 1,

      Model == "Dominico Hartón: Plant_Condition" ~ 2,

      Model == "Gros Michel: Plant_Condition" ~ 3,

      TRUE ~ 99
    ),

    Term_order = case_when(

      Term == "Crop" ~ 1,

      Term == "Plant Condition" ~ 2,

      Term == "Crop × Plant Condition" ~ 3,

      TRUE ~ 99
    )
  ) %>%

  arrange(
    Model_order,
    Term_order
  )


# ============================================================
# Publication-ready rows
# ============================================================
```

```
# Global model
table4_global <- table4_raw %>%

  filter(
    Model_order == 1
  ) %>%

  transmute(

    Model = if_else(
  row_number() == 1,
  "GLOBAL MODEL: Crop × Plant Condition",
  NA_character_
),

    Term,

    `Pillai's trace` = sprintf(
      "%.3f",
      `Pillai's trace`
    ),

    `Approx. F` = sprintf(
      "%.3f",
      `Approx. F`
    ),

    `Num Df` = as.character(
      `Num Df`
    ),

    `Den Df` = as.character(
  as.integer(round(`Den Df`))
),

    `p-value`,

    sig
  )


# ------------------------------------------------------------
# Section separator
# ------------------------------------------------------------

table4_separator <- tibble(

  Model = "WITHIN CULTIVAR",

  Term = NA_character_,

  `Pillai's trace` = NA_character_,

  `Approx. F` = NA_character_,

  `Num Df` = NA_character_,

  `Den Df` = NA_character_,

  `p-value` = NA_character_,

  sig = NA_character_
)


# ------------------------------------------------------------
# Within-cultivar analyses
# ------------------------------------------------------------

table4_crop <- table4_raw %>%

  filter(
    Model_order %in% c(
      2,
      3
    )
```

```
  ) %>%

  transmute(

    Model = Model_group,

    Term,

    `Pillai's trace` = sprintf(
      "%.3f",
      `Pillai's trace`
    ),

    `Approx. F` = sprintf(
      "%.3f",
      `Approx. F`
    ),

    `Num Df` = as.character(
      `Num Df`
    ),

    `Den Df` = sprintf(
      "%.1f",
      `Den Df`
    ),

    `p-value`,

    sig
  )


# ============================================================
# Final table
# ============================================================

table4_rows <- bind_rows(

  table4_global,

  table4_separator,

  table4_crop

)


# ============================================================
# Title and footnote
# ============================================================

table4_title <- paste0(

  "Table 4. MANOVA results for soil physicochemical variables in ",

  "Non symptomatic and Symptomatic banana and plantain plants ",

  "(Pillai's test)."

)


table4_note <- paste0(

  "Plant Condition: Non symptomatic vs. Symptomatic. ",

  "Crop: Gros Michel vs. Dominico Harton. ",

  "CEC was excluded from the MANOVA response set because of severe ",

  "multicollinearity identified during the assumption diagnostics; ",

  "11 physicochemical response variables were included in the final analysis. ",

  "Pillai's trace, approximate F statistic, numerator degrees of freedom ",
```

```
  "(Num Df), denominator degrees of freedom (Den Df), and p-values are ",
  "reported separately. ",
  "sig: *** p < 0.001; ** p < 0.01; * p < 0.05; ns, not significant."
)

# ============================================================
# QC
# ============================================================

cat("\n==========================================\n")

cat("TABLE 4 — FINAL MANOVA\n")

cat("==========================================\n")

print(
  table4_rows
)

cat("==========================================\n")
```

CHUNK 9 — export TSV + consolidated Excel + Table S3

r chunk: export_results — options: message=TRUE, warning=TRUE

```
# ---------- Full TSV files ----------
write_tsv(
  manova_global_mv,
  file.path(out_dir, "MANOVA_global_multivariate.tsv")
)

write_tsv(
  manova_global_uv,
  file.path(out_dir, "MANOVA_global_univariate.tsv")
)

write_tsv(
  manova_bycrop_mv,
  file.path(out_dir, "MANOVA_byCrop_multivariate.tsv")
)

write_tsv(
  manova_bycrop_uv,
  file.path(out_dir, "MANOVA_byCrop_univariate.tsv")
)

write_tsv(
  paper_mv,
  file.path(out_dir, "MANOVA_paper_multivariate.tsv")
)

write_tsv(
  paper_uv,
  file.path(out_dir, "MANOVA_paper_univariate_with_padjust.tsv")
)

# ---------- Reviewer-ready assumption diagnostics ----------
write_tsv(manova_outliers, file.path(out_dir, "MANOVA_assumption_multivariate_outliers.tsv"))
write_tsv(manova_boxm, file.path(out_dir, "MANOVA_assumption_BoxM.tsv"))
write_tsv(manova_correlations, file.path(out_dir, "MANOVA_assumption_response_correlations.tsv"))
write_tsv(manova_high_cor_pairs, file.path(out_dir, "MANOVA_assumption_high_correlation_pairs.tsv"))
write_tsv(manova_collinearity_summary, file.path(out_dir,
 ↪  "MANOVA_assumption_multicollinearity_summary.tsv"))

# ---------- Consolidated Excel ----------
wb <- createWorkbook()

# ---------- Variables included/excluded from MANOVA ----------

manova_variable_selection <- tibble(
```

```
  Variable = vars_edaphic_all,
  Variable_nice = dplyr::coalesce(
    unname(nice_names[vars_edaphic_all]),
    vars_edaphic_all
  ),
  Status = if_else(
    vars_edaphic_all %in% excluded_collinear_vars,
    "Excluded",
    "Included"
  ),
  Reason = if_else(
    vars_edaphic_all %in% excluded_collinear_vars,
    paste0(
      "Excluded because of severe multicollinearity in the initial ",
      "MANOVA diagnostics; CEC was strongly correlated with exchangeable ",
      "Ca and, in some datasets, Mg. Its exclusion was also consistent ",
      "with the VIF-based filtering used for dbRDA."
    ),
    "Retained in the MANOVA response set."
  )
)

addWorksheet(
  wb,
  "Variable_selection"
)

writeData(
  wb,
  "Variable_selection",
  manova_variable_selection
)

addWorksheet(wb, "Global_multivariate")
writeData(wb, "Global_multivariate", manova_global_mv)

addWorksheet(wb, "Global_univariate")
writeData(wb, "Global_univariate", manova_global_uv)

addWorksheet(wb, "ByCrop_multivariate")
writeData(wb, "ByCrop_multivariate", manova_bycrop_mv)

addWorksheet(wb, "ByCrop_univariate")
writeData(wb, "ByCrop_univariate", manova_bycrop_uv)

addWorksheet(wb, "Paper_multivariate")
writeData(wb, "Paper_multivariate", paper_mv)

addWorksheet(wb, "Paper_univariate")
writeData(wb, "Paper_univariate", paper_uv)

addWorksheet(wb, "Assump_Mahalanobis")
writeData(wb, "Assump_Mahalanobis", manova_outliers)

addWorksheet(wb, "Assump_BoxM")
writeData(wb, "Assump_BoxM", manova_boxm)

addWorksheet(wb, "Assump_Correlations")
writeData(wb, "Assump_Correlations", manova_correlations)

addWorksheet(wb, "Assump_HighCorPairs")
writeData(wb, "Assump_HighCorPairs", manova_high_cor_pairs)

addWorksheet(wb, "Assump_Collinearity")
writeData(wb, "Assump_Collinearity", manova_collinearity_summary)

# ============================================================
# TABLE 4 — publication-ready MANOVA
# ============================================================

addWorksheet(
  wb,
  "Table4_MANOVA"
)
```

```r
# ------------------------------------------------------------
# Title
# ------------------------------------------------------------

writeData(
  wb,
  sheet = "Table4_MANOVA",
  x = table4_title,
  startRow = 1,
  startCol = 1,
  colNames = FALSE
)

mergeCells(
  wb,
  sheet = "Table4_MANOVA",
  cols = 1:8,
  rows = 1
)


# ------------------------------------------------------------
# Header
# ------------------------------------------------------------

table4_headers <- c(
  "Model",
  "Term",
  "Pillai's trace",
  "Approx. F",
  "Num Df",
  "Den Df",
  "p-value",
  "sig"
)

writeData(
  wb,
  sheet = "Table4_MANOVA",
  x = t(table4_headers),
  startRow = 3,
  startCol = 1,
  colNames = FALSE
)


# ------------------------------------------------------------
# Data
# ------------------------------------------------------------

writeData(
  wb,
  sheet = "Table4_MANOVA",
  x = table4_rows,
  startRow = 4,
  startCol = 1,
  colNames = FALSE
)


# ------------------------------------------------------------
# Footnote
# ------------------------------------------------------------

table4_note_row <- 4 + nrow(table4_rows) + 1

writeData(
  wb,
  sheet = "Table4_MANOVA",
  x = table4_note,
  startRow = table4_note_row,
  startCol = 1,
  colNames = FALSE
)

mergeCells(
```

```
  wb,
  sheet = "Table4_MANOVA",
  cols = 1:8,
  rows = table4_note_row
)


# ============================================================
# Styles
# ============================================================

table4_title_style <- createStyle(

  fontName = "Times New Roman",
  fontSize = 11,
  textDecoration = "bold",

  halign = "left",
  valign = "center",

  wrapText = TRUE
)


table4_header_style <- createStyle(

  fontName = "Times New Roman",
  fontSize = 11,
  textDecoration = "bold",

  fgFill = "#D9E2F3",

  halign = "center",
  valign = "center",

  border = c(
    "top",
    "bottom",
    "left",
    "right"
  ),

  borderColour = "#808080"
)


table4_body_style <- createStyle(

  fontName = "Times New Roman",
  fontSize = 11,

  valign = "center",

  border = c(
    "top",
    "bottom",
    "left",
    "right"
  ),

  borderColour = "#B7B7B7"
)


table4_section_style <- createStyle(

  fontName = "Times New Roman",
  fontSize = 11,

  textDecoration = "bold",

  fgFill = "#EDEDED",

  halign = "left",

  border = c(
```

```
    "top",
    "bottom",
    "left",
    "right"
  ),

  borderColour = "#B7B7B7"
)


table4_note_style <- createStyle(

  fontName = "Times New Roman",
  fontSize = 10,

  textDecoration = "italic",

  wrapText = TRUE,

  valign = "top"
)


# ------------------------------------------------------------
# Apply styles
# ------------------------------------------------------------

addStyle(
  wb,
  sheet = "Table4_MANOVA",
  style = table4_title_style,
  rows = 1,
  cols = 1:8,
  gridExpand = TRUE
)


addStyle(
  wb,
  sheet = "Table4_MANOVA",
  style = table4_header_style,
  rows = 3,
  cols = 1:8,
  gridExpand = TRUE
)


data_rows_table4 <- 4:(
  3 + nrow(table4_rows)
)

addStyle(
  wb,
  sheet = "Table4_MANOVA",
  style = table4_body_style,
  rows = data_rows_table4,
  cols = 1:8,
  gridExpand = TRUE
)


# ------------------------------------------------------------
# WITHIN CULTIVAR separator
# ------------------------------------------------------------

within_row <- which(
  table4_rows$Model == "WITHIN CULTIVAR"
) + 3

if (length(within_row) == 1) {

  addStyle(
    wb,
    sheet = "Table4_MANOVA",
    style = table4_section_style,
    rows = within_row,
```

```
    cols = 1:8,
    gridExpand = TRUE
  )

  mergeCells(
    wb,
    sheet = "Table4_MANOVA",
    cols = 1:8,
    rows = within_row
  )
}


# ------------------------------------------------------------
# Bold model-start rows
# ------------------------------------------------------------

model_start_rows <- which(
  table4_rows$Model %in% c(
    "GLOBAL MODEL: Crop × Plant Condition",
    "Dominico Harton",
    "Gros Michel"
  )
) + 3

model_bold_style <- createStyle(

  fontName = "Times New Roman",
  fontSize = 11,

  textDecoration = "bold"
)

for (rr in model_start_rows) {

  addStyle(
    wb,
    sheet = "Table4_MANOVA",
    style = model_bold_style,
    rows = rr,
    cols = 1,
    stack = TRUE
  )
}


# ------------------------------------------------------------
# Footnote
# ------------------------------------------------------------

addStyle(
  wb,
  sheet = "Table4_MANOVA",
  style = table4_note_style,
  rows = table4_note_row,
  cols = 1:8,
  gridExpand = TRUE
)


# ============================================================
# Dimensions
# ============================================================

setColWidths(
  wb,
  sheet = "Table4_MANOVA",
  cols = 1:8,
  widths = c(
    36,  # Model
    28,  # Term
    15,  # Pillai
    12,  # F
    10,  # Num df
    10,  # Den df
    12,  # p
```

```
    8    # sig
  )
)


setRowHeights(
  wb,
  sheet = "Table4_MANOVA",
  rows = 1,
  heights = 35
)


setRowHeights(
  wb,
  sheet = "Table4_MANOVA",
  rows = table4_note_row,
  heights = 55
)


freezePane(
  wb,
  sheet = "Table4_MANOVA",
  firstActiveRow = 4
)

saveWorkbook(
  wb,
  file.path(out_dir, "MANOVA_edaphic_results.xlsx"),
  overwrite = TRUE
)

# ---------- Table S3 as a separate file ----------
wb_s3 <- createWorkbook()

addWorksheet(wb_s3, "TableS3_MAnovaUniva")

# Title
writeData(
  wb_s3,
  sheet = "TableS3_MAnovaUniva",
  x = table_s3_title,
  startRow = 1,
  startCol = 1,
  colNames = FALSE
)

mergeCells(
  wb_s3,
  sheet = "TableS3_MAnovaUniva",
  cols = 1:7,
  rows = 1
)

# Header
headers <- c(
  "Response variable",
  "Term",
  "Df",
  "F value",
  "p-value",
  "p-adjust",
  "sig"
)

writeData(
  wb_s3,
  sheet = "TableS3_MAnovaUniva",
  x = t(headers),
  startRow = 2,
  startCol = 1,
  colNames = FALSE
)

# Data
```

```r
writeData(
  wb_s3,
  sheet = "TableS3_MAnovaUniva",
  x = table_s3_rows,
  startRow = 3,
  startCol = 1,
  colNames = FALSE
)

note_row <- 3 + nrow(table_s3_rows) + 1

writeData(
  wb_s3,
  sheet = "TableS3_MAnovaUniva",
  x = table_s3_note,
  startRow = note_row,
  startCol = 1,
  colNames = FALSE
)

mergeCells(
  wb_s3,
  sheet = "TableS3_MAnovaUniva",
  cols = 1:7,
  rows = note_row
)

# ---------- Styles ----------
title_style <- createStyle(
  textDecoration = "bold",
  halign = "left",
  valign = "center",
  wrapText = TRUE
)

header_style <- createStyle(
  textDecoration = "bold",
  halign = "center",
  valign = "center",
  border = "Bottom"
)

section_style <- createStyle(
  textDecoration = "bold",
  halign = "left"
)

note_style <- createStyle(
  textDecoration = "italic",
  wrapText = TRUE,
  valign = "top"
)

addStyle(
  wb_s3,
  sheet = "TableS3_MAnovaUniva",
  style = title_style,
  rows = 1,
  cols = 1:7,
  gridExpand = TRUE
)

addStyle(
  wb_s3,
  sheet = "TableS3_MAnovaUniva",
  style = header_style,
  rows = 2,
  cols = 1:7,
  gridExpand = TRUE
)

# Detect and style the section/model rows
section_rows <- which(
  table_s3_rows$Term %>% is.na()
) + 2
```

```r
for (rr in section_rows) {
  addStyle(
    wb_s3,
    sheet = "TableS3_MAnovaUniva",
    style = section_style,
    rows = rr,
    cols = 1:7,
    gridExpand = TRUE
  )

  mergeCells(
    wb_s3,
    sheet = "TableS3_MAnovaUniva",
    cols = 1:7,
    rows = rr
  )
}

addStyle(
  wb_s3,
  sheet = "TableS3_MAnovaUniva",
  style = note_style,
  rows = note_row,
  cols = 1:7,
  gridExpand = TRUE
)

setColWidths(
  wb_s3,
  sheet = "TableS3_MAnovaUniva",
  cols = 1:7,
  widths = c(22, 25, 8, 12, 12, 12, 10)
)

setRowHeights(
  wb_s3,
  sheet = "TableS3_MAnovaUniva",
  rows = 1,
  heights = 35
)

setRowHeights(
  wb_s3,
  sheet = "TableS3_MAnovaUniva",
  rows = note_row,
  heights = 45
)

freezePane(
  wb_s3,
  sheet = "TableS3_MAnovaUniva",
  firstActiveRow = 3
)

saveWorkbook(
  wb_s3,
  file.path(out_dir, "TableS3_MAnovaUniva_padjust.xlsx"),
  overwrite = TRUE
)

cat("\nFiles exported to:\n", out_dir, "\n")
cat("- MANOVA_edaphic_results.xlsx\n")
cat("- TableS3_MAnovaUniva_padjust.xlsx\n")
cat("- Full TSV files and manuscript tables\n")
```

CHUNK 10 — summary

r chunk: resumen — options: echo=FALSE

```r
cat(glue("
This R Markdown document performs MANOVA on edaphic variables using:
- Metadata_Completo_plus_edaphic.xlsx

Analyses included:
```

```
1) Global MANOVA:
   Y ~ Crop * Plant_Condition

2) MANOVA by crop:
   Y ~ Plant_Condition
   - Dominico Hartón
   - Gros Michel

3) Follow-up univariate ANOVAs:
   - nominal p-value
   - p-adjust using {p_adjust_method}
   - significance based on p-adjust

Correction families:
- Global / Crop: edaphic variables tested
- Global / Plant Condition: edaphic variables tested
- Global / Crop × Plant Condition: edaphic variables tested
- Dominico Hartón / Plant Condition: edaphic variables tested
- Gros Michel / Plant Condition: edaphic variables tested

The multivariate MANOVA is not corrected across response variables;
Pillai's trace is reported as the primary statistic. Assumption diagnostics are also exported:
 ↪  multivariate outliers via Mahalanobis distance (alpha = 0.001), homogeneity of covariance matrices via
 ↪  Box's M, and multicollinearity via the correlation matrix, pairs with |r| >= 0.90, and the condition
 ↪  number of the correlation matrix.

Exported:
- multivariate tables (Wilks, Pillai, Hotelling-Lawley, Roy)
- full univariate tables with p-adjust
- simplified tables for the manuscript
- consolidated Excel file
- separate Table S3 with p-value and p-adjust

Output:
./Análisis_Quimico/06_Quimica_PCA/MANOVA/
"))
```

CHUNK 11 — sessionInfo

r chunk: session_info

```r
sessionInfo()
```